CSCI 566: Deep Learning and its Applications

Yue Zhao
Thomas Lord Department of Computer Science
University of Southern California

Credits to previous versions of USC CSCI566, Stanford CS 229, 231n CMU 10-623

Recap

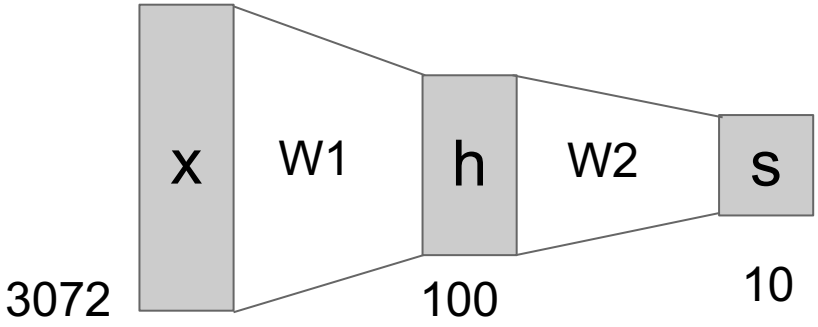
Neural Networks

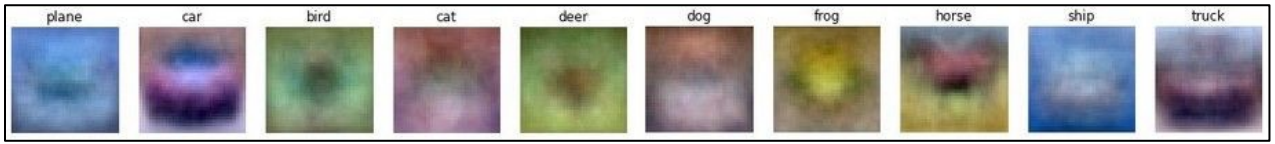
Linear score function:

$$f = Wx$$

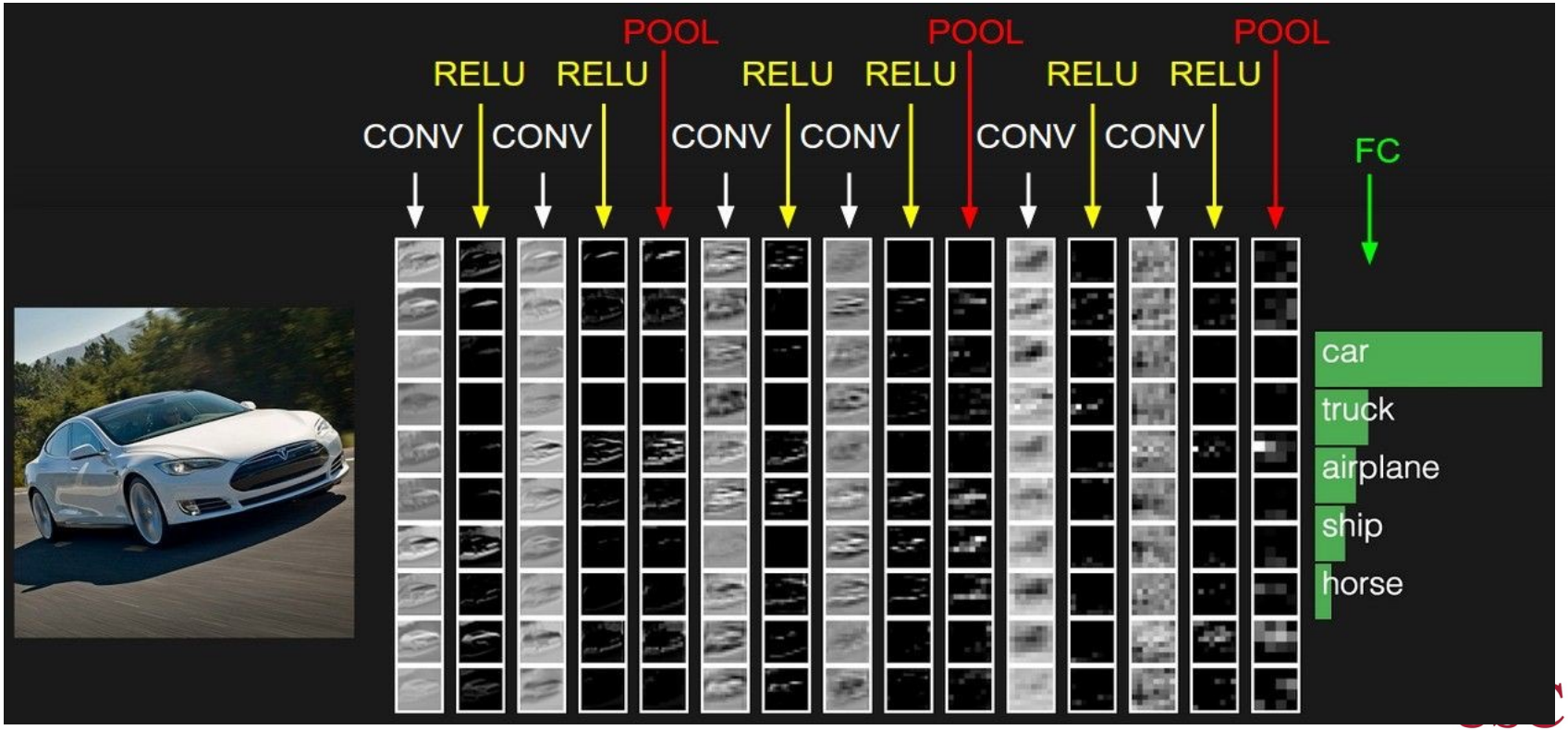
2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

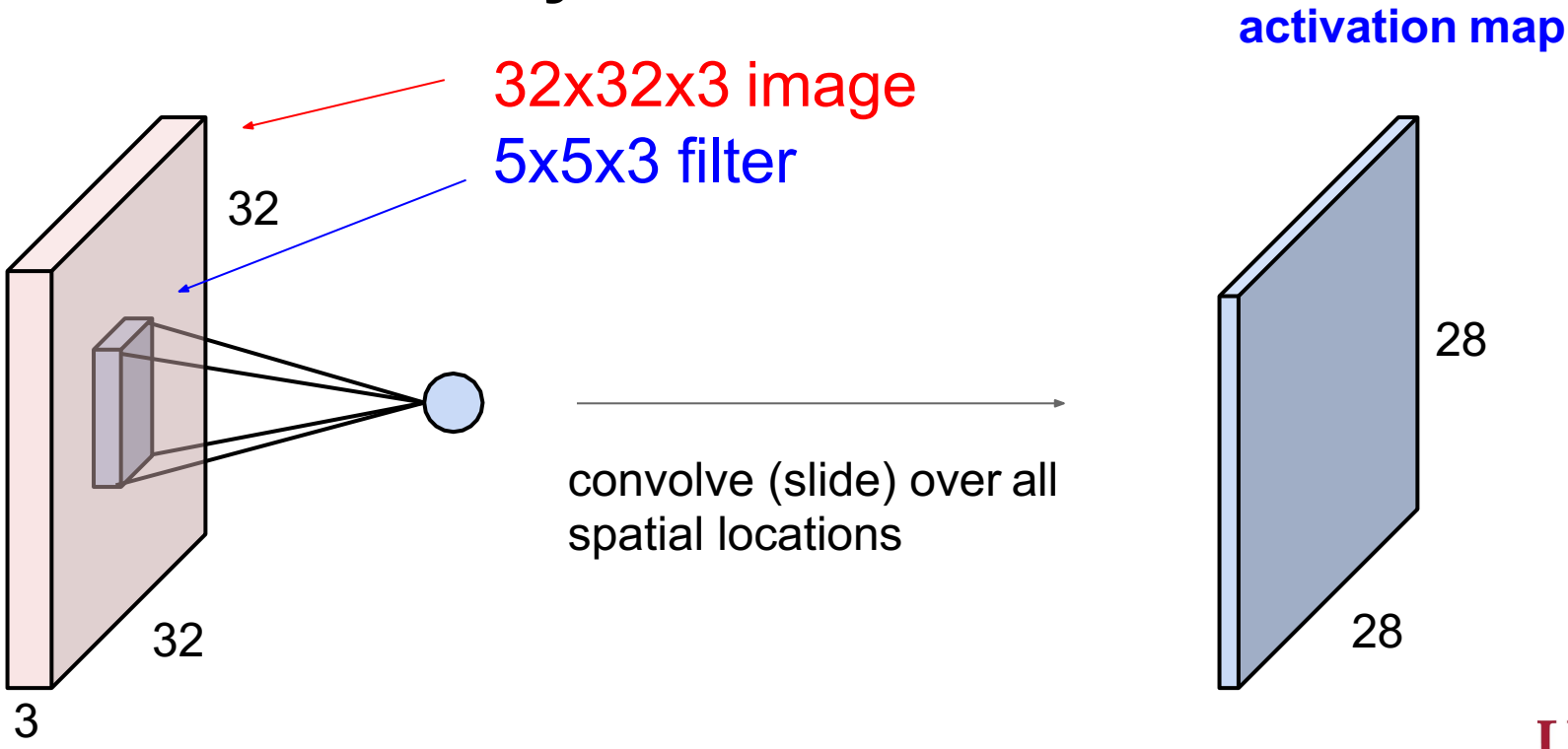




Convolutional Neural Networks

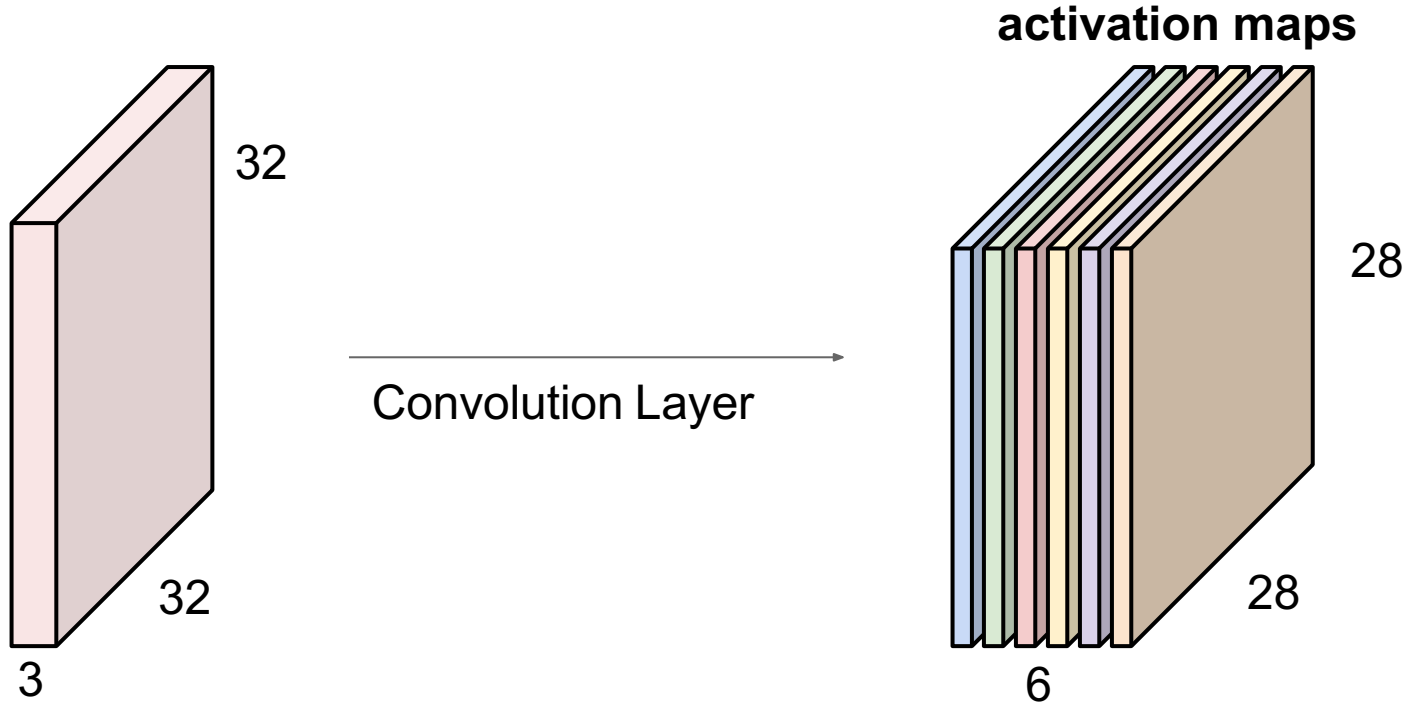


Convolutional Layer



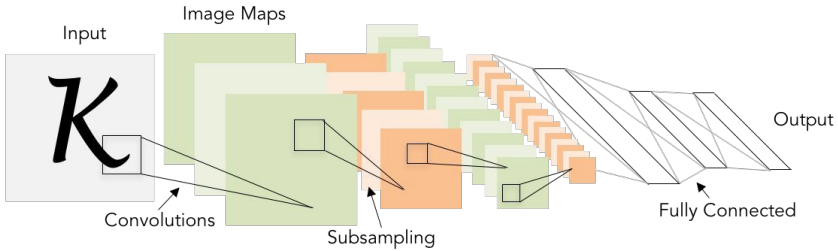
Convolutional Layer

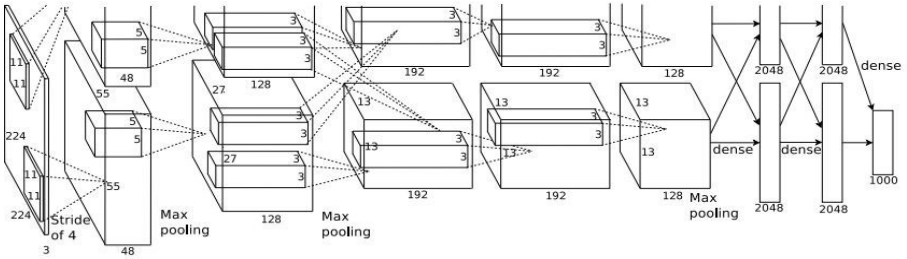
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

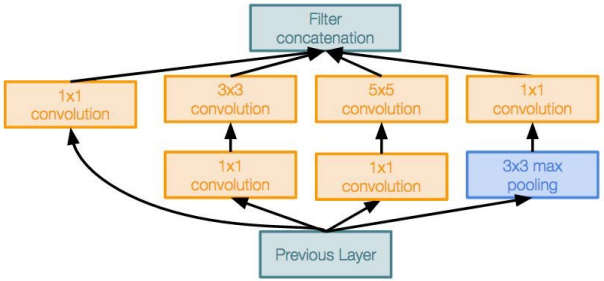


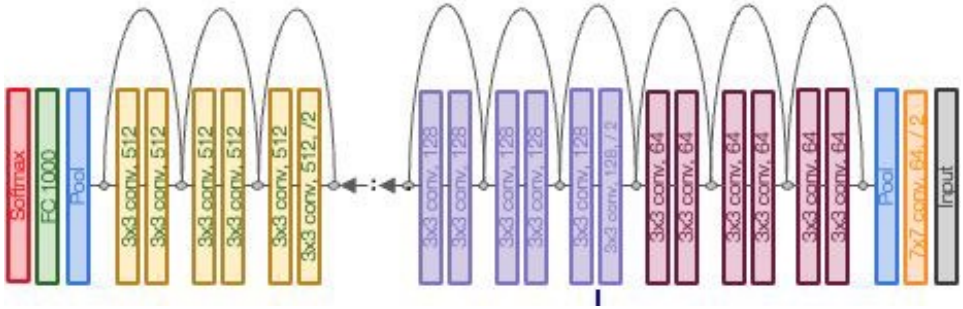
We stack these up to get a "new image" of size 28x28x6!

CNN Architectures

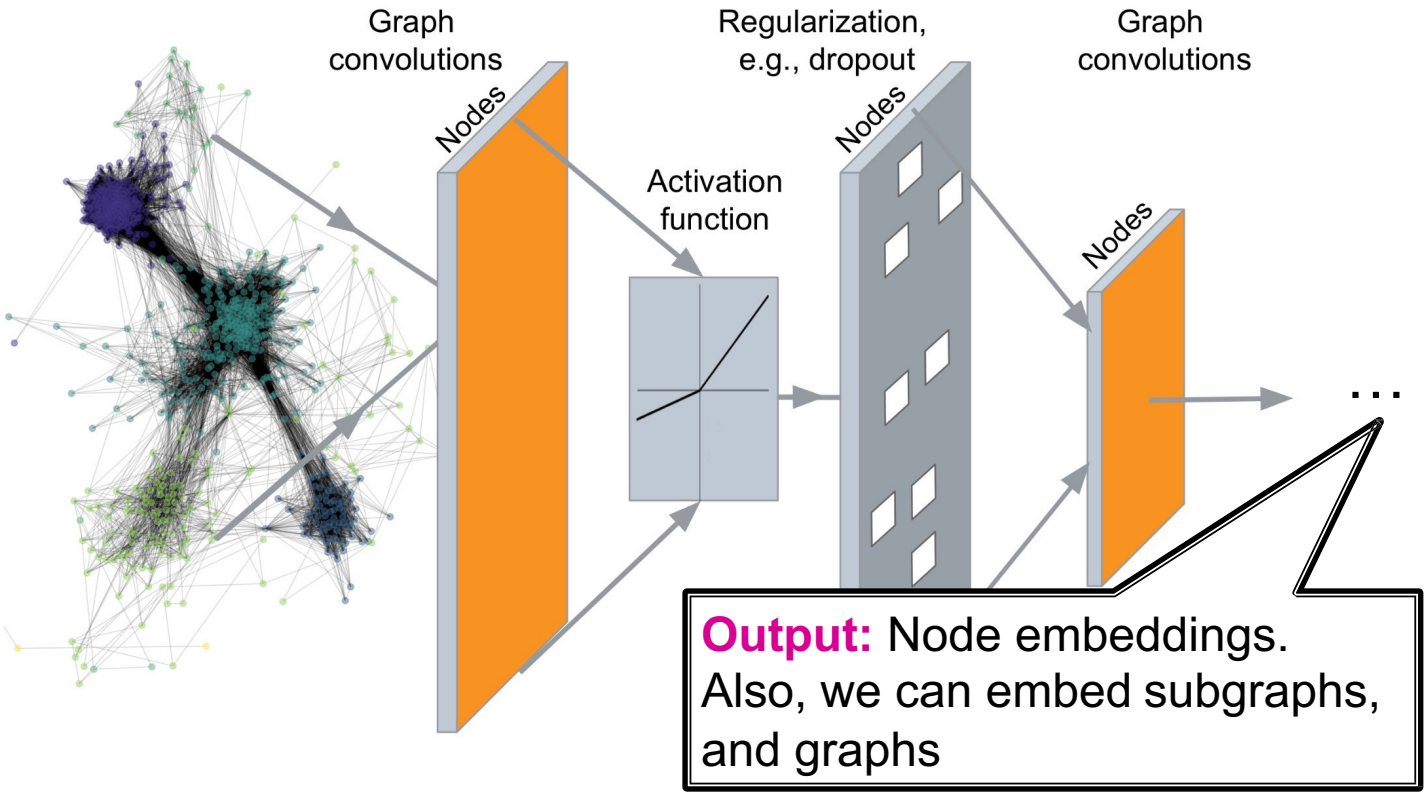








Deep Graph Encoders

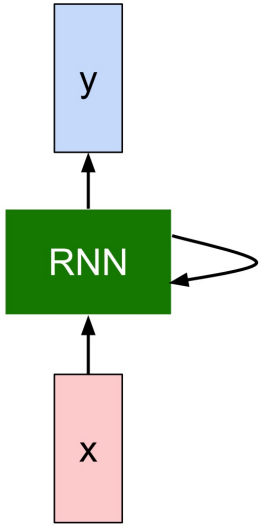


(Vanilla) Recurrent Neural Network

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

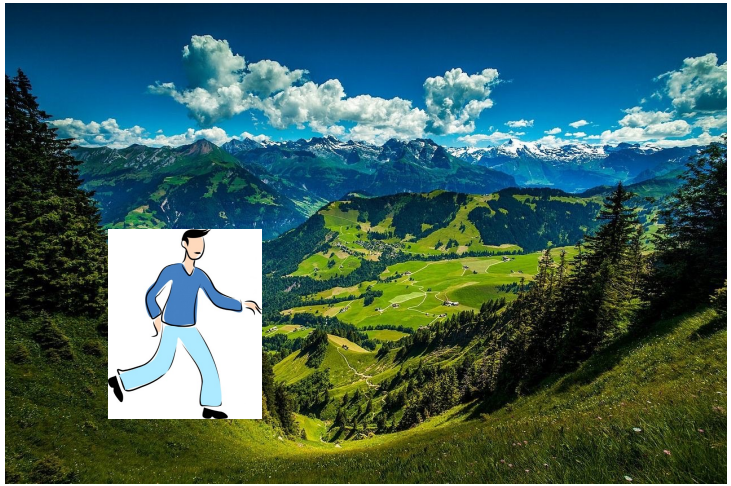
$$y_t = W_{hy}h_t$$

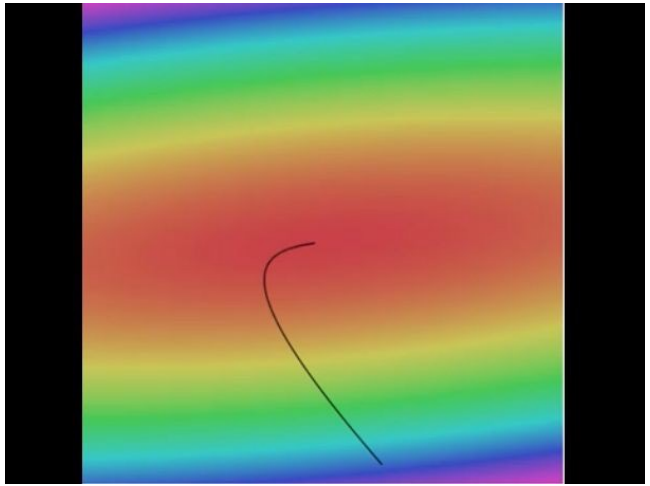


The state has a single “hidden” vector **h**

Slide credit: Stanford CS231n

Learning network parameters through optimization





```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```

Landscape image is CC0 1.0 public domain
Walking man image is CC0 1.0 public domain

Training a Model That Works

We will follow one small image-classification project.

Make learning work	Data, activations, initialization and optimization
Check generalization	Training and validation curves, regularization
Reuse a trained model	Frozen features, fine-tuning and LoRA

Each method answers a problem we can observe.

Course teaching example. Sources and assumptions in speaker notes.

Our Running Example: Flower Classification

A small dataset contains labeled photos of three flower species.

Success means classifying new photos correctly.

Course teaching example. Sources and assumptions in speaker notes.

One Flower Prediction and Its Loss

True label: rose. Compare the probability assigned to that label.

Prediction	Daisy	Rose	Tulip	Loss
Before: daisy	0.60	0.30	0.10	1.20
Improved: rose	0.20	0.70	0.10	0.36

Cross-entropy = −ln P(rose). These are illustrative predictions.

Higher probability for the correct label gives lower loss.

Course schematic. Sources and assumptions in speaker notes.

One Training Step

The label tells us how to score the current prediction.

Forward computes the loss. Backward computes gradients.

Course teaching example. Sources and assumptions in speaker notes.

Mini-Batches, Updates and Epochs

Suppose the training split has 320 flower photos.

A mini-batch contains 32 photos. One update uses that group.

10 mini-batches cover 320 photos: one epoch.

Shuffle the training photos, then begin another epoch.

An epoch counts data passes. An update changes parameters.

Course schematic. Sources and assumptions in speaker notes.

Training Neural Networks

Basics

Three Data Splits, Three Jobs

Split the flower photos before fitting preprocessing or models.

Training	Fit model weights and preprocessing statistics.
Validation	Choose settings and a checkpoint (saved model weights).
Test	Evaluate the final selected model.

Keep near-duplicate photos in the same split.

Course teaching example. Sources and assumptions in speaker notes.

Feature Scale Changes the Optimization Problem

Consider two numeric inputs to the same small classifier.

Feature A ranges from 0 to 1. Feature B ranges from 0 to 1,000.

Put comparable numeric features on sensible scales.

Fit the transformation on training data, then reuse it.

Scaling changes how parameter updates affect predictions.

Course teaching example. Sources and assumptions in speaker notes.

Data preprocessing

- Zero-center data
- Normalize data
- PCA Whitening

Data preprocessing

Why do we need preprocessing?

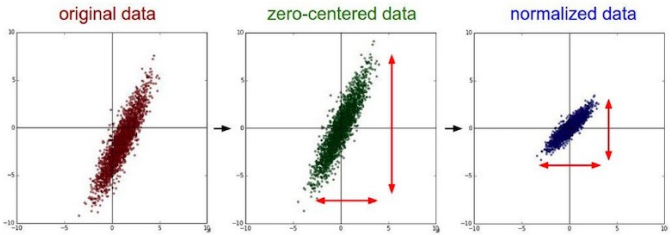
- Zero-center data
- Normalize data
- PCA Whitening

USC Viterbi School of Engineering

Data preprocessing

Why do we need preprocessing?

- Zero-center data
- Normalize data
- PCA Whitening



Centering can reduce imbalance in layer inputs.

ReLU keeps positive inputs. Input scale and offsets can affect which units activate.

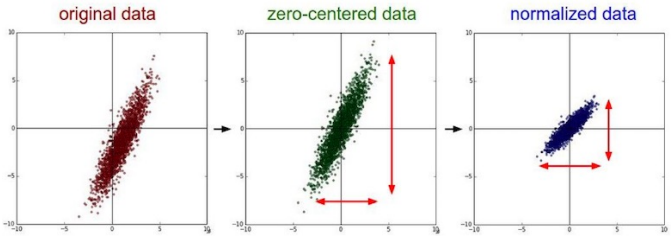
Figure from Stanford cs231n lecture slides

Data preprocessing

Why do we need preprocessing?

- Zero-center data
- Normalize data
- PCA Whitening

Different feature magnitudes can create uneven gradient scales.




```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

Figure from Stanford cs231n lecture slides

The Validation Set Uses Training Statistics

Suppose the training-set mean of one feature is 100.

Training example 120 becomes $120 - 100 = 20$.

Validation example 130 becomes $130 - 100 = 30$.

We do not recompute a new centering rule from validation data.

The fitted preprocessing rule is part of the model pipeline.

Course teaching example. Sources and assumptions in speaker notes.

Normalization Inside the Network

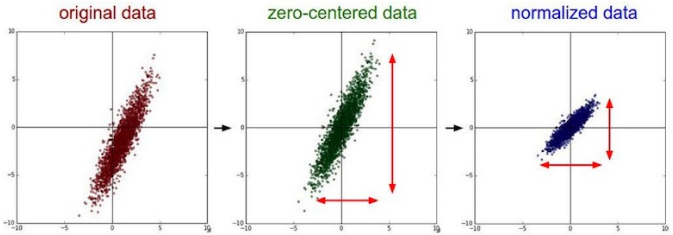
Input preprocessing happens once. Hidden activations change during training.

Normalization also changes the optimization behavior.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

Normalization

- Normalize the distribution of activations to have zero mean and unit variance



Center and scale hidden activations inside the network.

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

BatchNorm: A Complete Numeric Example

One feature across four examples: 2, 4, 6 and 8.

Input	2	4	6	8
Subtract mean 5	−3	−1	1	3
Divide by √5	−1.34	−0.45	0.45	1.34

Variance = 5. For this illustration, $\gamma = 1$, $\beta = 0$; ε is omitted.

Training learns γ and β after this normalization.

Course schematic. Sources and assumptions in speaker notes.

Normalization Adjusts Mean and Scale

Centering and scaling do not force an arbitrary distribution to become Gaussian.

What changes

Subtract the mean.
Divide by a scale estimate.

What does not follow

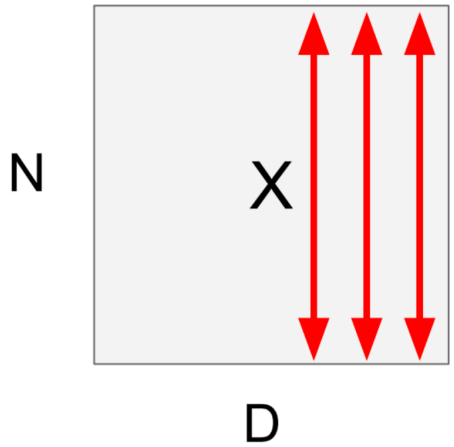
The resulting values need not follow a bell-shaped distribution.

BatchNorm then learns an additional scale and shift.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

Batch normalization

- Compute the empirical mean and variance independently for each dimension and normalize.



USC Viterbi School of Engineering

Batch normalization

- Vanilla Normalization:

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

Batch normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

[Ioffe and Szegedy, 2015]

Batch normalization

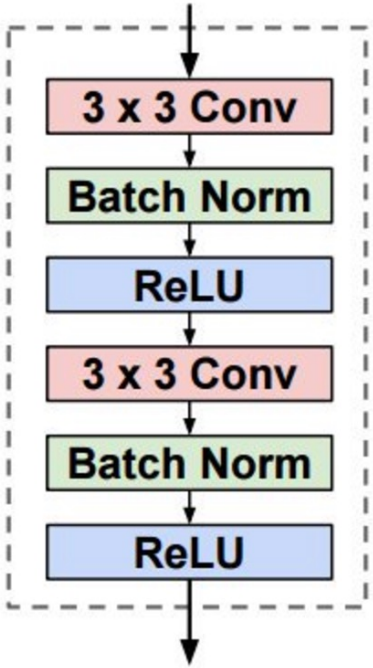
- Possible benefits, depending on the training setup:
  - Can improve optimization and gradient flow
  - Can allow a higher learning rate
  - Can reduce sensitivity to initialization
  - Batch statistics can add training-time noise

Batch normalization

- At test time
  - The mean and variance are not computed based on the batch.
  - Use stored running means and variances from training.

Batch normalization

- Usually added after convolutional layers or fully connected layers
- before non-linearities



Johnson, Justin, Alexandre Alahi, and Li Fei-Fei. "Perceptual losses for real-time style transfer and super-resolution." *European Conference on Computer Vision*. Springer International Publishing, 2016.

BatchNorm Behaves Differently at Evaluation

The flower classifier may receive one new photo at a time.

During training

Use mini-batch statistics.
Update running estimates.
Learn scale and shift.

During evaluation

Usually use stored running statistics.
Keep the same learned scale and shift.

Switch the model to evaluation mode when evaluating.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

BatchNorm and LayerNorm Group Values Differently

Both normalize activations, using different groups of values.

BatchNorm

Compare one feature across examples in a mini-batch.
Common in CNNs.

LayerNorm

Compare features within one example or token.
Common in Transformers.

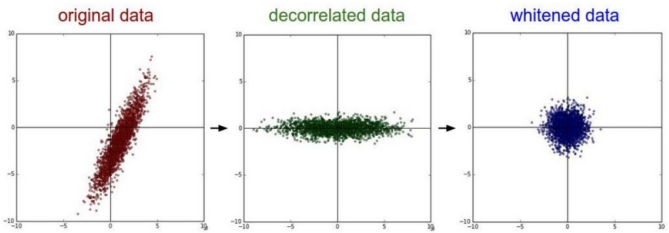
Identify which values supply the normalization statistics.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

Data preprocessing

Why do we need preprocessing?

- Zero-center data
- Normalize data
- PCA Whitening



Remove correlation between input features

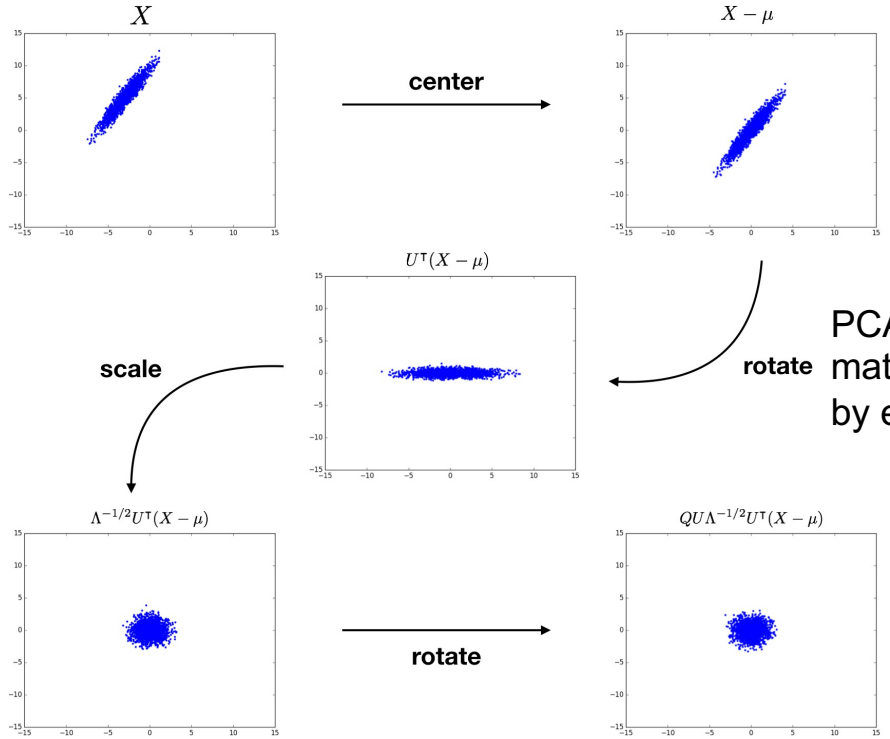
Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

Data preprocessing

Whitening transforms the data into a space where the features are uncorrelated, and all have the same variance

Whitening: inverse square root of the eigenvalue matrix. normalizes the variance across all dimensions



Not common to use PCA whitening

Image Preprocessing Follows the Model

For a pretrained classifier, use its expected input convention.

Image size	Resize and crop using the model specification.
Channel values	Use the expected numeric range and channel order.
Normalization	Apply the specified per-channel mean and scale.

A consistent input pipeline is part of successful transfer.

Course teaching example. Sources and assumptions in speaker notes.

Important Interview Questions

Question: How would you handle missing data in a dataset?

Answer: Missing data can be handled in several ways:

1. **Imputation**: Replace missing values with a statistic like mean, median, or mode, or use a model to predict missing values.
2. **Removal: Check how deletion changes data coverage. Fit any imputation rule on training data.**
3. **Indicator Variable**: For some algorithms, it can be useful to create a new variable indicating the presence of missing data along with imputation.

Important Interview Questions

Question: How do you ensure that preprocessing steps applied to the training data are also correctly applied to the test data?

Answer: To ensure consistency, preprocessing steps should be defined and fitted on the training data, then applied to the test data.

For example, if scaling is applied, the scale parameters (like mean and standard deviation for standardization) should be derived from the training data and then used to scale the test data. This prevents data leakage and ensures that the model's performance is evaluated on the same terms as it was trained.

Important Interview Questions

The `preprocessing` module provides the `StandardScaler` utility class, which is a quick and easy way to perform the following operation on an array-like dataset:

```
>>> from sklearn import preprocessing
>>> import numpy as np
>>> X_train = np.array([[ 1., -1.,  2.],
...                     [ 2.,  0.,  0.],
...                     [ 0.,  1.,  1.]])
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> scaler
StandardScaler()

>>> scaler.mean_
array([1. ..., 0. ..., 0.33...])

>>> scaler.scale_
array([0.81..., 0.81..., 1.24...])

>>> X_scaled = scaler.transform(X_train)
>>> X_scaled
array([[ 0.  ..., -1.22...,  1.33...],
       [ 1.22...,  0.  ..., -0.26...],
       [-1.22...,  1.22..., -1.06...]])
```

Scaled data has zero mean and unit variance:

USC Viterbi School of Engineering

Training Neural Networks

Activation Functions (Recap Note 3)

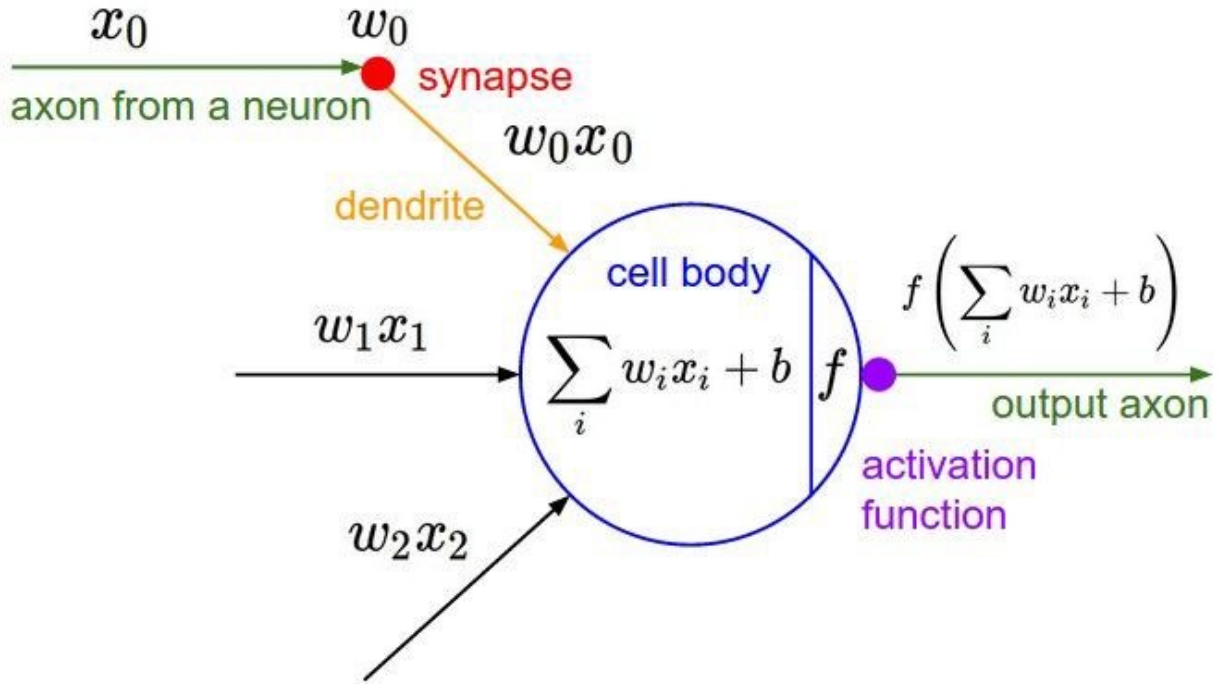
Why Put an Activation Between Linear Layers?

Two linear layers without a nonlinearity are still one linear map.

The activation lets stacked layers model nonlinear patterns.

Course teaching example. Sources and assumptions in speaker notes.

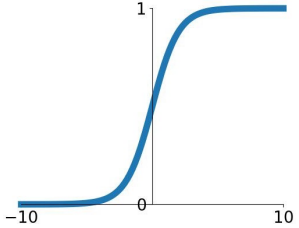
Activation Functions



Activation Functions

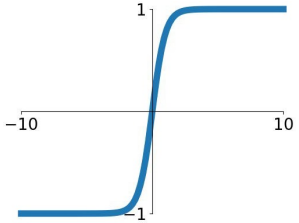
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



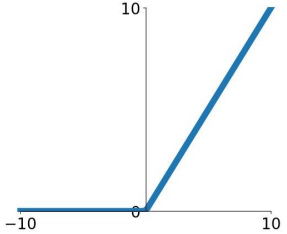
tanh

$$\tanh(x)$$



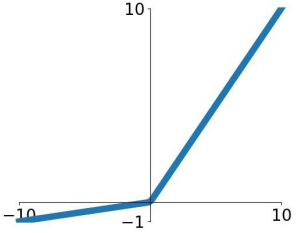
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

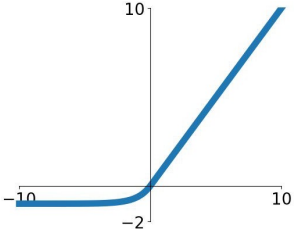


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

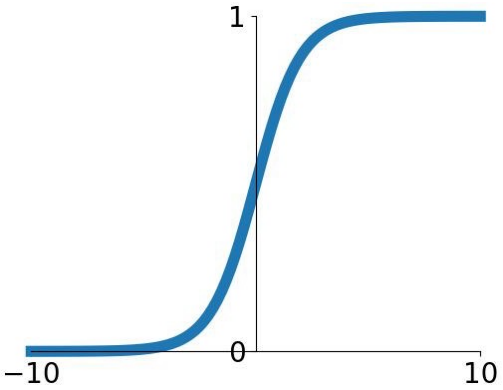
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



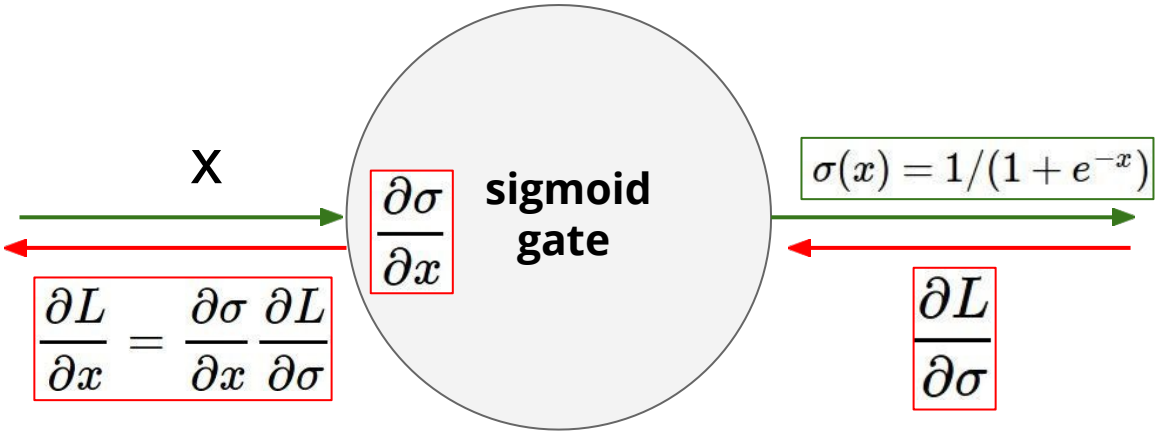
Activation Functions

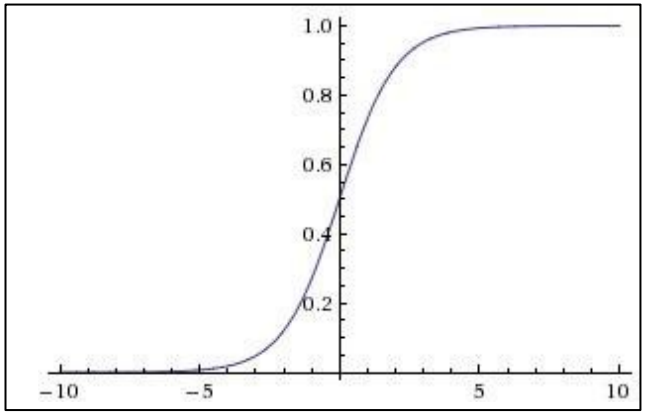
$$\sigma(x) = 1/(1 + e^{-x})$$



Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron





$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

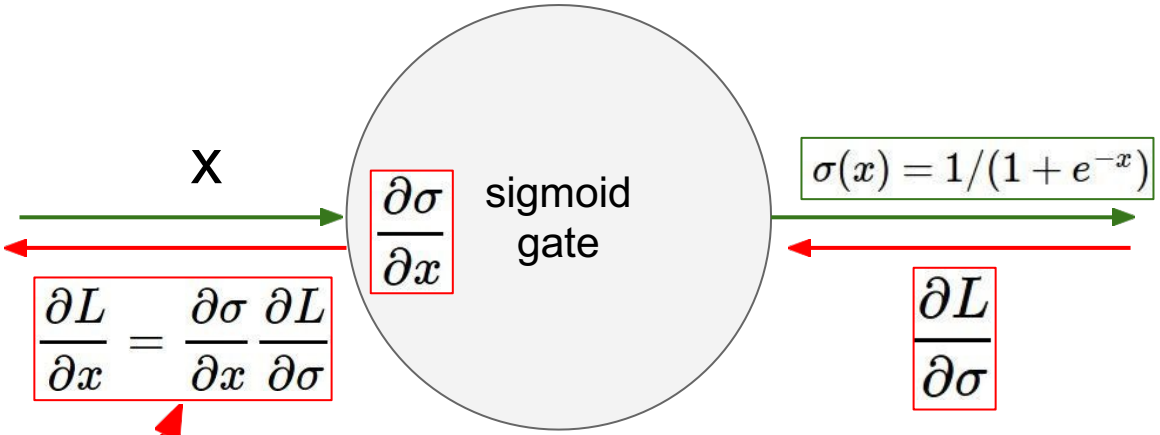
Sigmoid Saturation and Gradients

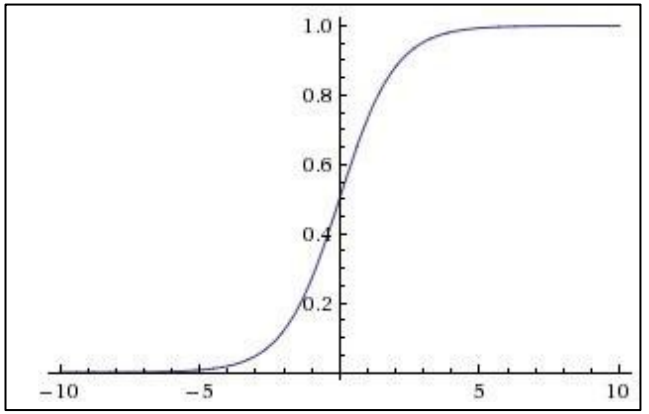
The local derivative tells us how much gradient passes through this gate.

Input x	Sigmoid output	Local derivative
−10	≈ 0.000045	≈ 0.000045
0	0.50	0.25
10	≈ 0.999955	≈ 0.000045

Near saturation, very little gradient passes backward.

Course schematic. Sources and assumptions in speaker notes.





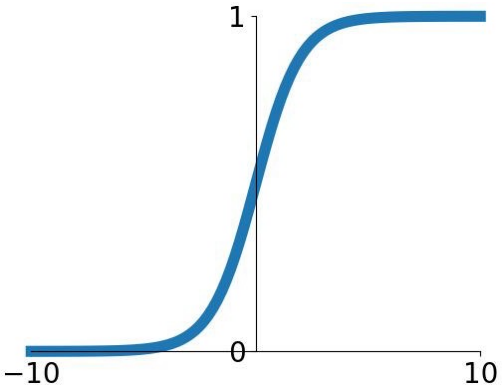
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

Why is this a problem?

A saturated sigmoid passes a very small gradient, making upstream updates slow.

Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$



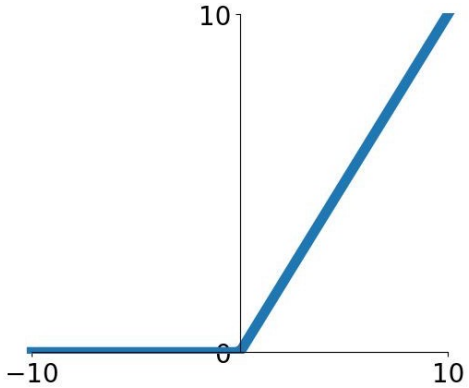
Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

3 problems:

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered
3. exp() is a bit compute expensive

Activation Functions



ReLU
(Rectified Linear Unit)

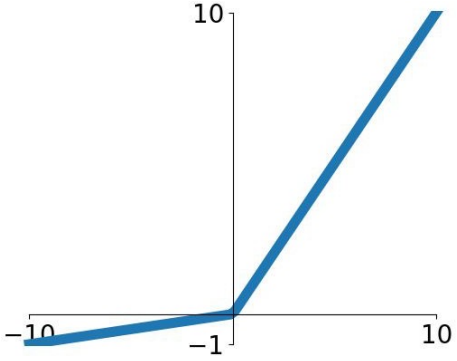
- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

[Krizhevsky et al., 2012]

Activation Functions

[Mass et al., 2013]
[He et al., 2015]



- Does not saturate
- Computationally efficient
- A small negative slope preserves a local gradient path.

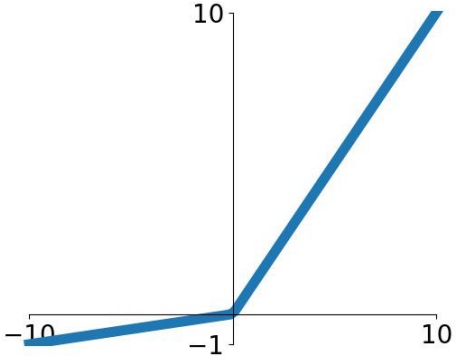
- Compare activation choices using validation results.

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Activation Functions

[Mass et al., 2013]
[He et al., 2015]



- Does not saturate
- Computationally efficient
- Leaky fixes α; PReLU learns α.
- **Negative inputs pass a gradient.**

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Parametric Rectifier (PReLU)

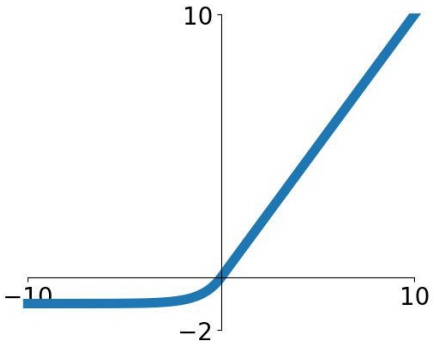
$$f(x) = \max(\alpha x, x)$$

backprop into α (parameter)

[Clevert et al., 2015]

Activation Functions

Exponential Linear Units (ELU)



- All benefits of ReLU
- Closer to zero mean outputs
- Negative saturation regime compared with Leaky ReLU adds some robustness to noise

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\left(\exp(x) - 1\right) & \text{if } x \leq 0 \end{cases}$$

(Alpha default = 1)

- Computation requires exp()

Activation Choice Depends on the Architecture

Choose an activation that fits the role of the layer.

CNN / MLP hidden layers	ReLU-family activations are common starting points.
Transformer hidden layers	GELU or gated SiLU variants are common choices.
Outputs and gates	Sigmoid or tanh can still have specific roles.

Match initialization to the activation and architecture.

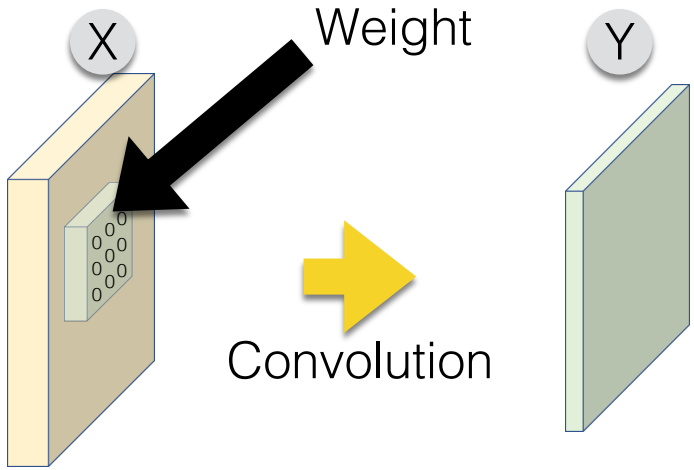
Course teaching example. Sources and assumptions in speaker notes.

Training Neural Networks

Weight W Initialization

Weight initialization

If all weights are 0?



Slide credit: CS 231N

USC Viterbi School of Engineering

Why Identical Hidden Units Stay Identical

Suppose two hidden units start with exactly the same parameters.

Same start

Same inputs and parameters produce the same outputs.

Same update

Under symmetric connections, they receive the same gradients.

Different random starts let hidden units learn different features.

Course teaching example. Sources and assumptions in speaker notes.

Zero Initialization and Hidden-Unit Symmetry

Consider several hidden units with identical incoming and outgoing connections.

Identical hidden units

They can remain identical through subsequent updates.

Other parameter roles

Zero biases can be valid.
LoRA also uses a zero factor with a nonzero random factor.

Initialization must account for the parameter's role.

Hu et al., LoRA (ICLR 2022). Course schematic.

Weight initialization

- Small random numbers
- Xavier
- He / Kaiming: ReLU-aware scale

Slide credit: CS 231N

Small Random Numbers

- Sample from a gaussian distribution with 0 mean and given standard deviation (such as 0.01)

$$W \sim \mathcal{N}(\mu, \sigma^2)$$

- A fixed scale such as 0.01 can shrink signals in a deep network.

```
W = 0.01 * np.random.randn(Din, Dout)
```

Signals Must Survive Many Layers

Before useful learning begins, check the scale of activations.

Too small	Signals or gradients can shrink across depth.
Too large	Signals or gradients can grow or saturate.
A suitable scale	Xavier / He rules account for layer size and activation.

Initialization gives optimization a usable starting point.

Course teaching example. Sources and assumptions in speaker notes.

Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

What will happen to the activations for the last layer?

Slide credit: CS 231N

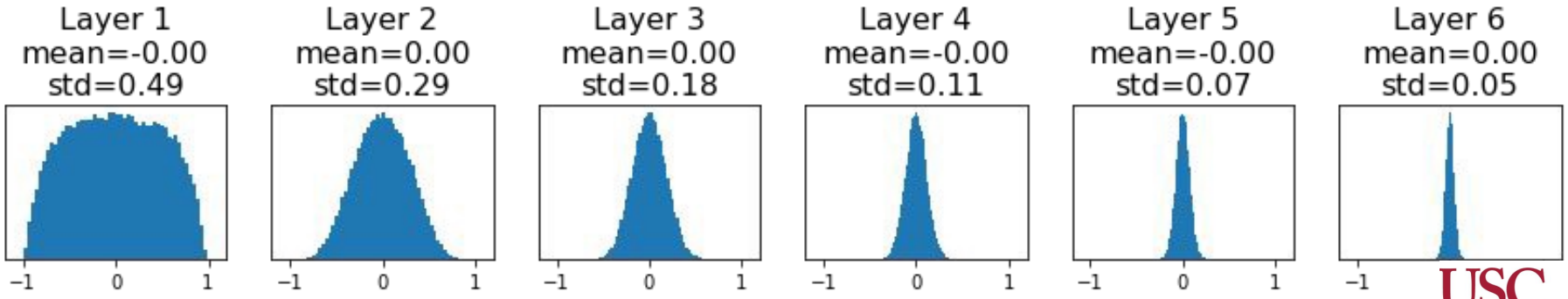
Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

All activations tend to go zero for deeper network layers

Q: What do the gradients dL/dW look like?



Slide credit: CS 231N

Weight Initialization: Activation statistics

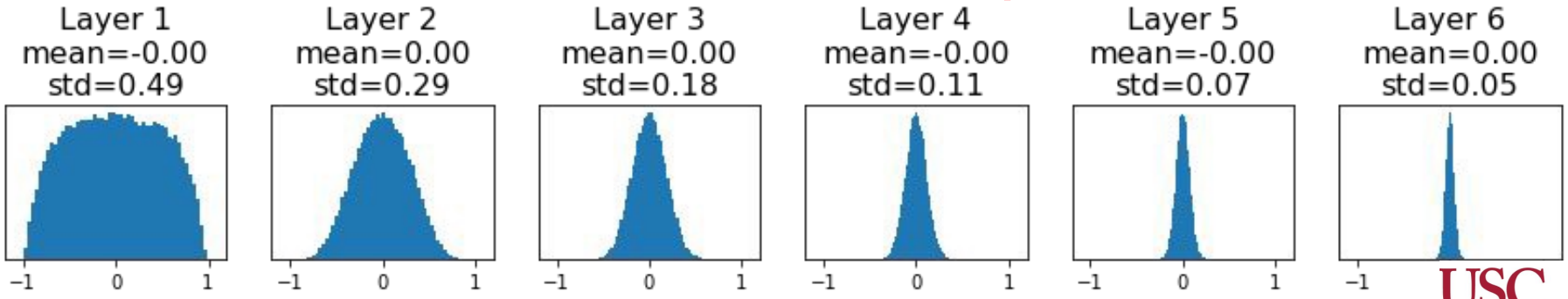
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

All activations tend to zero for deeper network layers

Q: What do the gradients dL/dW look like?

A: Tiny gradients; slow updates.



Slide credit: CS 231N

Xavier Initialization

- Set the variance according to # of input neurons

- Intuition: balance signal scales across layers.

$$W \sim \mathcal{N}(\mu, \sigma^2) \longleftarrow Var(W) = \frac{1}{n_{in}}$$

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

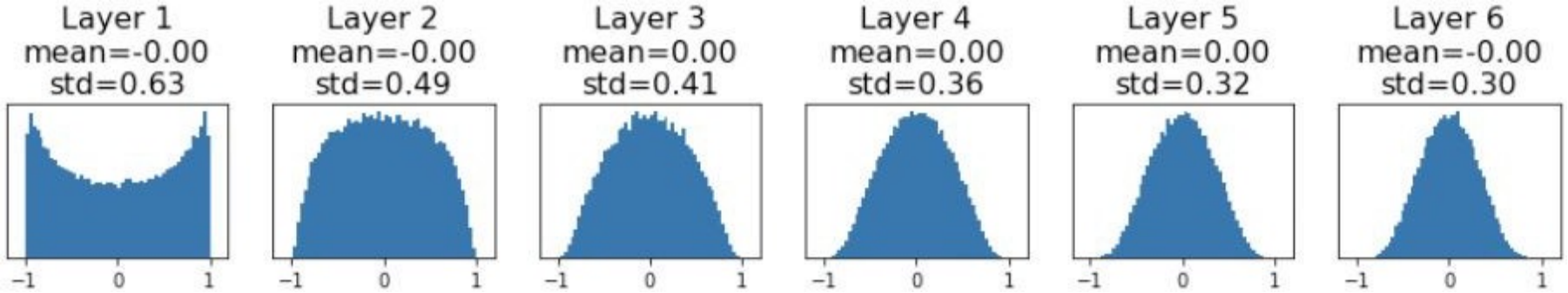
USC Viterbi School of Engineering

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!



Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

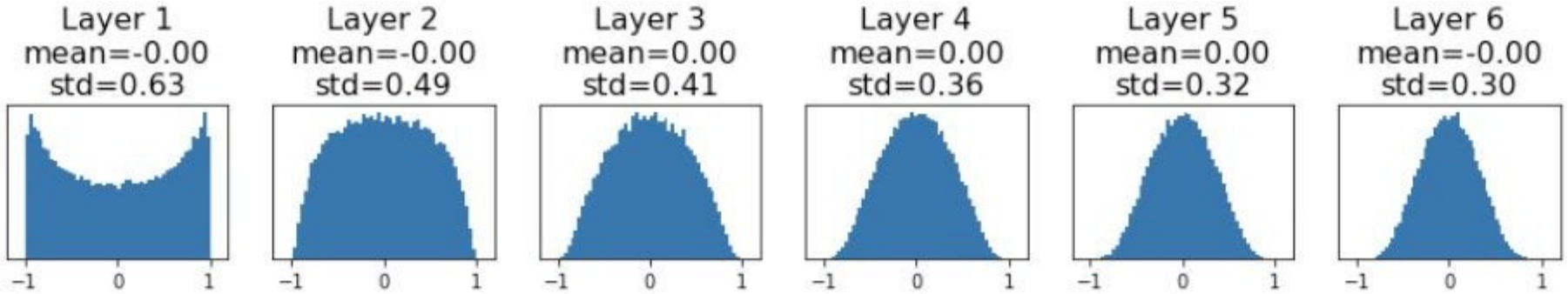
Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels



Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= \ldots=Var(x_{Din})$

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+...+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= \ldots=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= \ldots=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din})$
[substituting value of y]

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+...+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= ...=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1w_1+x_2w_2+...+x_{Din}w_{Din})$

$= Din\ Var(x_iw_i)$

[Assume all x_i, w_i are iid]

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+...+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= ...=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$$Var(y) = Var(x_1w_1+x_2w_2+...+x_{Din}w_{Din})$$
$$= Din\ Var(x_iw_i)$$
$$= Din\ Var(x_i)\ Var(w_i)$$

[Assume all x_i, w_i are zero mean]

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+...+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= ...=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$$Var(y) = Var(x_1w_1+x_2w_2+...+x_{Din}w_{Din})$$
$$= Din\ Var(x_iw_i)$$
$$= Din\ Var(x_i)\ Var(w_i)$$

[Assume all x_i, w_i are iid]

So, $Var(y) = Var(x_i)$ only when $Var(w_i) = 1/Din$

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: What about ReLU?

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Change from tanh to ReLU

Slide credit: CS 231N

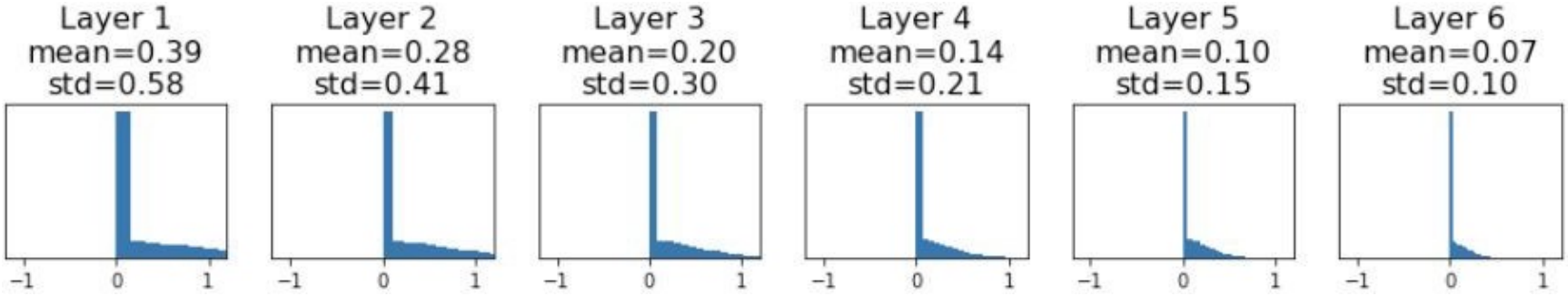
Weight Initialization: What about ReLU?

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Change from tanh to ReLU

ReLU removes negative inputs, changing the signal scale.

Signal scale shrinks here.



Slide credit: CS 231N

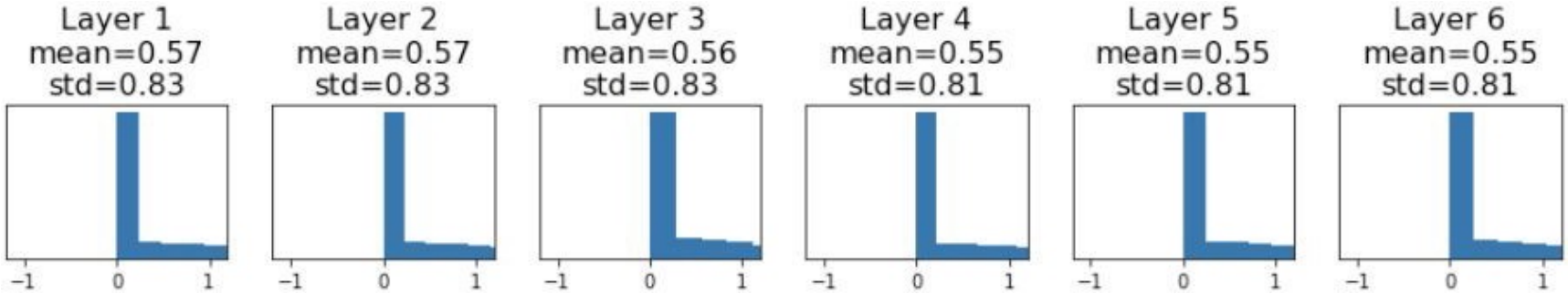
USC Viterbi School of Engineering

Weight Initialization: Kaiming / MSRA Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

ReLU correction: std = sqrt(2 / Din)

"Just right": Activations are nicely scaled for all layers!



He et al, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015

Slide credit: CS 231N

He Initialization Accounts for ReLU Layers

ReLU removes negative activations, changing the signal's second moment.

He scaling is a common starting point for ReLU layers.

Course teaching example. Sources and assumptions in speaker notes.

Kaiming Initialization

- MIT Professor
- Author of ResNet (covered in this our CNN lecture), Faster R-CNN, and more

Initialization Follows the Layer Design

Use the model or library initialization intended for that architecture.

ReLU-family layer	He / Kaiming scaling is a common choice.
Other feed-forward layers	Xavier scaling can be appropriate.
Pretrained backbone	Start from its learned weights when transferring.

Changing an activation may require changing initialization.

Course teaching example. Sources and assumptions in speaker notes.

Xavier and He Initialization

Choose a starting scale that matches the layer width and activation.

Xavier / Glorot

Balances input and output signal scales.
A common choice for tanh-like layers.

He / Kaiming

Accounts for ReLU removing negative values.
A common choice for ReLU layers.

The activation affects the suitable weight scale.

Course schematic. Sources and assumptions in speaker notes.

Initialization Helps Gradient Flow

Good scaling can reduce shrinking or growing signals across layers.

Initialization	Sets the starting scale of weights.
Architecture	Depth, activations and residual paths also matter.
Training	Inspect actual loss and gradient behavior.

A suitable initialization helps without guaranteeing convergence.

Course teaching example. Sources and assumptions in speaker notes.

Training Neural Networks

Optimization

Gradient Direction and Step Size

The gradient and the learning rate play different roles.

Current weight: 2.0 Gradient: −0.5 Learning rate: 0.1

$$\text{New weight} = 2.0 - 0.1 \times (-0.5) = 2.05$$

The gradient is local. The chosen step must still be checked.

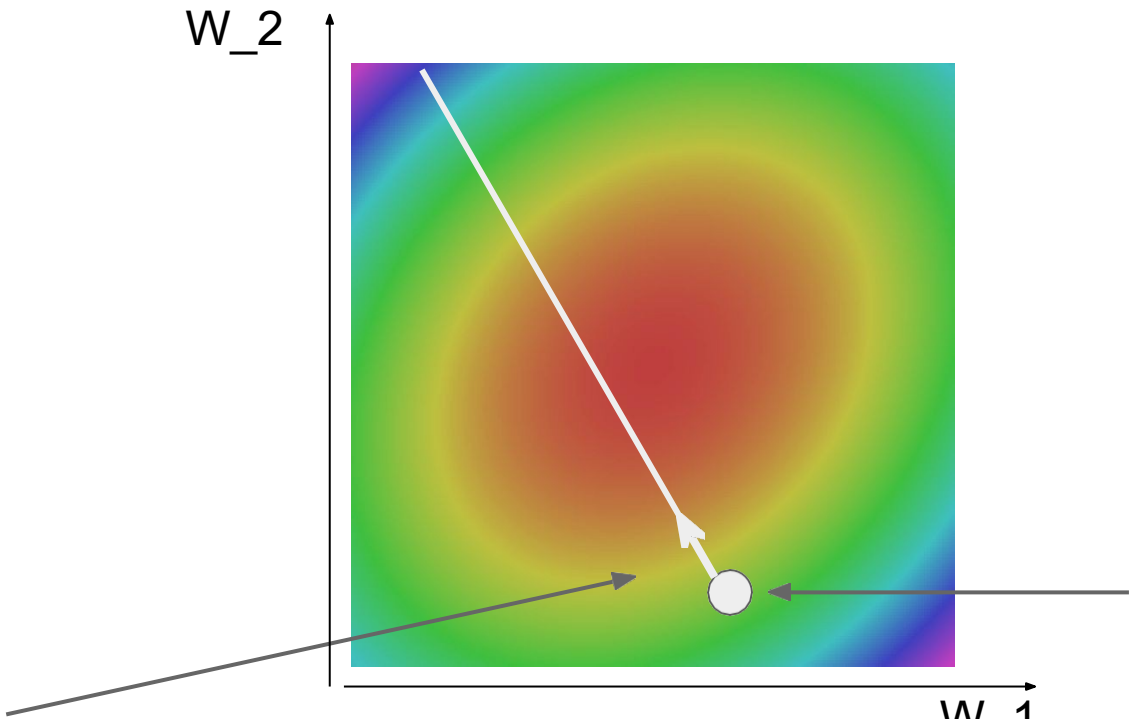
The learning rate controls how far we move.

Course teaching example. Sources and assumptions in speaker notes.

Gradient Descent

Use **all** data here to update the model weights

```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```


W_2
W_1
original W
negative gradient direction

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

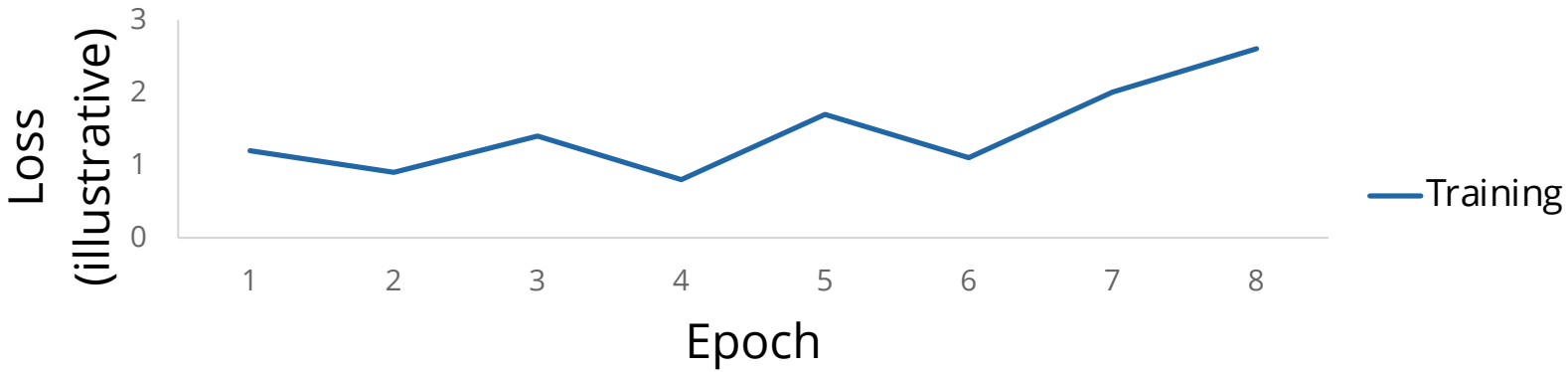
Full sum expensive when **N** is large!

Approximate sum using a **minibatch** of examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent

while True:
  data_batch = sample_training_data(data, 256) # sample 256 examples
  weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
  weights += - step_size * weights_grad # perform parameter update
```

A Learning Rate Can Be Too Large

A plausible training-loss trace for our flower classifier.

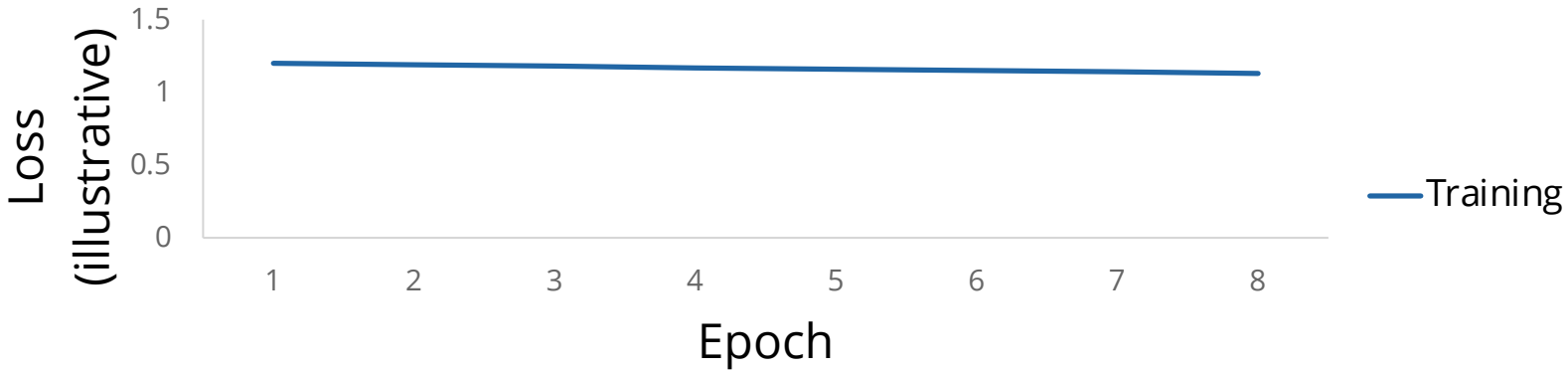


Large jumps can make training unstable.

Course teaching example. Sources and assumptions in speaker notes.

A Learning Rate Can Be Too Small

The same training budget can yield very little progress.

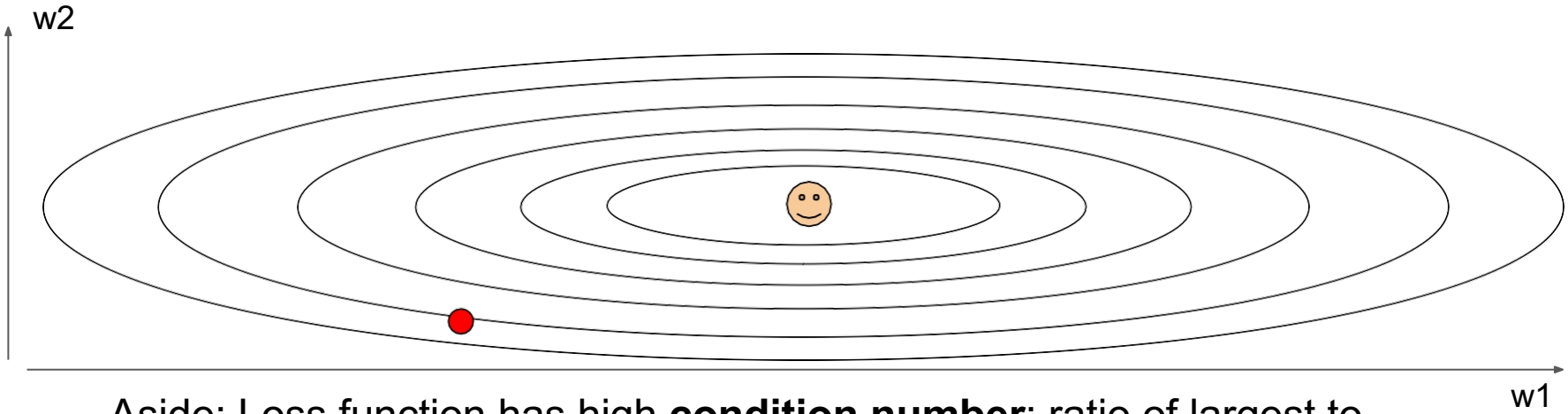


Slow progress motivates a controlled learning-rate check.

Course teaching example. Sources and assumptions in speaker notes.

Optimization: Problem #1 with SGD

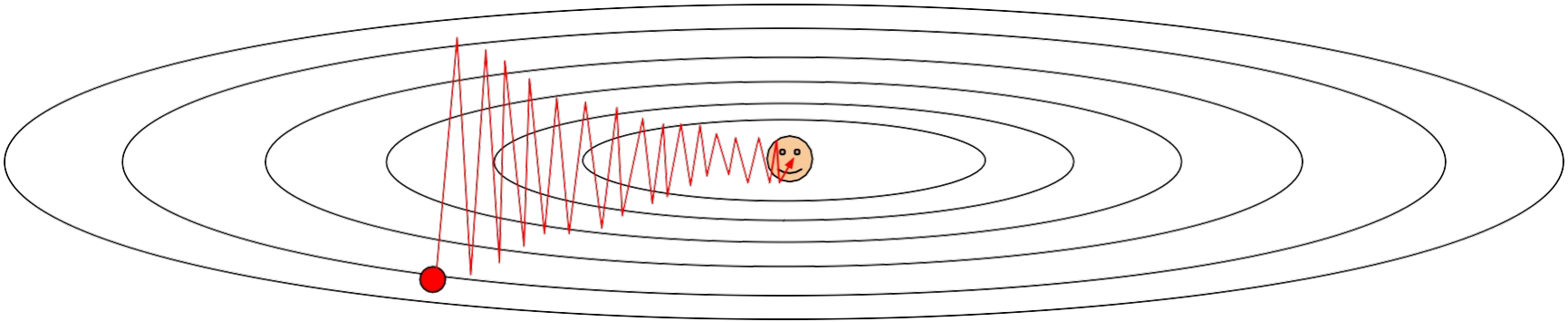
What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?



Aside: Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large. (**no need to know details though**)

Optimization: Problem #1 with SGD

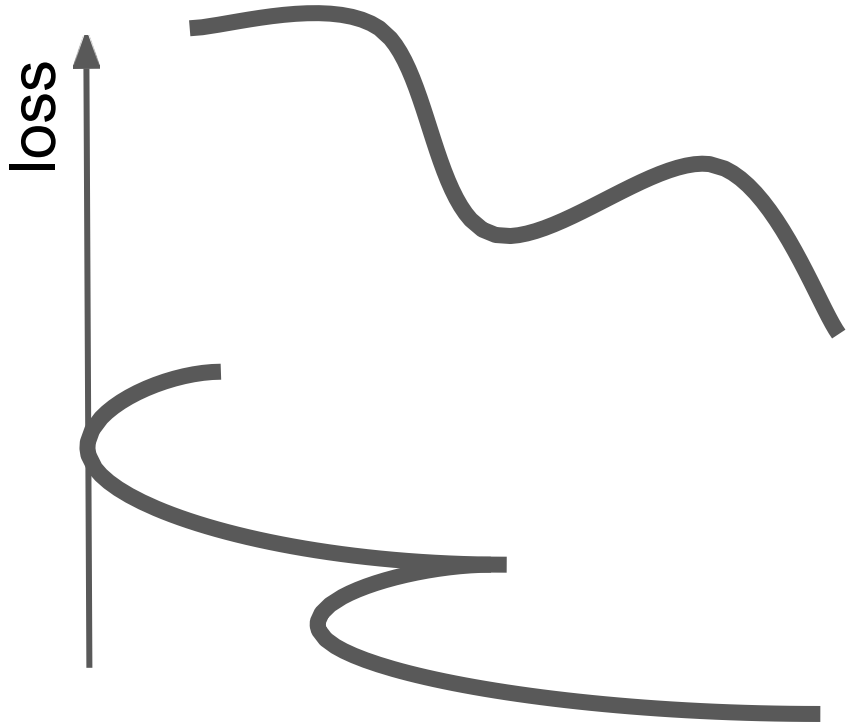
What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter (zigzag) along steep direction



Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

Optimization: Problem #2 with SGD

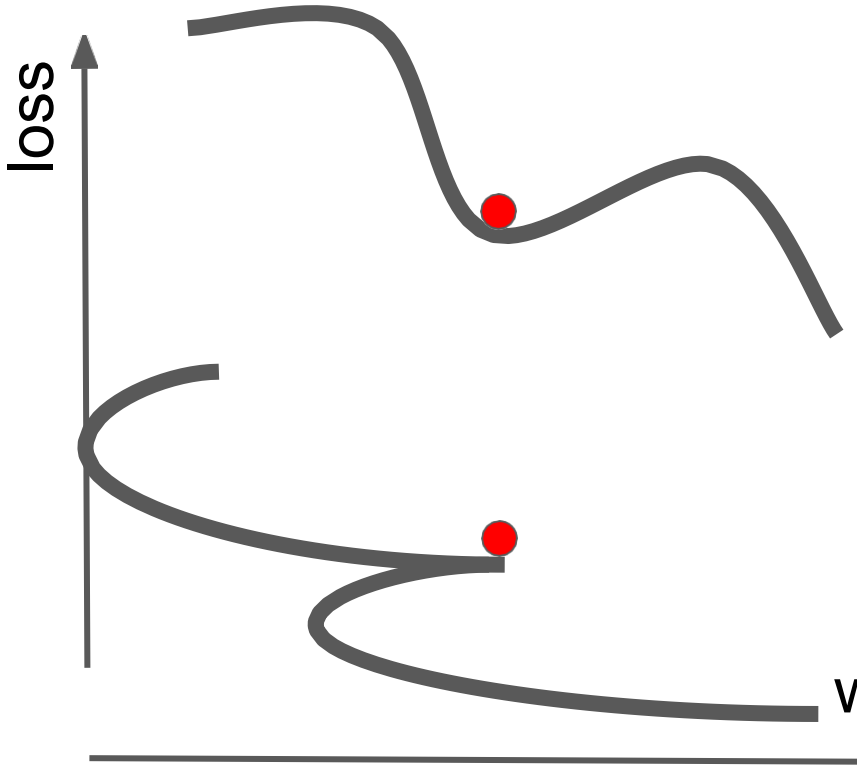
What if the loss function has a **local minima** or **saddle point**?



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

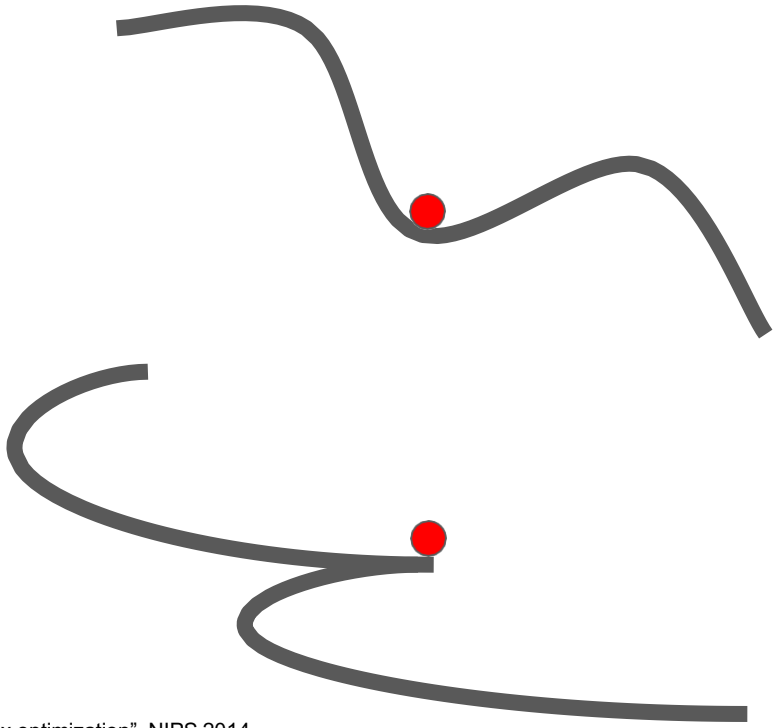
Zero gradient, gradient descent gets stuck



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

Saddle points much more common in high dimension



Dauphin et al, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”, NIPS 2014

Optimization: Problem #2 with SGD

saddle point in two dimension

$$f(x, y) = x^2 - y^2$$

$$\frac{\partial}{\partial x}(x^2 - y^2) = 2x \rightarrow 2(0) = 0$$

$$\frac{\partial}{\partial y}(x^2 - y^2) = -2y \rightarrow -2(0) = 0$$

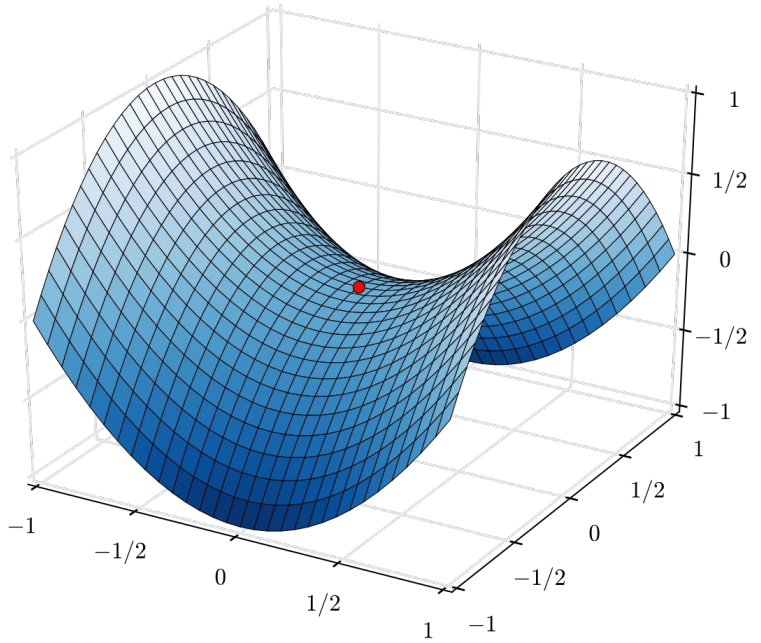


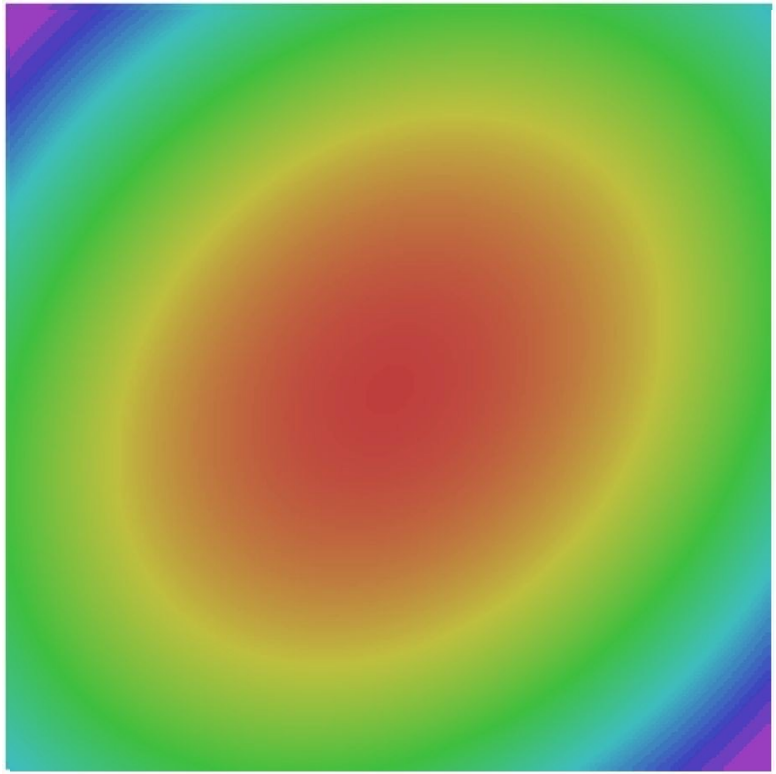
Image source: https://en.wikipedia.org/wiki/Saddle_point

Optimization: Problem #3 with SGD

Our gradients come from minibatches so they can be noisy!

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$



Mini-Batches Give Different Gradient Estimates

One batch may contain mostly easy photos, another difficult photos.

A single mini-batch

A cheap but noisy estimate of the training objective.

Across many mini-batches

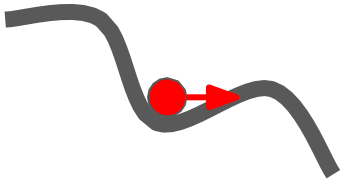
Repeated updates can make progress
without every step lowering loss.

Judge the trend over many updates.

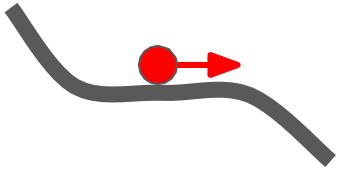
Course teaching example. Sources and assumptions in speaker notes.

SGD + Momentum

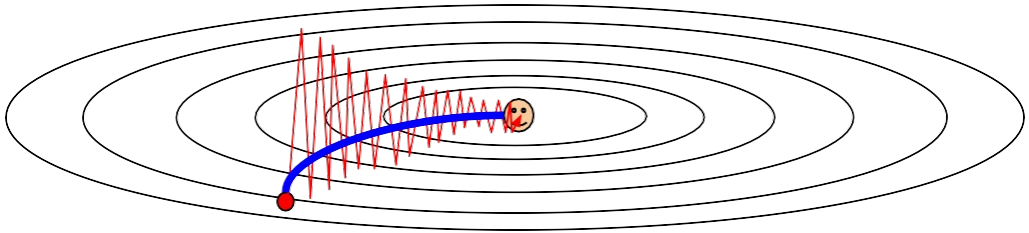
Local Minima



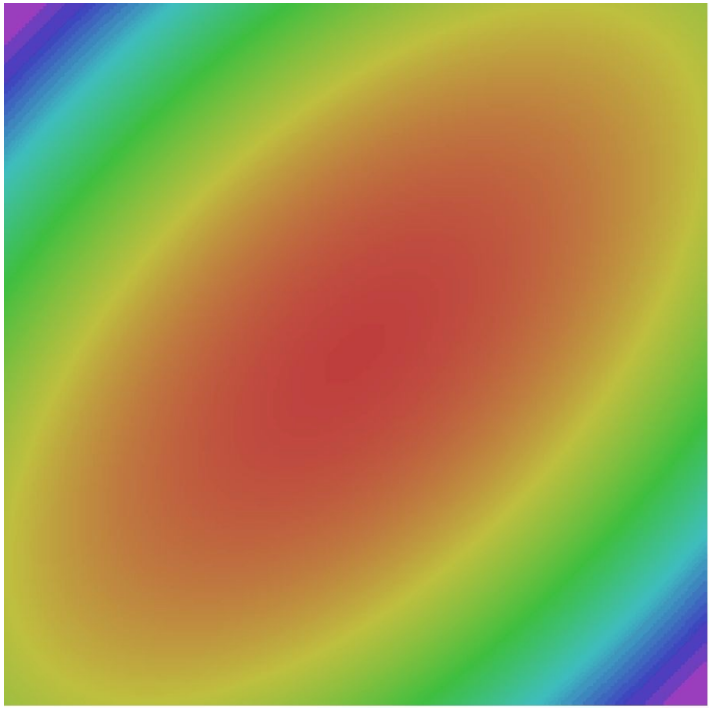
Saddle points



Poor Conditioning



Gradient Noise



SGD SGD+Momentum

Momentum Reuses Recent Gradient Direction

Updates can zigzag even when the long-term direction is consistent.

Momentum smooths the update direction over time.

Course teaching example. Sources and assumptions in speaker notes.

SGD: the simple two line update code

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99

Reduce the oscillations in the rapid change directions.

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

SGD + Momentum:
alternative equivalent formulation

SGD+Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t)$$
$$x_{t+1} = x_t + v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx - learning_rate * dx
  x += vx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

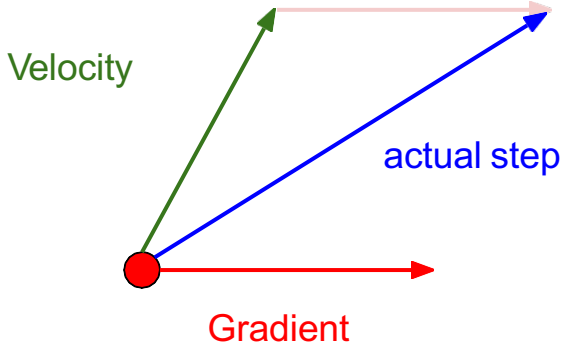
```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

You may see SGD+Momentum formulated different ways, but they are equivalent - give same sequence of x

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

SGD+Momentum

Momentum update:



Combine gradient at current point with velocity to get step used to update weights

Nesterov, “A method of solving a convex programming problem with convergence rate O(1/k^2)”, 1983
Nesterov, “Introductory lectures on convex optimization: a basic course”, 2004
Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

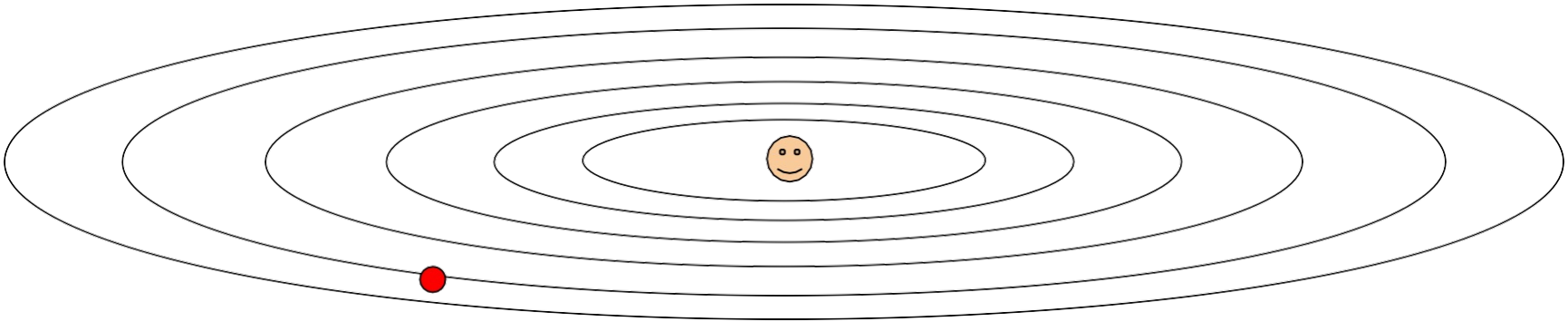
Added element-wise scaling of the gradient based on the historical sum of squares in each dimension

"Per-parameter learning rates"
or "adaptive learning rates"

Duchi et al, "Adaptive subgradient methods for online learning and stochastic optimization", JMLR 2011

AdaGrad

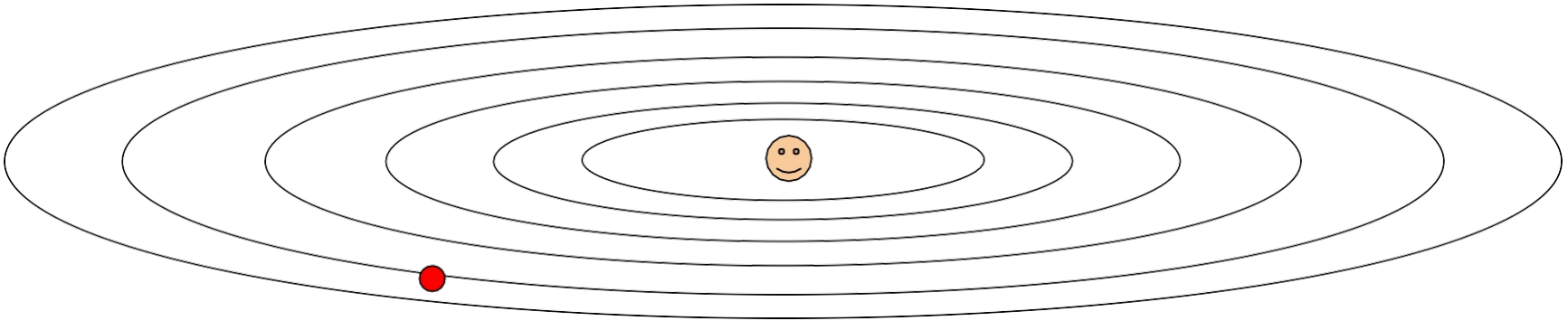
```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Q: What happens with AdaGrad?

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

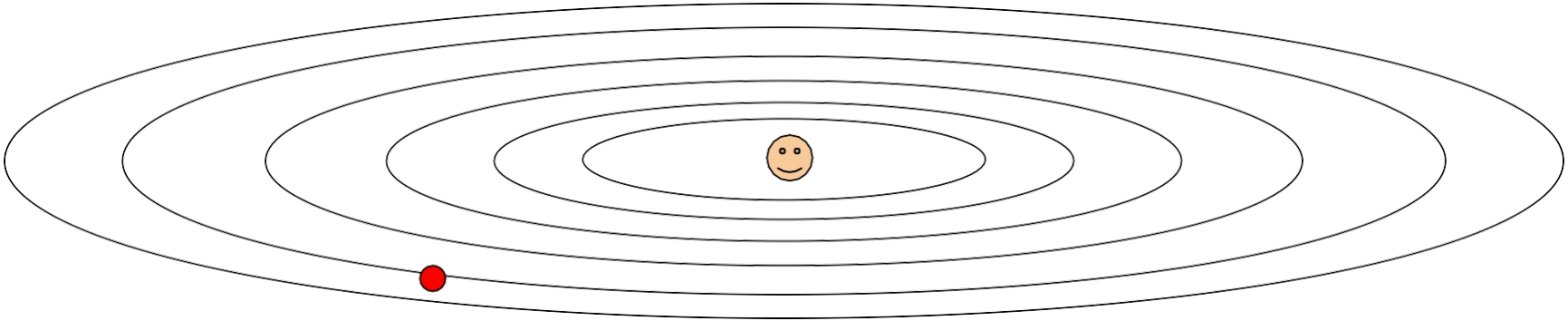


Q: What happens with AdaGrad?

Progress along “steep” directions is damped; progress along “flat” directions is accelerated

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

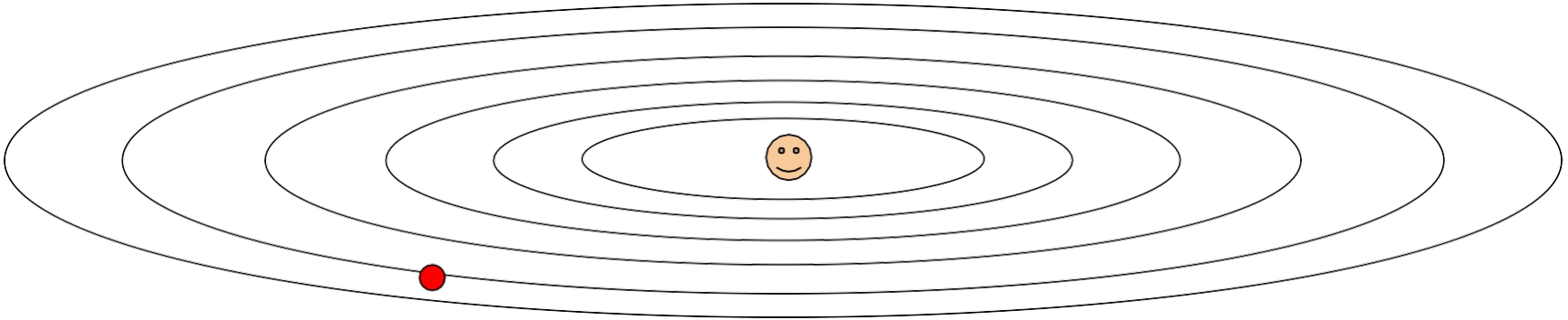


Q2: What happens to the step size over long time?

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Steps may become tiny.



Q2: What happens to the step size over long time?

RMSProp: “Leaky AdaGrad”

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

RMSProp

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Tieleman and Hinton, 2012

Adaptive Optimizers Rescale Coordinates

Different weights can experience very different gradient magnitudes.

Momentum estimate	Track a smoothed gradient direction.
Magnitude estimate	Track recent squared-gradient magnitude.
Adaptive update	Rescale each coordinate using these estimates.

Adam still needs a learning rate and a training schedule.

Course teaching example. Sources and assumptions in speaker notes.

Adam (almost)


```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Kingma and Ba, “Adam: A method for stochastic optimization”, ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Momentum

AdaGrad / RMSProp

Sort of like RMSProp with momentum

Q: What happens at first timestep?

Kingma and Ba, "Adam: Amethod for stochastic optimization", ICLR 2015

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Kingma and Ba, "Adam: Amethod for stochastic optimization", ICLR 2015

Optimizer Comparison

Method	What It Remembers	How It Updates
SGD	Current gradient	Update using this gradient.
SGD + Momentum	Recent signed gradients	Smooth the update direction.
AdaGrad	All past squared gradients	Scale each parameter's step. Steps can become tiny.
RMSProp	Recent squared gradients	Scale each parameter's step. Older gradients fade.
Adam	Recent signed gradients and squared gradients	Smooth the direction and scale each parameter's step.

All five still need a learning rate.

Duchi et al. (2011). Tieleman & Hinton (2012). Kingma & Ba (2015).

AdamW Separates Weight Decay From Adaptation

The gradient-based update and parameter shrinkage have different jobs.

L2 penalties and weight decay differ under adaptive updates.

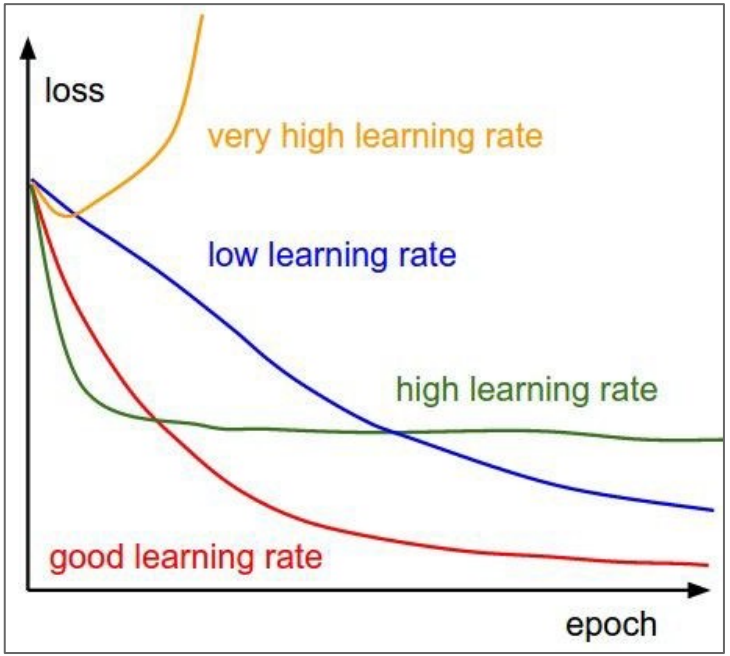
Loshchilov & Hutter, Decoupled Weight Decay (ICLR 2019).

Learning rate schedules

```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```

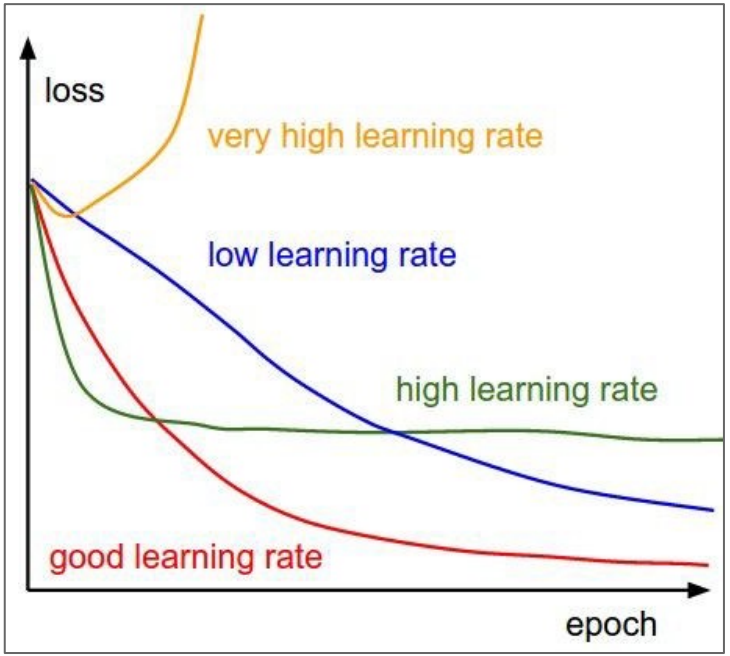
Learning rate

SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



Q: Which one of these learning rates is best to use?

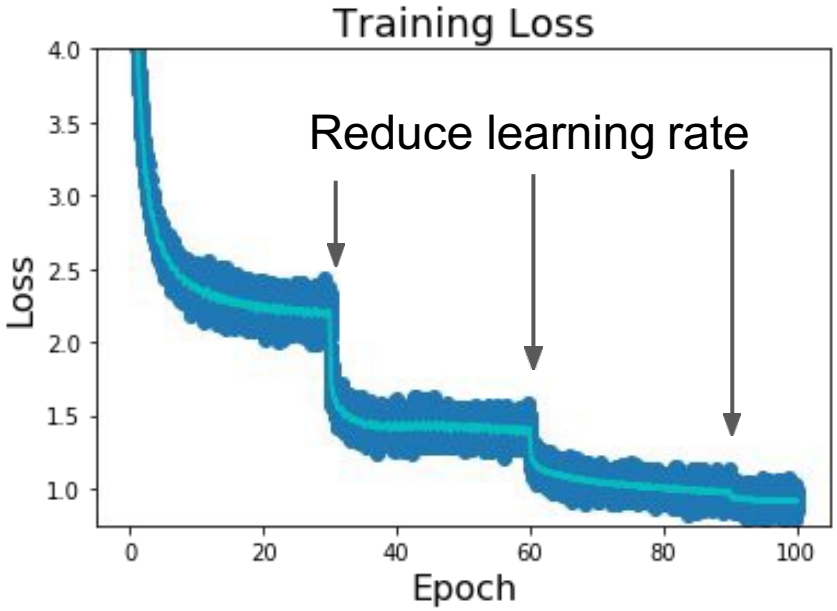
SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



Q: Which one of these learning rates is best to use?

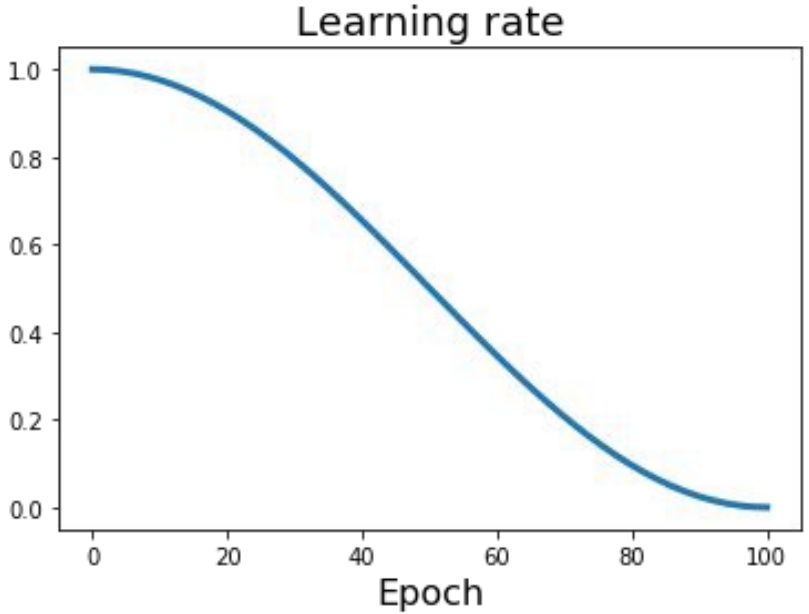
Compare validation performance within the same training budget.

Learning rate decays over time



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:

$$\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$$

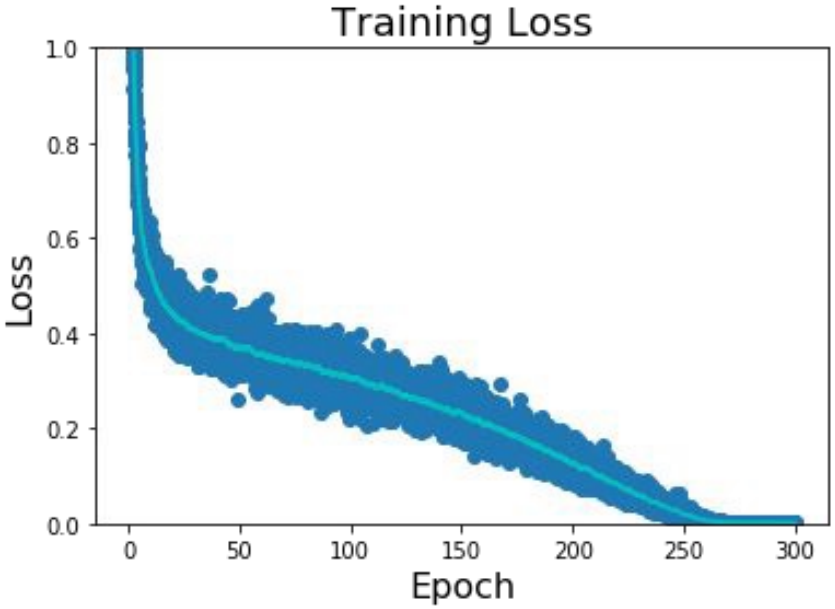
α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
Child at al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:

$$\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$$

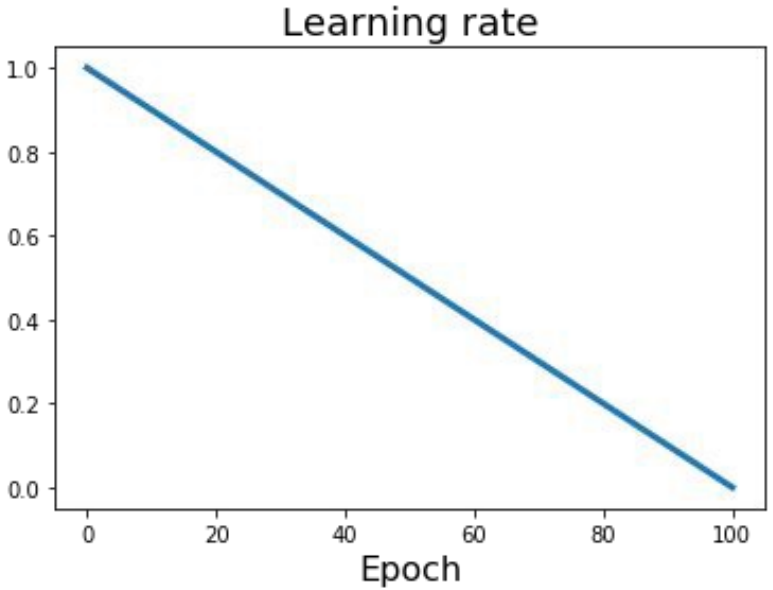
α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
Child at al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

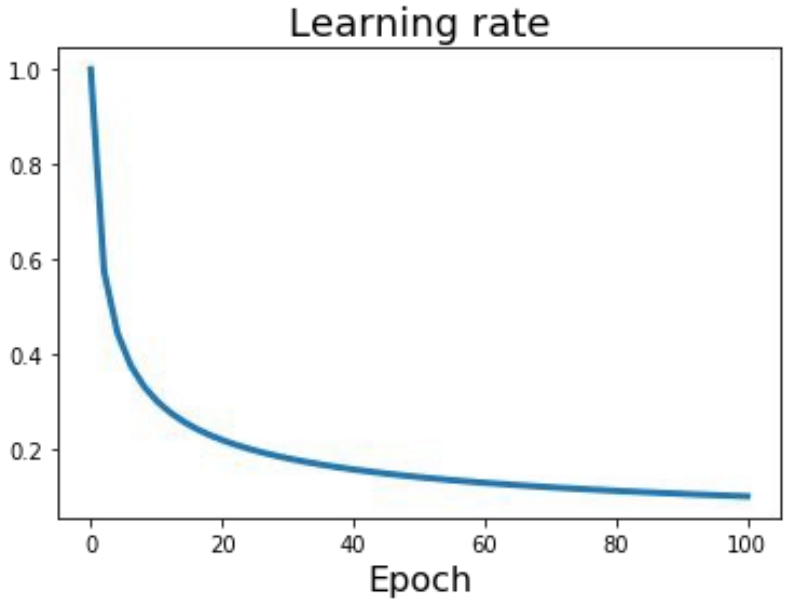
Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Devlin et al, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, 2018

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: **Linear:** **Inverse**

sqrt:

$$\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$$

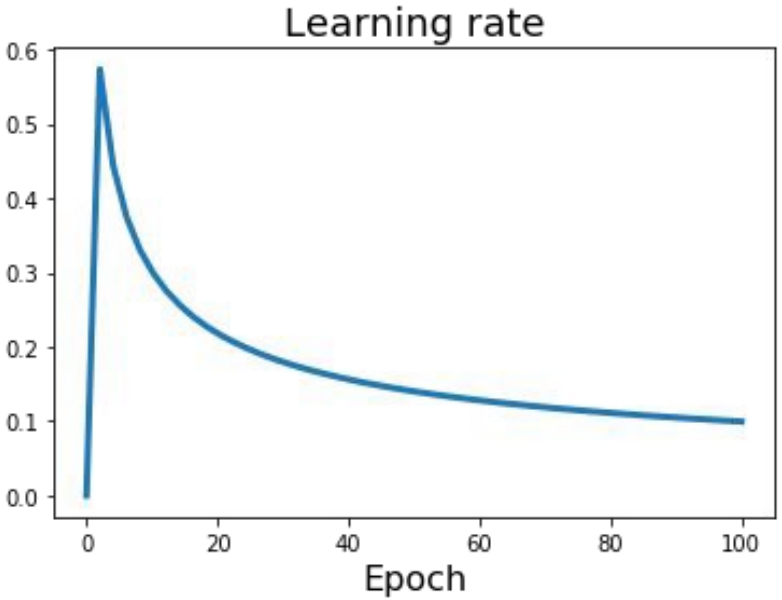
$$\alpha_t = \alpha_0(1 - t/T)$$

$$\alpha_t = \alpha_0/\sqrt{t}$$

α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Vaswani et al, “Attention is all you need”, NIPS 2017

Learning Rate Decay: Linear Warmup



Warmup gradually raises the learning rate to avoid unstable early updates. Its duration is a choice.

Linear batch-size scaling is an ImageNet SGD example, not a universal rule.

Goyal et al, “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour”, arXiv 2017

Warmup and Decay Change the Step Size Over Time

Early training and late refinement need not use the same learning rate.

A schedule is a planned change in learning rate.

Course teaching example. Sources and assumptions in speaker notes.

Learning Rate: Cyclical Learning Rates

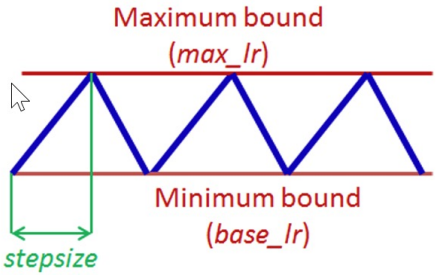


Figure 2. Triangular learning rate policy. The blue lines represent learning rate values changing between bounds. The input parameter *stepsize* is the number of iterations in half a cycle.

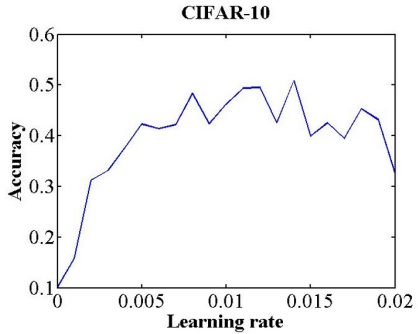


Figure 3. Classification accuracy as a function of increasing learning rate for 8 epochs (LR range test).

Smith, Leslie N. "Cyclical learning rates for training neural networks." 2017 IEEE v
conference on applications of computer vision (WACV)

Docs > torch.optim

lr_scheduler.LRScheduler	Adjusts the learning rate during optimization.
lr_scheduler.LambdaLR	Sets the initial learning rate.
lr_scheduler.MultiplicativeLR	Multiply the learning rate of each parameter group by the factor given in the specified function.
lr_scheduler.StepLR	Decays the learning rate of each parameter group by gamma every step_size epochs.
lr_scheduler.MultiStepLR	Decays the learning rate of each parameter group by gamma once the number of epoch reaches one of the milestones.
lr_scheduler.ConstantLR	Multiply the learning rate of each parameter group by a small constant factor.
lr_scheduler.LinearLR	Decays the learning rate of each parameter group by linearly changing small multiplicative factor.
lr_scheduler.ExponentialLR	Decays the learning rate of each parameter group by gamma every epoch.

An Optimizer Is One Part of the Training Setup

Compare complete runs under the same evaluation protocol.

SGD + momentum	A useful baseline for many vision models.
AdamW	A useful baseline for many Transformer models.
For either choice	Tune learning rate, decay, batch size and schedule.

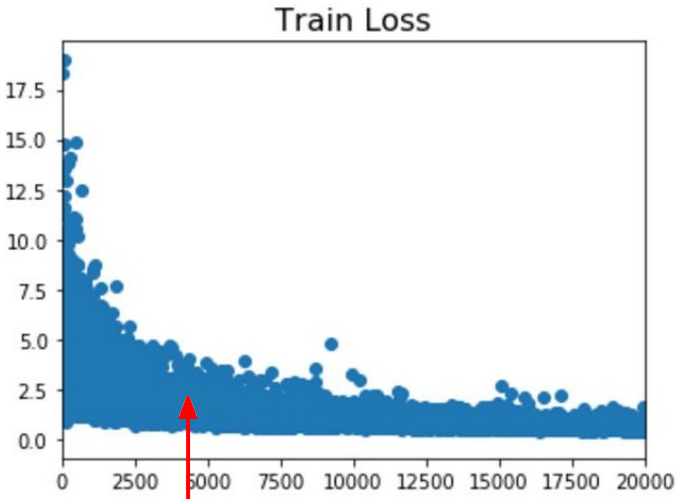
Use validation results to compare the complete setup.

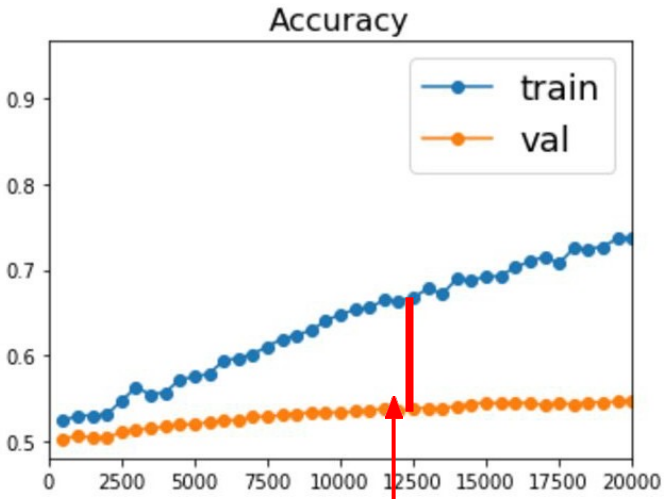
Loshchilov & Hutter, Decoupled Weight Decay (ICLR 2019).

Training Neural Networks

Training vs. Test Error

Beyond Training Error



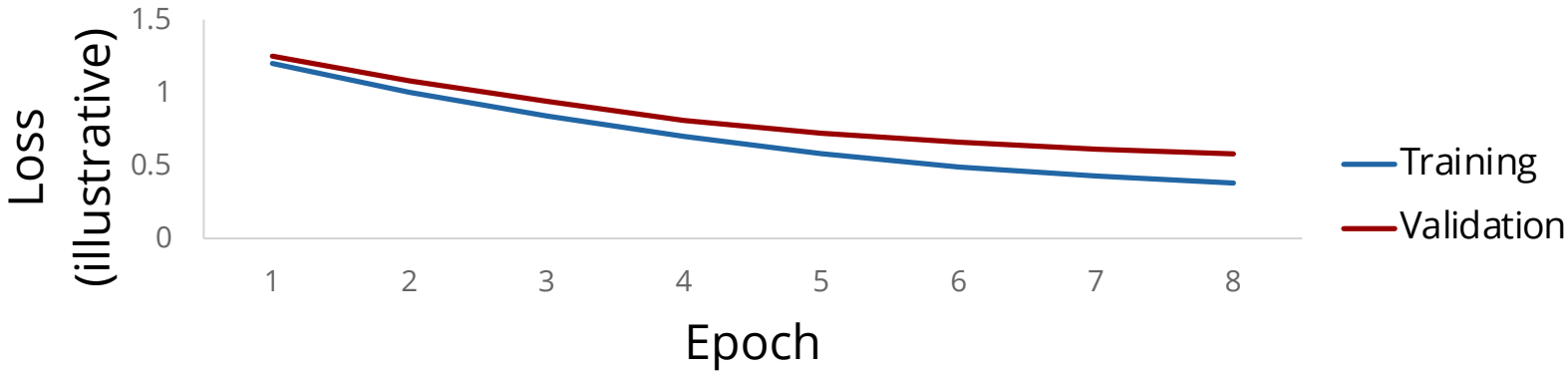


Better optimization algorithms help reduce training loss

But we really care about error on new data - how to reduce the gap?

When Training and Validation Improve Together

Evaluate both curves using comparable loss definitions and modes.

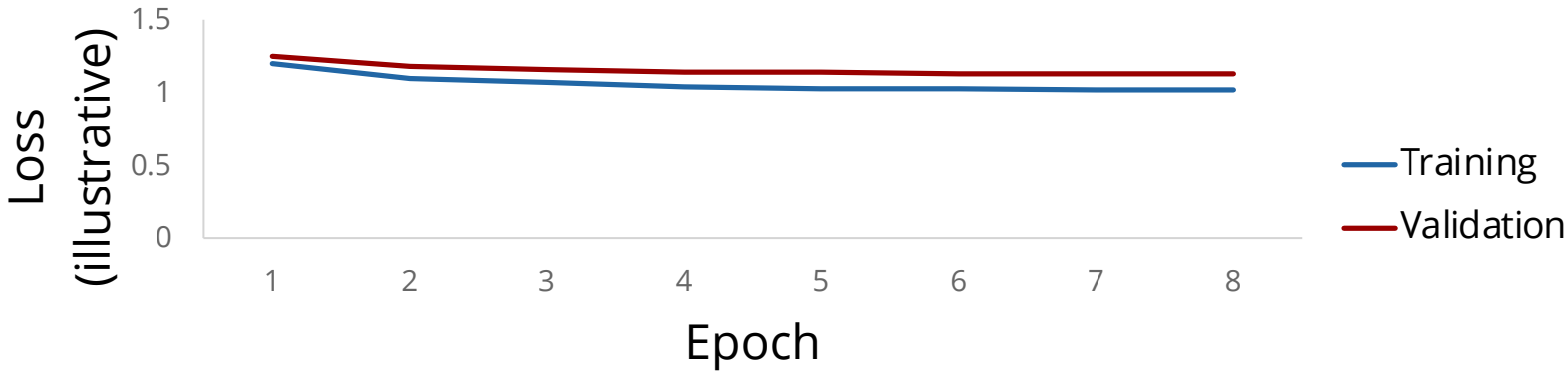


The model is learning patterns that transfer to held-out data.

Course teaching example. Sources and assumptions in speaker notes.

When Both Curves Stay High

The model has not yet fit even the training examples well.

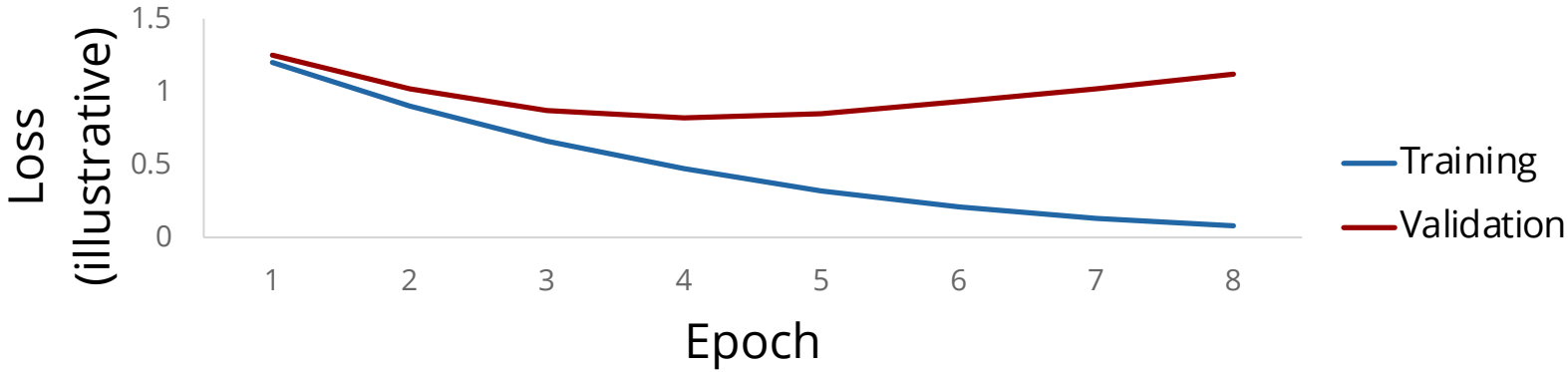


Check the training pipeline before adding more regularization.

Course teaching example. Sources and assumptions in speaker notes.

When Training Improves but Validation Worsens

The best training checkpoint may not be the best held-out checkpoint.



Select checkpoints using the chosen validation metric.

Course teaching example. Sources and assumptions in speaker notes.

Checkpoint Selection and Early Stopping

One noisy validation dip does not require stopping immediately.

Save the best checkpoint	Keep the best value of the selected validation metric.
Wait for a pattern	A patience window can tolerate short-term noise.
Use a fixed protocol	Keep the test set out of stopping decisions.

Choose the stopping rule before inspecting test performance.

Course teaching example. Sources and assumptions in speaker notes.

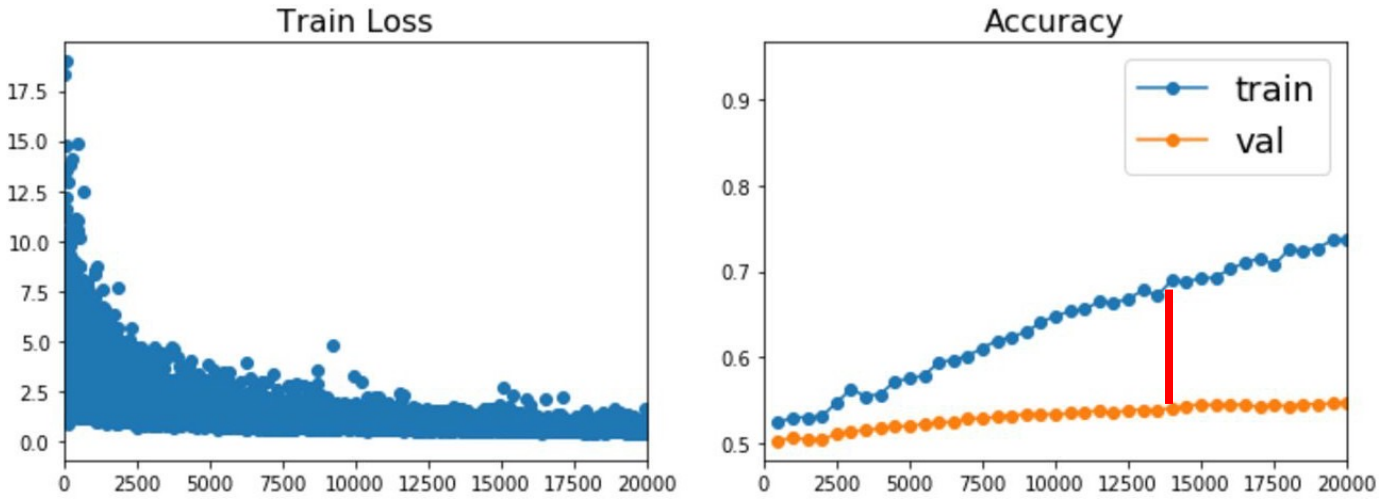
An Ensemble Combines Several Predictions

Suppose three trained classifiers disagree on one flower photo.

Measure any accuracy gain together with serving cost.

Course teaching example. Sources and assumptions in speaker notes.

How to improve single-model performance?



Regularization

The Purpose of a Regularization Penalty

The training loss rewards fitting labels. A penalty also constrains the weights.

Objective = data loss + λ × weight penalty

L2 penalty = sum of squared weights

λ controls the tradeoff. Choose it using validation data.

The next page lists different choices of weight penalty.

Course schematic. Sources and assumptions in speaker notes.

Regularization: Add term to loss

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$ (Weight decay)

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

Dropout Changes Which Features Are Available

During training, some hidden activations are randomly set to zero.

Training step A

Some feature activations are kept.
Others are temporarily zeroed.

Training step B

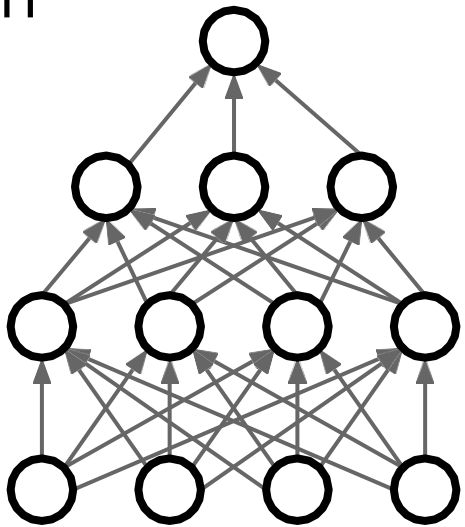
A new random mask keeps a different subset of activations.

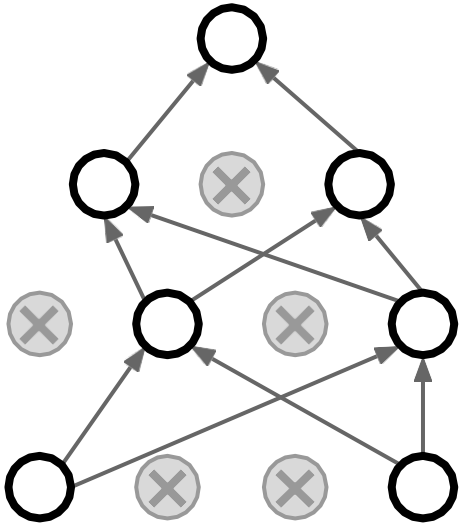
The weights remain. The activation mask changes.

Course teaching example. Sources and assumptions in speaker notes.

Regularization: Dropout

In each forward pass, randomly set some neurons to zero Probability of dropping is a hyperparameter; 0.5 is common



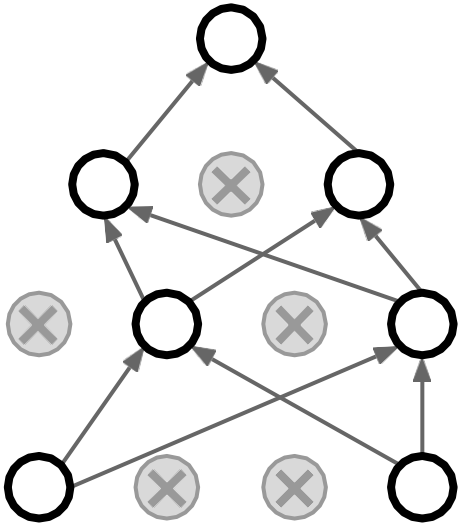


Srivastava et al, “Dropout: A simple way to prevent neural networks from overfitting”, JMLR 2014

Regularization: Dropout

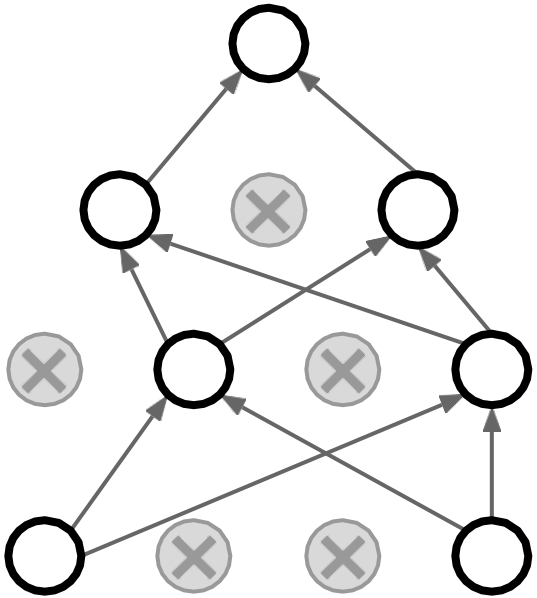
```
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)
```

Example forward pass with a 3-layer network using dropout



Regularization: Dropout

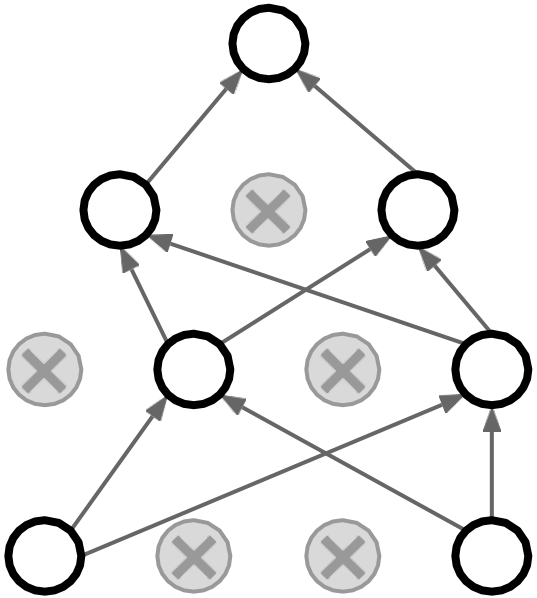
How can this possibly be a good idea?



Forces the network to have a redundant representation;
Prevents co-adaptation of features

Regularization: Dropout

How can this possibly be a good idea?



Another interpretation:

Dropout is training a large **ensemble** of models (that share parameters).

Each binary mask is one model

An FC layer with 4096 units has $2^{4096} \sim 10^{1233}$ possible masks!
Only $\sim 10^{82}$ atoms in the universe...

Dropout: Test time

Dropout makes our output random!

Output (label) Input (image)

$$y = f_W(x, z)$$

Random mask

Want to "average out" the randomness at test-time

$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$

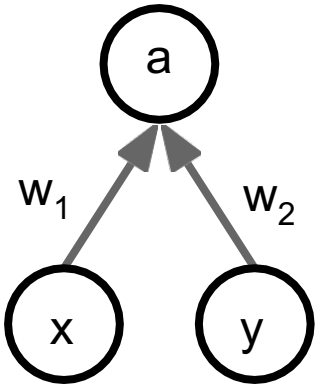
But this integral seems hard …

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

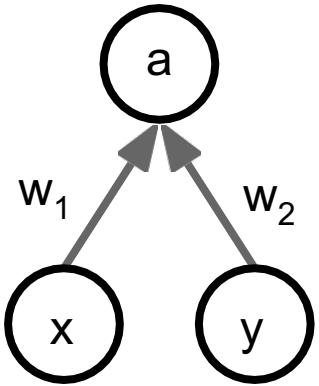
Consider a single neuron.



Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



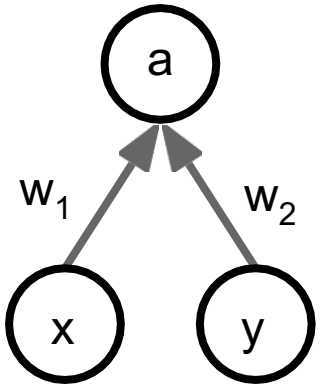
Consider a single neuron.

At test time we have: $E\big[a\big] = w_1 x + w_2 y$

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



Consider a single neuron with two inputs x and y.

At test time we have: $E\big[a\big] = w_1 x + w_2 y$

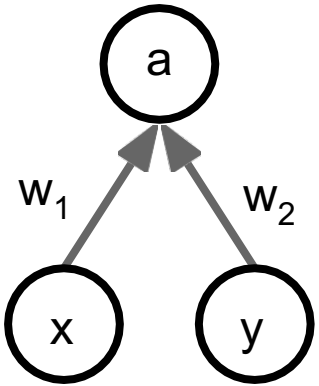
During training we have:

$$\begin{aligned} E\big[a\big] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



Consider a single neuron.

At test time we have: $E\big[a\big] = w_1 x + w_2 y$

During training we have:

$$\begin{aligned} E\big[a\big] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Classic convention: multiply by the KEEP probability.

Dropout: Test time

```
def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

Evaluation disables the random dropout mask.
Match each activation's expected scale. A nonlinear network output need not equal its full mask-averaged prediction.

Dropout Scaling Conventions

Let q be the probability of keeping an activation. Dropping probability is 1 − q.

Classic dropout

Training: keep it unchanged or set it to zero.
Evaluation: multiply it by q.

Inverted dropout

Training: keep it divided by q or set it to zero.
Evaluation: use it unchanged.

Each convention matches training and evaluation scales.

Course schematic. Sources and assumptions in speaker notes.

More common: "Inverted dropout"

```python
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = (np.random.rand(*H1.shape) < p) / p # first dropout mask. Notice /p!
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = (np.random.rand(*H2.shape) < p) / p # second dropout mask. Notice /p!
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) # no scaling necessary
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  out = np.dot(W3, H2) + b3
```

test time is unchanged!

Inverted Dropout Preserves the Expected Scale

A hidden activation is 2. The keep probability is 0.5.

Kept: output 2 / 0.5 = 4. Dropped: output 0.

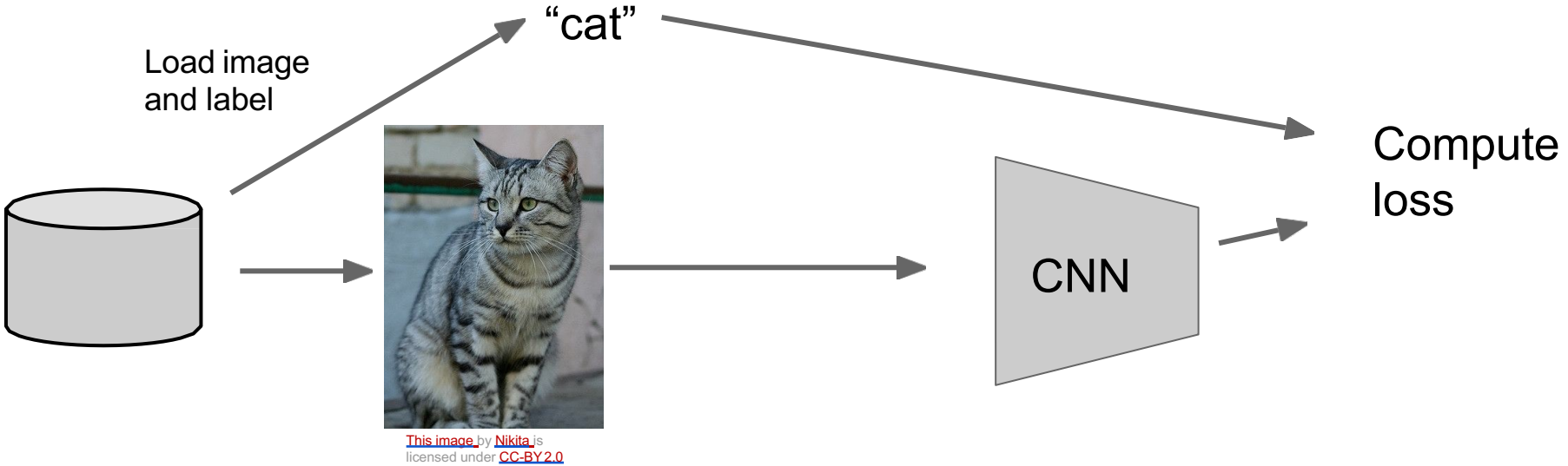
Expected training output = 0.5 × 4 + 0.5 × 0 = 2

At evaluation: keep the activation as 2, with no dropout mask.

The preserved quantity is an expectation across masks.

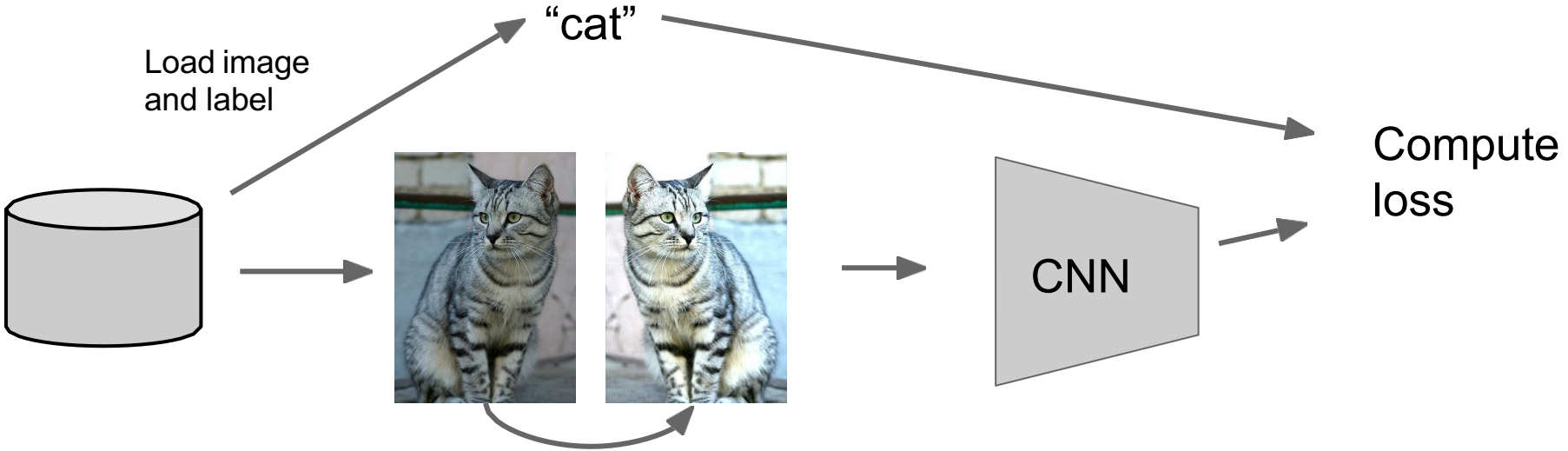
Course teaching example. Sources and assumptions in speaker notes.

Regularization: Data Augmentation



This image by Nikita is licensed under CC-BY 2.0

Regularization: Data Augmentation



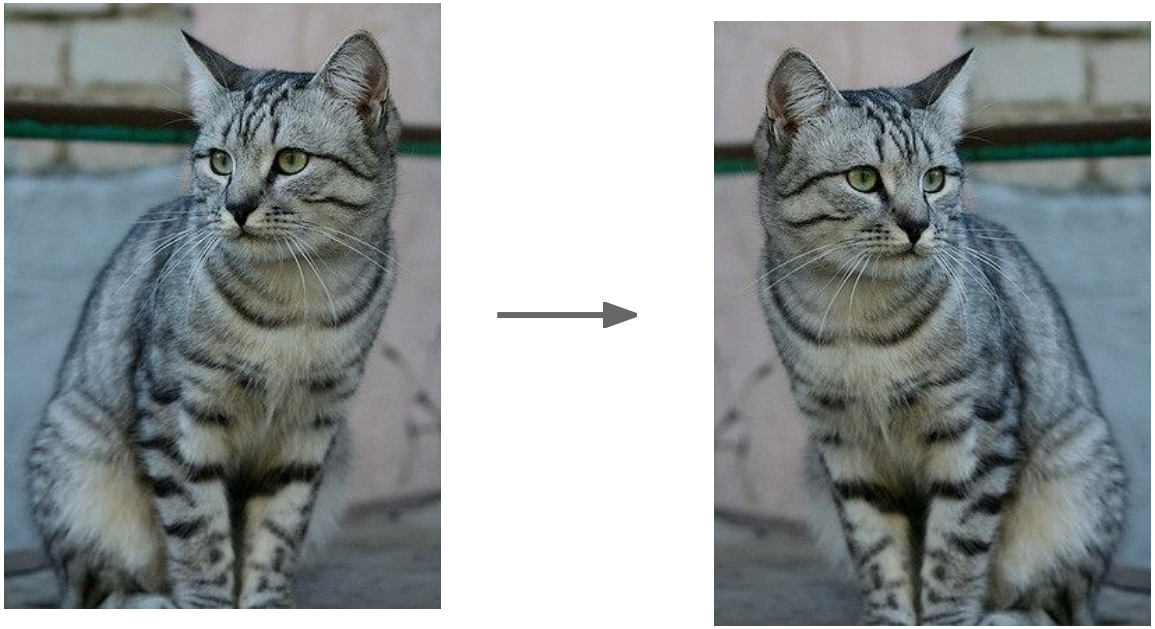
An Augmented Example Must Keep a Valid Label

For flower classification, consider a slightly different view of the same flower.

Choose transformations that preserve the task label.

Course teaching example. Sources and assumptions in speaker notes.

Data Augmentation
Horizontal Flips



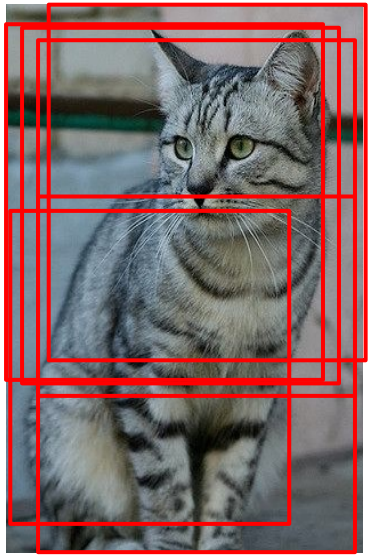
Data Augmentation

Random crops and scales

Training: sample random crops / scales

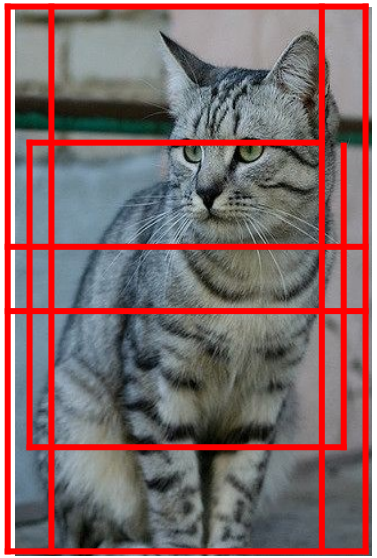
ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch



Data Augmentation

Random crops and scales



Training: sample random crops / scales

ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch

Testing: average a fixed set of crops

ResNet:

1. Resize image at 5 scales: {224, 256, 384, 480, 640}
2. For each size, use 10 224 x 224 crops: 4 corners + center, + flips

Data Augmentation

Color Jitter

Simple: Randomize
contrast and brightness

Data Augmentation
Color Jitter

Simple: Randomize contrast and brightness

More Complex:

1. Apply PCA to all [R, G, B] pixels in training set
2. Sample a "color offset" along principal component directions
3. Add offset to all pixels of a training image

(As seen in *[Krizhevsky et al. 2012],* ResNet, etc)

Data Augmentation

Choose transformations that keep labels valid.

Examples of data augmentations:

- translation
- rotation
- stretching
- shearing,
- task-valid lens distortions

The Same Transformation Can Break Another Task

Rotating a digit image can change what the image means.

Flower photo

A rotated view may still show the same flower species.

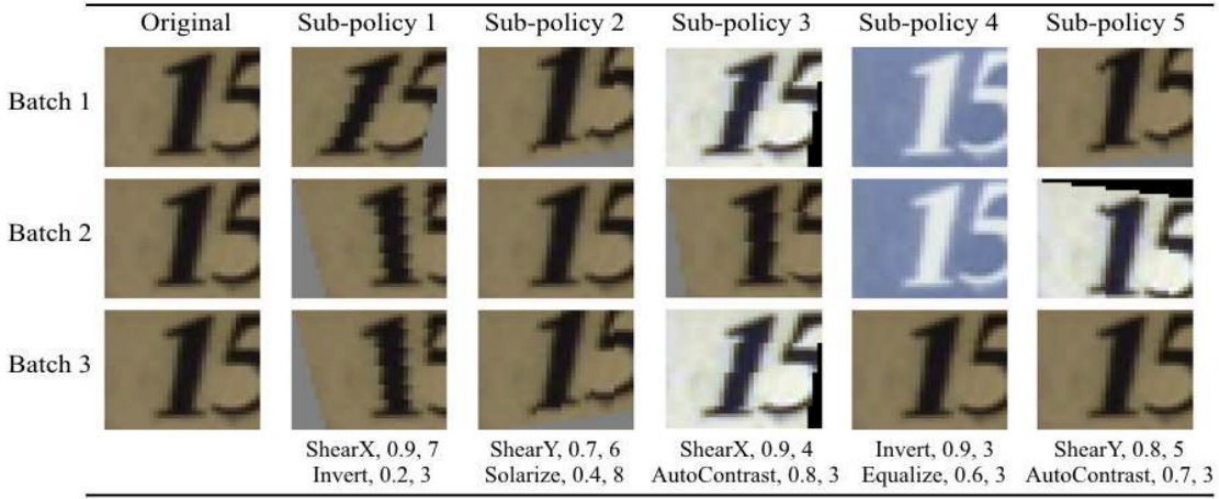
Digit recognition

A rotated 6 can resemble a 9. Keeping label 6 can be wrong.

Augmentation depends on the task semantics.

Course teaching example. Sources and assumptions in speaker notes.

Automatic Data Augmentation



Cubuk et al., "AutoAugment: Learning Augmentation Strategies from Data", CVPR 2019

Training Randomness and Evaluation

Each method has its own evaluation rule.

Dropout	Turn off the random mask; apply the chosen scaling convention.
BatchNorm	Use stored training statistics in the usual running-statistics mode.
Augmentation	Use a fixed evaluation transform, or a predefined averaging protocol.

Decide the evaluation protocol before comparing models.

Course schematic. Sources and assumptions in speaker notes.

Regularization: DropConnect

Training: Drop connections between neurons (set weights to 0)

Testing: Use all the connections

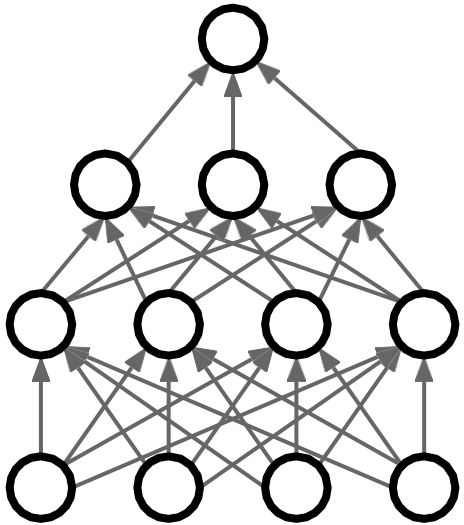
Examples:

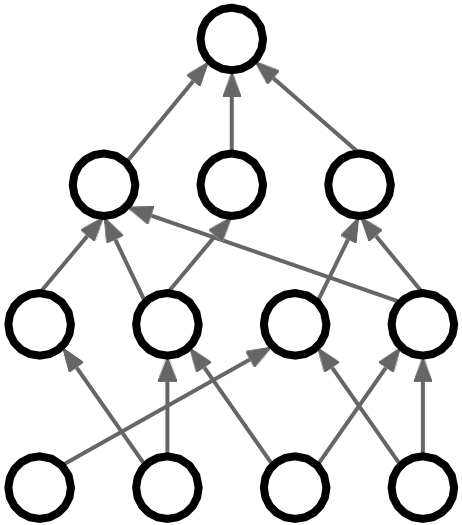
Dropout

Batch Normalization

Data Augmentation

DropConnect





Wan et al, “Regularization of Neural Networks using DropConnect”, ICML 2013

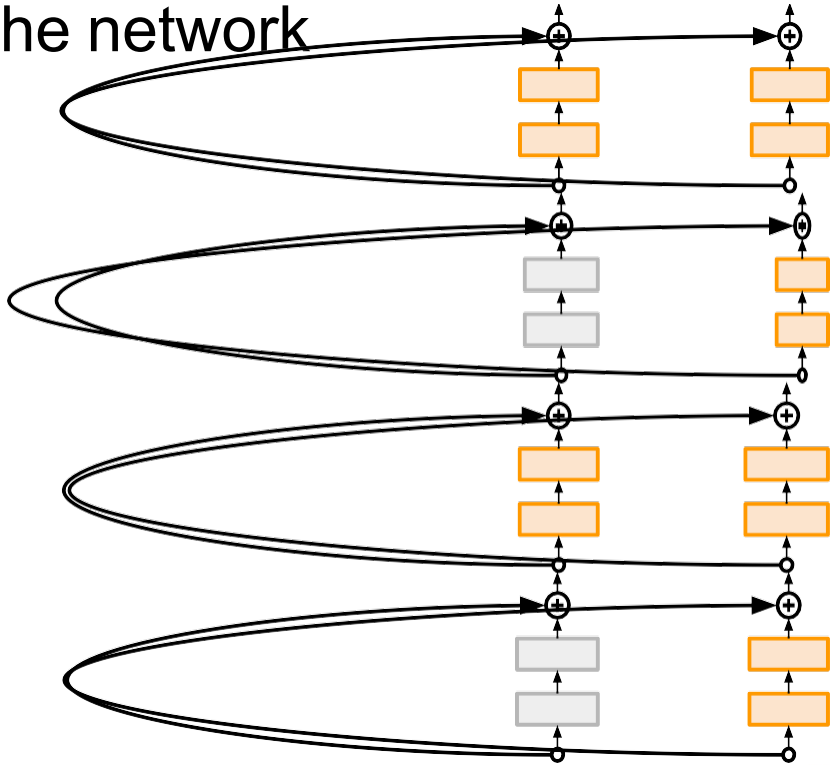
Regularization: Stochastic Depth

Training: Skip some layers in the network

Testing: Use all the layer

Examples:

Dropout
Batch Normalization
Data Augmentation
DropConnect
Fractional Max Pooling
Stochastic Depth



Huang et al, “Deep Networks with Stochastic Depth”, ECCV 2016

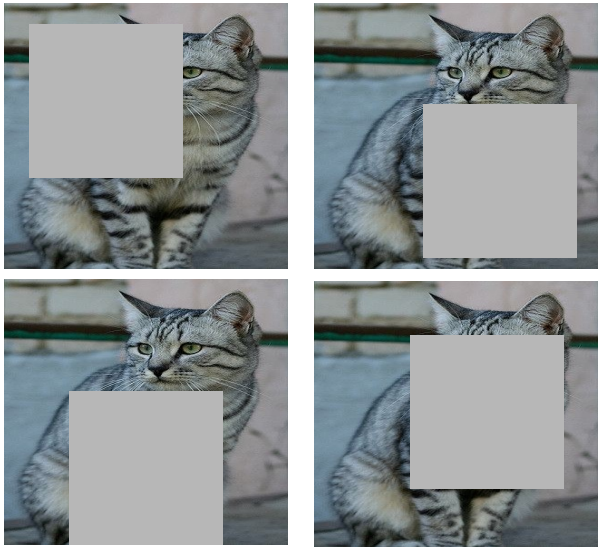
Regularization: Cutout

Training: Set random image regions to zero

Testing: Use full image

Examples:

Dropout
Batch Normalization
Data Augmentation
DropConnect
Fractional Max Pooling
Stochastic Depth
Cutout / Random Crop



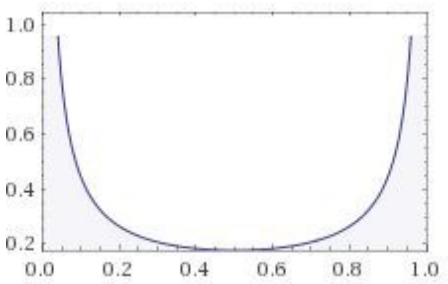
Works very well for small datasets like CIFAR, less common for large datasets like ImageNet

DeVries and Taylor, “Improved Regularization of Convolutional Neural Networks with Cutout”, arXiv 2017

Regularization: Mixup

Training: Train on random blends of images

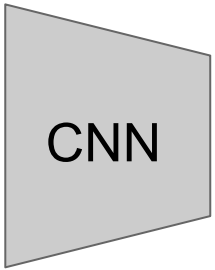
Testing: Use original images



Examples:

Dropout
Batch Normalization
Data Augmentation
DropConnect
Fractional Max Pooling
Stochastic Depth
Cutout / Random Crop
Mixup

Randomly blend the pixels of pairs of training images, e.g. 40% cat, 60% dog



Target label:
cat: 0.4
dog: 0.6

Zhang et al, "*mixup*: Beyond Empirical Risk Minimization", ICLR 2018

Regularization: Diagnose, Then Choose

Return to the gap between training and validation performance.

More or better data	Improve coverage and inspect labels or duplicates.
Augmentation / weight decay	Test task-appropriate transformations and constraints.
Dropout / early stopping	Use when they fit the architecture and validation protocol.

Compare one controlled change at a time.

Course teaching example. Sources and assumptions in speaker notes.

Training Neural Networks

Choosing Hyperparameters

One Experiment Should Answer One Question

Keep the same data split, metric and training budget.

A controlled comparison

Same backbone and preprocessing.
Change the learning rate.
Compare validation results.

A hard-to-interpret comparison

Change data, model and optimizer
at the same time.
Which change caused the result?

Record the setting and the outcome of every run.

Course teaching example. Sources and assumptions in speaker notes.

Choosing Hyperparameters

Hyperparameters: usually set beforehand

Parameters: learnable, e.g., model weights

Choosing Hyperparameters

Step 1: Check initial loss

Turn off weight decay, sanity check loss at initialization
For uniform class probabilities: loss = ln(C); three classes give ≈1.10.

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Fit a tiny sample as a debugging check

Try to train to 100% training accuracy on a small sample of training data (~5-10 minibatches); fiddle with architecture, learning rate, weight initialization

Loss not going down? LR too low, bad initialization
Loss explodes to Inf or NaN? LR too high, bad initialization

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down

Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~100 iterations

Good learning rates to try: 1e-1, 1e-2, 1e-3, 1e-4

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs

Choose a few values of learning rate and weight decay around what worked from Step 3, train a few models for ~1-5 epochs.

Good weight decay to try: 1e-4, 1e-5, 0

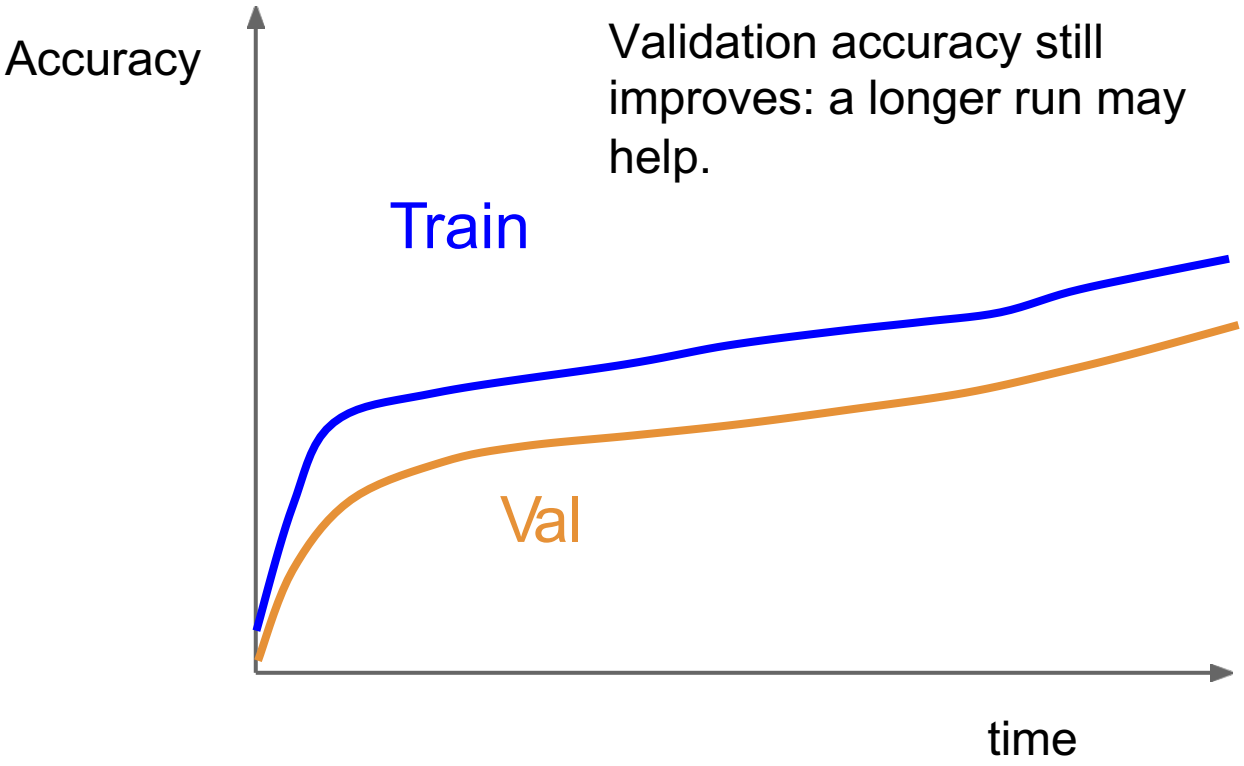
Choosing Hyperparameters

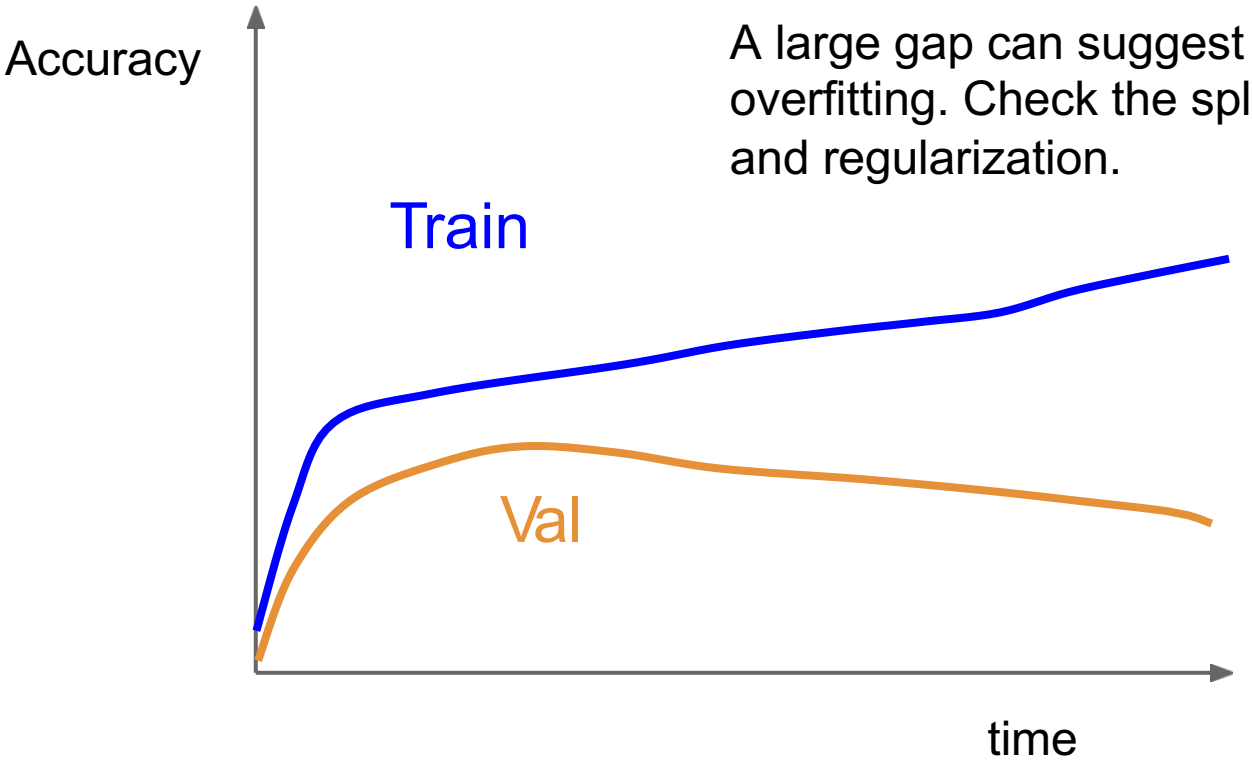
Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs
Step 5: Refine grid, train longer

Pick best models from Step 4, train them for longer (~10-20 epochs) without learning rate decay

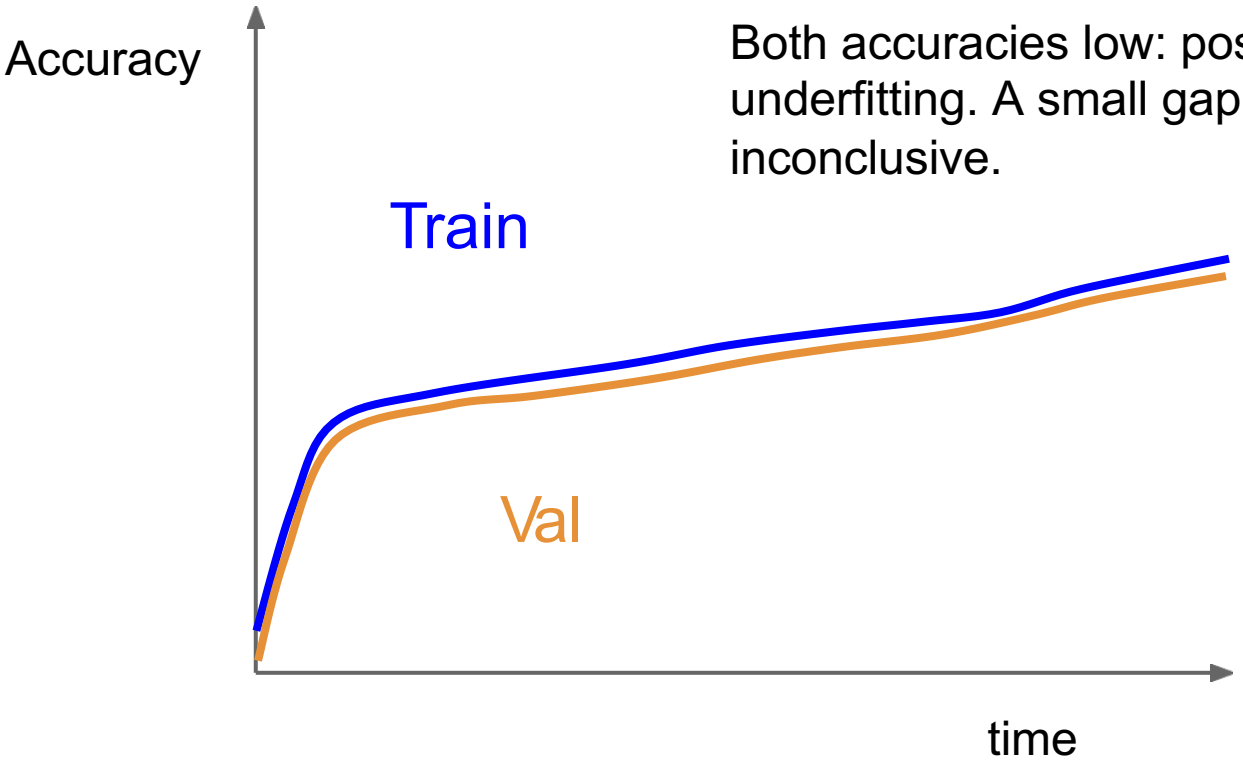
Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs
Step 5: Refine grid, train longer
Step 6: Look at loss and accuracy curves


Accuracy
Validation accuracy still improves: a longer run may help.
Train
Val
time



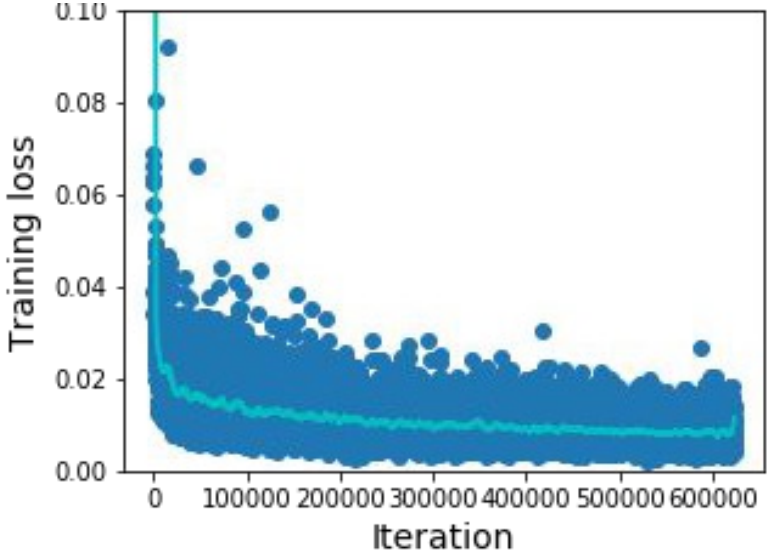
A large gap can suggest overfitting. Check the split, labels and regularization.



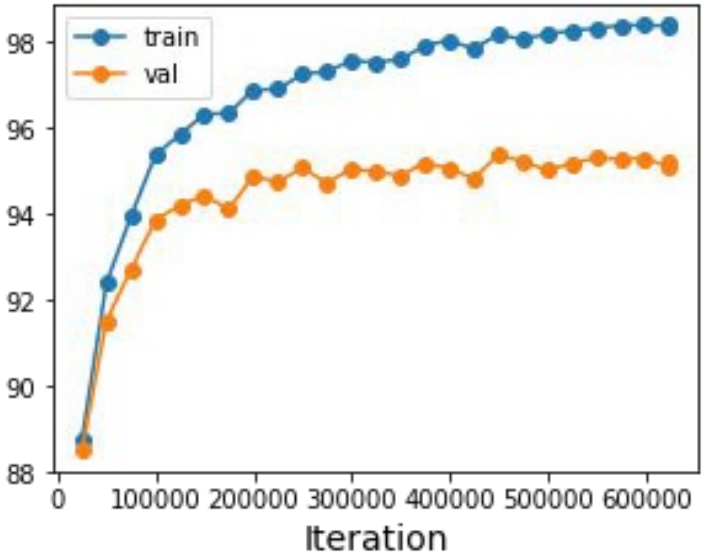
Both accuracies low: possible underfitting. A small gap alone is inconclusive.

Look at learning curves!

Training Loss

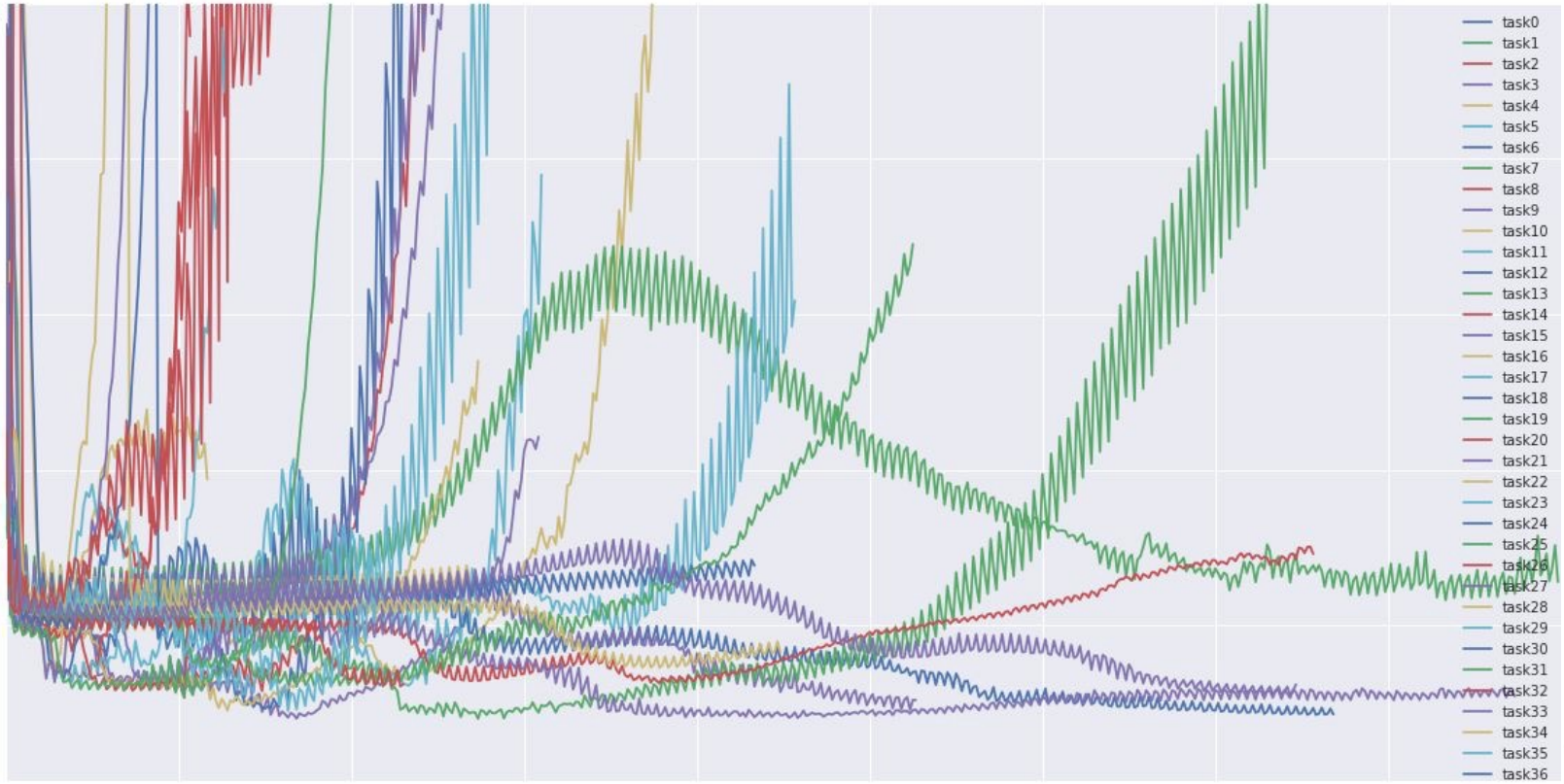


Train / Val Accuracy



Losses may be noisy, use a scatter plot and also plot moving average to see trends better

You can plot all your loss curves for different HPs on a single plot



Don't look at accuracy or loss curves for too long!

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs
Step 5: Refine grid, train longer
Step 6: Look at loss and accuracy curves
Step 7: GOTO step 5

Search Learning Rates Across Scales

A broad first pass checks orders of magnitude.

Example candidate rates: 0.0001, 0.001, 0.01, 0.1

Find a useful region, then search more closely.

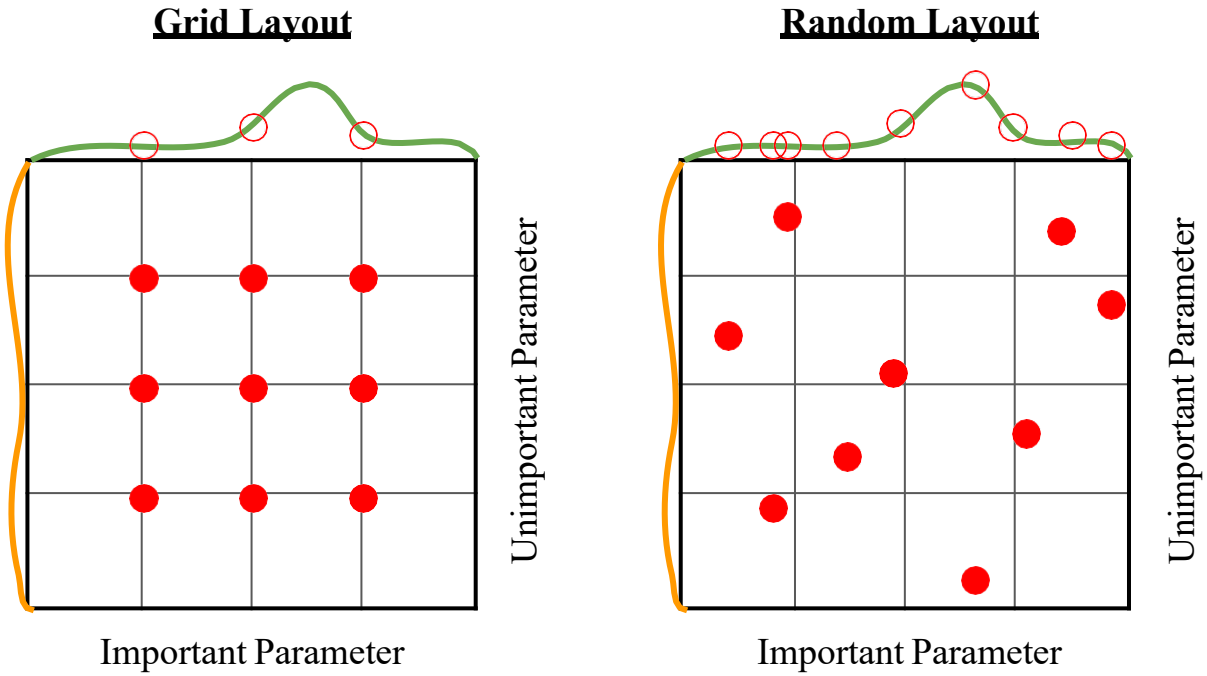
These values illustrate spacing, not a universal recommendation.

A fixed compute budget makes comparisons meaningful.

Course teaching example. Sources and assumptions in speaker notes.

Random Search vs. Grid Search

Random Search for Hyper-Parameter Optimization
Bergstra and Bengio, 2012



Summary

- Improve your training error:
  - Optimizers
  - Learning rate schedules
- Improve generalization, choosing by validation:
  - Regularization
  - Choosing Hyperparameters

Repeated Validation Can Overfit the Selection Process

Trying many settings also uses information from the validation set.

Model selection is part of the learning process.

Course teaching example. Sources and assumptions in speaker notes.

Beyond Training Neural Networks

Transfer Learning

Limited Data Changes the Starting Point

Our flower dataset is small. A pretrained model already encodes visual patterns.

Train from scratch

Learn all representations from this dataset.

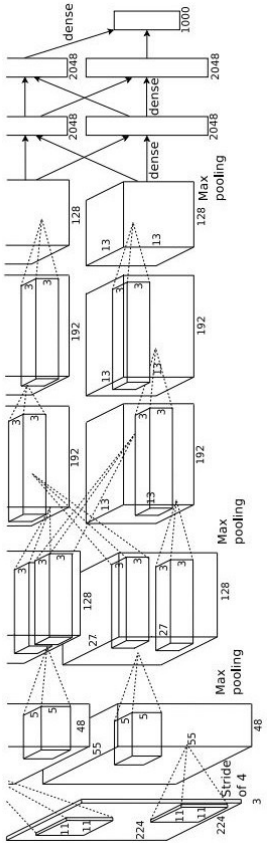
Transfer learning

Start with representations learned from other data.

The source and target data determine how useful transfer is.

Course teaching example. Sources and assumptions in speaker notes.

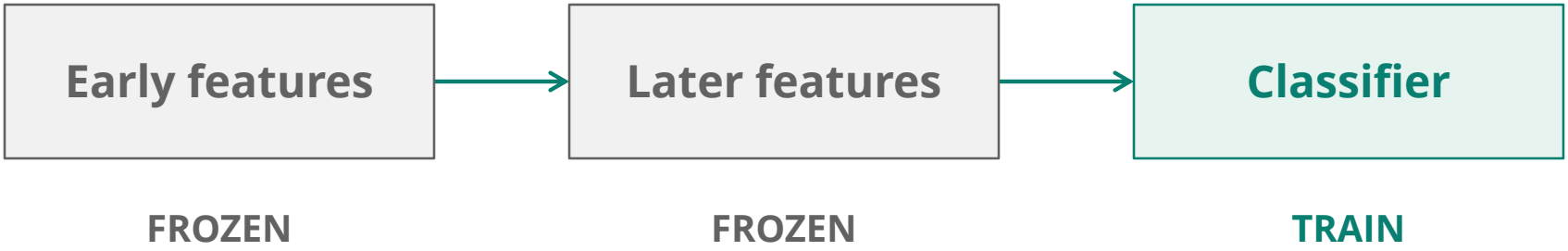
Transfer Learning with CNNs



Start by Training a New Classifier

Replace the original label predictor with a three-class flower head.

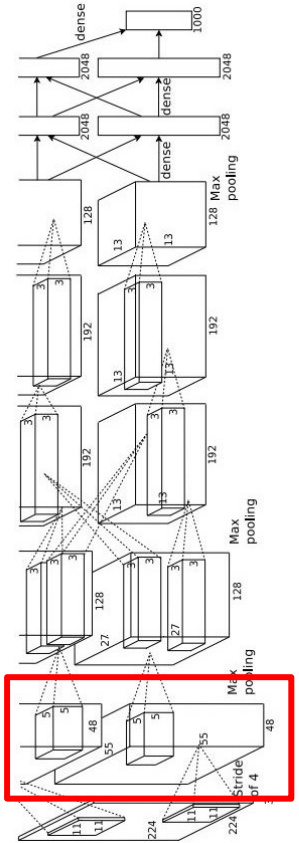
Stage 1: backbone = the pretrained feature extractor. Keep it fixed.

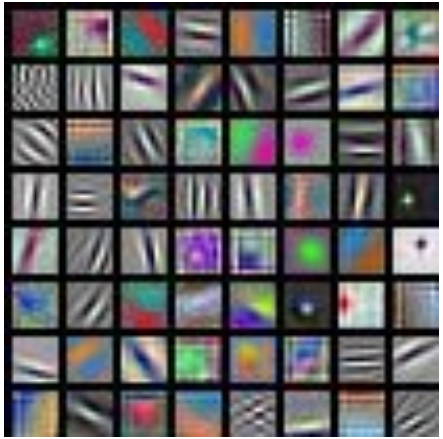


Frozen features still participate in the forward pass.

Course teaching example. Sources and assumptions in speaker notes.

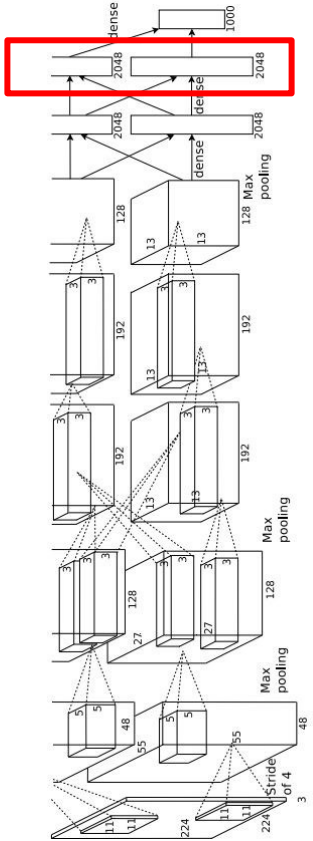
Transfer Learning with CNNs





AlexNet:
64 x 3 x 11 x 11

Transfer Learning with CNNs



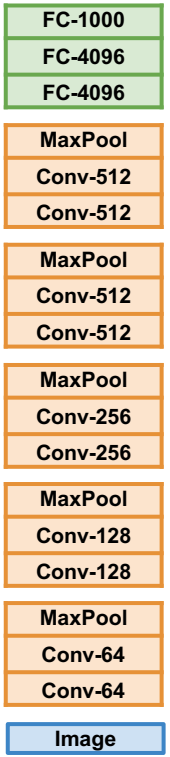
Test image

L2 Nearest neighbors in feature space

Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

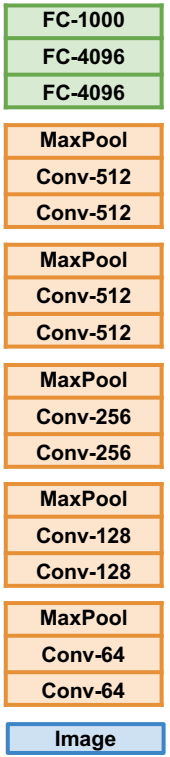
1. Train on Imagenet



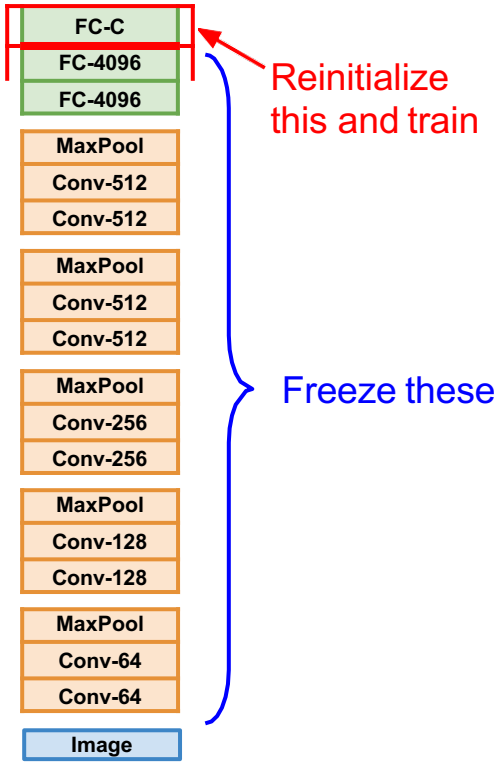
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

1. Train on Imagenet



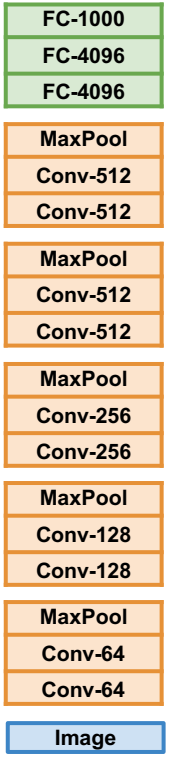
2. Small Dataset (C classes)



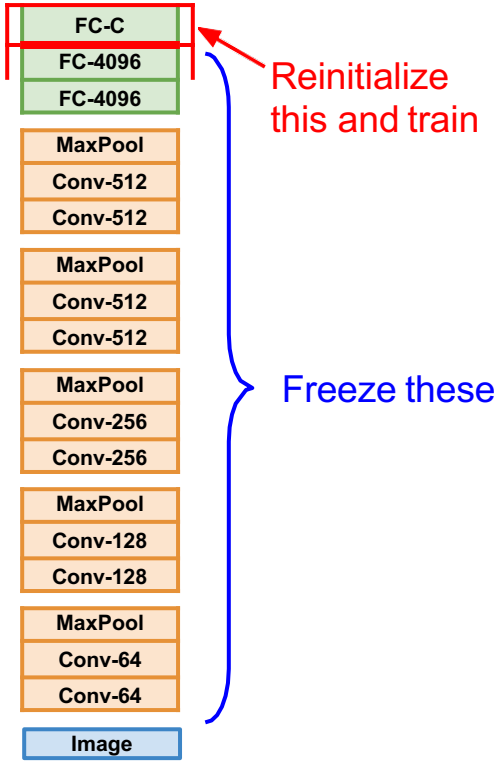
Transfer Learning with CNNs

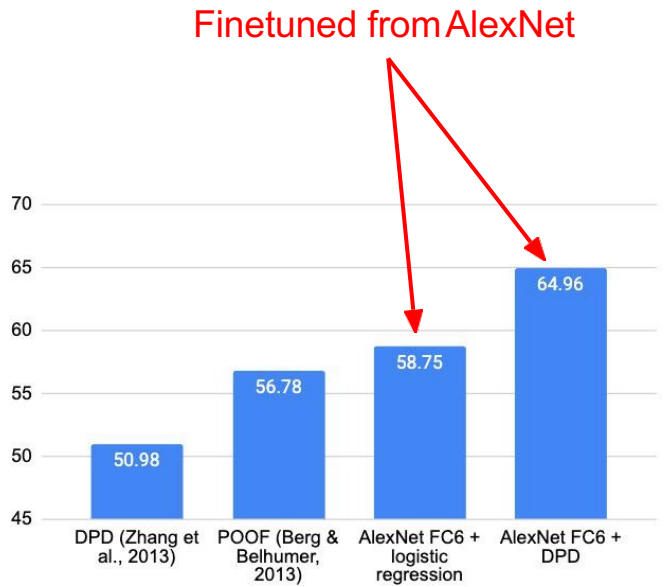
Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

1. Train on Imagenet



2. Small Dataset (C classes)





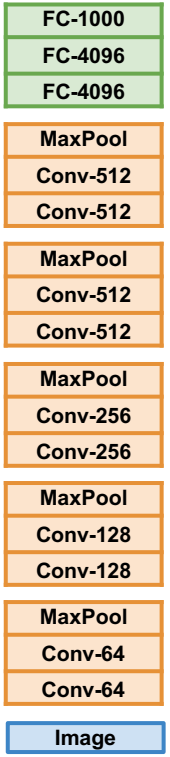
Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014

USC Viterbi School of Engineering

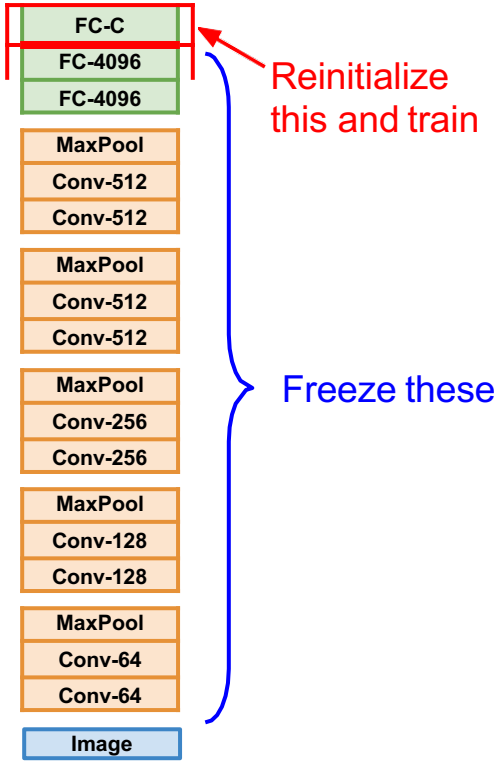
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

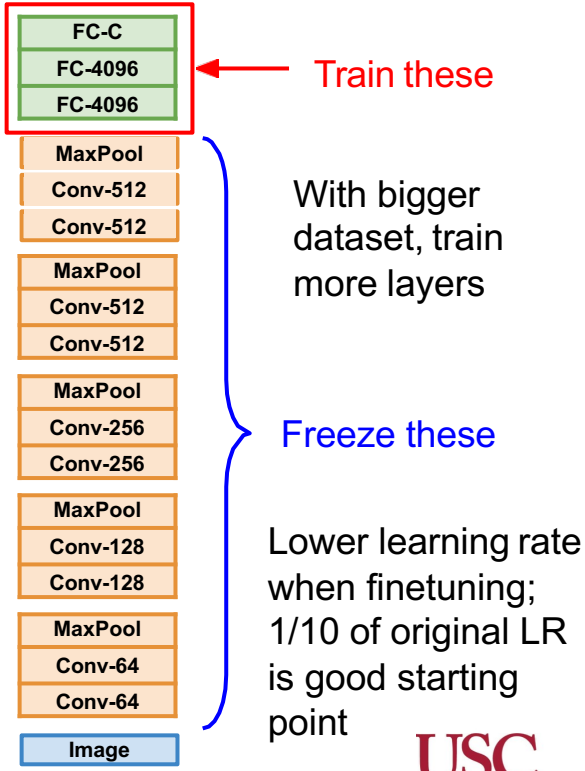
1. Train on Imagenet



2. Small Dataset (C classes)



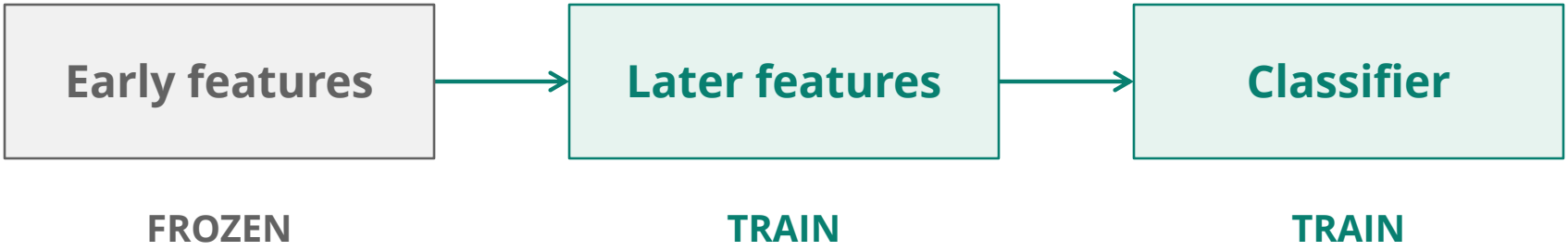
3. Bigger dataset



Then Adapt Some of the Backbone

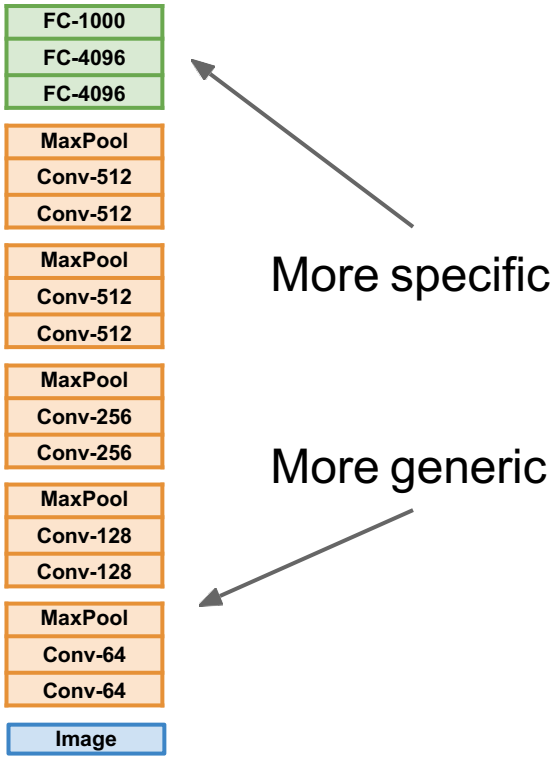
If the fixed features are insufficient, unfreeze selected layers.

Stage 2: adapt later features and the classifier.

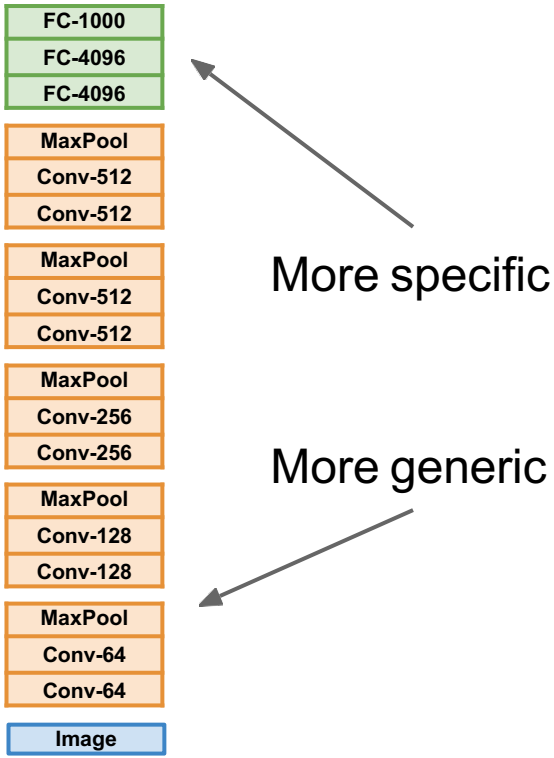


Fine-tuning changes selected pretrained parameters.

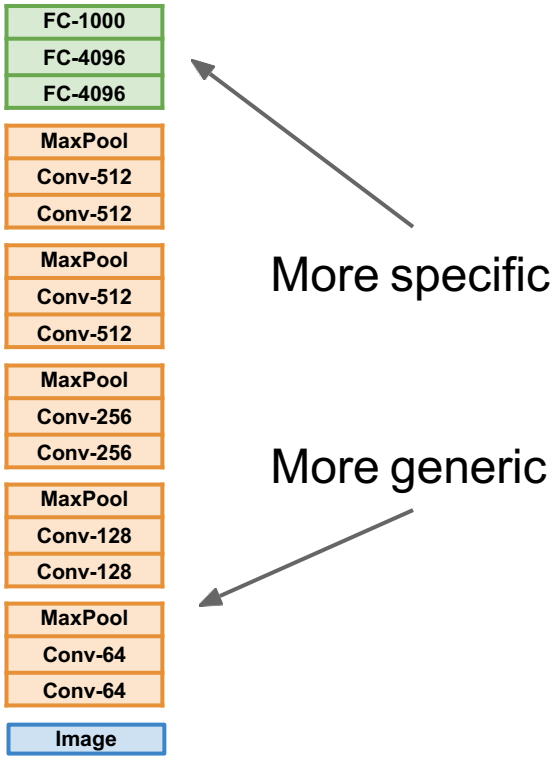
Course teaching example. Sources and assumptions in speaker notes.



	very similar dataset	very different dataset
very little data	?	?
quite a lot of data	?	?



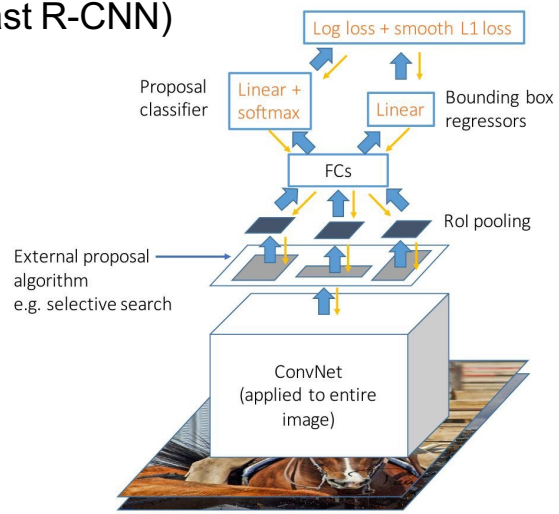
	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	?
quite a lot of data	Finetune a few layers	?



	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

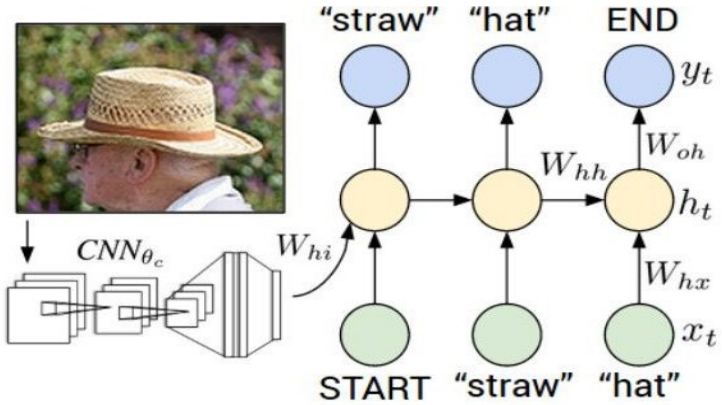
Transfer learning with CNNs is pervasive…
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

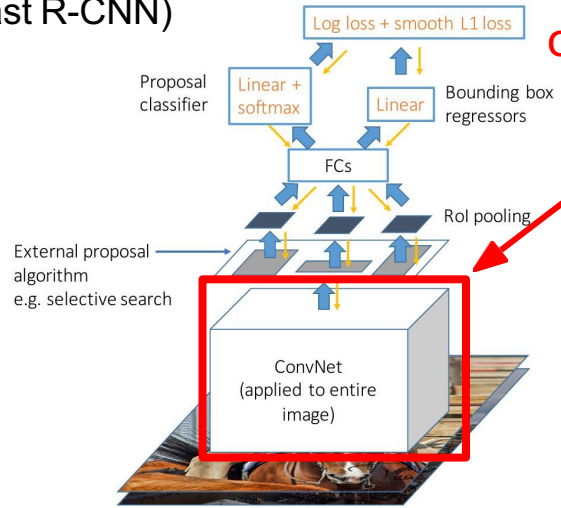
Image Captioning: CNN + RNN



Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

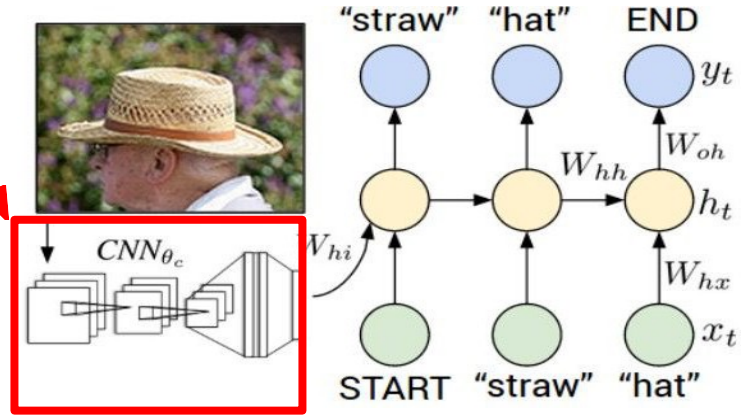
Transfer learning with CNNs is pervasive…
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained on ImageNet

Image Captioning: CNN + RNN

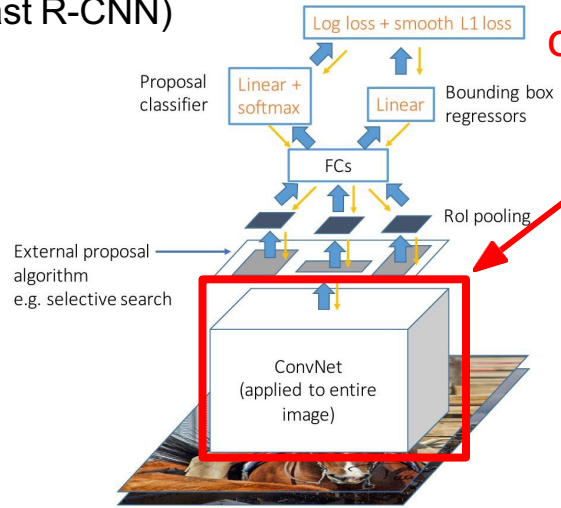


Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

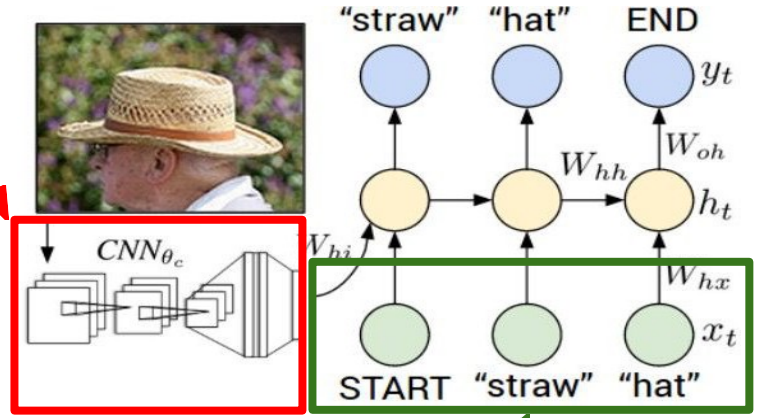
Transfer learning with CNNs is pervasive…
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained on ImageNet

Image Captioning: CNN + RNN

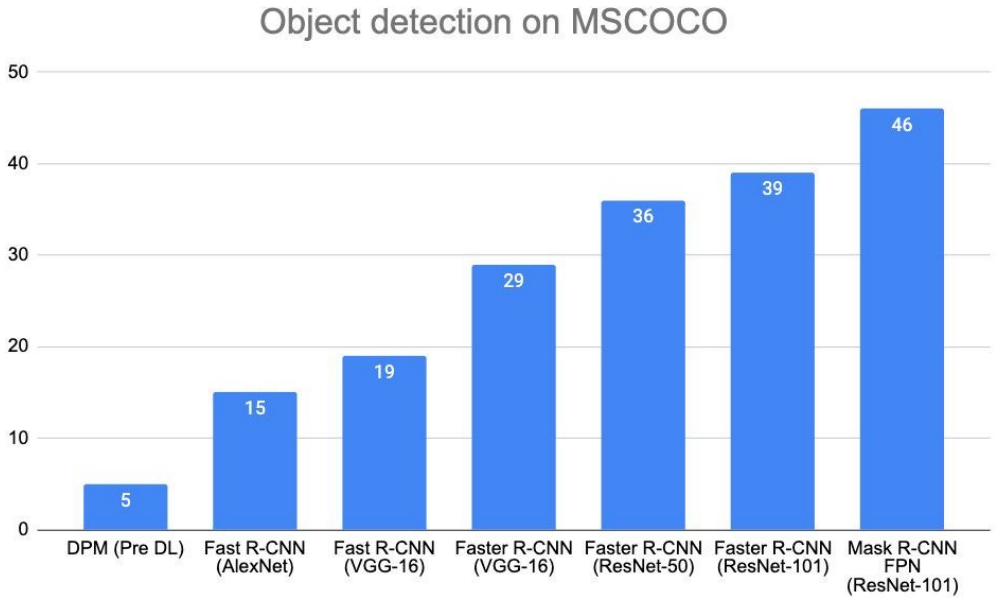


Word vectors pretrained with word2vec

Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

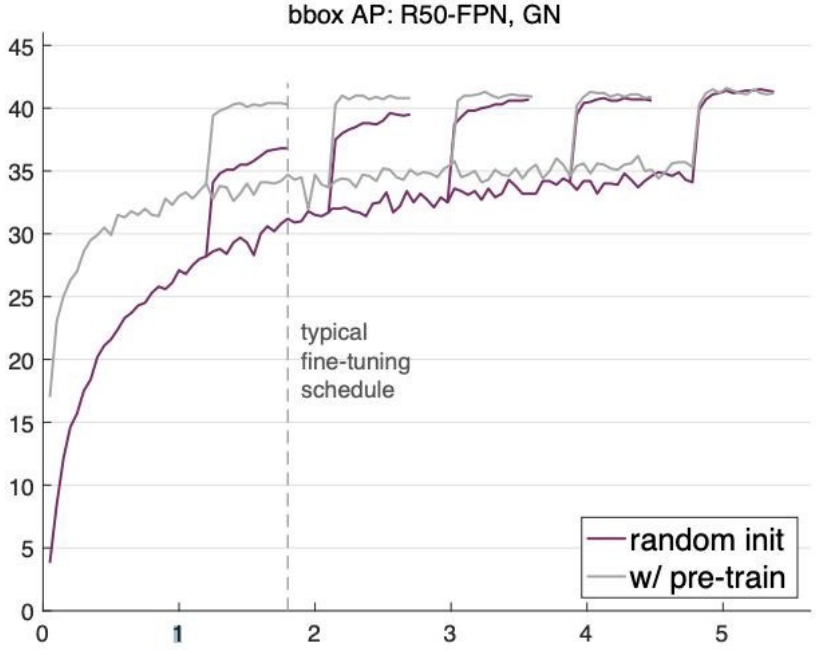
Transfer learning with CNNs - Architecture matters



Girshick, "The Generalized R-CNN Framework for Object Detection", ICCV 2017 Tutorial on Instance-Level Visual Recognition

Transfer learning with CNNs is pervasive…

Evidence: object detection, ICCV 2019.



He et al, "Rethinking ImageNet Pre-training", ICCV 2019
Figure copyright Kaiming He, 2019. Reproduced with permission.

Training from scratch can work just as well as training from a pretrained ImageNet model for object detection

But it takes 2-3x as long to train.

They also find that collecting more data is better than finetuning on a related task

How Much of the Model Should We Train?

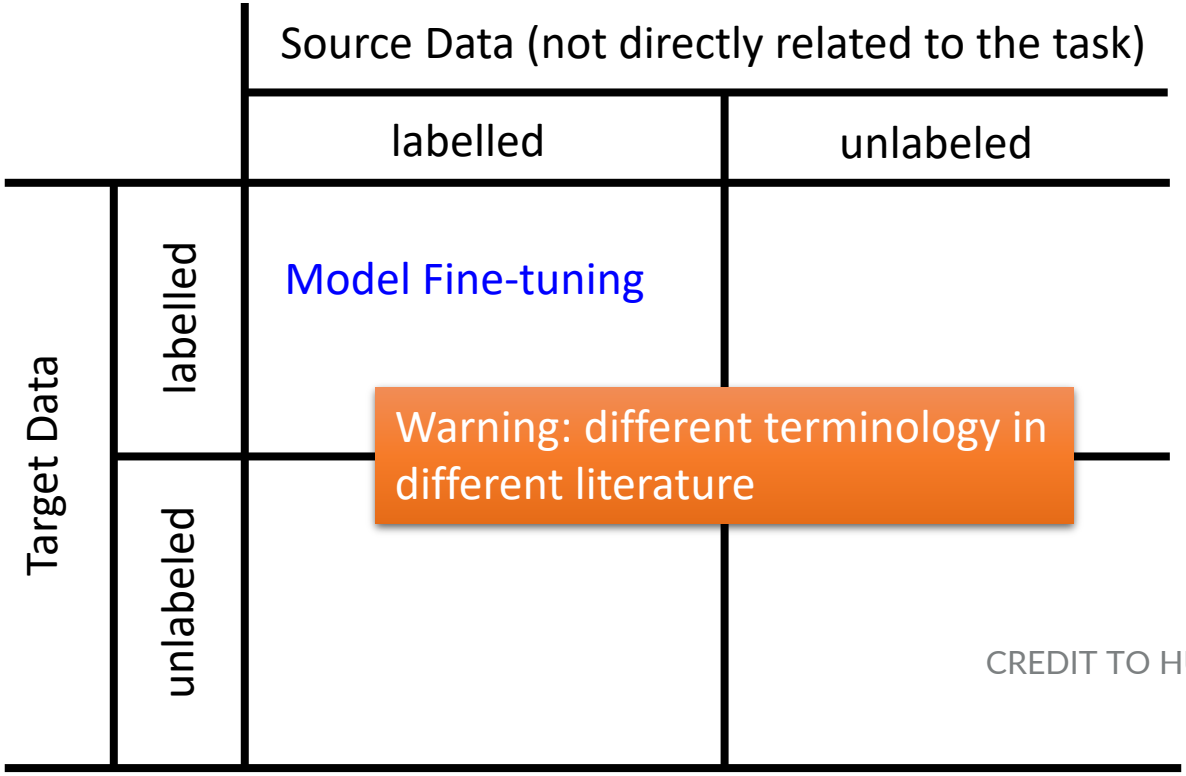
Data size, domain similarity and compute budget affect the choice.

Frozen backbone + new head	A low-cost baseline for the new labels.
Partial fine-tuning	Adapt some features while keeping others fixed.
Full fine-tuning	More flexibility, with greater training cost.

Compare these options using the same validation protocol.

Course teaching example. Sources and assumptions in speaker notes.

Transfer Learning - Overview



CREDIT TO HUNG-YI LEE@NTU

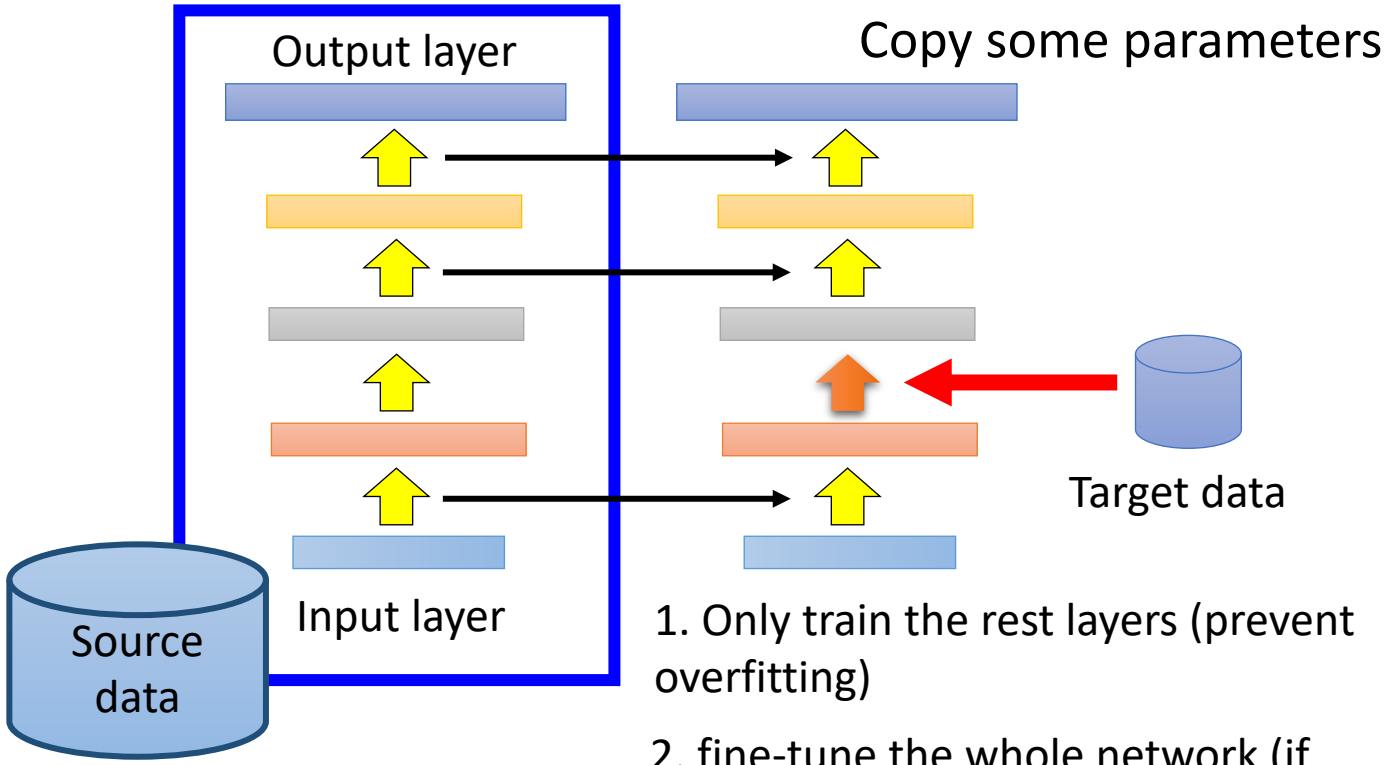
USC Viterbi School of Engineering

Model Fine-tuning

Few-shot: a few examples.
One-shot: one per class/task.

- Task description
  - Source data: (x^s, y^s) ⬅ A large amount
  - Target data: (x^t, y^t) ⬅ Very little
- Example: (supervised) speaker adaption
  - Source data: audio data and transcriptions from many speakers
  - Target data: audio data and its transcriptions of specific user
- Idea: training a model by source data, then fine-tune the model by target data
  - Challenge: only limited target data, so be careful about overfitting

Layer Transfer

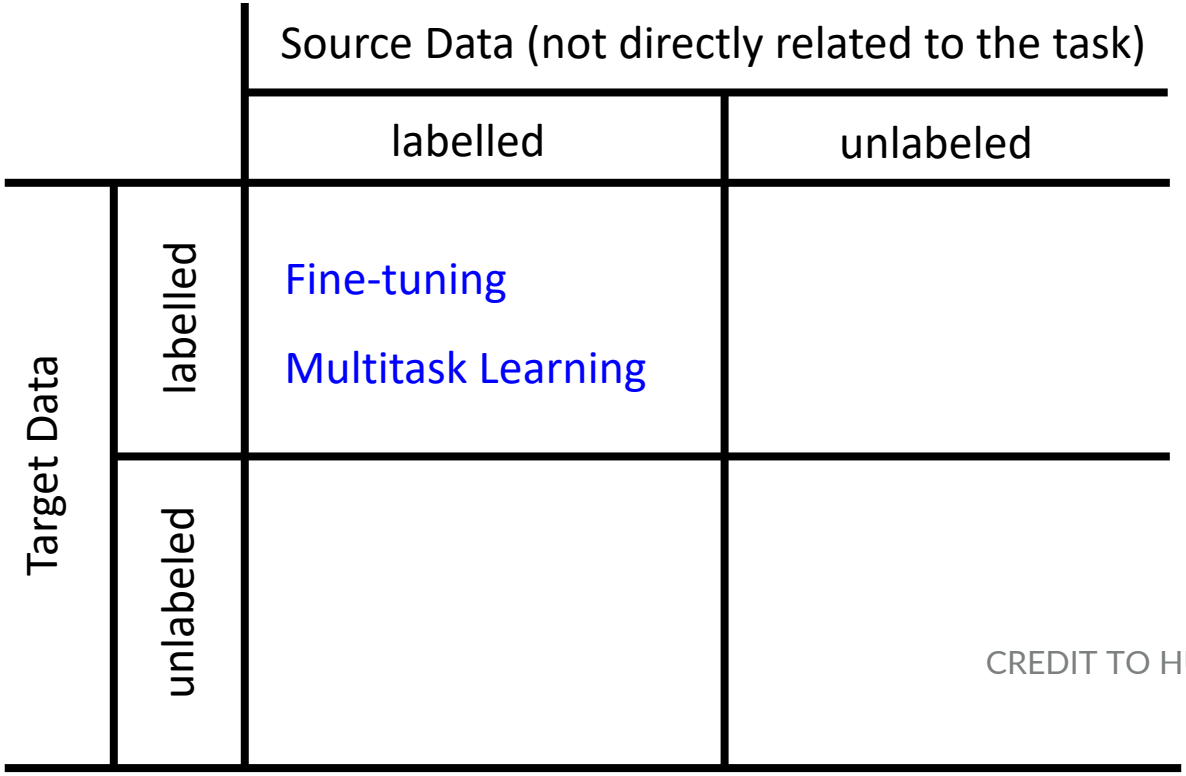


1. Only train the rest layers (prevent overfitting)

2. fine-tune the whole network (if there is sufficient data)

CREDIT TO HUNG-YI LEE@NTU

Transfer Learning - Overview



CREDIT TO HUNG-YI LEE@NTU

USC Viterbi School of Engineering

Transfer Can Fail When the Domain Changes

A model trained on everyday photos may miss specialized visual features.

Similar source and target

Some shapes, textures and object parts
may transfer usefully.

Different source and target

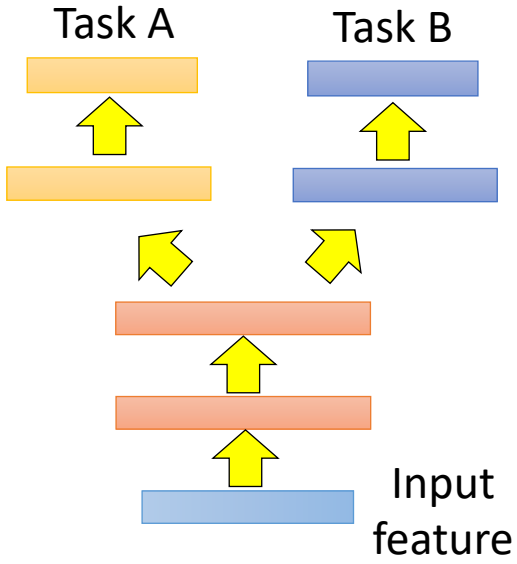
Microscopy or unusual sensors may need different representations.

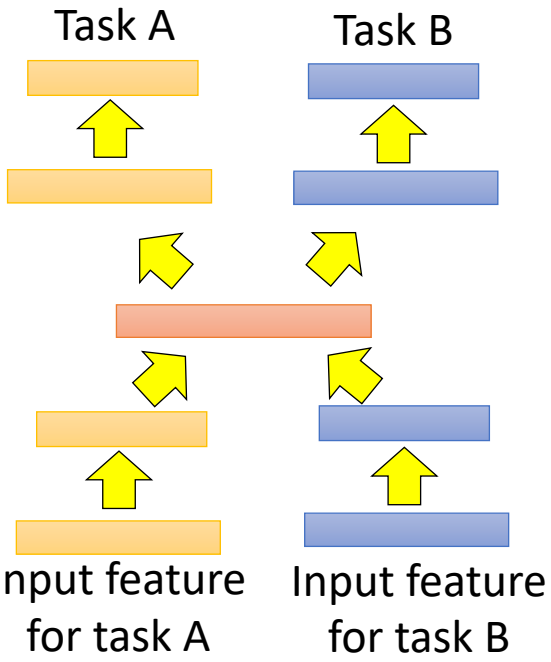
Check target-domain performance before trusting reuse.

Course teaching example. Sources and assumptions in speaker notes.

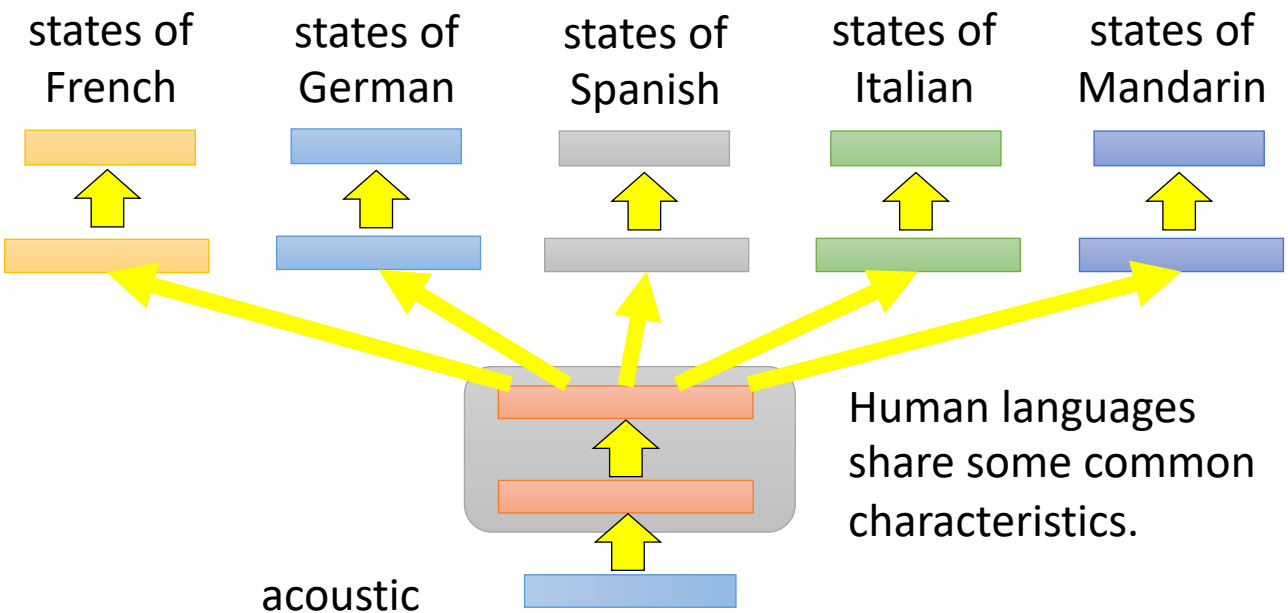
Multitask Learning

- The multi-layer structure makes NN suitable for multitask learning



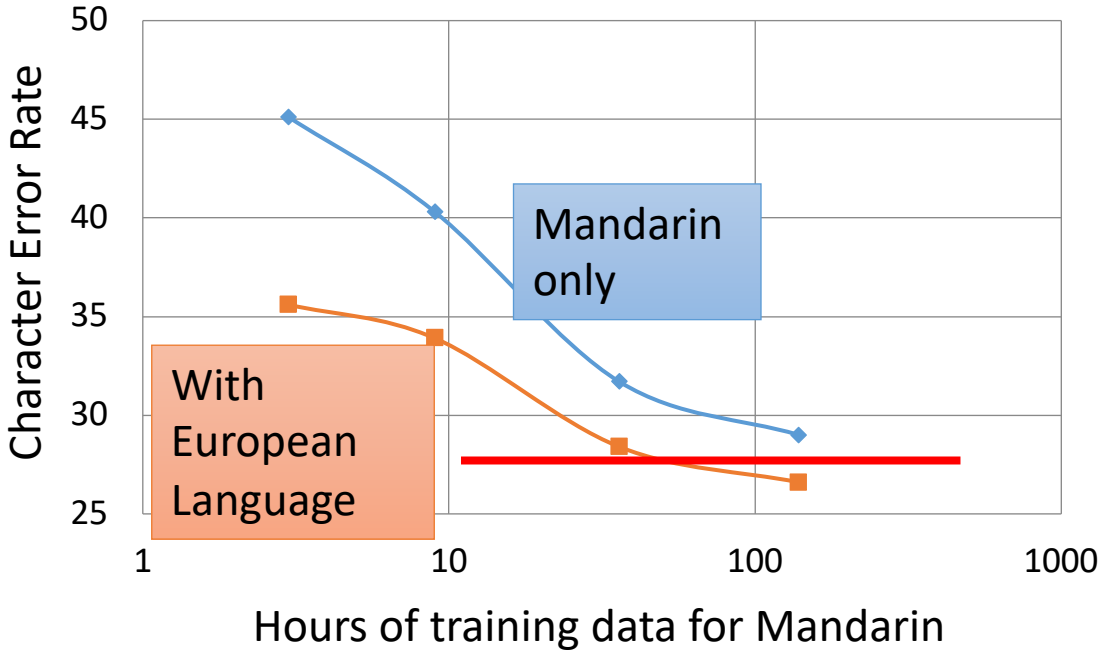


Multitask Learning
- Multilingual Speech Recognition



Similar idea in translation: Daxiang Dong, Hua Wu, Wei He, Dianhai Yu and Haifeng Wang, "Multi-task learning for multiple language translation.“, ACL 2015

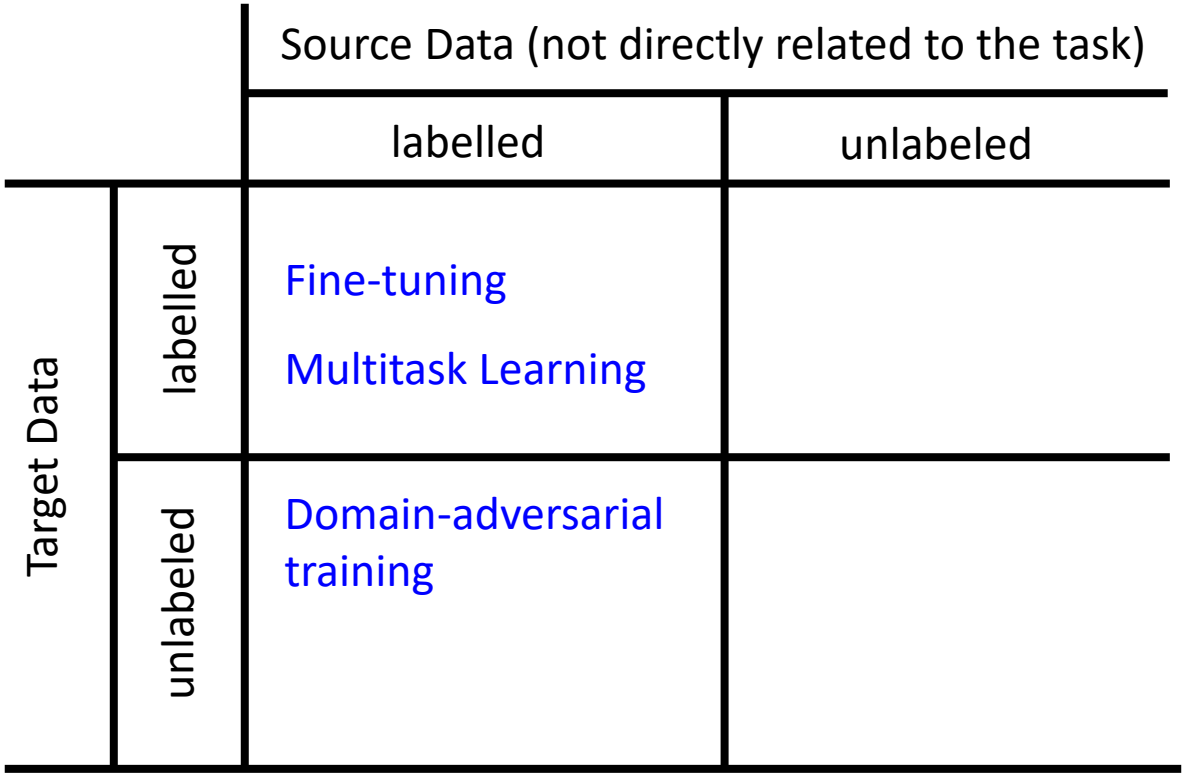
Multitask Learning - Multilingual



Huang, Jui-Ting, et al. "Cross-language knowledge transfer using multilingual deep neural network with shared hidden layers." *ICASSP, 2013*

USC Viterbi School of Engineering

Transfer Learning - Overview



USC Viterbi School of Engineering

Task description

- Source data: (x^s, y^s) → Training data
- Target data: (x^t) → Testing data

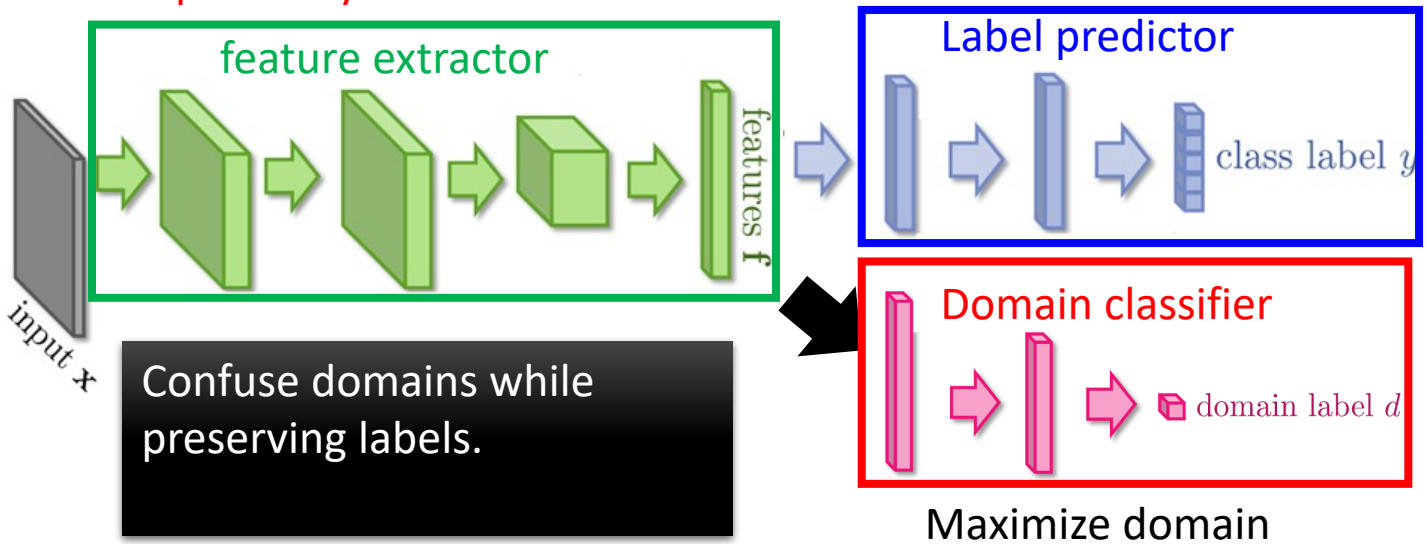
} Same task, mismatch

CREDIT TO HUNG-YI LEE@NTU

Domain-adversarial training

Make features useful for labels and difficult to separate by domain.

Maximize label classification accuracy

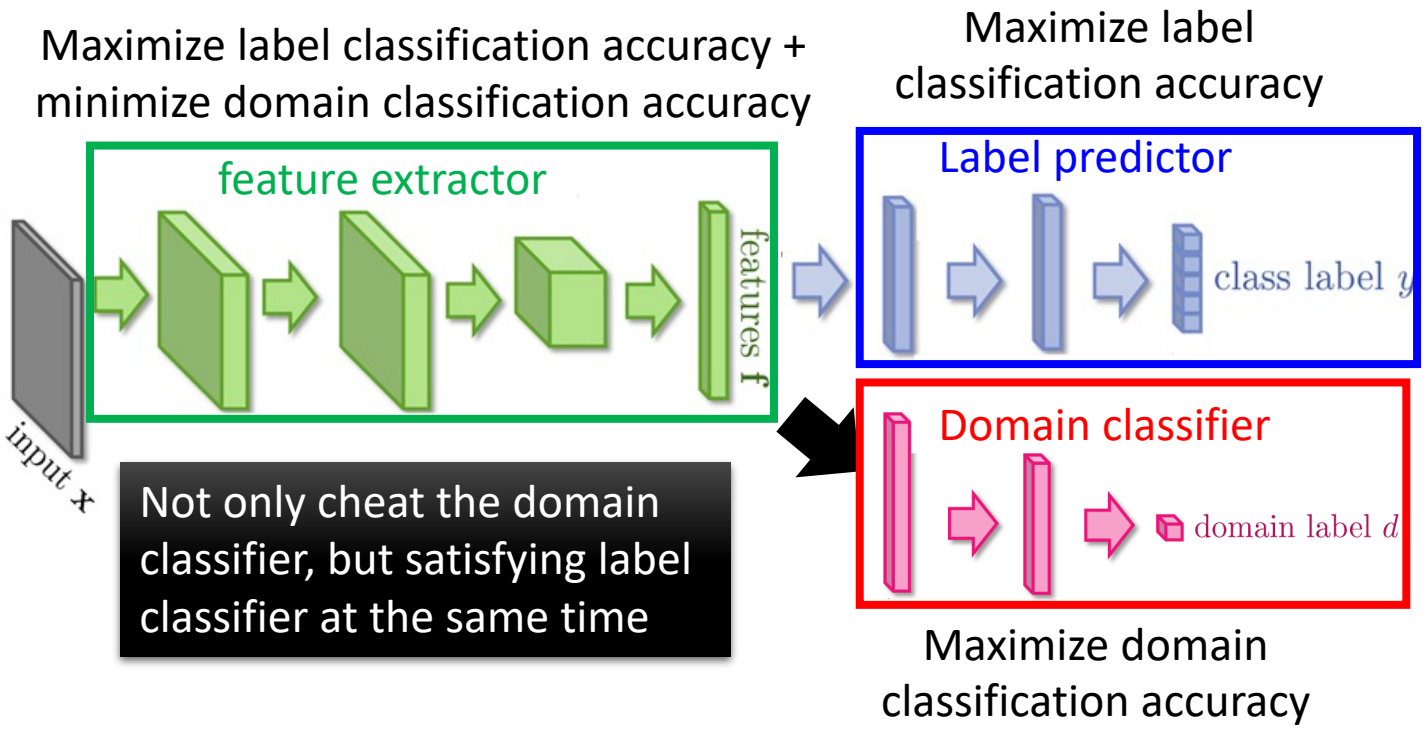


Maximize domain classification accuracy

This is a big network, but different parts have different goals.

USC Viterbi School of Engineering

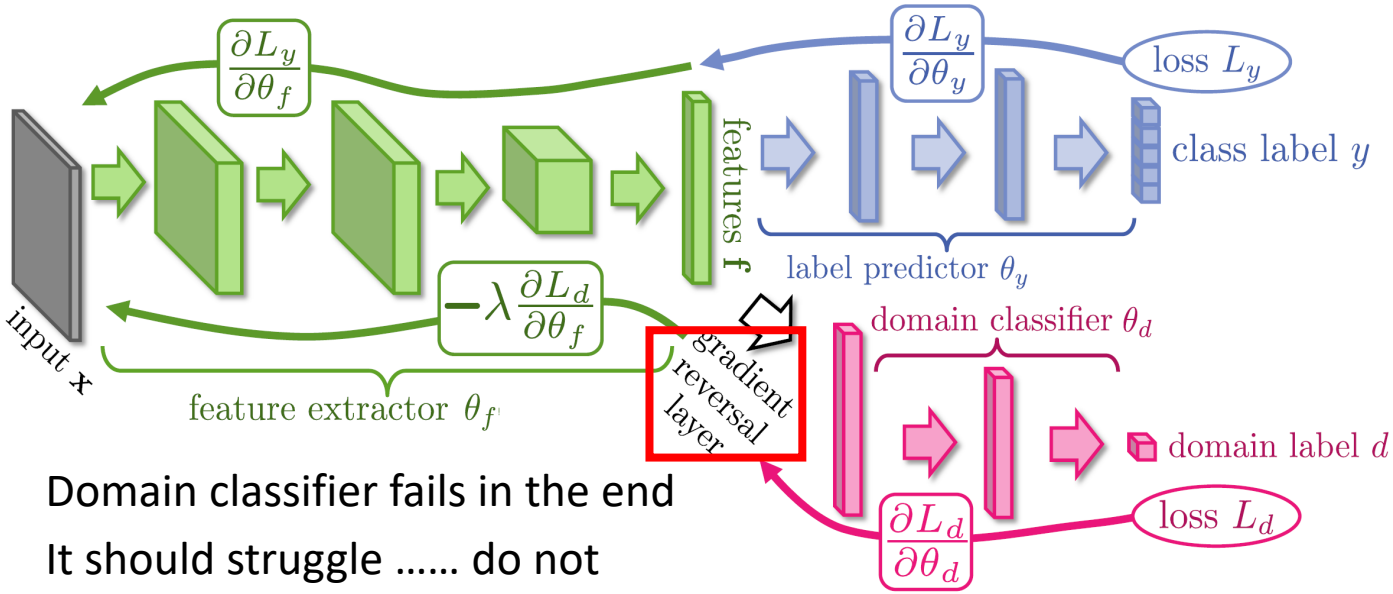
Domain-adversarial training



This is a big network, but different parts have different goals.

USC Viterbi School of Engineering

Domain-adversarial training



Domain classifier fails in the end

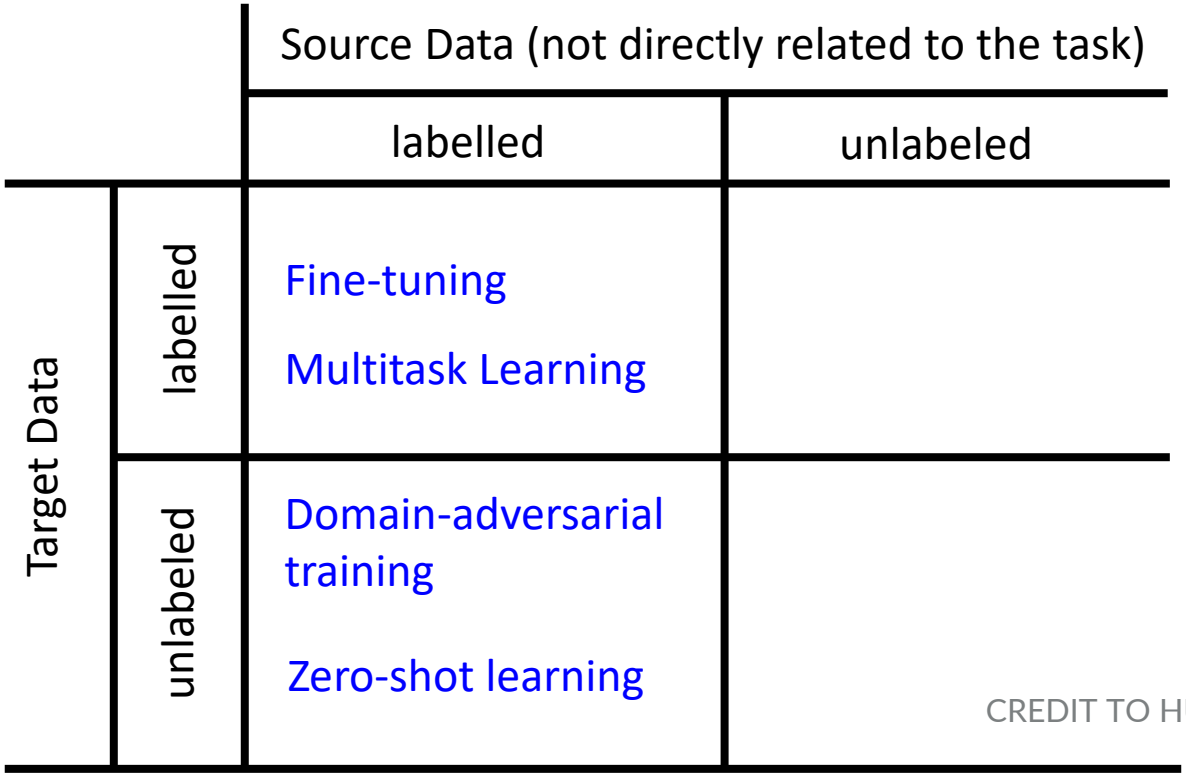
It should struggle …… do not know if it from source or target

Yaroslav Ganin, Victor Lempitsky, Unsupervised Domain Adaptation by Backpropagation, ICML, 2015

Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, Domain-Adversarial Training of Neural Networks, JMLR, 2016

USC Viterbi School of Engineering

Transfer Learning - Overview



CREDIT TO HUNG-YI LEE@NTU

USC Viterbi School of Engineering

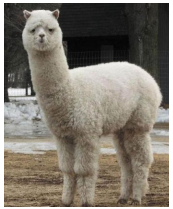
Zero-Shot Learning With Semantic Labels

- Source data: (x^s, y^s) → Training data
- Target data: (x^t) → Testing data

Different tasks

x^s: ……

y^s: cat dog ……

x^t : 

Unseen classes need semantic descriptions shared with known classes.

Example: alpaca = woolly coat, long neck, hooves.

Training and Transfer in One View

The same classifier can start from random or pretrained weights.

Starting point	Initialize a new model or load pretrained weights.
Trainable part	Choose the full model, selected layers, or a new head.
Evaluation	Select by validation performance and test the final choice.

Initialization, optimization and transfer fit one training loop.

Course teaching example. Sources and assumptions in speaker notes.

Finetune LLMs

Thanks to CMU 10-623 Material

Adapting a Language Model to a New Task

We now switch from flower photos to the sentiment of flower-shop reviews.

Input text	“The flowers arrived fresh.”
Target label	Positive. A wilted-flower complaint could be Negative.
Reusable model	A pretrained Transformer supplies text representations.

The adaptation question stays the same: what will change?

Course schematic. Sources and assumptions in speaker notes.

Examples in the Prompt or Updates to the Weights?

Both can change behavior, through different mechanisms.

In-context learning

Put examples in the input prompt.
The usual inference call keeps model parameters fixed.

Fine-tuning

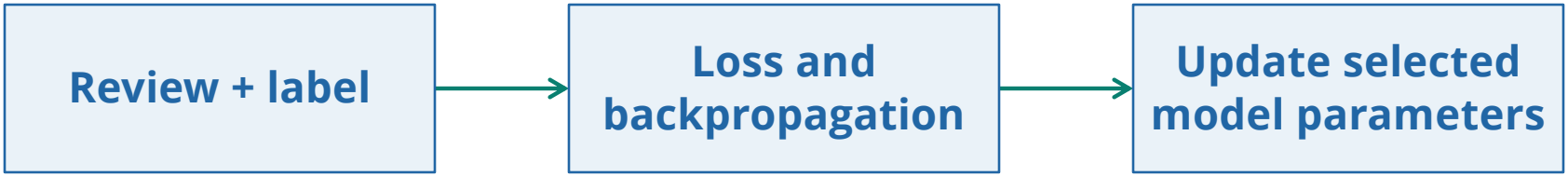
Use training examples and loss to update selected parameters before later inference.

Ask where the adaptation is stored.

Hu et al., LoRA (ICLR 2022). Course schematic.

Fine-Tuning With Labeled Reviews

Use the same supervised training loop we used for flower photos.



The learned adaptation is stored in the updated parameters.

Course schematic. Sources and assumptions in speaker notes.

Examples Inside the Prompt

At inference, the ordinary in-context-learning call keeps model parameters fixed.

“Fresh flowers.” → Positive
“Arrived wilted.” → Negative
“The bouquet was lovely.” → ?

Examples use context space. Dense attention compares many token pairs.

The prompt supplies task examples for this prediction.

Course schematic. Sources and assumptions in speaker notes.

Fine-Tuning vs. In-Context Learning

- Why would we ever bother with fine-tuning if it's so inefficient?
- Because, even for very large LMs, fine-tuning often beats in-context learning

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Question:

Why did fine-tuning of GPT-3 do so much better on these two tasks than few-shot learning?

Answer:

- *Few-shot details:* GPT-3's context length is only 2048 tokens. So the MNLI-m setting above only uses 6 few-shot examples in total.
- *Fine-tuned details:* MNLI-m has 393,000 training examples and GPT-3 is fine-tuned for 2 epochs.

Figure from http://arxiv.org/abs/2106.09685

Fine-Tuning vs. In-Context Learning

- Why would we ever bother with fine-tuning if it's so inefficient?
- Because, even for very large LMs, fine-tuning often beats in-context learning
- In a fair comparison of fine-tuning (FT) and in-context learning (ICL), we find that FT outperforms ICL for most model sizes on RTE and MNLI

		FT						
		125M	350M	1.3B	2.7B	6.7B	13B	30B
ICL	125M	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	350M	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	1.3B	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	2.7B	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	6.7B	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	13B	−0.04	−0.02	−0.01	−0.00	0.09	0.11	0.05
	30B	−0.11	−0.09	−0.08	−0.08	0.02	0.03	−0.02

(a) RTE

		FT						
		125M	350M	1.3B	2.7B	6.7B	13B	30B
ICL	125M	−0.00	0.00	0.02	0.01	0.10	0.11	0.07
	350M	−0.00	0.00	0.02	0.01	0.10	0.11	0.07
	1.3B	−0.01	−0.00	0.01	0.01	0.10	0.11	0.07
	2.7B	−0.01	−0.00	0.01	0.01	0.09	0.10	0.07
	6.7B	−0.01	−0.01	0.01	0.00	0.09	0.10	0.06
	13B	−0.03	−0.03	−0.02	−0.02	0.07	0.08	0.04
	30B	−0.07	−0.07	−0.05	−0.06	0.03	0.04	0.00

(b) MNLI

Table 1: Difference between average **out-of-domain performance** of ICL and FT on RTE (a) and MNLI (b) across model sizes. We use 16 examples and 10 random seeds for both approaches. For ICL, we use the `gpt-3` pattern. For FT, we use pattern-based fine-tuning (PBFT) and select checkpoints according to in-domain performance. We perform a Welch's t-test and color cells according to whether: ICL performs significantly better than FT, FT performs significantly better than ICL. For cells without color, there is no significant difference.

Figure from https://aclanthology.org/2023.findings-acl.779.pdf

Fine-Tuning vs. In-Context Learning

- Why would we ever bother with fine-tuning if it's so inefficient?
- Because, even for very large LMs, fine-tuning often beats in-context learning
- In a fair comparison of fine-tuning (FT) and in-context learning (ICL), we find that FT outperforms ICL for most model sizes on RTE and MNLI

		FT						
		125M	350M	1.3B	2.7B	6.7B	13B	30B
ICL	125M	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	350M	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	1.3B	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	2.7B	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	6.7B	−0.00	0.01	0.02	0.03	0.12	0.14	0.09
	13B	−0.04	−0.02	−0.01	−0.00	0.09	0.11	0.05
	30B	−0.11	−0.09	−0.08	−0.08	0.02	0.03	−0.02

(a) RTE

		FT						
		125M	350M	1.3B	2.7B	6.7B	13B	30B
ICL	125M	−0.00	0.00	0.02	0.01	0.10	0.11	0.07
	350M	−0.00	0.00	0.02	0.01	0.10	0.11	0.07
	1.3B	−0.01	−0.00	0.01	0.01	0.10	0.11	0.07
	2.7B	−0.01	−0.00	0.01	0.01	0.09	0.10	0.07
	6.7B	−0.01	−0.01	0.01	0.00	0.09	0.10	0.06
	13B	−0.03	−0.03	−0.02	−0.02	0.07	0.08	0.04
	30B	−0.07	−0.07	−0.05	−0.06	0.03	0.04	0.00

(b) MNLI

Table 1: Difference between average **out-of-domain performance** of ICL and FT on RTE (a) ar
model sizes. We use 16 examples and 10 random seeds for both approaches. For ICL, we use
For FT, we use pattern-based fine-tuning (PBFT) and select checkpoints according to in-do
We perform a Welch's t-test and color cells according to whether: ICL performs significantly
performs significantly better than ICL. For cells without color, there is no significant difference.

At least this was the general wisdom in 2023.

We might have a different story to tell since 2024.

Figure from https://aclanthology.org/2023.findings-acl.779.pdf

Why Train Only a Small Part of a Large Model?

Full fine-tuning stores more than the pretrained weights.

Weights	The model parameters used in the forward pass.
Gradients and optimizer state	Additional values used to update trainable parameters.
Activations	Intermediate values needed for backpropagation.

Fewer trainable parameters can reduce training-state cost.

Hu et al., LoRA (ICLR 2022). Course schematic.

Parameter Efficient Fine-Tuning

- **Goal**: perform fine-tuning of fewer parameters, but achieve performance on a downstream task that is comparable to fine-tuning of all parameters
- **Various approaches**:
  - **Subset**: Pick a subset of the parameters and fine-tune only those (e.g. only the top K layers of a K+L layer deep neural network)
  - **Adapters**: add additional layers that have few parameters and tune only the parameters of those layers, keeping all others fixed
  - **Prefix Tuning**: for a Transformer LM, pretend as if there exist many tokens that came before your sequence and tune the keys/values corresponding to those tokens
  - **LoRA**: learn a small delta for the each of the parameter matrices with the delta chosen to be low rank

Three Places to Put Trainable Parameters

Keep the same pretrained model as the reference.

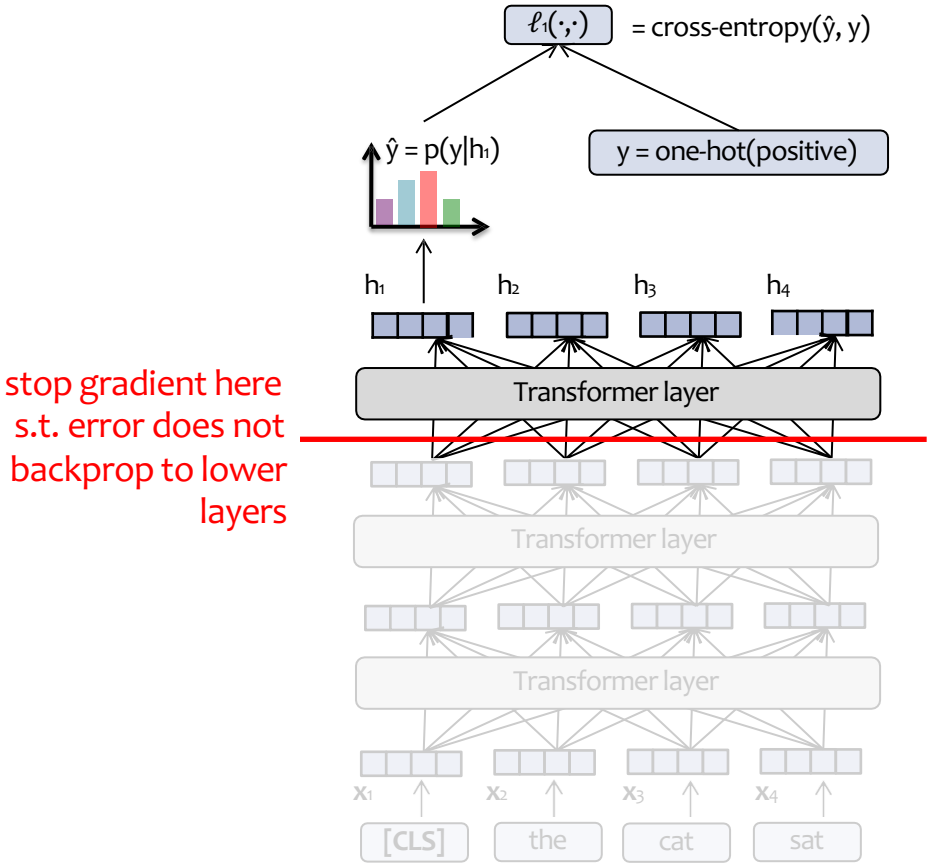
New head	Learn a new output mapping.
Adapter modules	Add small trainable transformations inside the model.
LoRA updates	Learn low-rank updates to selected weight matrices.

Choose the trainable part explicitly.

Hu et al., LoRA (ICLR 2022). Course schematic.

Fine-Tuning the Top Layers Only

- Simple baseline for PEFT:
  - keep all parameters fixed except for the top K layers
  - gradients only need to flow through K layers instead of K+L total layers
  - reduced memory usage b/c we don't need to store the adjoints (gradient of the loss with respect to each parameter) for the full computation graph
- Can easily be applied to most deep neural networks



Why Adapt Only the Top Layers?

Pretraining may already provide useful features for the new task.

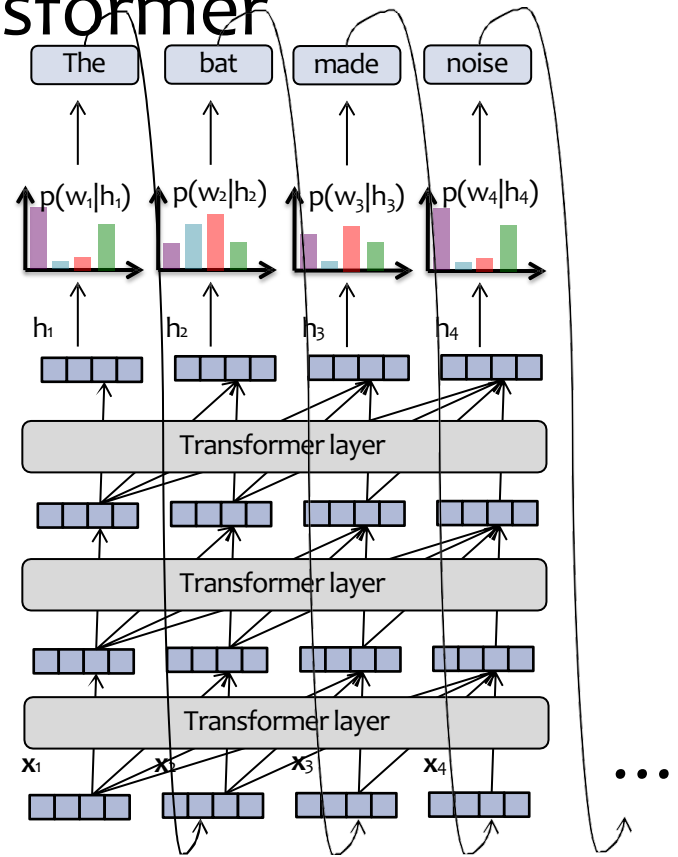
Frozen lower layers	Still compute features in the forward pass.
Trainable upper layers	Learn a new mapping from those features to the target labels.
Gradient boundary	Backward can stop below all trainable layers in this arrangement.

Check whether this restricted update is sufficient on validation data.

Course schematic. Sources and assumptions in speaker notes.

ADAPTERS

Decoder-only Transformer



Each layer of a Transformer LM consists of several **sublayers:**

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer.**

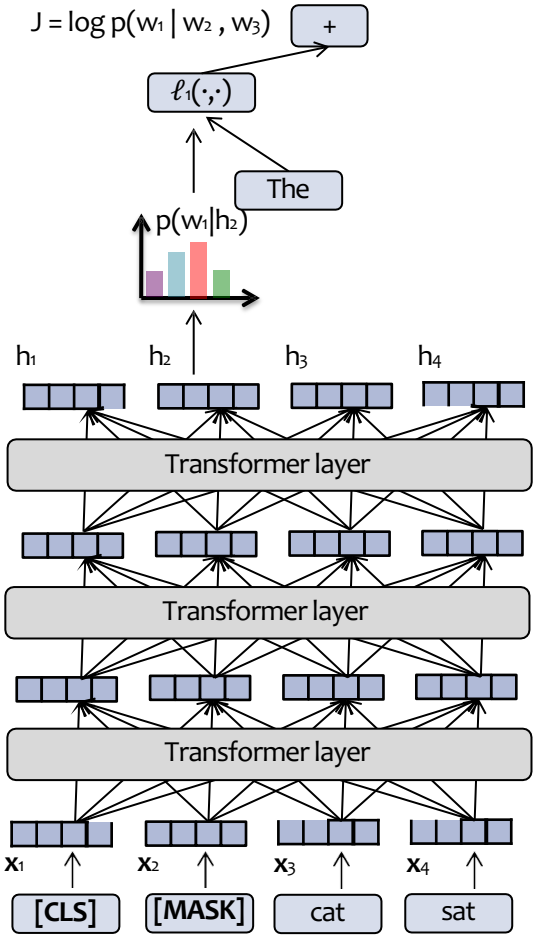
The language model part is just like an RNN-LM.

Encoder-only Transformer

BERT popularized this encoder-only Transformer architecture and style of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones…
- …mask out a word (or a few words) and predict the missing words from the remaining ones



…

Each layer of an encoder-only Transformer consists of several **sublayers:**

1. non-causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks at the the hidden vectors of **all timesteps in the previous layer.**

The distribution over words is used for **masked language model (MLM) pre-training** (cf. BERT)

Adapters Module

- An adapter layer is simply a feed-forward neural network with one hidden layer, and a residual connection
- For input dimension, d, the adapter layer also has output dimension d, but bottlenecks to a lower dimension r in the middle

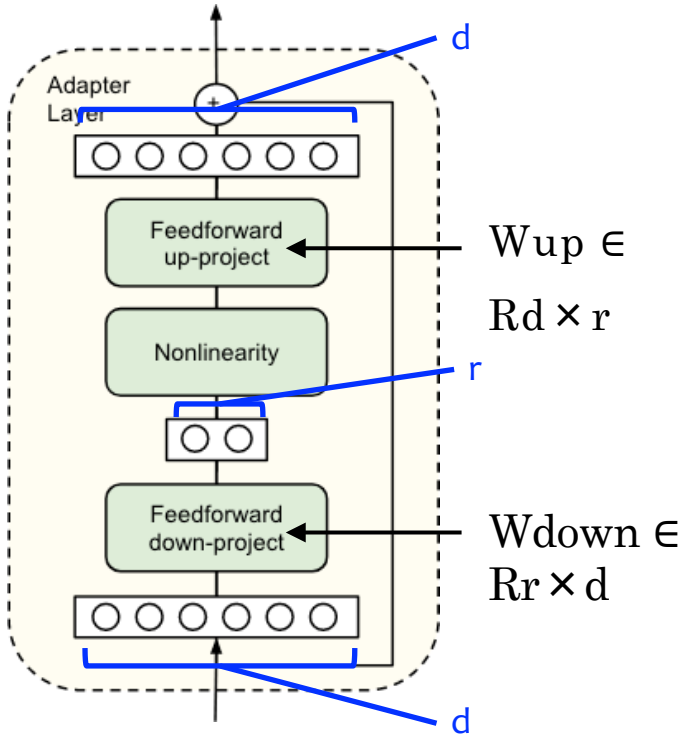
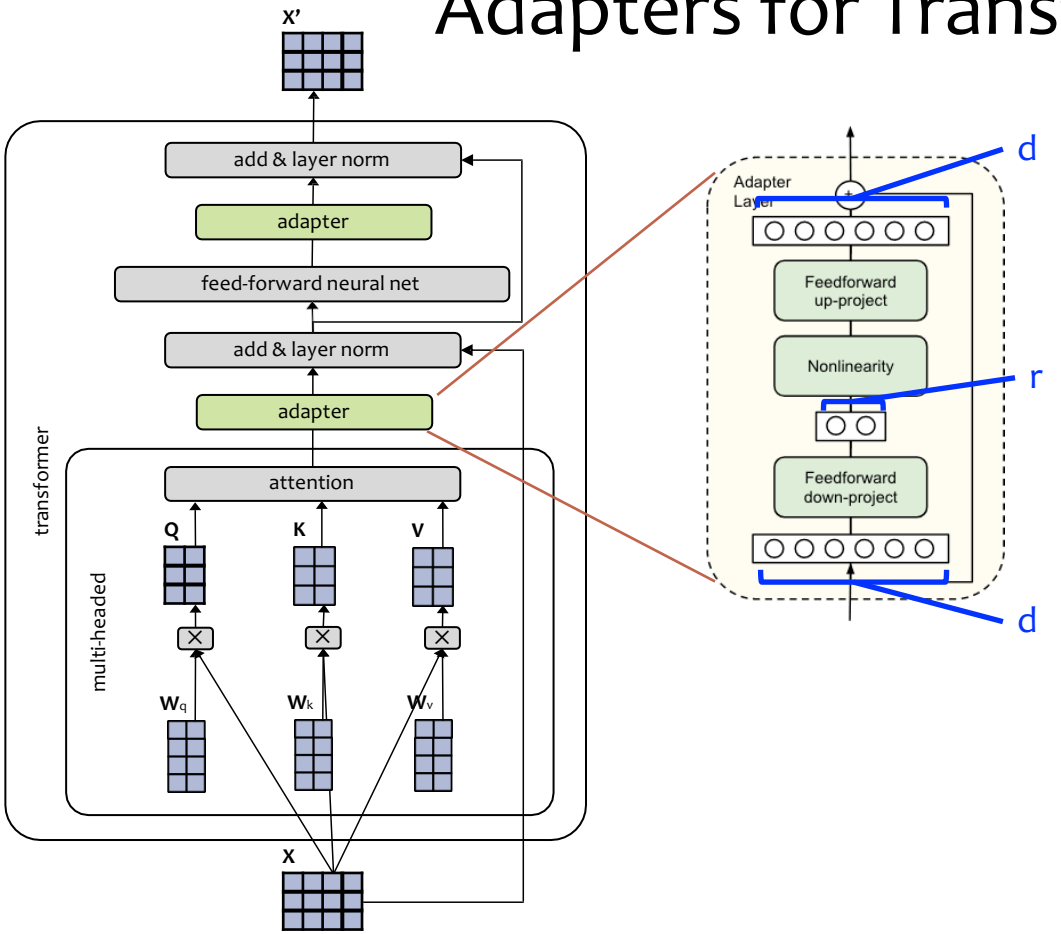


Figure from https://arxiv.org/pdf/1902.00751

Adapters for Transformer

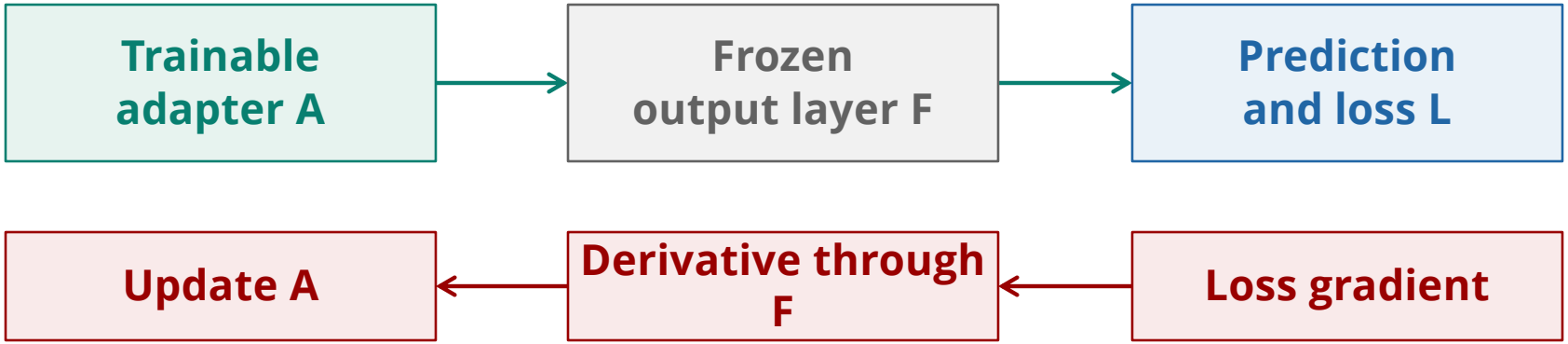


- In practice, r is chosen s.t. r << d and the adapter layers contain only 0.5% – 8% of the total parameters
- When added to a deep neural network (e.g. Transformer) all the other parameters of the pretrained model are kept fixed, and only the adapter layer parameters are fine-tuned
- Interesting: it works even though the grey modules are kept fixed!

Figure inspired by He et al. (2022) https://arxiv.org/pdf/2110.04366

Frozen Weights Can Pass Gradients

A frozen layer can sit between a trainable adapter and the loss.



Frozen means no weight update; gradients can still pass through.

Course schematic. Sources and assumptions in speaker notes.

Adapter Results

- **Pretrained Model:** BERT-Large
- **Baseline Method:** fine-tune only the top K layers of BERT-Large
- Adapters achieve nearly the performance (i.e. 0% delta) of full fine-tuning but with substantially fewer parameters
- Sometimes adapters even outperform full fine-tuning

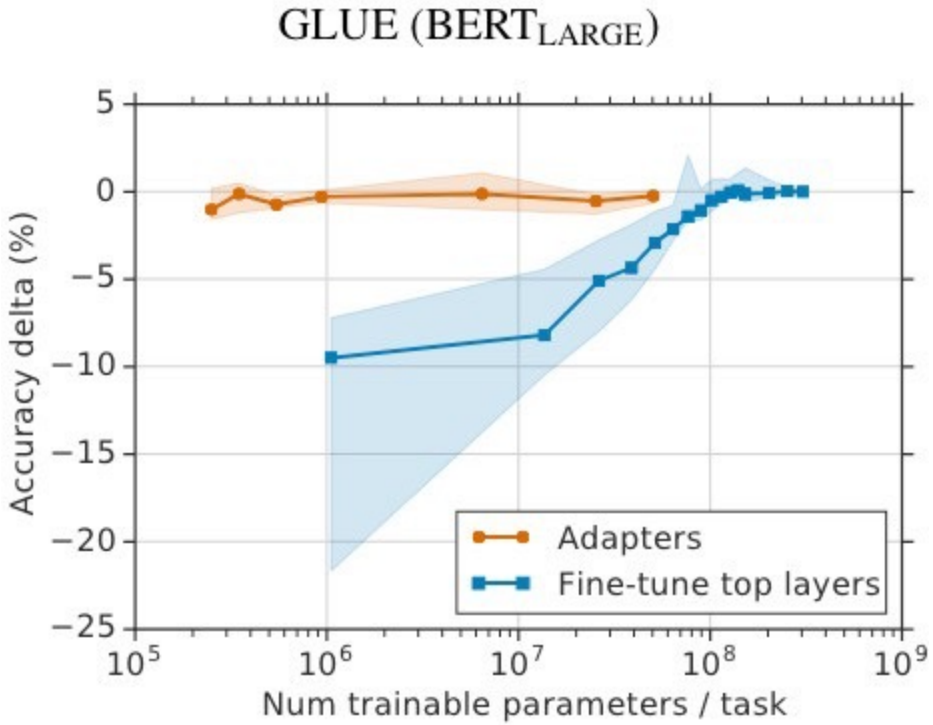


Figure from https://arxiv.org/pdf/1902.00751

Adapter Results

- **Pretrained Model:** BERT-Large
- **Baseline Method:** fine-tune only the top K layers of BERT-Large
- Adapters achieve nearly the performance (i.e. 0% delta) of full fine-tuning but with substantially fewer parameters
- Sometimes adapters even outperform full fine-tuning

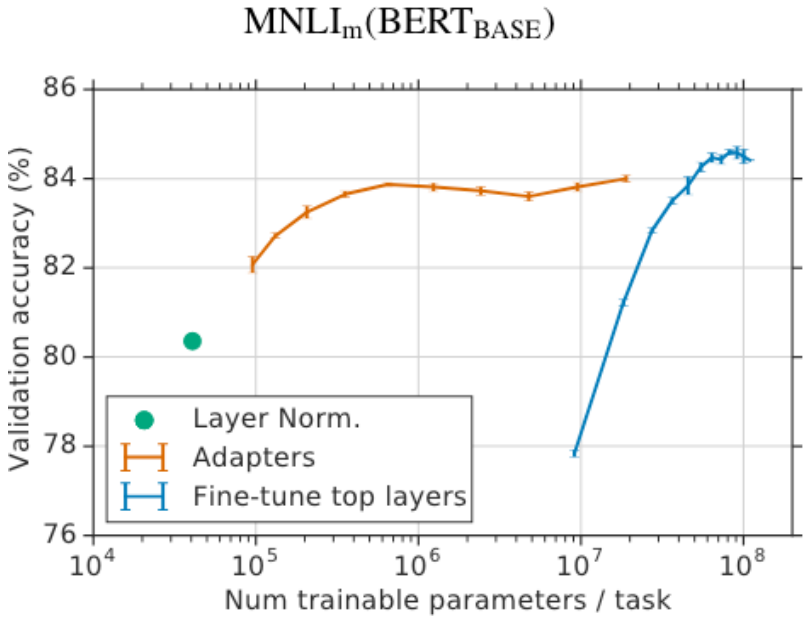


Figure from https://arxiv.org/pdf/1902.00751

Adapter Results

- **Pretrained Model:** BERT-Large
- **Baseline Method:** fine-tune only the top K layers of BERT-Large
- Adapters achieve nearly the performance (i.e. 0% delta) of full fine-tuning but with substantially fewer parameters
- Sometimes adapters even outperform full fine-tuning

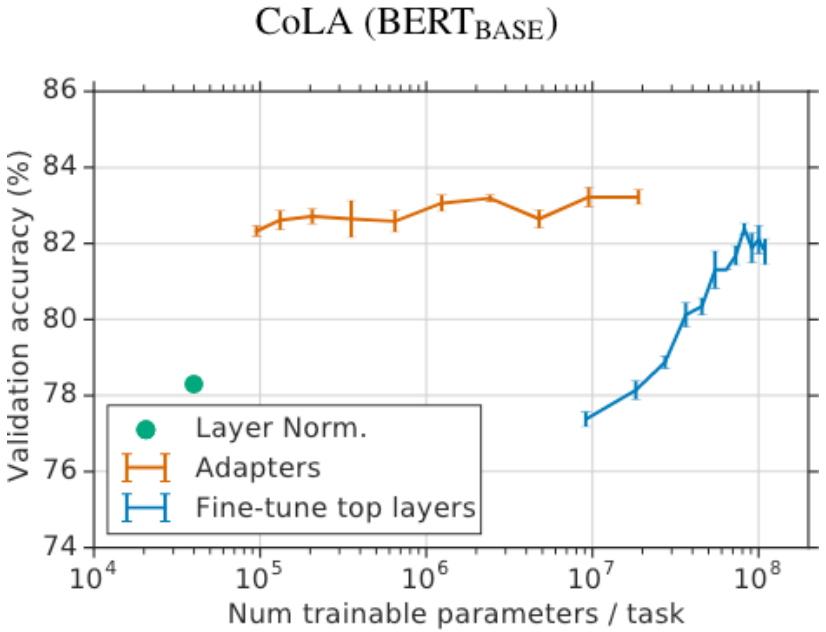


Figure from https://arxiv.org/pdf/1902.00751

LOW-RANK ADAPTATION (LORA)

LoRA Learns a Change to a Frozen Weight Matrix

For a selected linear layer, keep the pretrained matrix W fixed.

Full fine-tuning can change every entry of W.

LoRA uses $W + \Delta W$, with $\Delta W = B \times A$.

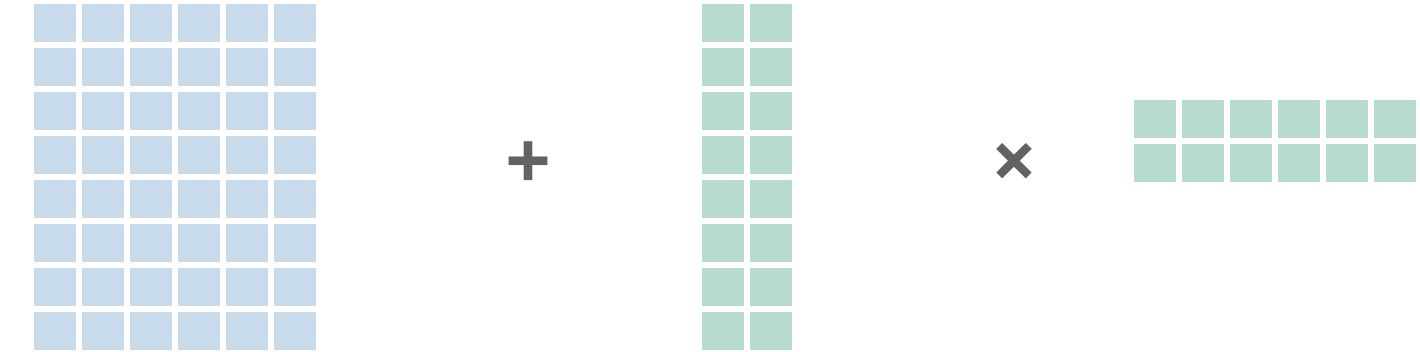
Train A and B. Keep the original W frozen.

The low-rank object is the update ΔW.

Hu et al., LoRA (ICLR 2022). Course schematic.

A Small LoRA Example

W maps 6 input features to 8 output features. Choose rank 2.



The original W remains intact while B × A changes its output.

Hu et al., LoRA (ICLR 2022). Course schematic.

Count the Trainable Parameters

Use the same 8-by-6 layer and rank-2 update.

Full matrix: 8 × 6 = 48 trainable entries.

LoRA: (8 × 2) + (2 × 6) = 28 trainable entries.

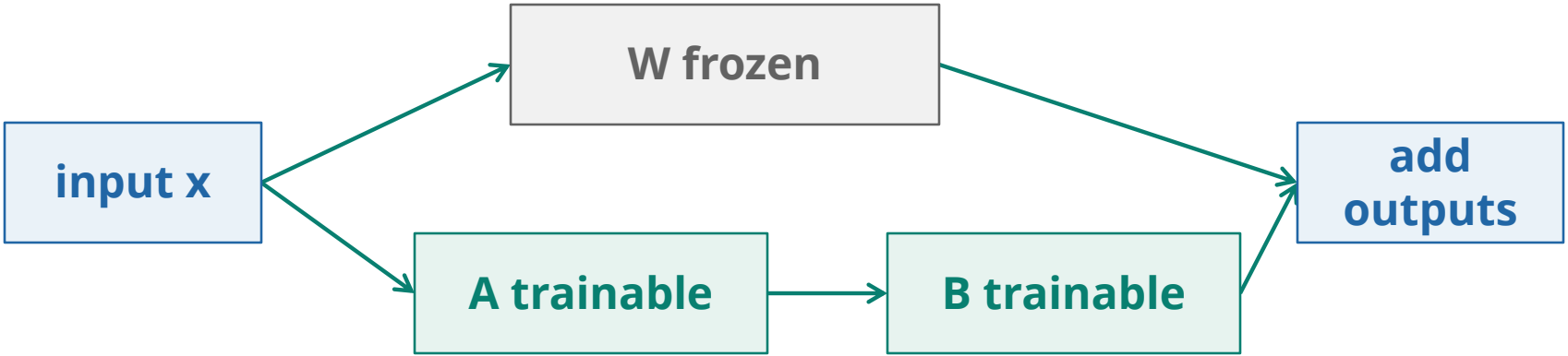
The 48 frozen entries of W still need to be stored.

The saving depends on rank and matrix dimensions.

Hu et al., LoRA (ICLR 2022). Course schematic.

The Frozen and Trainable Paths Add Together

For a column input x, the layer computes W x + B(A x).



Both paths use the same input x.

Hu et al., LoRA (ICLR 2022). Course schematic.

How large are LLMs?

Recall…

Comparison of some **large language models** (LLMs)

Model	Creators	Year	Training tokens	Parameters
GPT-2	OpenAI	2019	~10 billion (40Gb)	1.5 billion
GPT-3 (cf. ChatGPT)	OpenAI	2020	300 billion	175 billion
PaLM	Google	2022	780 billion	540 billion
Chinchilla	DeepMind	2022	1.4 trillion	70 billion
LaMDA (cf. Bard)	Google	2022	1.56 trillion	137 billion
LLaMA	Meta	2023	1.4 trillion	65 billion
LLaMA-2	Meta	2023	2 trillion	70 billion
GPT-4	OpenAI	2023	?	Not disclosed
Gemini (Ultra)	Google	2023	?	Not disclosed
Llama 3.1	Meta	2024	15 trillion	405 billion

How large are the linear layers in a Transformer?

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-paramet (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 ion tokens.

Size of linear layer in GPT-3:
12k * 12k

Figure from https://arxiv.org/pdf/2005.14165.pdf

A Restricted Space for Fine-Tuning

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Question:

Can useful adaptation fit inside a small update space?

Hypothesis:

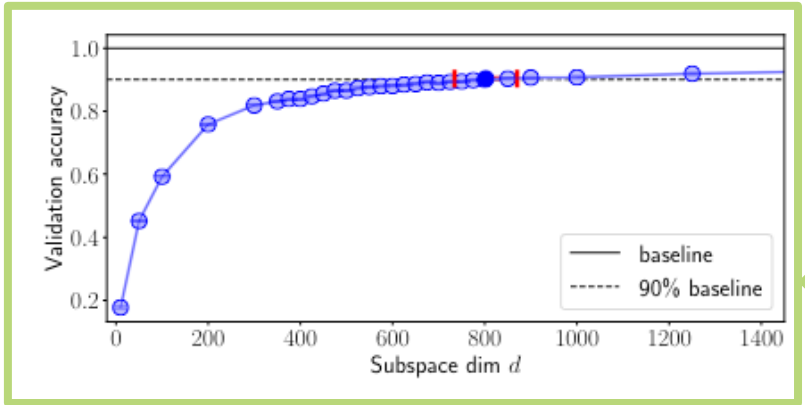
Small update spaces can suffice on some tasks.

Figure from http://arxiv.org/abs/1804.08838

Intrinsic Dimensionality

Motivation

- Maybe the number of parameters in a model is **not** a great measure of how many degrees of freedom are needed to successfully learn some problem
- How could we measure the number of degrees of freedom you **really** need?



Intrinsic Dimension

Definition from Li et al. (2018):

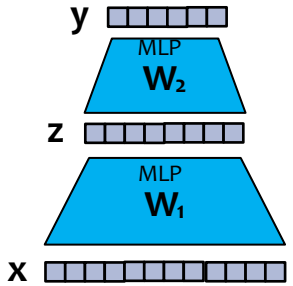
- Learn a neural network with *D* parameters in a random lower dimensional subspace, *d*
- Then repeat, gradually increasing the dimensionality, *d*
- Let the **intrinsic dimension** be the value of *d* when good solutions (above 90% threshold of full parameterization) start to appear

For MNIST digit recognition, original neural network has *D*=199,210 parameters but the intrinsic dimension is only *d*=750

Intrinsic Dimensionality

How do we learn in a low dimensional subspace?

$$\theta^{(D)} = \text{concat}(\text{flatten}(W_1), \text{flatten}(W_2))$$



Standard optimization, which we will refer to hereafter as the *direct* method of training, entails evaluating the gradient of a loss with respect to $\theta^{(D)}$ and taking steps directly in the space of $\theta^{(D)}$. To train in a random subspace, we instead define $\theta^{(D)}$ in the following way:

$$\theta^{(D)} = \theta_0^{(D)} + P\theta^{(d)} \tag{2}$$

where P is a randomly generated $D \times d$ projection matrix[1] and $\theta^{(d)}$ is a parameter vector in a generally smaller space $\mathbb{R}^d$. $\theta_0^{(D)}$ and P are randomly generated and frozen (not trained), so the system has only d degrees of freedom. We initialize $\theta^{(d)}$ to a vector of all zeros, so initially $\theta^{(D)} = \theta_0^{(D)}$.

Figure from http://arxiv.org/abs/1804.08838

Intrinsic Dimensionality

- Aghajanyan et al. (2020) estimate the update dimension needed for a task.
- d90: directions needed to reach 90% of full fine-tuning performance.

- Number of parameters:
  - BERT-Base: 110 million
  - BERT-Large: 340 million

	SAID		DID	
Model	MRPC	QQP	MRPC	QQP
BERT-Base	1608	8030	1861	9295
BERT-Large	1037	1200	2493	1389
RoBERTa-Base	896	896	1000	1389
RoBERTa-Large	**207**	**774**	322	**774**

Table 1: Estimated d_{90} intrinsic dimension for a set of sentence prediction tasks and common pre-trained models. We present both the *SAID* and *DID* methods.

Aghajanyan et al. (2020), arXiv:2012.13255.

LoRA Restricts the Update Space

The pretrained matrix stays full size while its learned change has limited rank.

Full fine-tuning

Each selected weight can change independently.

LoRA

The change must be expressible as B × A with the chosen rank.

More capacity does not guarantee better accuracy.

Hu et al., LoRA (ICLR 2022). Course schematic.

LoRA

Key Idea

- Keep the original pretrained parameters $\mathbf{W}_0$ fixed during fine-tuning
- Learn an additive modification to those parameters $\Delta\mathbf{W}$
- Define $\Delta\mathbf{W}$ via a low rank decomposition:

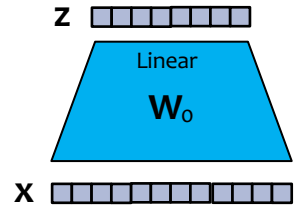
$$\Delta W = BA$$

where **BA** has rank r, which is **much less** than the input dimension k or the output dimension d

Standard Linear Layer

$$z = W_0 x$$

$$W_0 \in R^{d \times k}, x \in R^k, z \in R^d$$



LoRA Linear Layer

$$z = W_0 x + BAx$$
$$= (W_0 + BA)x$$

$$W_0 \in R^{d \times k},$$

$$A \in R^{r \times k}, B \in R^{d \times r}$$

where $r << \min(d, k)$

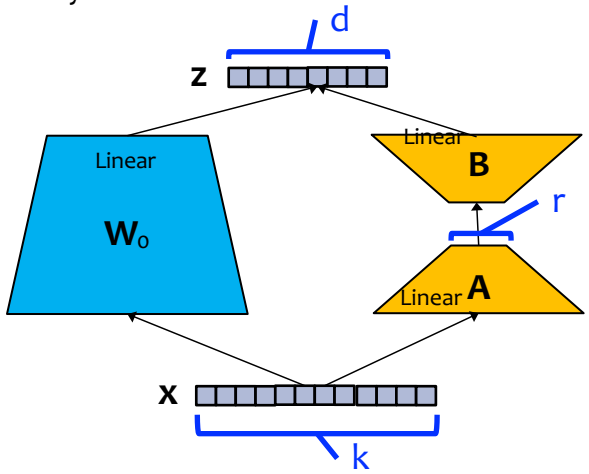


Figure inspired by He et al. (2022)

Rank Controls the Space of Allowed Updates

A rank-2 update has fewer degrees of freedom than a full matrix update.

Smaller rank

Fewer trainable parameters.
A more restricted set of changes.

Larger rank

More trainable parameters.
More possible changes to the layer.

Validation decides whether extra capacity helps.

Hu et al., LoRA (ICLR 2022). Course schematic.

Adapters and LoRA Modify Different Computations

Both can keep the pretrained weights frozen during adaptation.

Adapter

Insert a small trainable module. Its computation remains in the adapted forward pass.

LoRA

Add B × A to selected weights. For a compatible fixed adapter, merge it into W for inference.

LoRA learns an update rather than factorizing the original W.

Hu et al., LoRA (ICLR 2022). Course schematic.

Hyperparameter Tuning and HyperTuning Differ

The next research example introduces a separately named method.

Hyperparameter tuning

Search settings such as learning rate,
weight decay or LoRA rank.

HyperTuning method

A trained hypernetwork generates
adaptation parameters from task demonstrations.

Separate choosing settings from generating model parameters.

Course teaching example. Sources and assumptions in speaker notes.

HyperTuning: Adapting LLMs without Backpropagation

Authors: Jason Phang, Yi Mao, Pengcheng He, Weizhu Chen
Conference: ICML 2023

HyperTuning: Toward Adapting Large Language Models without Back-propagation

Jason Phang [1 2] Yi Mao [3] Pengcheng He [3] Weizhu Chen [3]

Abstract

Fine-tuning large language models for different tasks can be costly and inefficient, and even methods that reduce the number of tuned parameters still require full gradient-based optimization. We propose HyperTuning, a novel approach to model adaptation that uses a hypermodel to generate task-specific parameters for a fixed downstream model. We demonstrate a simple setup for hypertuning with HyperT5, a T5-based hypermodel that produces soft prefixes or LoRA parameters for a frozen T5 model from few-shot examples. We train HyperT5 in two stages: first, hyperpretraining with a modified conditional language modeling objective that trains a hypermodel to generate parameters; second, multi-task fine-tuning (MTF) on a large number of diverse language tasks. We evaluate HyperT5 on P3, MetaICL and Super-NaturalInstructions datasets, and show that it can effectively generate parameters for unseen tasks. Moreover, we show that using hypermodel-generated parameters as initializations for further parameter-efficient fine-tuning improves performance. HyperTuning can thus be a flexible and efficient way to leverage large language models for diverse downstream applications.

1. Introduction

While language models (LMs) have achieved remarkable capabilities with increasing model size (Brown et al., 2020), fine-tuning them on specific downstream tasks introduces significant engineering challenges and computational costs. Although large models can perform zero-shot, instruction-prompted, and few-shot learning (Sanh et al., 2022; Wei et al., 2022), they are usually outperformed by fully fine-tuned models when sufficient training data is available.

To reduce the computational and memory overhead of fine-tuning LMs, parameter-efficient fine-tuning (PEFT) methods have been proposed, such as adapters (Houlsby et al., 2019), prefix tuning (Li & Liang, 2021), and prompt tuning (Lester et al., 2021). These methods update only a small subset of (possibly new) parameters of the LM, and have achieved competitive performance with full fine-tuning (Ding et al., 2022). However, PEFT methods still require full back-propagation through the LM during training, which is computationally expensive and memory intensive. Given that (1) only a small number of parameters need to be updated to adapt an LM to a given task, and (2) very large LMs have demonstrated strong in-context learning capabilities on a forward pass, we hypothesize that it is possible to train a separate model to perform the optimization or adaptation procedure entirely using only a forward pass.

To avoid the costly computation of back-propagating through the LM to produce parameter updates, especially for thousands or millions of iterations during training, we propose a new paradigm of **hypertuning**: using a *hypermodel* to adapt a *downstream* LM to a desired application. As a concrete proof of concept, we explore a simple setup where hypermodels take as input a set of few-shot examples from a given task, and output the PEFT parameters corresponding to that task in a single forward pass.

To demonstrate the feasibility of this approach, we train *HyperT5*: a set of T5-based hypermodels that output soft prefixes (Li & Liang, 2021) or LoRA parameters (Hu et al., 2022), to be incorporated into a frozen downstream T5 LM. To train HyperT5, we introduce a two-stage procedure for training hypermodels: *hyperpretraining*, where we adapt a pretrained LM to generate PEFT parameters via a modified language modeling objective, followed by *multi-task fine-tuning* (MTF) the hypermodel. HyperT5 models can take few-shot examples from unseen tasks and generate the corresponding PEFT parameters, allowing us to adapt a downstream LM without back-propagation. We show that hypertuning is effective on P3, Super-NaturalInstructions and MetaICL datasets. Furthermore, we show that when the hypermodel-generated parameters are used as initializations for further parameter-efficient fine-tuning, they achieve

[*]Equal contribution [1]Center for Data Science, New York University, NY, USA [2]EleutherAI [3]Microsoft Azure AI, WA, USA. Correspondence to: Jason phang <jasonphang@nyu.edu>.

Proceedings of the 40^{th} International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 202, 2023. Copyright 2023 by the author(s).

USC Viterbi School of Engineering

Motivation & Problem Statement

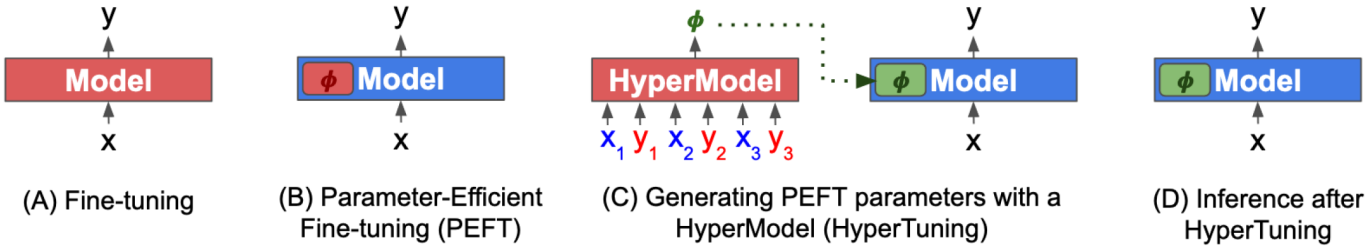


Figure 1: Overview of HyperTuning. (A) Fine-tuning, where all model parameters are updated (red). (B) Parameter-efficient fine-tuning (PEFT), where all model parameters are frozen (blue) and only a small number of parameters, ϕ, are updated. (C) HyperTuning, where a hypermodel is used to generate parameters ϕ for a frozen downstream model. For instance, a hypermodel may take a set of few-shot examples to determine what ϕ to generate. Only the hypermodel's parameters are updated during training. (D) At inference time, the parameters ϕ only need to be generated once, and thereafter only need to store ϕ, with no need to retain the few-shot examples.

- **Fine-tuning LLMs is expensive** and memory-intensive.
- **Parameter-efficient fine-tuning (PEFT)** reduces parameter updates but still requires full backpropagation.
- **HyperTuning** proposes using a hypermodel to generate task-specific parameters in a single forward pass.

Key Idea: HyperTuning Framework

- Uses a hypermodel to generate PEFT parameters for a frozen LLM.
- New-task generation uses a forward pass.
- Demonstrations generate parameters; they need not all stay in later prompts.

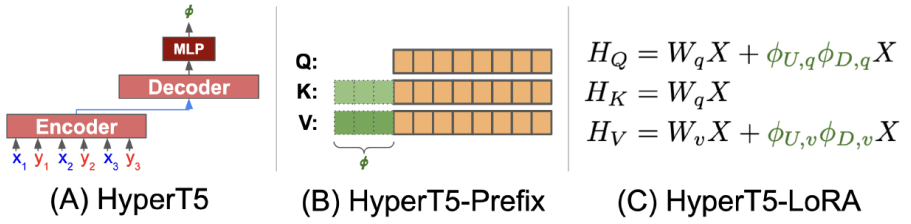


Figure 2: Overview of HyperT5. (A) HyperT5 takes as input few-shot examples and outputs PEFT parameters ϕ. The model is initialized from an LM-adapted T5. (B) In HyperT5-Prefix, ϕ are key and value prefixes for every attention layer. (C) In HyperT5-LoRA, ϕ are additive low-rank modifications to the query and value linear maps.

HyperTuning: Generate an Adaptation

A separately trained hypermodel reads a few labeled task examples.

The target model uses the generated task parameters.

Phang et al., HyperTuning (ICML 2023).

The Hypermodel Must Be Trained

Separate the earlier training stage from adaptation to a new task.

Earlier training	Backpropagation trains the hypermodel using pretraining and many tasks.
New-task adaptation	A forward pass generates parameters from task demonstrations.
Optional further training	Generated parameters can initialize later gradient-based adaptation.

The no-backprop claim applies to the generation step.

Phang et al., HyperTuning (ICML 2023).

HyperTuning and the Training Workflow

Compare where the task-specific adaptation comes from.

Ordinary fine-tuning

Optimize target parameters using examples, loss and gradients.

HyperTuning

Run a trained hypermodel on examples to generate target parameters.

Judge accuracy and cost under the chosen task and setup.

Phang et al., HyperTuning (ICML 2023).

The Flower Classifier: A Complete Workflow

Return to the original goal of classifying new photos.

Establish a baseline	Fix the split, metric, preprocessing and starting model.
Diagnose and change	Use learning curves to select a controlled intervention.
Select and evaluate	Choose by validation, then evaluate the final model.

Explain which problem each training choice addresses.

Course teaching example. Sources and assumptions in speaker notes.

Optional Research Examples

The training and adaptation workflow is complete.

LLM behavior	A research example of how adaptation changes behavior.
TOD systems	A research example of efficient outlier-detection systems.
AutoML	A later lecture extends our manual experiment loop.

These sections can be taught separately from the core workflow.

Course teaching example. Sources and assumptions in speaker notes.

LLMs for
Human Behavior Simulation

Our work: A Large-Scale Simulation on LLMs for Decision-Making in Political Science

Outlines

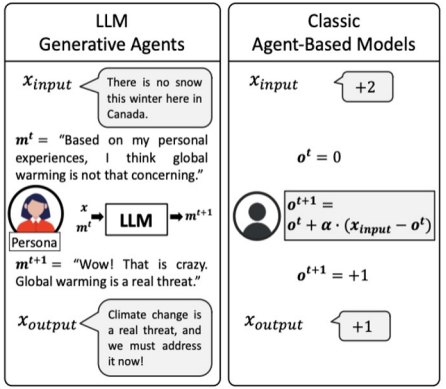
- **General Ideas**
- **Our Approach: LLMs' Simulation for Decision-Making in Political Science**
- **Pipelines - Version I & II & III**
- **Results evaluation (accurate simulation)**
    1. Public Benchmark ANES
    2. On Synthetic Data (2020,2024)
- **Results evaluation (Beyond Accuracy)**
    1. Systematic Bias
    2. Reinforcement of Demographic Stereotypes
    3. Hyper-accuracy Distortion
- **Discussion & New directions**

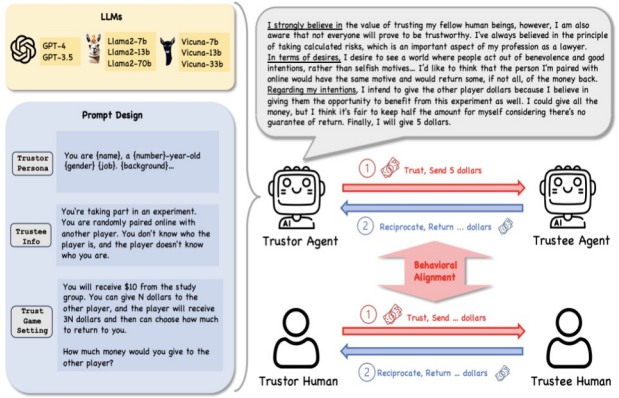
General Ideas

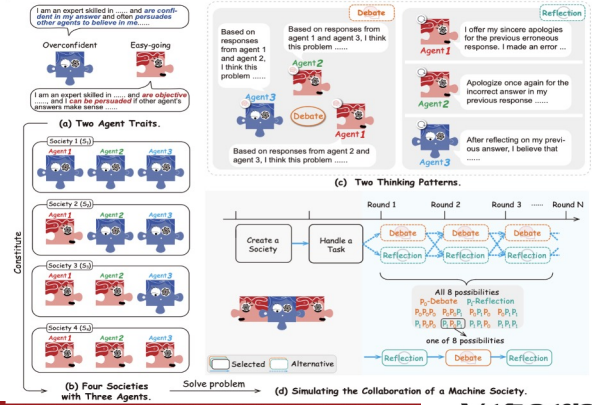
- **Give LLMs a specific persona –Let LLMs simulate the corresponding behaviours**

- E.g relevant studies:

1. Public Opinion - confirmation bias (*Chuang et al., 2024)*
2. Economic - utility theory (*Xie et al., 2024)*
3. Social Psychology - collaboration (Zhang et al., 2024)
4. … …



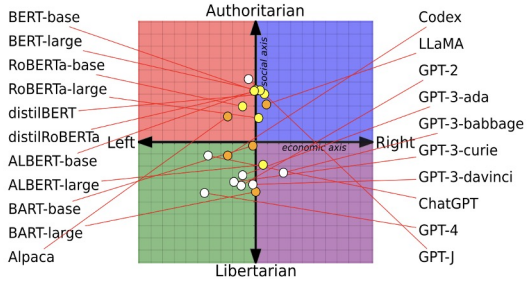




Our Approach: LLMs' Simulation for Decision-Making in Political Science

- **Inspirations:** Given specific persona under political science – **If LLMs can Vote like Real Human?**
  - Why did we choose the **voting simulation task**?
    1. A Complex Decision-making Task (demographic, ideological, spatial, and external factors…)
    2. Easiness in evaluation (state- and county-level election results)

- **Difficulties:**
  1. LLMs' Bias from Pretrain Corpus (Feng et al, 2023)
  2. Lack of Voter-level Data / shifting political context …



Research Questions

- RQ1: How can LLMs be used to simulate human decision-making in political science?
- RQ2: How do different LLMs perform, and how robust are their election simulations?
- RQ3: What limitations arise when using LLMs to model political decision-making?

USC Viterbi School of Engineering

Our Approach: LLMs' Simulation for Decision-Making in Political Science

Datasets

- Public ANES (real human samples)
- 1:1 Synthetic data (nati

Time range

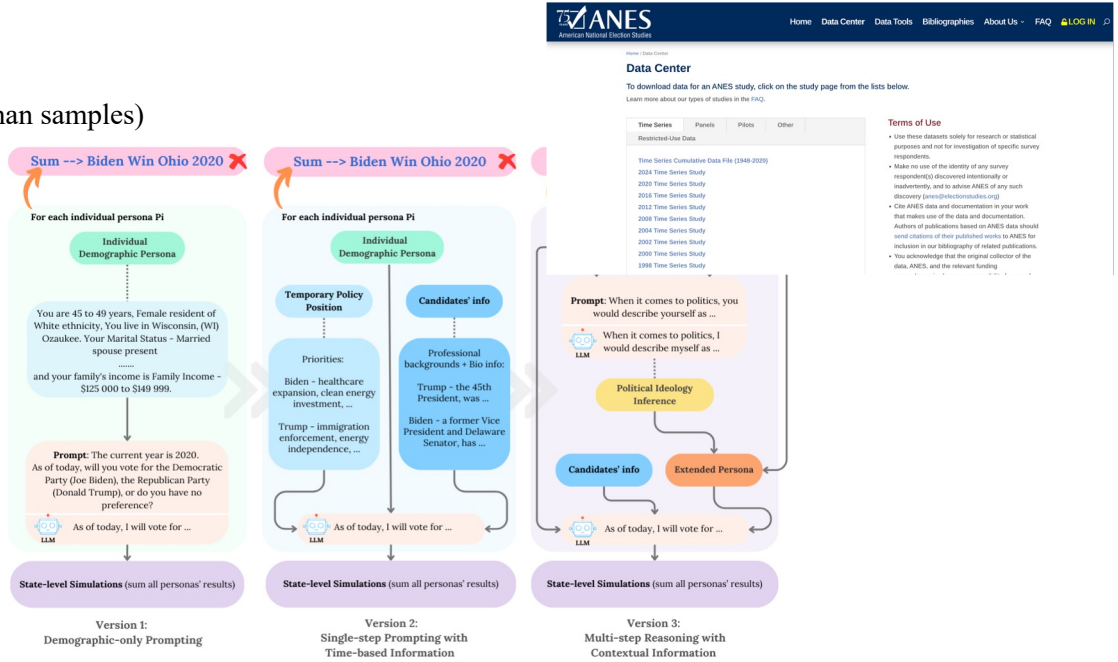
- Evaluation Within preti
- Test Beyond pretrain c

Pipelines

- V1: direct prompt on e
- V2: direct prompt on e
- **V3: Multi-Step Reaso**

LLMs

- GPT-4o
- Llama3.1-70B
- Qwen-72B
- DeepSeek-V3



USC Viterbi School of Engineering

Pipelines - Version I & II

V1: Direct Prompt on Each Persona

- Given LLMs a specific persona & let it vote in one single prompt (Argyle et al., 2023)

V2: w/ Time-Sensitive Information

- Direct Prompt
- + **timely info** from *Ballotpedia*:
  - campaign agendas
  - key policy positions
  - candidates' biographies

Task: You are persona [age, gender, ethnicity, marital status, household size, presence of children, education level, occupation, individual income, family income, and place of residence.] The current year is [year]. /v2 + **[additional timely info]**

Please answer the following question as if you were the resident:
As of today, will you vote for the Democratic Party (Joe Biden), the Republican Party (Donald Trump), or do you have no preference?

Options: Democratic, Republican, No Preference

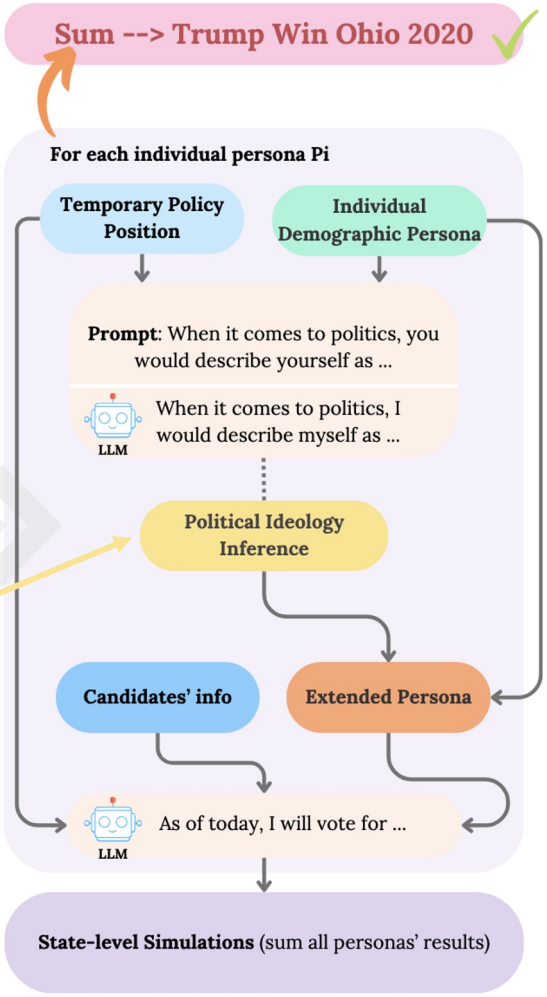
Pipelines - Version III Multi-Step Reasoning with Ideology Inference

- Theoretical Backgrounds:
  - A significant increase of polarization
  - Political Ideology <-> Party Identification
    (Levendusky, 2009; Abramowitz and Saunders, 2008; Bornschier et al.,2021)

- Design: Break the thinking chain into multiple steps
  - **intermediate step**: generating political ideology

- Voting Behavior Simulation- **Significant Improvement**!

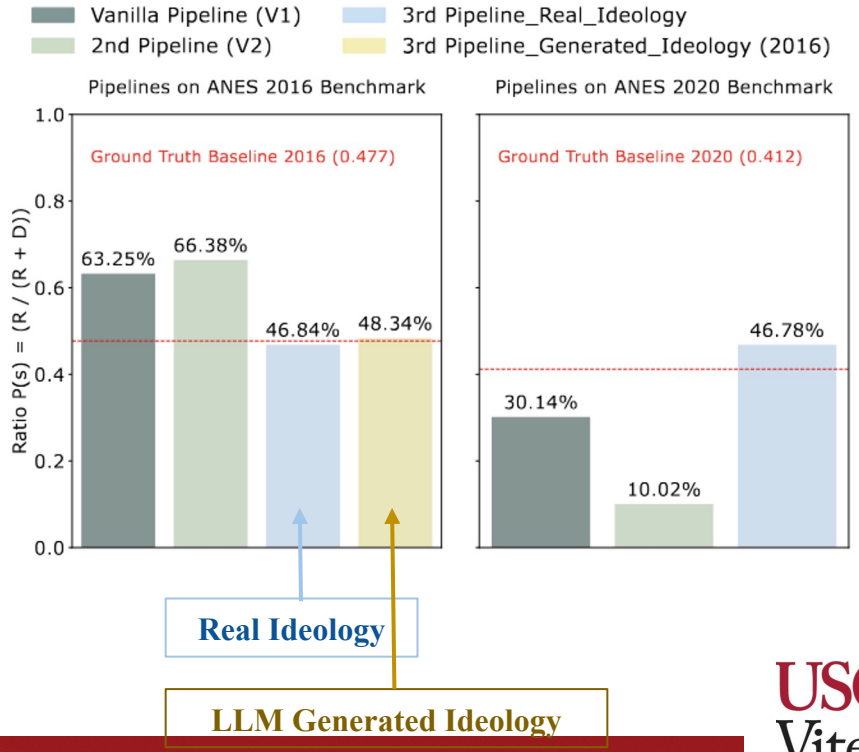
  (both public benchmark validation and state-level simulations.)



Results: Evaluation on Public Benchmark

ANES (real human samples)

- 4,270 / 8,280 samples
- Demographic info, political ideology, voting records from human respondents
- LLM on Generated Ideology performs almost same as on Real Ideology
- (2020's variables is partially missing)



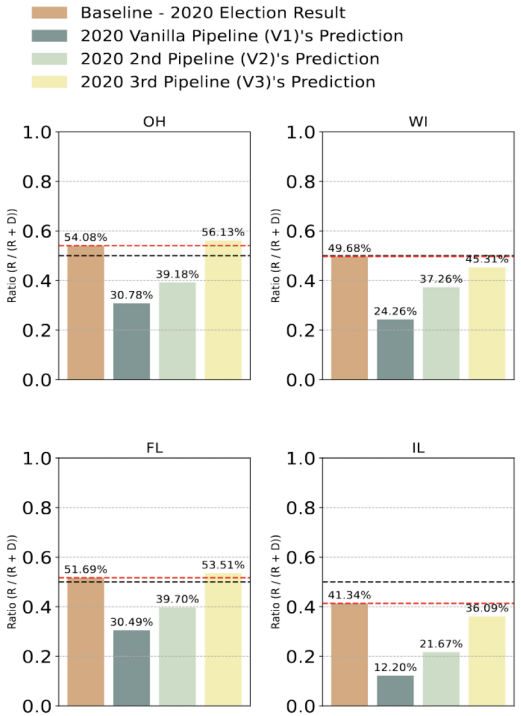
Results: Evaluation On Synthetic Data
(2020 U.S. Election Simulation)

- 1/1000 of total population ~ 330,000 personas
- V3 Simulate in all ‘red’ and ‘blue’ states accurately
- V3 correctly predicted in 9 out of 11 swing and tipping point states

  with only minor deviations in North Carolina (NC) and Arizona (AZ)

Aggregate Results

GPT-4o Simulation of 2020 U.S. Election			
Metric	**V1 (%)**	**V2 (%)**	**V3 (%)**
WAE	22.78	14.97	**5.24**
WMSE	5.46	2.34	**0.37**
Bias Metric (BM)	-22.78	-14.97	**0.34**



State-level Results (subset)

- WAE & WMSE: smaller the number, more accurate the simulation

USC Viterbi School of Engineering

Results: Evaluation On Synthetic Data
(2024 U.S. Election Simulation)

- 2024 evaluation period. Model-specific training cutoffs and data leakage need separate checks.
- Cross pipelines & Cross models evaluation
- GPT-4o V3 pipeline outperforms than others (V1, V2 under same LLM)

GPT-4o Simulation of 2020 U.S. Election			
Metric	**V1 (%)**	**V2 (%)**	**V3 (%)**
WAE	22.78	14.97	**5.24**
WMSE	5.46	2.34	**0.37**
Bias Metric (BM)	-22.78	-14.97	**0.34**

Multi-LLM Simulation of 2024 U.S. Election									
Metric	**GPT-4o V1**	**GPT-4o V2**	**GPT-4o V3**	**LLaMA3-70B V1**	**LLaMA3-70B V2**	**LLaMA3-70B V3**	**Qwen-72B V3**	**DeepSeek-V3 V3**	**Zhang et al., 2024**
WAE	21.35	25.96	**3.49**	19.97	10.23	**6.88**	10.66	14.57	4.70
WMSE	4.83	6.89	**0.22**	4.15	1.42	**0.68**	1.47	2.43	0.30
BM	-21.35	-25.96	-2.95	-19.97	-10.23	-5.44	-9.47	-14.57	1.26

RQ3: ***What limitations arise when using LLMs to model political decision-making?***

Discussion : ***How to fulfilling the gap***

Beyond Accuracy: Some Interesting Findings

- **Systematic Bias**
  - Bias Metrics (BM): A metric which quantifies systematic bias by measuring whether simulated personas consistently favor one party over the other.
  - BM > 0: Republican-leaning, BM < 0: Democratic-leaning.

GPT-4o Simulation of 2020 U.S. Election			
Metric	**V1 (%)**	**V2 (%)**	**V3 (%)**
WAE	22.78	14.97	**5.24**
WMSE	5.46	2.34	**0.37**
Bias Metric (BM)	-22.78	-14.97	**0.34**

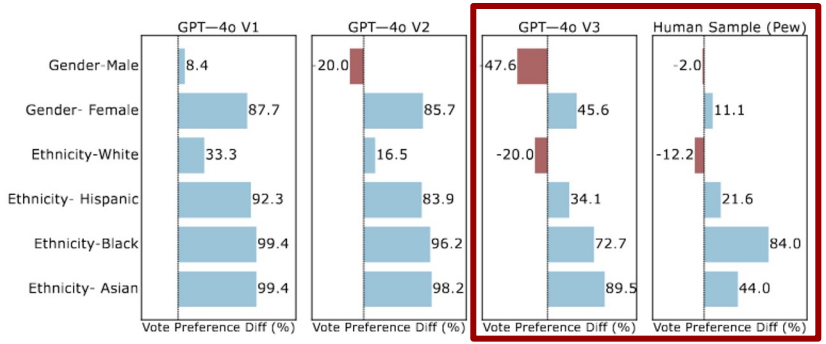
4. Bias Metric (BM)

$$\mathrm{BM} = \frac{\sum_{s \in S} E(s) \cdot (P(s) - R(s))}{\sum_{s \in S} E(s)} \qquad \text{(A6)}$$

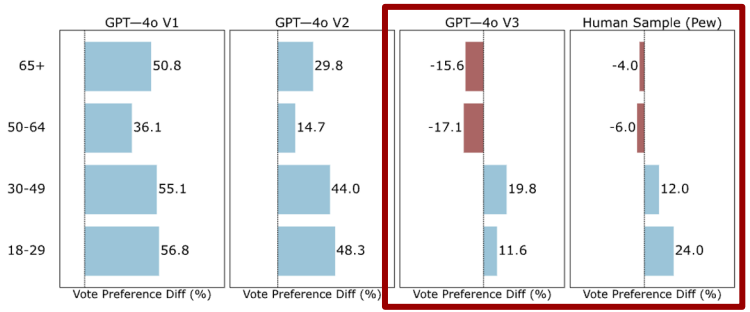
Multi-LLM Simulation of 2024 U.S. Election									
Metric	**GPT-4o V1**	**GPT-4o V2**	**GPT-4o V3**	**LLaMA3-70B V1**	**LLaMA3-70B V2**	**LLaMA3-70B V3**	**Qwen-72B V3**	**DeepSeek-V3 V3**	**Zhang et al., 2024**
WAE	21.35	25.96	**3.49**	19.97	10.23	**6.88**	10.66	14.57	4.70
WMSE	4.83	6.89	**0.22**	4.15	1.42	**0.68**	1.47	2.43	0.30
BM	-21.35	-25.96	-2.95	-19.97	-10.23	-5.44	-9.47	-14.57	1.26

Beyond Accuracy: Some Interesting Findings

- **Stereotypes: Over-amplification of Demographic Voting Patterns.**



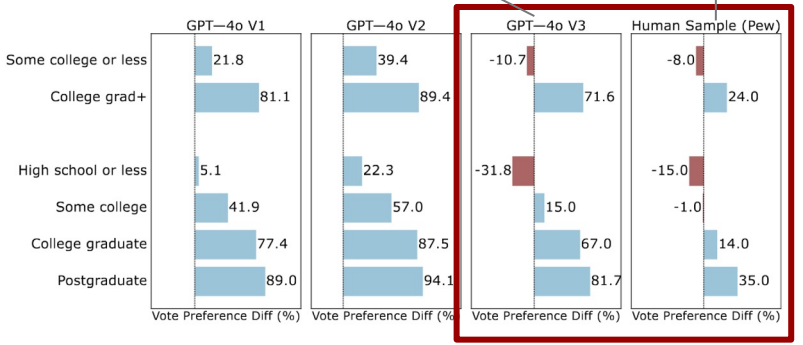
Gender & Ethnicity



Age

Real humans' voting pattern (pew 2020)

LLM generated voting pattern



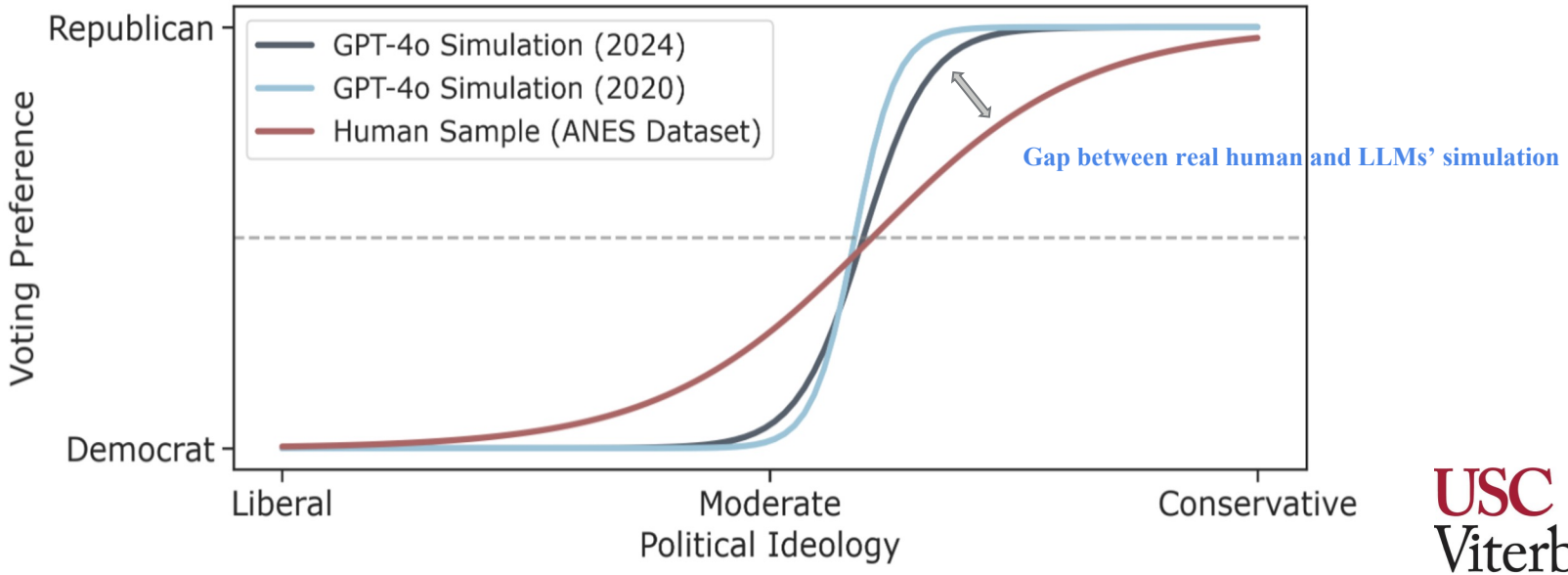
Education

"Large language models that replace human participants can harmfully misportray and flatten identity groups" (wang et al., 2025)

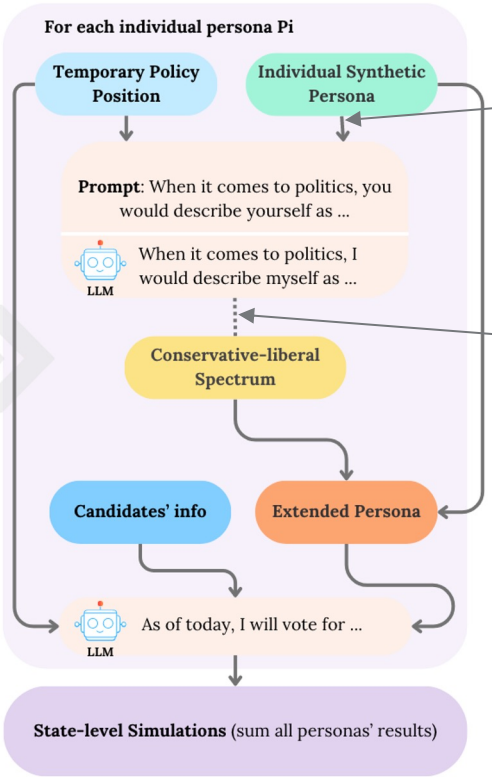
USC Viterbi School of Engineering

Beyond Accuracy: Some Interesting Findings

- **Hyper-accuracy Distortion: Overestimation of ideological influence.**
  - logistic regression: predict voting outcome over Political Ideology
  - Flatter real human regression: human decision making process is more complex & dynamic
  - LLM oversimplified this decision-making process – like a shortcut



How to reduce political bias of LLM (agents)?



Social Identity Theory
"As [a young, suburban moderate voter] in [Pennsylvania], you are part of a community that includes both Republican and Democratic supporters. How do social interactions shape your decision?"

Information Processing Theory
"Recently, the [candidate] from [party] have initiated [policy issue]. How does this affect your stance?"

Goal: Improve realism by aligning synthetic personas with real-world voter concerns.

Experiment with the new reasoning framework on sample personas from swing states.

USC Viterbi School of Engineering

Building ML *Systems* for Scalable Learning

TOD: A Tensor-based, Multi-GPU OD System

VLDB' 23 [Zhao, Chen, Jia]

USC Viterbi School of Engineering

Efficiency and Data Dependency Bottleneck of OD

- **Detection algorithms** are with high time/space complexity.

Algorithm	Time Complexity	Space Complexity
kNN	$O(n^2)$	$O(n^2)$
COF	$O(n^3)$	$O(n^2)$
LOF	$O(n^2)$	$O(n^2)$
KDE	$O(n^3)$	$O(n^2)$
LOCI	$O(n^2)$	$O(n^2)$
OCSVM	$O(n^3)$	$O(n^2)$

Efficiency: $O(n^2)$ **or higher complexity**

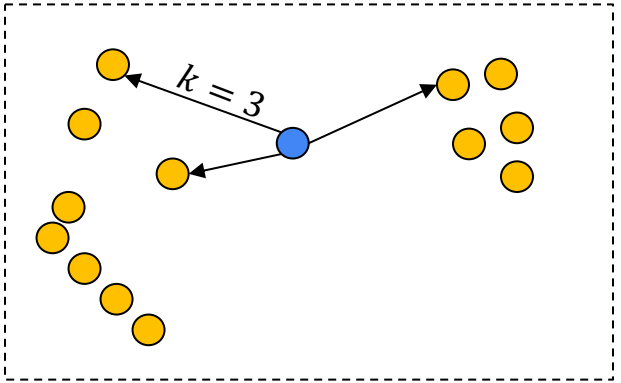
Efficiency and Data Dependency Bottleneck of OD

- **Detection algorithms** are with high time/space complexity.

Algorithm	Time Complexity	Space Complexity
kNN	$O(n^2)$	$O(n^2)$
COF	$O(n^3)$	$O(n^2)$
LOF	$O(n^2)$	$O(n^2)$
KDE	$O(n^3)$	$O(n^2)$
LOCI	$O(n^2)$	$O(n^2)$
OCSVM	$O(n^3)$	$O(n^2)$

kNN: k-th nearest neighbor distance as the outlier score.

Dependency: Need ***all*** samples for pairwise dist.

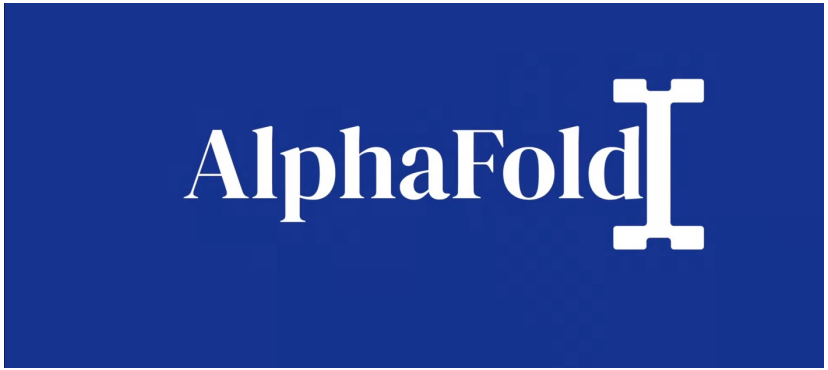


TOD: 1st Tensor-based GPU Outlier Detection System

- **Opportunity with GPUs**:

1. **Success in deep neural networks** with advancement in hardware

Visual object recognition (2012)

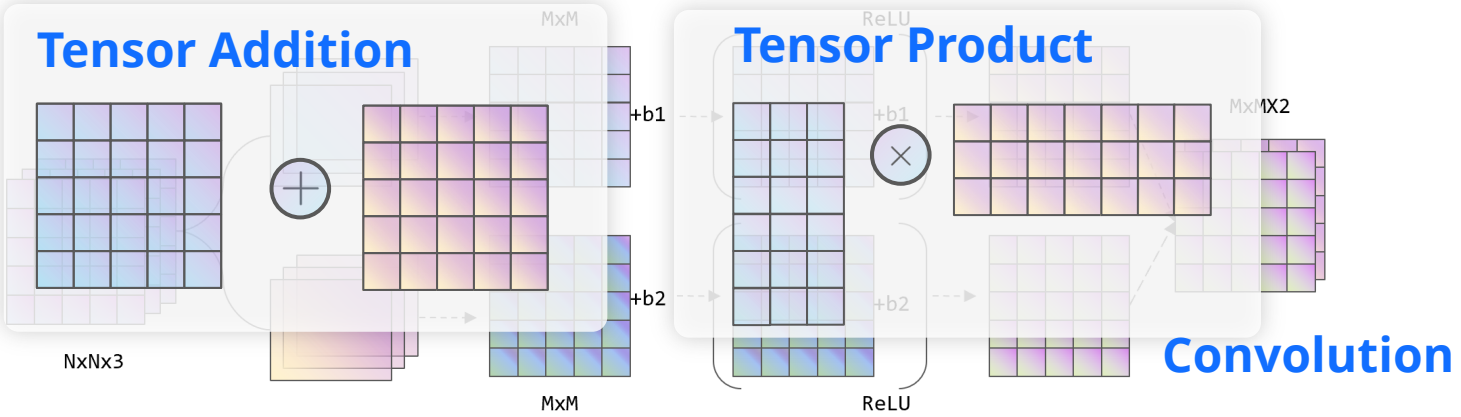


Protein structure prediction (2021)

TOD: 1st Tensor-based GPU Outlier Detection System

- **Opportunity with GPUs**:

1. **Success in deep neural networks** with advancement in hardware
    - **Fast computation** with **tensor operations/algebra**



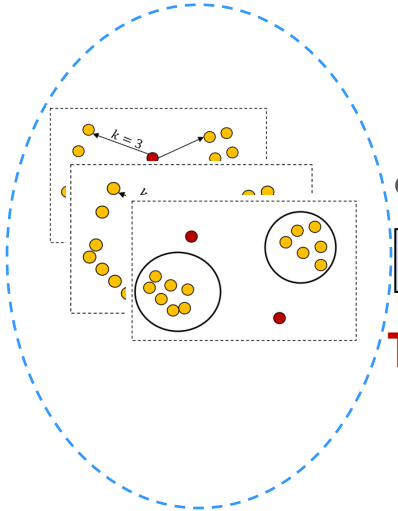
TOD: 1st Tensor-based GPU Outlier Detection System

- **Opportunity with GPUs**:
1. **Success in deep neural networks** with advancement in hardware
2. **Maturity in learning systems**

Deep optimization & **Neural nets on GPUs**

TOD: 1st Tensor-based GPU Outlier Detection System

Data dependency ☹ →
✖batches of tensor algebra



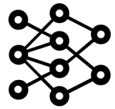
Diverse OD algorithms

● **Transpiler**

TOD

Translator

☺ Neural nets: tensor algebra in batches



mxnet

DL4J

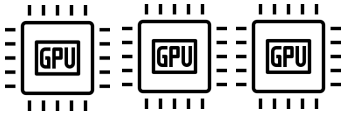
ONNX

PyTorch

TensorFlow

PaddlePaddle

GPU/TPU advancement



TOD: 1st Tensor-based GPU Outlier Detection System

●**Limitations of existing OD GPU systems**:

●(1) implement ***one*** (family of) OD algorithm(s) (2) without deep optimization

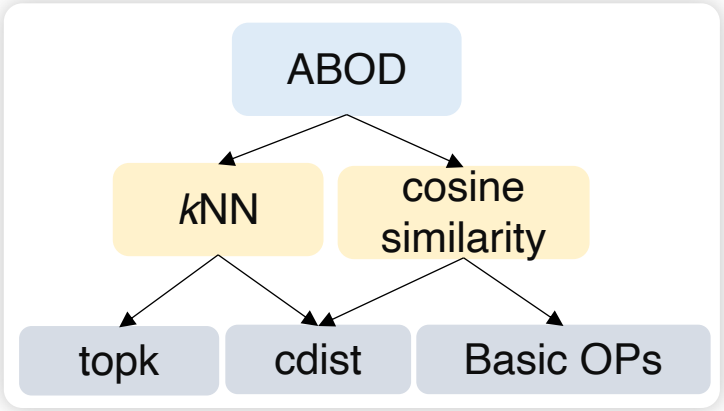
●**TOD (ours)**:

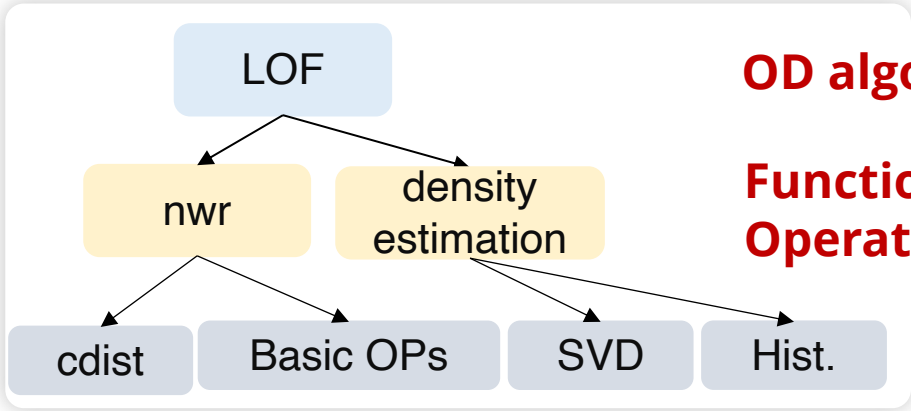
●(1) support ***diverse*** OD algorithms via **tensor transpilation** →

●(2) **leverage** GPUs and ***optimized*** GPU systems

Challenge 1: Representing OD as Tensor Operations

- **Decompose** an **OD algorithm** into **tensor operations:**





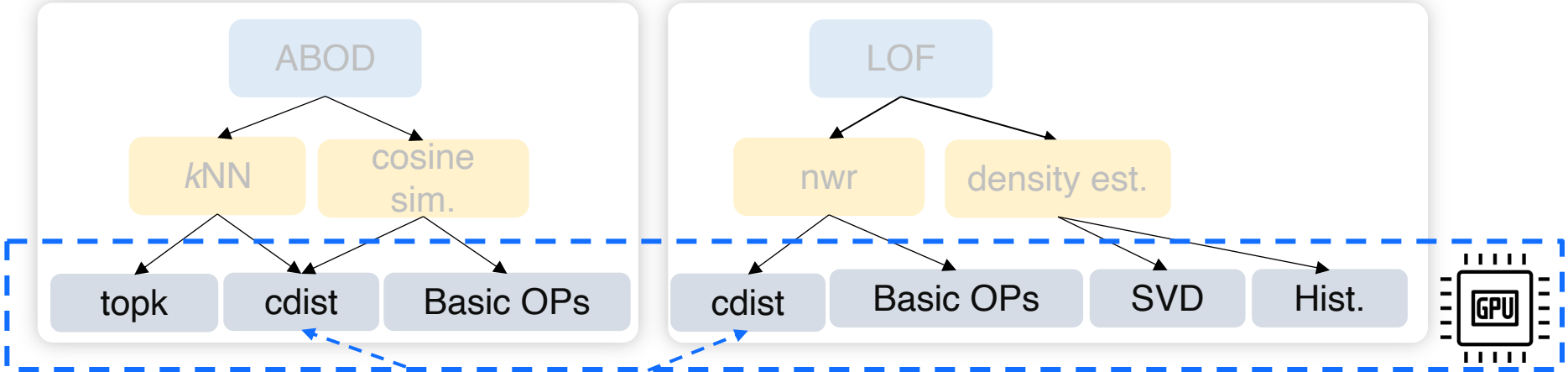
OD algorithms

Functional Operators

Basic tensor operators

Challenge 1: Representing OD as Tensor Operations

- **Decompose** an **OD algorithm** into **tensor operations:**

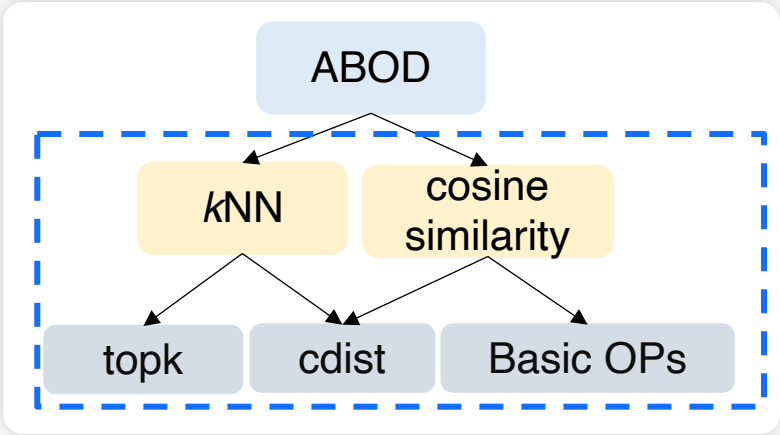


Note 1: Decomposable as basic tensor operators for GPUs !

Note 2: Shared tensor operators across algorithms!

Solution 1: Tensor Transpilation Interface

- **[Ours] decompose** *complex* and *diverse* OD algorithms into **a small set of**:
- **shared** *basic tensor operators*



USC Viterbi School of Engineering

Solution 1: Tensor Transpilation Interface

Outlier Detection Algorithms

*k*NNOD | ABOD | COF | LOF | LOCI | HBOS | SOD | COPOD | ECOD

FB | LSCP | MetaOD | SUOD | LODA | PCA | ... | ... | ...

Basic Functional Operators

*k*NN | shared neighbors | neighbors within Range | cosine similarity | dimension reduction | density estimation

Basic Tensor Operators

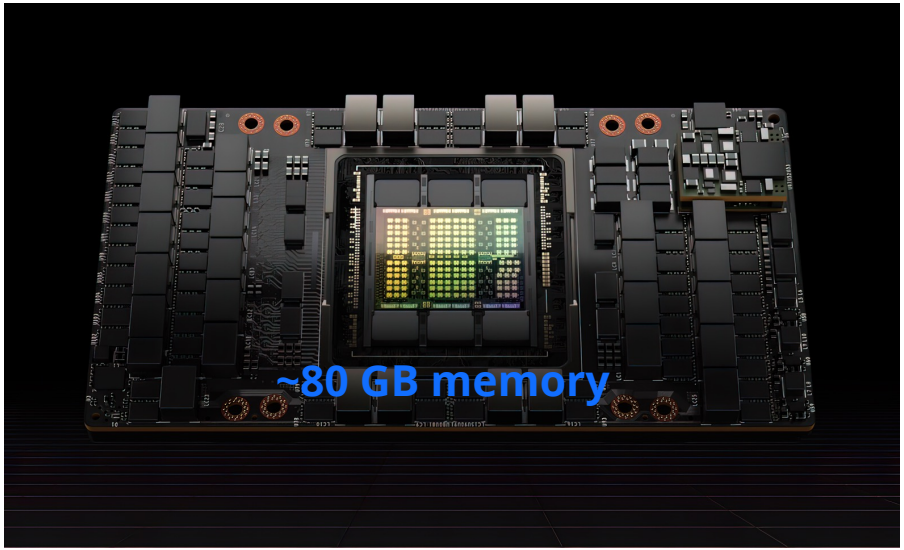
cdist | topk | intersect | SVD | feature sampler | histogram | aggregation

Advantage 1: lower implementation and optimization effort

Advantage 2: extends to new fields and algorithms

Challenge 2: Operator Optimization for GPU Memory

- **Limitation of GPU memory**:
- Example: the NVIDIA A100 shown here has 80 GB of device memory.
- OD operators are costly:
  - **~160GB**: distance mat for 200k samples in 32-bit



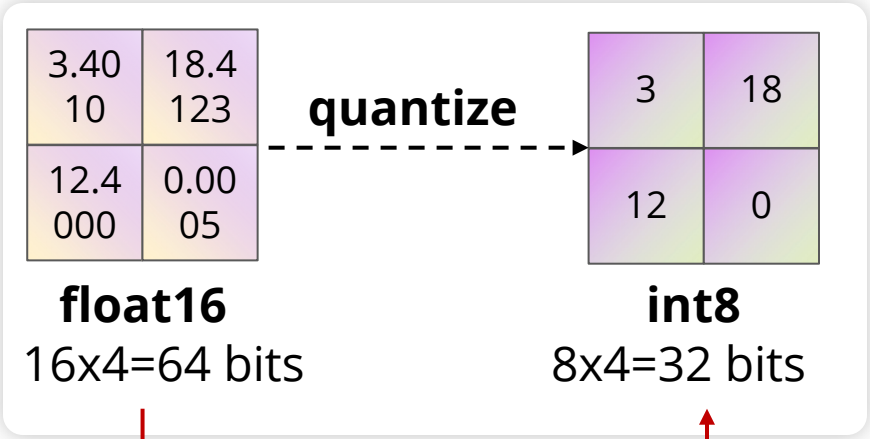
Nvidia A100 GPU

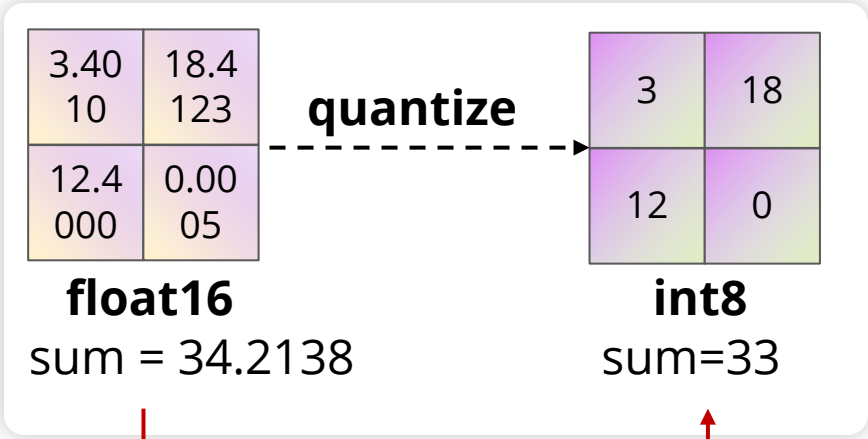
Solution 2: Operator Optimization for GPU Memory

- **Optimizations on basic operators**:

1. **Provable quantization: use lower precision, verify the result, recompute if needed.**

Operator Optimization: Provable Quantization





Save memory but cause result difference

Operator Optimization: Provable Quantization

- **Our proposal:**
- For select operators, can we **quantize them** *without* **accuracy change?**

Operator Optimization: Provable Quantization

- **Examples of accuracy preservation quantization:**
  - $\boldsymbol{a} = [1.011, 1.011]$
  - $sum(\boldsymbol{a}) > 2$? Correct answer: Yes

- ✓ **Correct result**
  - **Quantization into 2 decimals**, $sum(\boldsymbol{a}) = sum([1.01, 1.01]) = 2.02 > 2$

The key is to quantify **the rounding error** in **lower precisions**!

- ✗ **Inaccurate result (due to *the large rounding error*)**
  - **Quantization into 1 decimal**, $sum(\boldsymbol{a}) = sum([1.0, 1.0]) = 2$

$(1+\epsilon)$–property (Lee et al., 2018) for Error Bound

- **Quantify floating-point operation error at precision α** (e.g., 64-bit) :

- x, y are floating-point numbers
- Operator $*$ is one of the $\{+, -, \times, /\}$

- **With proper conditions**:

- $x \circledast y = (x * y)(1+\delta)$, where $|\delta| \leq \epsilon$,

Error in operation between x and y Bound by ϵ

- where ϵ **is a constant** that solely **depends on the precision level** α

Operator Optimization: Provable Quantization

- **The rounding error bound for selected operators:**
- $x \circledast y = (x * y)(1 + \epsilon)$; replace δ by ϵ for the max error

Measured by

max error in low-precision

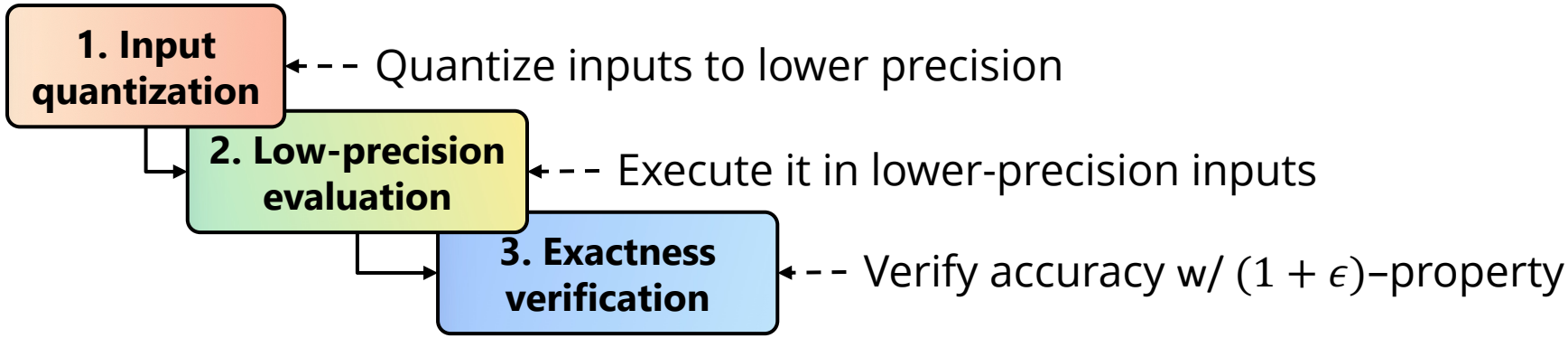
Affected?

the output of the operator

No

Quantization accuracy is provable ☺

Operator Optimization: Provable Quantization



- **Exactness verification:**
  - fix if needed: recalculate in higher precisions

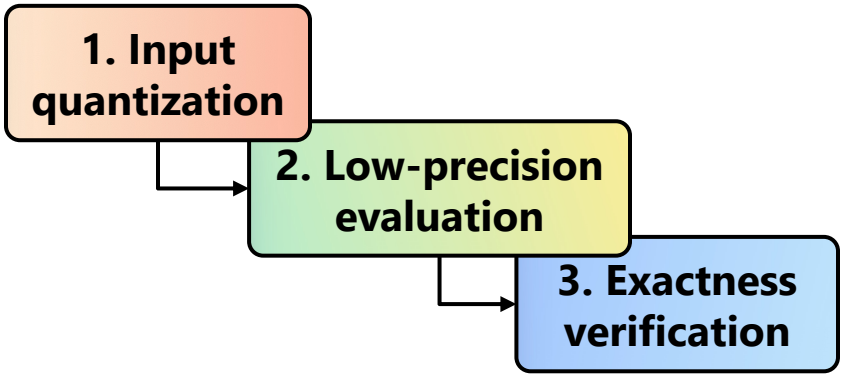
Operator Optimization: Provable Quantization

- **Why is it suited for OD operators?**
  - often output **ranks, indexes**, etc. (not floating points)
  - are mostly calculation-based – **no *iterative* error propagation**

Solution 2: Operator Optimization for GPU Memory

- Optimizations on basic operators:

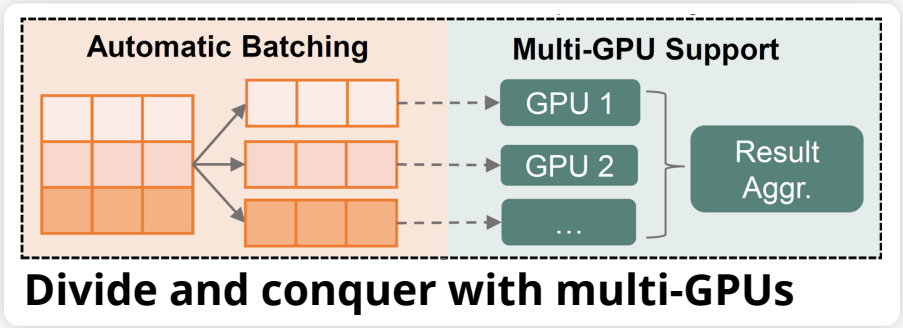
1. Provable quantization: use lower precision, verify the result, recompute if needed.



Solution 2: Operator Optimization for GPU Memory

- **Optimizations on basic operators**:

1. Provable quantization: lower precision with verification and recomputation.
2. **Automatic batching**: split large operations into smaller ones
3. **Multi-GPU support**: deploy small batches onto multiple GPUs



Hyperparameter Tuning

Automated ML

AutoML Automates Repeated Modeling Decisions

Suppose our manual experiment already records settings and validation scores.

The search still needs a trustworthy evaluation protocol.

Course teaching example. Sources and assumptions in speaker notes.

The Same Flower Task, Many Possible Settings

We could try different learning rates, regularizers and model choices.

AutoML organizes this repeated search.

Course teaching example. Sources and assumptions in speaker notes.

Classical ML Pipelines: Algorithms

The key elements in ML pipelines include (i) **algorithms** and (ii) hyperparameters.

Algorithms:

1. *Data preprocessing algorithms*, e.g., dimensionality reduction like PCA
2. *Classical ML algorithms*, e.g., kNN, logistic regression, random forests, etc.
3. *Modern ML algorithms* (a.k.a, deep learning), e.g., RNN, LSTM, and CNN

Classical ML Pipelines: Hyperparameters

Once an algorithm is chosen, we need to decide its associated **HPs**:

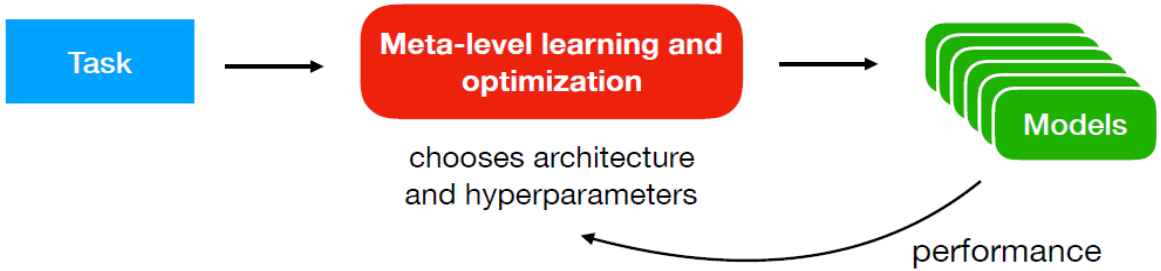
1. Data preprocessing algorithm, e.g., dimensionality reduction like PCA (**how many principal components**? i.e., what is the target dimension?)
2. Classical ML algorithms, e.g., kNN (**# nearest neighbors**), random forests (**# base trees**), etc.
3. Modern ML algorithms, like deep learning algorithms (e.g., **learning rate** and **dropout rate**)

We will revisit these definitions in a few slides…

How will you choose algorithm/hyperparameters for a given task?

Automated ML (AutoML) & Key components

The key of AutoML is to automate the selection process of algorithms and hyperparameters.



Unlike classical ML models, deep learning may have more flexible architectures, so people typically categorize AutoML into ***three*** buckets:

1. **HP tuning** and/or **algorithm selection: for a single dataset (covered in this lecture)**
2. Meta Learning (learning to learn): generalize to many different datasets
3. Neural architecture search (may be considered as a subarea/combination of 1&2)

What Can a Modeling System Automate?

An automated search still follows a task and evaluation protocol.

Preprocessing choices	Compare suitable transformations within the pipeline.
Model and training choices	Search architectures or training hyperparameters.
Resource allocation	Decide which trials deserve more compute.

People still define the task, data boundaries and success metric.

Course teaching example. Sources and assumptions in speaker notes.

Warning: Should We Always Use AutoML?

AutoML is not necessarily superior to human selection for many reasons:

- Human selection (regarding algorithm and hyperparameter) is indeed based on prior knowledge and historical information of the problem.
- It is not always possible to perform AutoML due to software and hardware limitations, especially the cost. AutoML is often costly to run.

Alternatives to AutoML:

- One may use stable & robust methods like ***ensemble learning*** to combine multiple models together. For instance, one may not know how to select the ML algorithms and hyperparameters. It is possible to randomly build multiple models with different hyperparameters and then combine them, e.g., taking the average.
    - ***Drawbacks***: building many diversified models may be slow for prediction.

Preliminaries: Algorithm & Hyperparameter Revisit

ML Algorithms, e.g., decision tree, kNN, etc.

Hyperparameters are specific settings of a selected **algorithm**, such as the tree depth of decision tree or the number of nearest neighbors for kNN.

Models are the **combination** of **algorithms** and **hyperparameters**:

- {Decision tree, tree_depth=5}
- {Decision tree, tree_depth=10}
- {kNN, n_neighbors=5}
- {kNN, n_neighbors=10}
- …

Preliminaries: Configuration

AutoML helps us to find an "**optimal**" model in large model space.

- **Situation 1 (reduced to hyperparameter search): Same algorithm, searching for best hyperparameters**

Models are referred as the **combination** of **algorithms** and **hyperparameters**:

- {Decision tree, tree_depth=3}
- {Decision tree, tree_depth=5}
- {Decision tree, tree_depth=10}

Preliminaries: Configuration

AutoML helps us to find an "**optimal**" model in large model space.

- **Situation 2 (reduced to algorithm search): Different algorithms with their default hyperparameters, searching for the best model (merely algorithm)**

Models are referred as the **combination** of **algorithms** and **hyperparameters**:

- {Decision tree, default hyperparameters}
- {kNN, default hyperparameters}
- {Logistic regression, default hyperparameters}
- …

Preliminaries: Configuration

AutoML helps us to find an "**optimal**" model in large model space.

- **Situation 3: Different algorithms with varying hyperparameters, searching for best model+ associated algorithm(s)**

Models are referred as the **combination** of **algorithms** and **hyperparameters**:

- {Decision tree, tree_depth=5}
- {Decision tree, tree_depth=10}
- {kNN, n_neighbors=5}
- {kNN, n_neighbors=10}
- …

Algorithms and Hyperparameters

Most of the methods introduced in this lecture can be used for **model search**, although they may be put under hyperparameter search or algorithm search.

1. You may think in this way, **the selection of algorithms (kNN, Decision Tree, etc.) can also be viewed as a "hyperparameter" of the ML pipeline**.
2. Selecting an algorithm and tuning its corresponding hyperparameters uses human knowledge (priors) to fix the algorithm, where the search space is restricted.

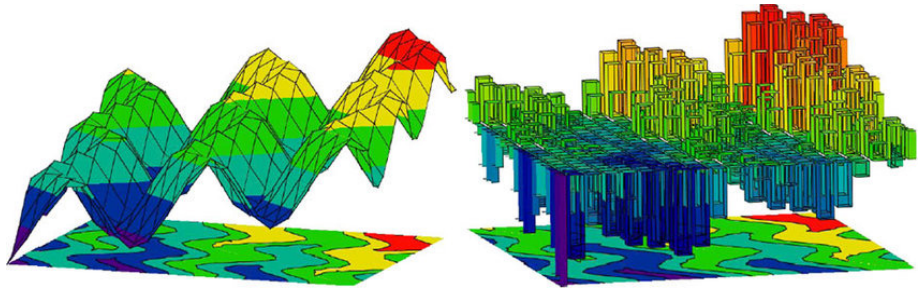
Why do we want to do hyperparameter search only?

1. The search space is smaller, so the optimization is easier
2. The diff between algorithms can be huge. If the algorithm choice is included, **the search process will be less smooth**, especially for learning-based algorithms.

Hyperparameter Search w/ a More Smooth Objective

Why do we want to do hyperparameter search only?

1. The search space is smaller, so the optimization is easier
2. The difference between algorithms can be significant. If the algorithm choice is included, **the search process will be less smooth**, especially for learning based algorithms.



Smooth(er) objective vs. nonsmooth objective

Part 1: Given a Dataset and an Algorithm, how to choose the best hyperparameter(s)?

This is the most common case we encounter in our daily work. In practice, **most real-world applications focus on one or just a (few) dataset(s)** and try to find the **hyperparameters** that maximize the prediction accuracy of a given algorithm.

1. **Random and Grid search**: if cheap evaluation is possible
2. **Bayesian optimization**: if evaluation is expensive, and the optimization is restricted

Other than these approaches, have you tried any other algorithms?

Hyperparameter Optimization

For the flower classifier, choose settings using validation performance.

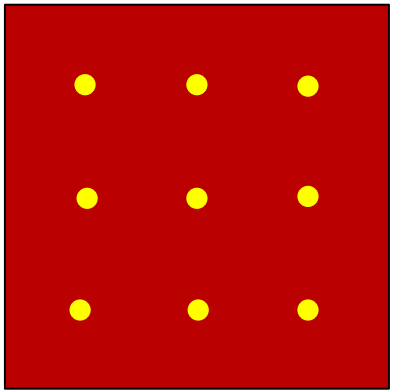
Training data	Fit each candidate model with its proposed learning rate and weight decay.
Validation data	Compare candidate settings under the same metric and budget.
Test data	Evaluate the selected procedure after tuning is complete.

The test score remains outside the search loop.

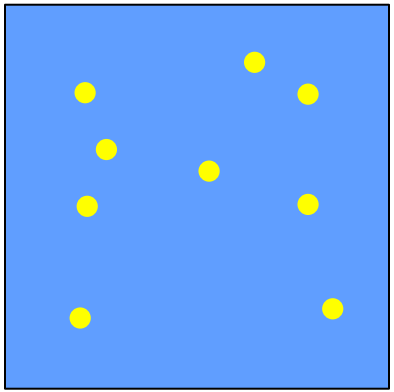
Course schematic. Sources and assumptions in speaker notes.

HPO: Grid Search & Random Search

The most straightforward way is to evaluate many different combinations of hyperparameters, either using **grid search (left) with a "plan"** or **random search (right). Output the best one!**



Grid Search



Random Search

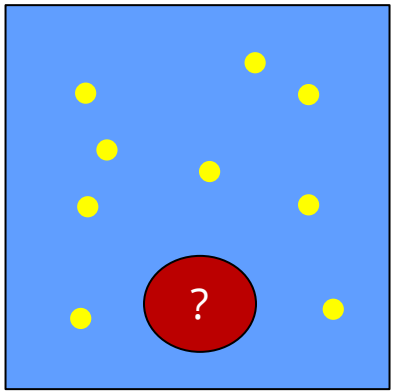
What are the advantages of grid search and random search?

HPO: Grid Search & Random Search

Grid Search: not all the hyperparameters are equally important. Grid search may overemphasize the importance of "unimportant hyperparameters".

Fix: decrease the granularity of unimportant hyperparameters

Random Search: its heuristic does not guarantee a good coverage of the searching space.



The red area is unexplored in random search

Summary: Grid Search & Random Search

Advantages:

1. Easy to set up
2. Appropriate when you have no knowledge of the hyperparameter space
3. It is a **parallel** process with no dependency, which can be efficiently parallelized

Disadvantages:

1. There is no learning process, but pure model evaluation (computation)
2. Can be extremely expensive when you have many hyperparameters. Given k hyperparameters with 2 possible values, it has exponential search space $O(2^k)$. In practice, most HPs are continuous, leading to an intractable computations.
3. Only select from the predefined space!

Takeaway: most of time, this should be sufficient for our daily jobs.

Summary: Grid Search & Random Search (advanced)

Speed-up scikit-learn based hyperparameter tuning:

1. Multi-thread: using n_jobs option in scikit-learn, see here

Even faster:

1. Distributed scikit-learn via dask: https://ml.dask.org/joblib.html

Fix the Limitations of Grid and Random Search

1. If trying different hyperparameters is expensive, **how to reduce the number of evaluations?**
2. **How to learn from the process?** Note grid and random search do not involve learning but pure evaluation.

How to do intelligent "searching" or hyperparameter tuning?

A Search Budget Is Part of the Problem

Suppose we can afford the compute of ten full training runs.

Evaluate fewer candidates fully

More evidence about each one.
Fewer settings explored.

Screen more candidates briefly

Broader exploration.
Early rankings may be misleading.

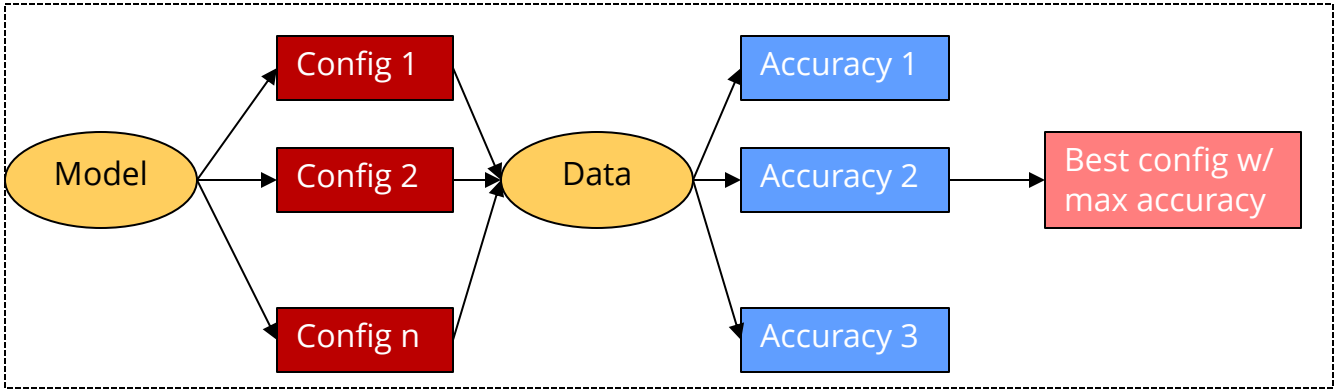
Cheap early evaluations can guide a later, longer evaluation.

Course teaching example. Sources and assumptions in speaker notes.

Fix the Limitations of Grid and Random Search

For the same classifier and split, try settings and measure validation accuracy.

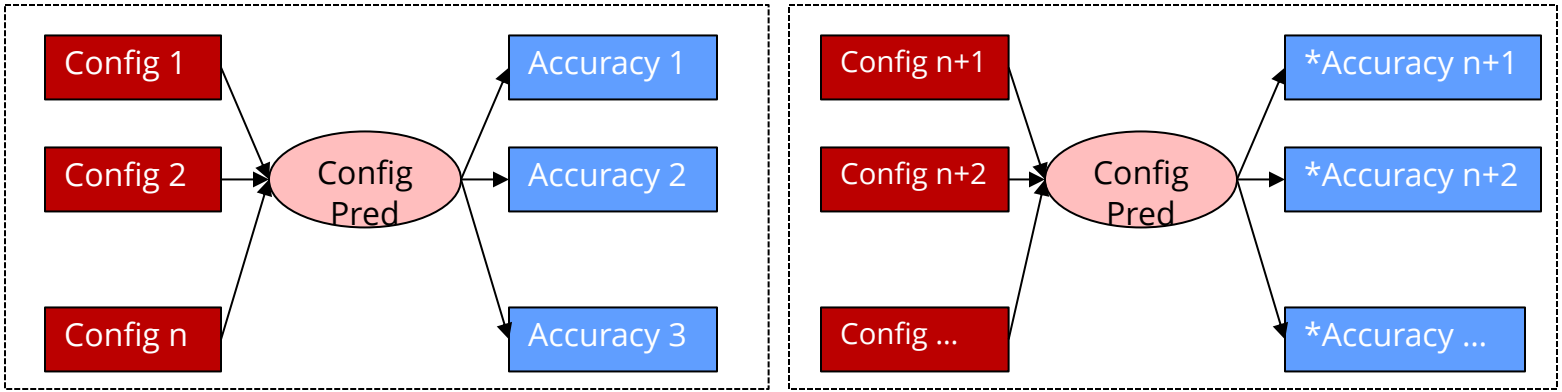
Fitting on the data is expensive! How to prevent this?



Fix the Limitations of Grid and Random Search

Prediction-based hyperparameter tunning:

1. First do some evaluations as before, and then build a **predictor** to forecast the performance of any hyperparameters!
    1. **learn** the relationship between the **hyperparameter configs** and the **performance**!
    2. Cheap predictions rank candidates within a chosen search space.



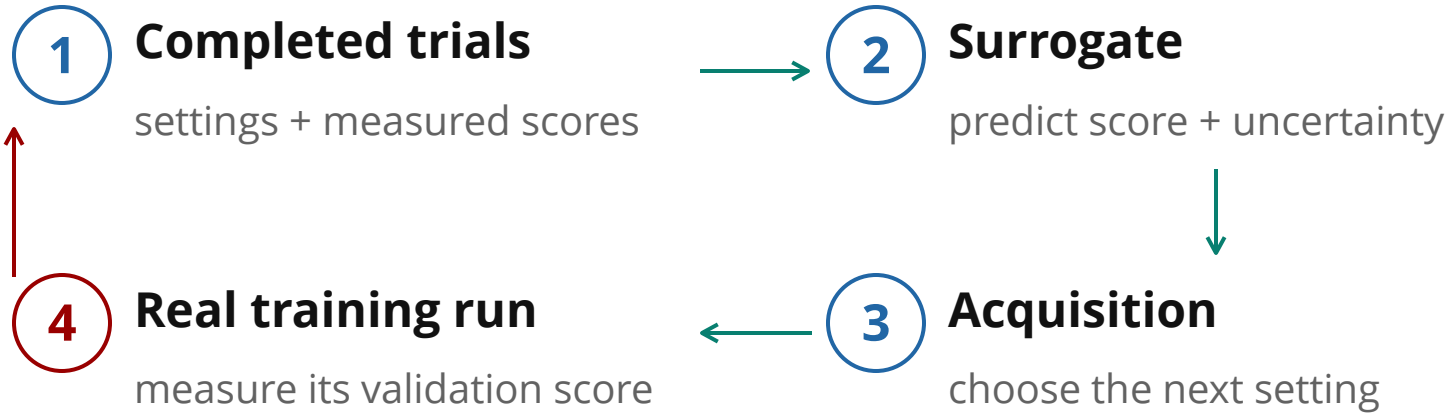
Prediction-based Hyperparameter Tunning

A few things to consider:

1. We still need to do some hyperparameter evaluation in the beginning, and we would expect the number of evaluations as small as possible.
2. **Goal: find useful settings within a fixed budget. Compare with random and grid search; improvement is not guaranteed.**
3. Can we **gradually** build this up and find a nice hyperparameter?
    1. Train the predictor in an iterative way
    2. Train the predictor in a smart way

Bayesian Optimization: One Search Loop

One evaluation trains a flower classifier and measures validation accuracy.

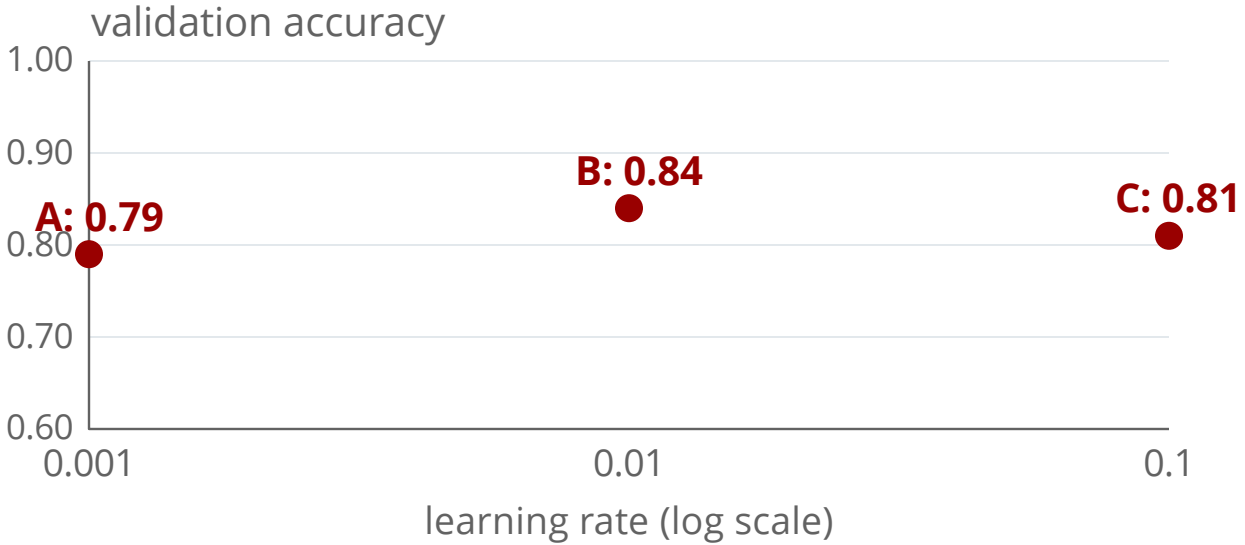


Repeat within the budget; return the best measured configuration.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Three Completed Flower-Classifier Trials

Keep the architecture, data split and training budget fixed. Change the learning rate.



TRIAL LOG

A 0.001 → 0.79

B 0.01 → 0.84

C 0.1 → 0.81

Other settings are unmeasured.

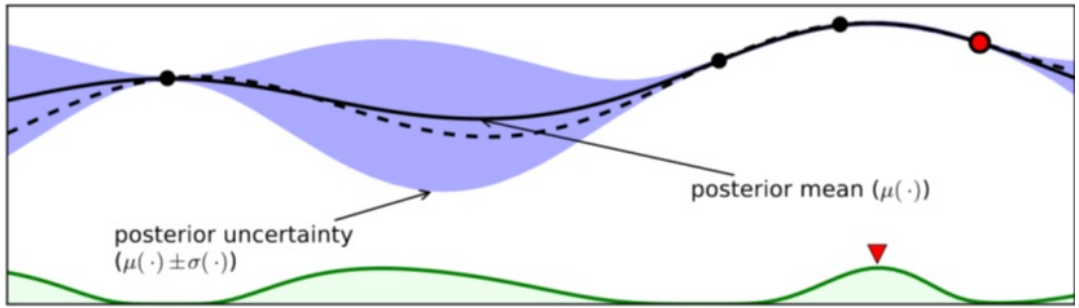
Best measured so far: B, validation accuracy 0.84.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Prediction-based Hyperparameter Tunning

One more thing to consider:

- Each time we update the performance predictor p, so that the prediction result will be different in each round.
- However**, it is likely the predictor does not change and always give you the same result!**
- This observation tells us, while selecting the next best hyperparameter to evaluate, we need to consider **how accurate it might be (accuracy)** and **how uncertain (so we do not get trapped)** it is!



Prediction-based Hyperparameter Tunning

To predict both the **accuracy** and the **uncertainty/variation** of a new hyperparameter, we need specialized ML models for this! Some algorithms can estimate the prediction uncertainty.

1. Random forest (need additional implementation)
2. **Gaussian process** -> scikit-learn directly support it (https://scikit-learn.org/stable/modules/gaussian_process.html). We will discuss this shortly.

predict(*X*, *return_std=False*, *return_cov=False*) [source]

Predict using the Gaussian process regression model.

We can also predict based on an unfitted model by using the GP prior. In addition to the mean of the predictive distribution, optionally also returns its standard deviation (`return_std=True`) or covariance (`return_cov=True`). Note that at most one of the two can be requested.

Parameters:

X : *array-like of shape (n_samples, n_features) or list of object*
Query points where the GP is evaluated.

return_std : *bool, default=False*
If True, the standard-deviation of the predictive distribution at the query points is returned along with the mean.

USC Viterbi School of Engineering

Prediction-based Hyperparameter Tunning

Basically, this sequential prediction-based method **keeps finding the next most promising hyperparameter** to evaluate and **improve the prediction quality gradually**.

The predictor guides candidates within the chosen domain. Real evaluations verify their scores.

With a predictive surrogate and acquisition rule, this becomes Bayesian optimization.

Two Different Optimization Loops

Inside a trial we update model weights. Outside a trial we choose its settings.

OUTER LOOP: CHOOSE A SETTING

BO proposes learning rate = 0.025

→

INNER LOOP: TRAIN THE MODEL

images → classifier → loss

backpropagation updates W

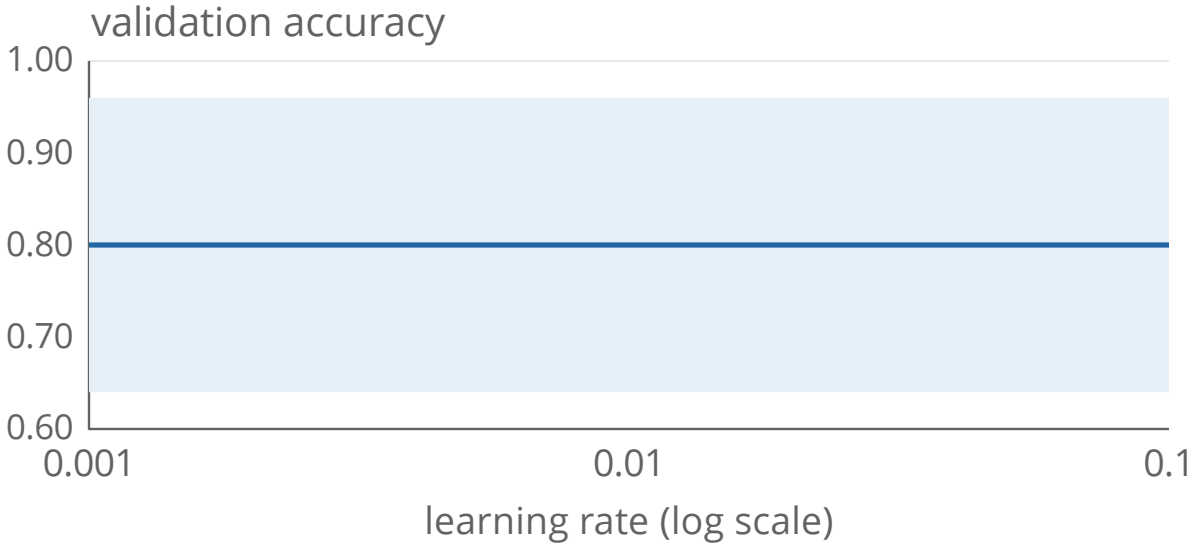
Measured validation score returns to BO

BO chooses the learning rate; SGD or Adam trains the classifier.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Before Trials: A Prior Over the Score Curve

A prior represents assumptions about scores before seeing this task's measurements.



Mean: 0.80

Wide uncertainty across untried settings

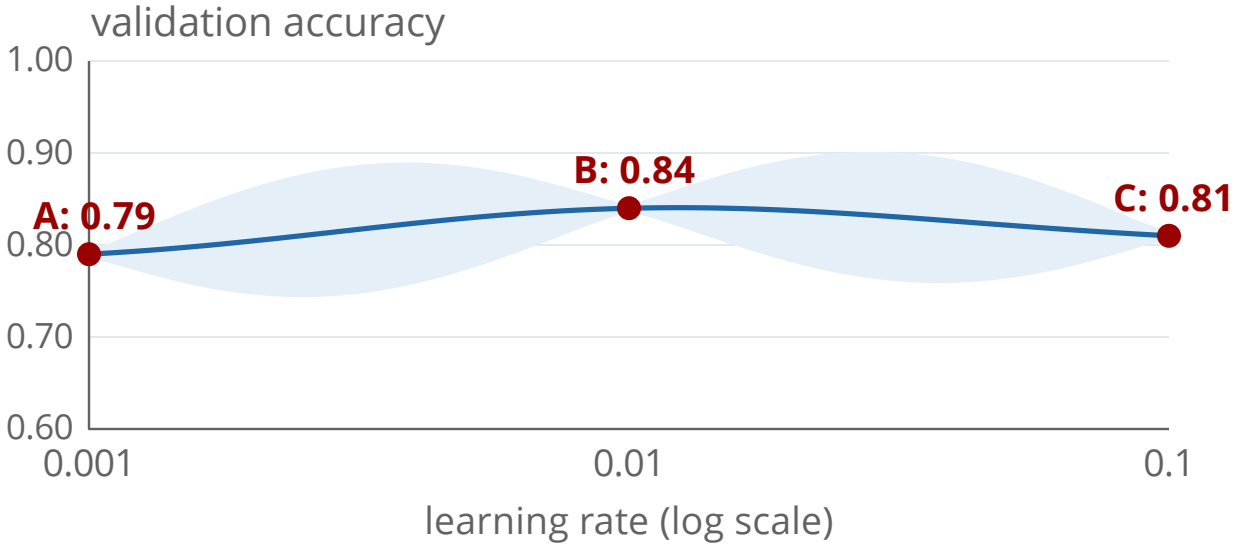
This toy model assumes nearby log learning rates have related scores.

The curve and band are predictions before any training result.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

After Trials: Update the Predictive Distribution

Condition the same model on A, B and C. The resulting prediction is the posterior.



- **measured trial**
- predicted mean
- mean ± 2 SD

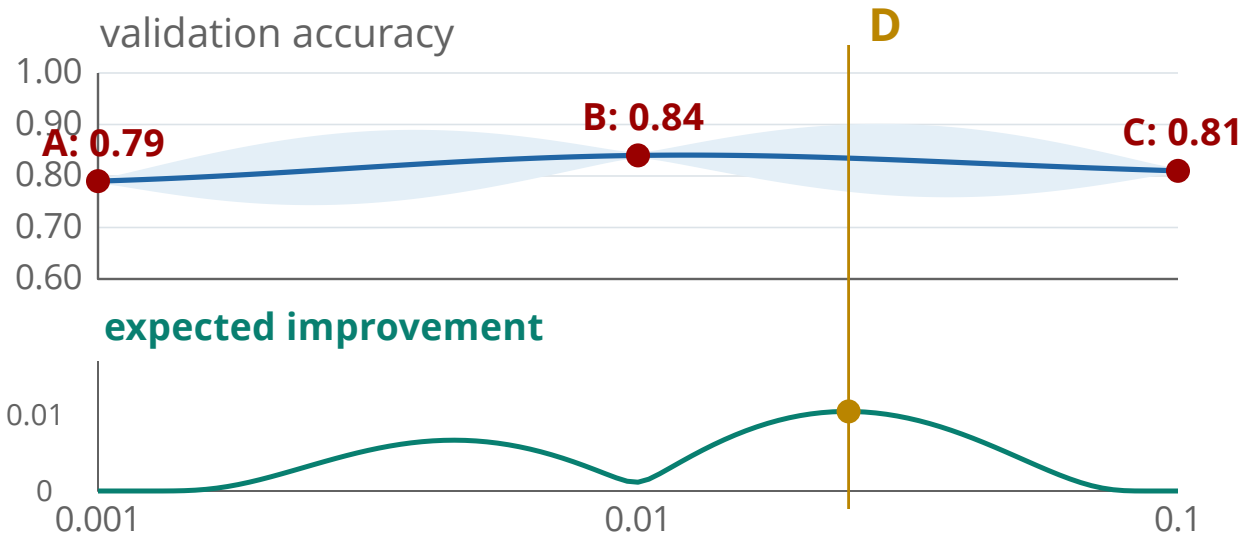
SD = model's predictive standard deviation. The band can be wrong.

Measurements narrow uncertainty near A, B and C.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

The Surrogate Predicts; the Acquisition Chooses

Expected improvement (EI) scores the possible gain over the best observed accuracy, 0.84.



Score prediction

Expected gain (EI)

Units: accuracy gain
D: predicted ≈ 0.835
EI ≈ 0.011
Actual score: unknown

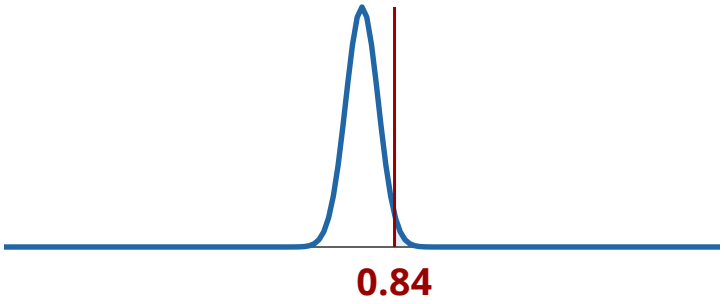
Try D (learning rate ≈ 0.025), then measure its score.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Why Can an Uncertain Setting Be Worth Trying?

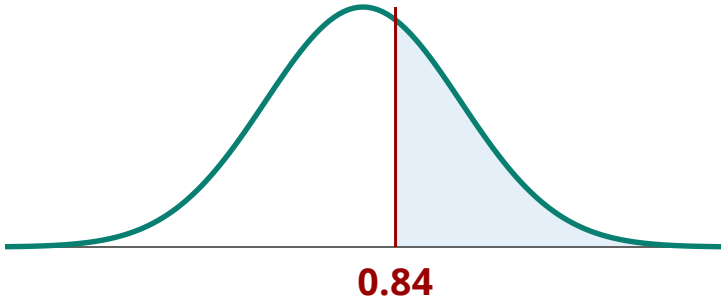
Two illustrative predictions have the same mean, 0.83, but different uncertainty.

NARROW PREDICTION



Most predicted outcomes are below 0.84.

WIDER PREDICTION



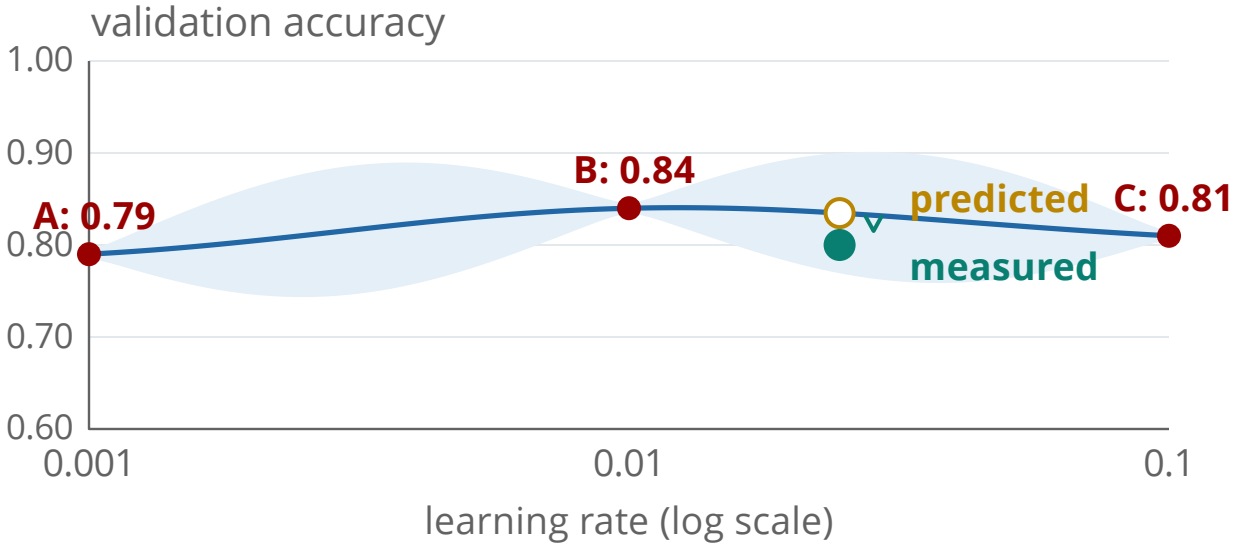
More outcomes exceed 0.84. Shaded region can improve.

Same mean; wider uncertainty allows more possible gain above 0.84.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

A Prediction Must Be Tested by a Real Trial

D was selected using a distribution of possible outcomes. Now train the classifier at D.



D: lr ≈ 0.025

Prediction: 0.835
Measured: 0.80

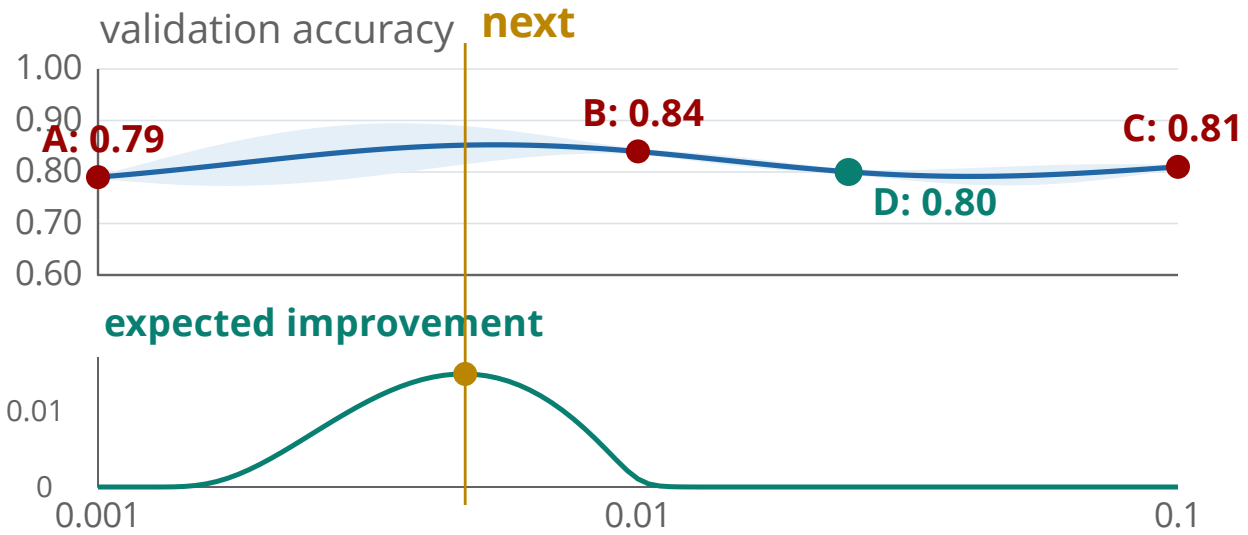
Add (D, 0.80) to the trial log.

D measures 0.80; B at 0.84 remains the best measured setting.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

D Changes the Next Prediction

Add the measured D = 0.80 and fit the surrogate again. Keep the same axes and assumptions.



D now has evidence

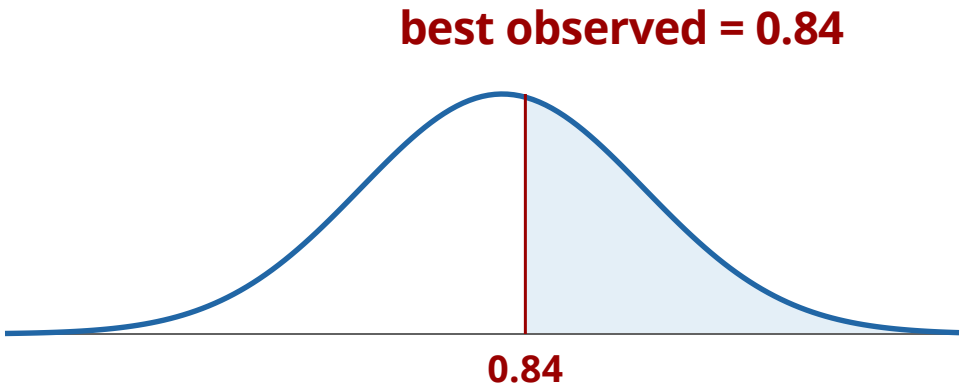
SD near D
0.033 → 0.003
B remains best: 0.84.
Another trial is proposed only if budget remains.

Update near D; next EI choice: learning rate ≈ 0.0048.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Expected Improvement Measures Possible Gain

Compare each possible score with the best observed score. Ignore outcomes that are worse.



Improvement for a score:

max(score − 0.84, 0)

Larger gains count more. Their probabilities also matter.

EI averages positive gains over the predictive distribution.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

A Small Expected-Improvement Example

For arithmetic only, suppose a candidate has three equally likely possible scores.

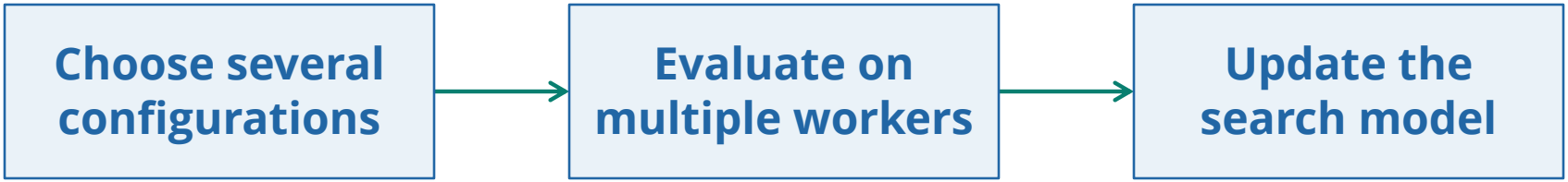
score 0.80	**score 0.84**	**score 0.90**
probability 1/3	probability 1/3	probability 1/3
↓	↓	↓
max(0.80 − .84, 0)	max(0.84 − .84, 0)	max(0.90 − .84, 0)
gain = 0.00	**gain = 0.00**	**gain = 0.06**

Expected gain: (0 + 0 + 0.06) / 3 = 0.02 accuracy, or 2 percentage points.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Batch Search Runs Several Trials in Parallel

Multiple workers can train different proposed configurations.



Parallel trials improve throughput within the available budget.

Course teaching example. Sources and assumptions in speaker notes.

The Whole BO Procedure and Its Final Choice

A prediction proposes a trial. A measurement becomes evidence for the next round.

1	**Measured log**	A .79 B .84 C .81
2	**Predict and choose**	mean + uncertainty → EI → D
3	**Train D and update**	D measures .80 → revise prediction
4	**Stop or repeat**	budget remains? propose another trial

At this stopping point, choose measured B: validation accuracy 0.84.

Snoek et al., NeurIPS 2012. Toy GP and scores for teaching.

Part 2: Given many datasets, how to learn a model to select model for datasets?

Meta-learning focuses on learning to learn. There are generally two "definitions" of meta-learning.

One is under the deep learning context: https://lilianweng.github.io/lil-log/2018/11/30/meta-learning.html; we will not cover it today.

Another goal of more classical meta-learning is to **learn a general model that can select model for arbitrary datasets**.

Meta-Learning Splits Whole Tasks

Earlier, we split photos within one task. Here, we also separate entire datasets.

Historical datasets

Learn how dataset properties relate to model performance.
Each dataset has its own evaluation protocol.

A new dataset

Test model selection on an unseen task.
Its hidden evaluation labels stay outside the selector.

Later X_test means a new dataset at the task level.

Course schematic. Sources and assumptions in speaker notes.

Outline of this Part

Overview

- What is unsupervised ML?
- Why does it matter?
- What are key challenges?

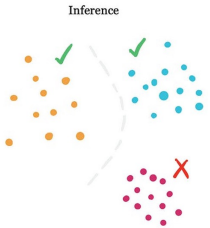
Model Selection by LLMs

- Leverage LLMs in model selection
- *ICLR 2025* (MetaOOD) & *preprint* (AD-LLM)

Second

First

Third



Unsupervised Model Selection

- Meta learning for automating OD/OOD model selection
- *NeurIPS 2021* (MetaOD)

USC Viterbi School of Engineering

Outliers and Distribution Shift

The following examples select models for detecting unusual observations.

Outlier detection (OD)	Find observations unusual relative to a reference dataset.
Out-of-distribution (OOD)	Identify inputs outside the reference distribution.
Model selection here	Choose a detector for a new dataset without its labels.

Historical labeled tasks can teach this selection rule.

Course schematic. Sources and assumptions in speaker notes.

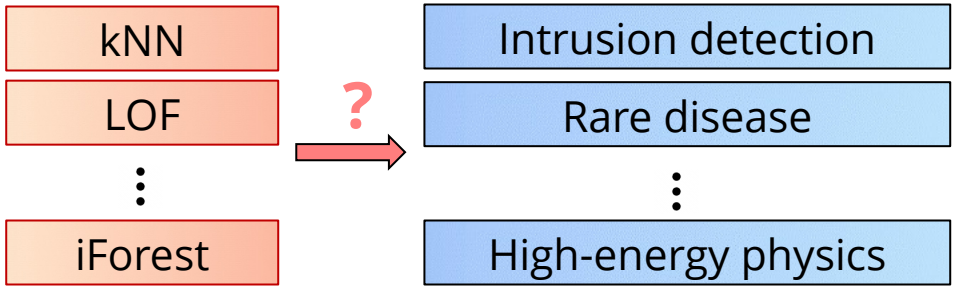
Part 1: How to *Automate* OD/OOD *Model Selection w/o* Labels?

MetaOD: Automatic Outlier Model Selection

NeurIPS' 21 [Zhao, Rossi, Akoglu]

The *Best* Algorithm Varies by Applications

No free lunch theorem: no learning algorithm always outperforms

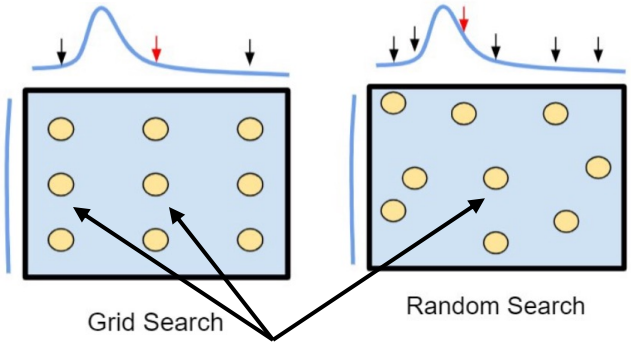


USC Viterbi School of Engineering

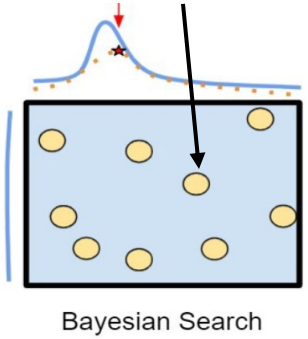
Supervised Model Selection with Labels

With **ground truth** y for evaluation strategy:

- grid & random search by evaluation
- bandit-based algorithm by evaluation
- …



Model evaluation w/ labels

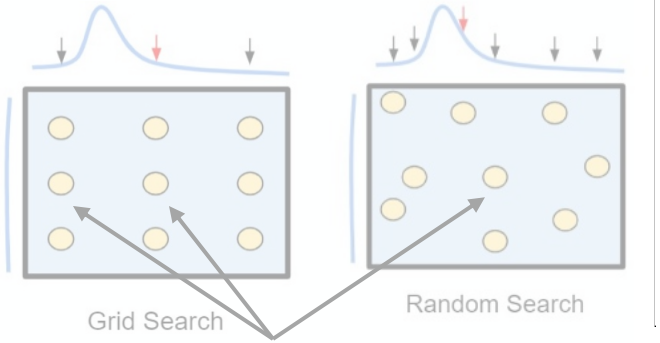


USC Viterbi School of Engineering

Supervised Model Selection with Labels

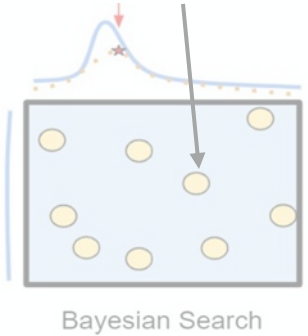
With **ground truth** y for evaluation strategy:

- grid & random search by evaluation
- bandit-based algorithm by evaluation
- …



Model evaluation w/ labels

For **unsupervised ML**, there is no labels **for evaluation** ☹



Meta-Learning for (Unsupervised) Model Selection

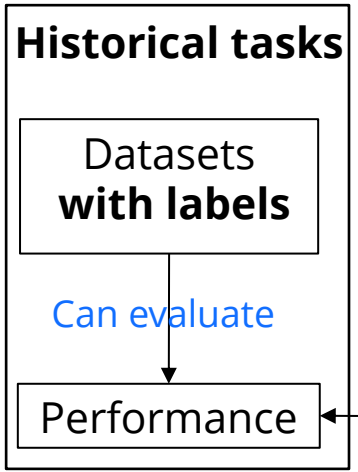
Meta-learning helps **select ML models for new tasks** by **prior experience.**

How to **select** a **good ML model** for wolf images *without* evaluation?

Meta-Learning for (Unsupervised) Model Selection

3. Dogs and wolves are **similar** → use **the same model** for wolves ☺

1. We have dog images and labels



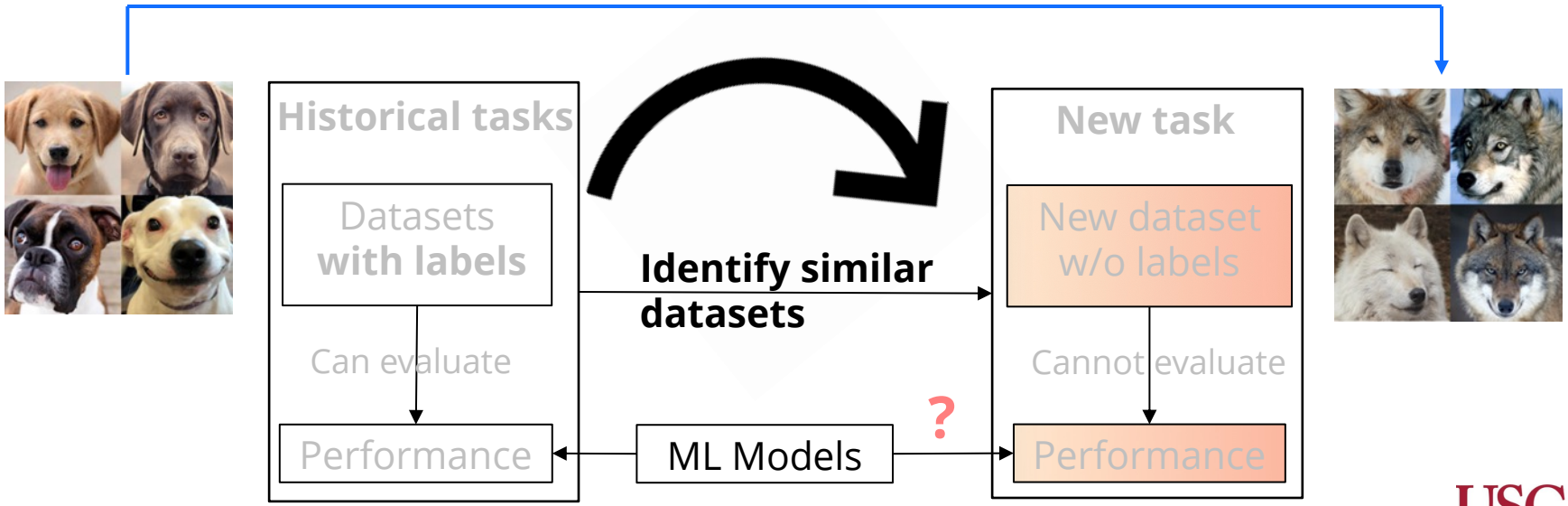
2. We know **the best model** for dog images ☺

Transfer the best model(s)

USC Viterbi School of Engineering

Meta-Learning for (Unsupervised) Model Selection

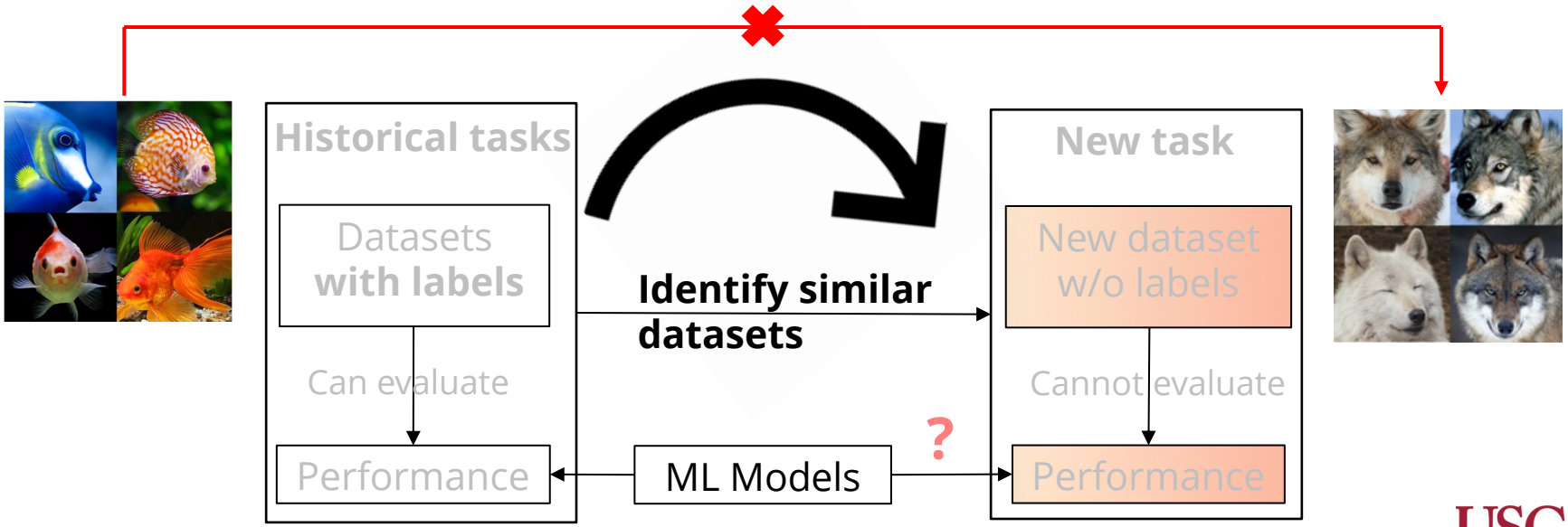
Meta-learning helps **new tasks** by transferring from **similar prior tasks.**



USC Viterbi School of Engineering

Meta-Learning for (Unsupervised) Model Selection

Do not transfer the model from **dissimilar datasets/tasks.**



The best model(s) differ

USC Viterbi School of Engineering

Unsupervised OD Model Selection via Meta-Learning

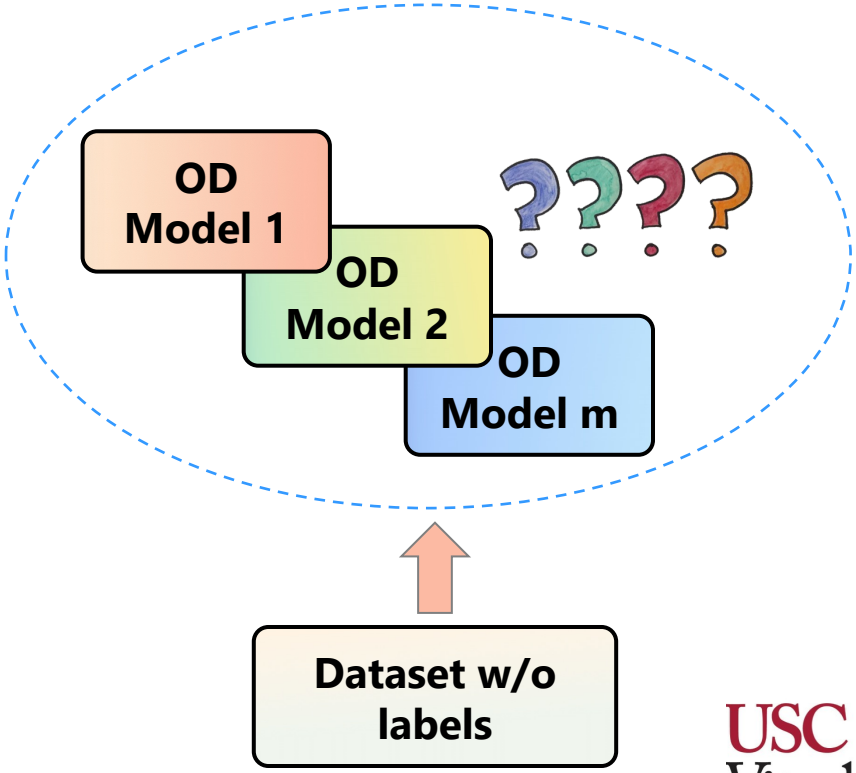
[New] Unsupervised OD model selection (UOMS):

Given a *new input dataset* with *no labels*, **select an OD model**.

[1st time] UOMS via meta-learning:

Use ***prior knowledge*** to facilitate ***model selection*** *on the* ***new dataset.***

The key is to measure *dataset/task similarity*.



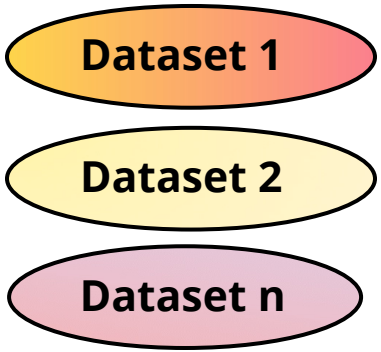
USC Viterbi School of Engineering

Unsupervised OD Model Selection via Meta-Learning

1. Training on historical datasets:

- **A meta-train database** with n datasets:

  $\mathbf{D}_{\text{train}} = \{\mathrm{D}_1, \mathrm{D}_2, \ldots, \mathrm{D}_n\}$, where $\mathrm{D}_\mathrm{i} = (\mathbf{X_i}, \mathbf{y_i})$



USC Viterbi School of Engineering

Unsupervised OD Model Selection via Meta-Learning

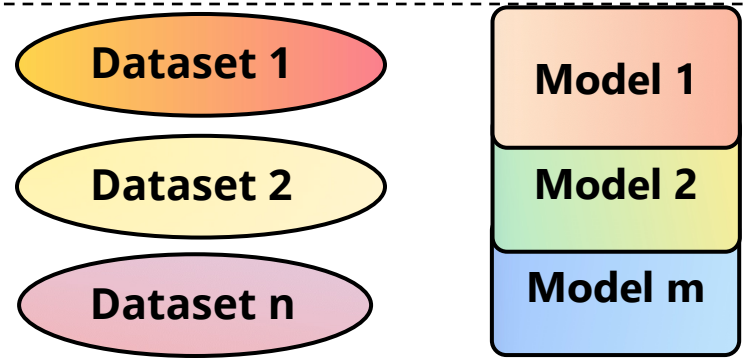
1. Training on historical datasets:

- **A meta-train database** with n datasets:

  $\mathbf{D}_{\text{train}} = \{\mathrm{D}_1, \mathrm{D}_2, \ldots, \mathrm{D}_n\}$, where $\mathrm{D}_i = (\mathbf{X_i}, \mathbf{y_i})$

- **A predefined OD model set** with m models**:**

  $\mathcal{M} = \{\mathrm{M}_1, \mathrm{M}_2 \ldots \mathrm{M}_m\}$



USC Viterbi School of Engineering

Unsupervised OD Model Selection via Meta-Learning

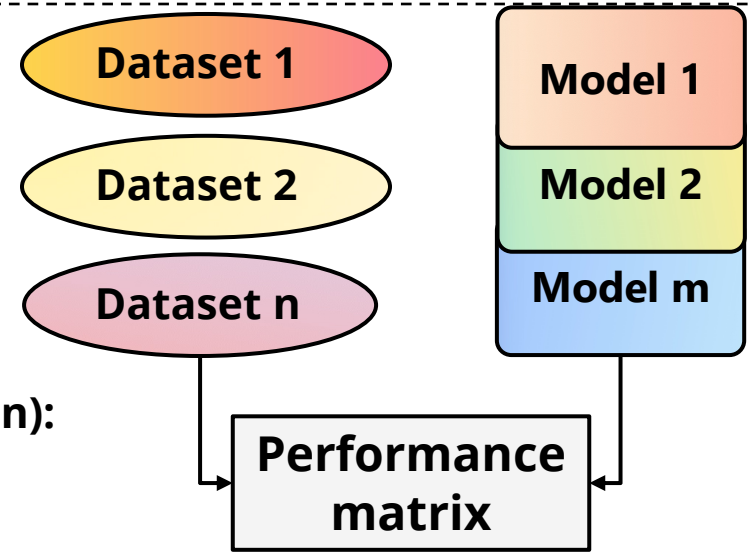
1. Training on historical datasets:

- **A meta-train database** with n datasets:

  $\mathbf{D}_{\text{train}} = \{D_1, D_2, \ldots, D_n\}$, where $D_i = (\mathbf{X_i}, \mathbf{y_i})$

- **A predefined OD model set** with m models:

  $\mathcal{M} = \{M_1, M_2 \ldots M_m\}$



Pairing n datasets with m models (by evaluation):

- **Performance matrix $\mathbf{P} \in \mathbb{R}^{n \times m}$**

USC Viterbi School of Engineering

Unsupervised OD Model Selection via Meta-Learning

1. Training on historical datasets:

- **A meta-train database** with n datasets:

  $\mathbf{D}_{\text{train}} = \{\mathrm{D}_1, \mathrm{D}_2, \ldots, \mathrm{D}_n\}$, where $\mathrm{D}_i = (\mathbf{X}_i, \mathbf{y}_i)$

- **A predefined OD model set** with m models:

  $\mathcal{M} = \{\mathrm{M}_1, \mathrm{M}_2 \ldots \mathrm{M}_m\}$

	M_1	M_2	…	M_m
D_1	0.8	0.7	0.3	0.6
D_2	0.5	1	0.6	0.2
…	…	…	…	…
D_n	0.1	0.2	0.3	0.2

Pairing n datasets with m models (by evaluation):

- **Performance matrix** $\mathbf{P} \in \mathbb{R}^{n \times m}$

Model 1's perf. on dataset 2

2. Model selection: new dataset $\mathbf{D}_{\text{test}} = (\mathbf{X}_{\text{test}}, \emptyset)$ with no labels, select a model from $\mathcal{M}$ with the highest perf?

Model selection on $\mathbf{X}_{\text{test}}$

	M_1	M_2	…	M_m
$\mathbf{D}_{\text{test}}$	?	?	?	?

Unsupervised OD Model Selection via Meta-Learning

1. Training on historical datasets:

- **A meta-train database** with n datasets:

  $\mathbf{D}_{\text{train}} = \{\mathrm{D}_1, \mathrm{D}_2, \ldots, \mathrm{D}_n\}$, where $\mathrm{D}_i = (\mathbf{X_i}, \mathbf{y_i})$

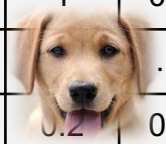
- **A predefined OD model set** with m models:

  $\mathcal{M} = \{\mathrm{M}_1, \mathrm{M}_2 \ldots \mathrm{M}_m\}$

Pairing n datasets with m models (by evaluation):

- **Performance matrix** $\mathbf{P} \in \mathbb{R}^{n \times m}$

	$\mathbf{M_1}$	$\mathbf{M_2}$	…	$\mathbf{M_m}$
$\mathbf{D_1}$	0.8	0.7	0.3	0.6
$\mathbf{D_2}$	0.5	1	0.6	0.2
…	…		…	…
$\mathbf{D_n}$	0.1		0.3	0.2



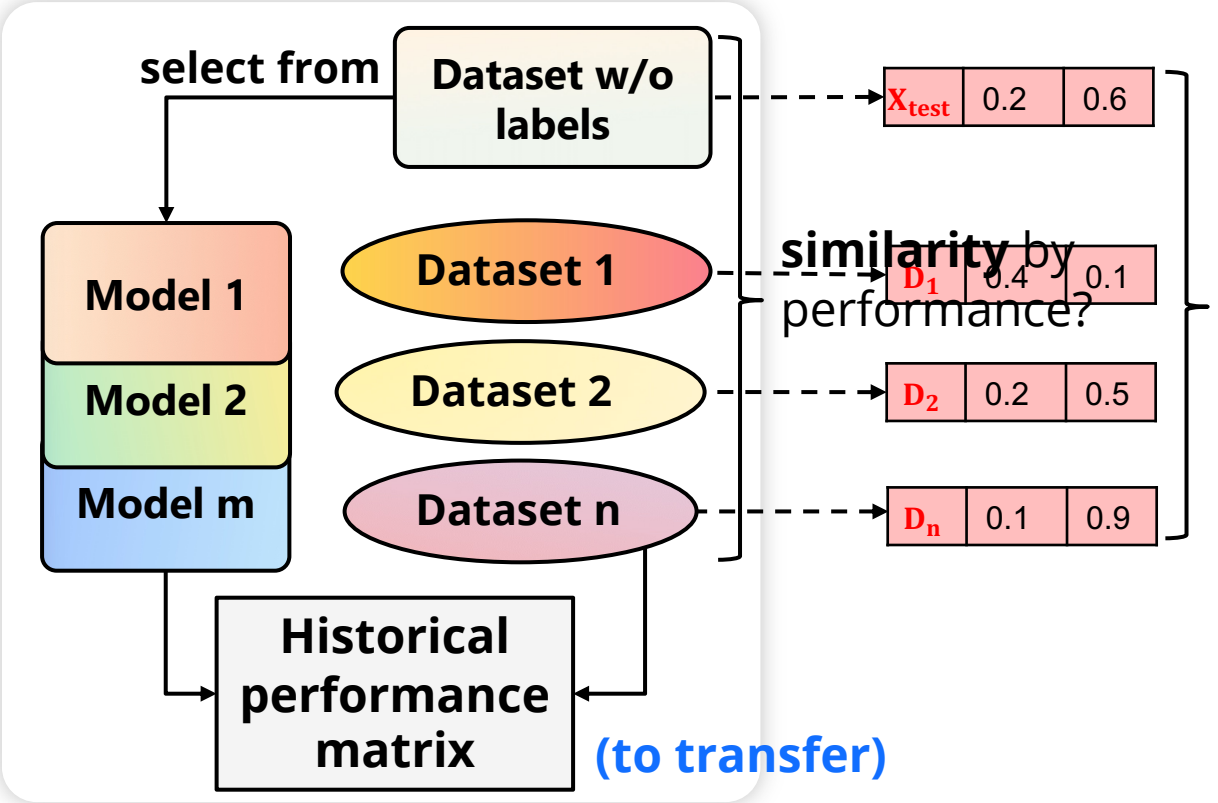
Transfer knowledge by meta-learning

2. Model selection: new dataset $\mathbf{D}_{\text{test}} = (\mathbf{X}_{\text{test}}, \emptyset)$ with no labels, select a model from $\mathcal{M}$ with the highest perf?

Model selection on $\mathbf{X}_{\text{test}}$

	$\mathbf{M_1}$		…	$\mathbf{M_m}$
$\mathbf{D}_{\text{test}}$	?		?	?

Challenge 1: Measure Dataset Similarity w.r.t Perf.



Unified embedding w.r.t. performance:

- matrix factorization
- neural net
- (recent) LLMs

USC Viterbi School of Engineering

Dataset Similarity by Model Performance (Matrix P)

m models

n datasets

	M_1	M_2	…	M_m
D_1	0.8	0.7	0.3	0.6
D_2	0.5	1	0.6	0.2
…	…	…	…	…
D_n	0.1	0.2	0.3	0.2

Performance matrix P

≈

k factors

D_1	0.2	0.2
D_2	0.4	0.7
D_3	0.8	0.2
D_4	0.5	0.1
…	…	…
D_n	0.3	0.1

U: dataset matrix for data similarity.

•

k factors

M_1	M_2	M_3	M_4	…	M_m
1	0.6	0.1	0.7	…	0.7
0.7	0.3	0.2	0.5	…	0.2

V^T: model matrix for model similarity

Non-negative ***low-rank*** **matrix factorization**/decomposition
(i.e., $k < m$ and $k < n$)

USC Viterbi School of Engineering

Dataset Similarity by Model Performance (Matrix P)

m models

n datasets

	M_1	M_2	…	M_m
D_1	0.8	0.7	0.3	0.6
D_2	0.5	1	0.6	0.2
…	…	…	…	…
D_n	0.1	0.2	0.3	0.2

Performance matrix P

≈

k factors

D_1	0.2	0.2
D_2	0.4	0.7
D_3	0.8	0.2
D_4	0.5	0.1
…	…	…
D_n	0.3	0.1

U: dataset matrix for data similarity.

•

M_1	M_2	M_3	M_4	…	M_m
1	0.6	0.1	0.7	…	0.7
0.7	0.3	0.2	0.5	…	0.2

k factors

V^T: model matrix for model similarity

Model embedding (transferable)

Dataset embedding

USC Viterbi School of Engineering

MetaOD: Predict Test Model Perf. Via Meta-Learning

Offline training

	M_1	M_2	…	M_m
D_1	0.8	0.7	0.3	0.6
D_2	0.5	1	0.6	0.2
…	…	…	…	…
D_n	0.1	0.2	0.3	0.2

P: performance matrix

$\approx$

U: dataset matrix

D_1	0.2	0.2
D_2	0.4	0.7
D_3	0.8	0.2
D_4	0.5	0.1
…	…	…
D_n	0.3	0.1

$\cdot$

V^T: model matrix

M_1	M_2	M_3	M_4	…	M_m
1	0.6	0.1	0.7	…	0.7
0.7	0.3	0.2	0.5	…	0.2

Transfer knowledge via meta-learning

How to get the embedding of the test data?

Online Model selection on X_{test}:

X_{test}	0.2	0.6

$\cdot$

M_1	M_2	M_3	M_4	…	M_m
1	0.6	0.1	0.7	…	0.7
0.7	0.3	0.2	0.5	…	0.2

V^T: model matrix

$\approx$

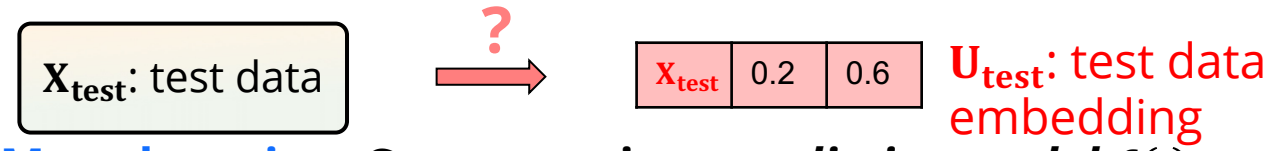
	M_1	M_2	…	M_m
X_{test}	0.7	0.3	…	0.2

$\widehat{P_{test}} := U_{test} V^T$: predicted model performance

U_{test}: test embed.

USC Viterbi School of Engineering

Challenge 2: Getting Dataset Embedding $\mathbf{U_{test}}$

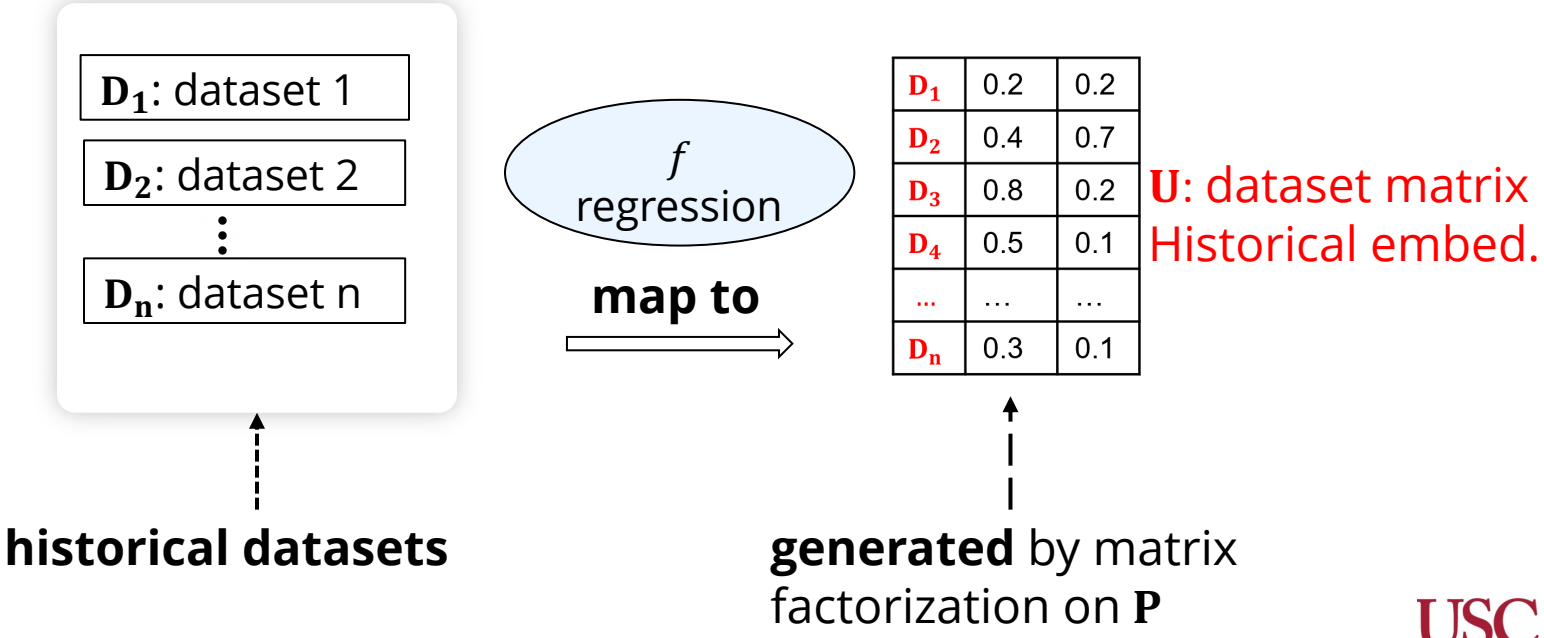


Meta-learning: Can we train *a predictive model* $f(\cdot)$ on the historical to predict to test data embedding?

USC Viterbi School of Engineering

Predicting Dataset Embedding $\mathbf{U}_{\text{test}}$

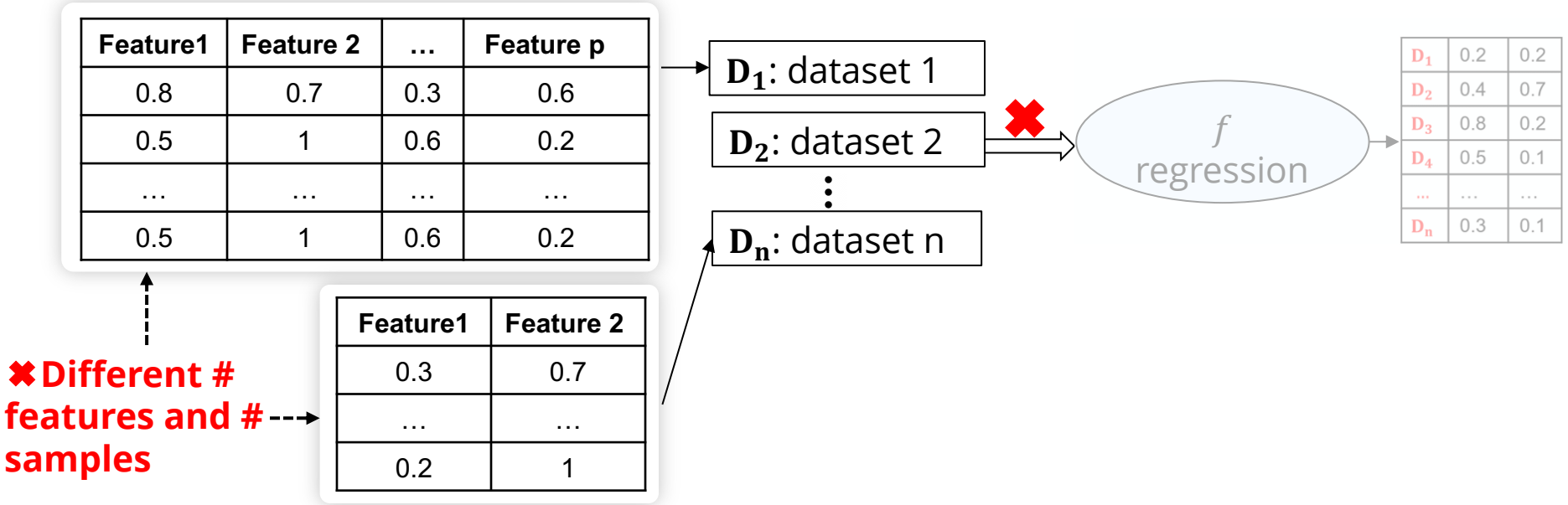
Training: regressor $f(\cdot)$ to predict dataset embeddings



USC Viterbi School of Engineering

Characterize Raw Datasets

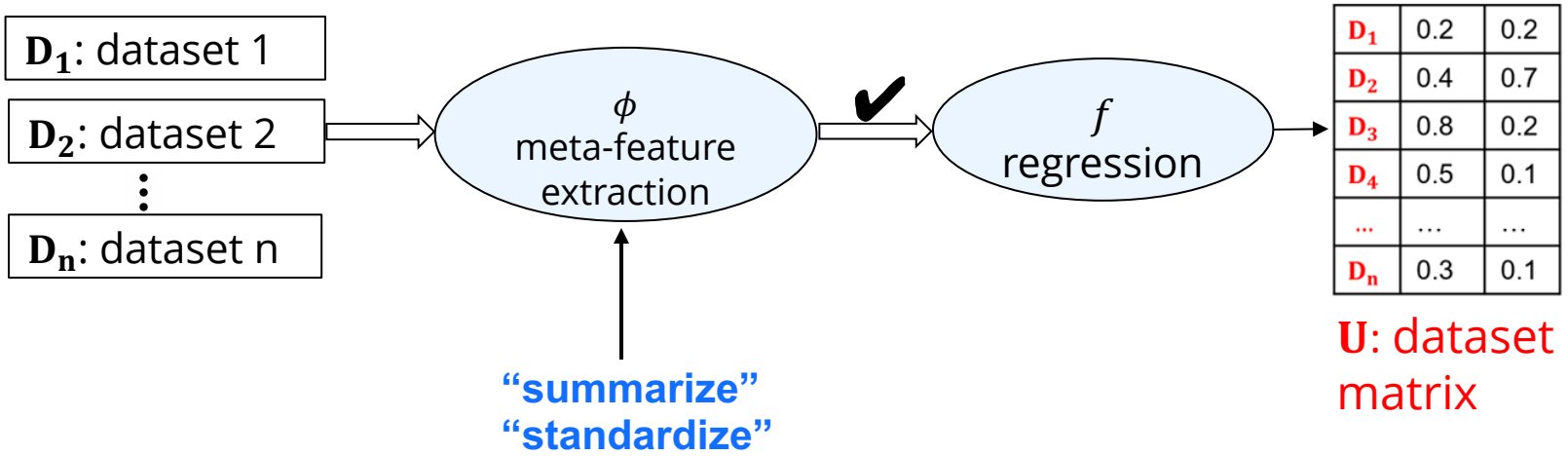
Training: regressor $f(\cdot)$ to predict dataset embeddings



USC Viterbi School of Engineering

Characterize Raw Datasets

Training: regressor $f(\cdot)$ to predict dataset embeddings

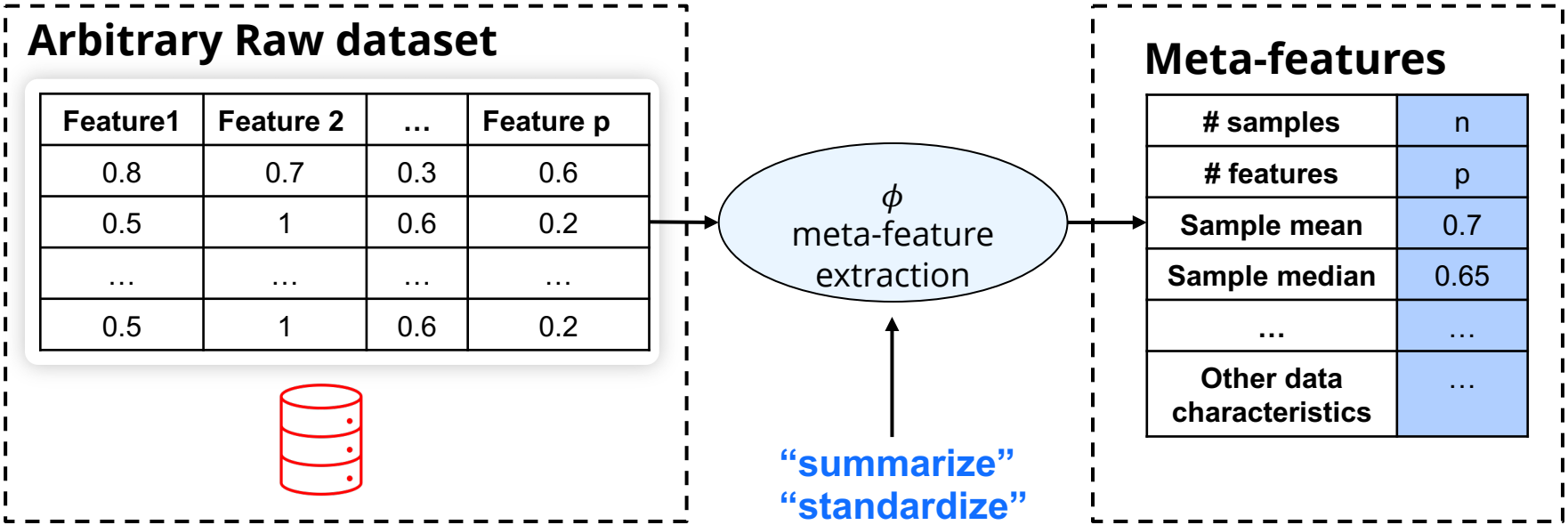


More recently, using deep learning or LLMs to extract embeddings

USC Viterbi School of Engineering

Characterize Raw Datasets by Meta-features

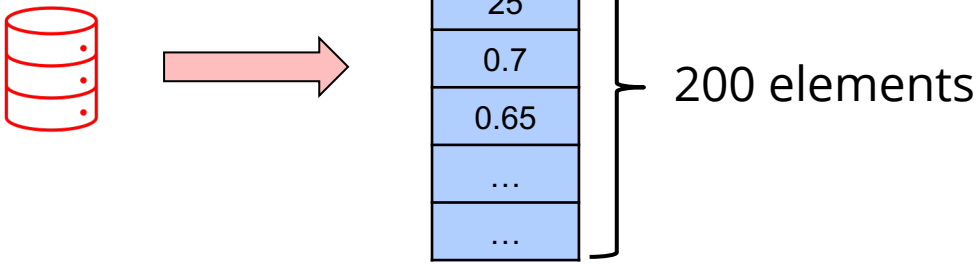
Meta-features: characteristics of a dataset



Automation

Characterize Raw Datasets by Meta-features

Meta-feature extraction: characterize an arbitrary dataset as a 200-element **meta-feature vector** ☺



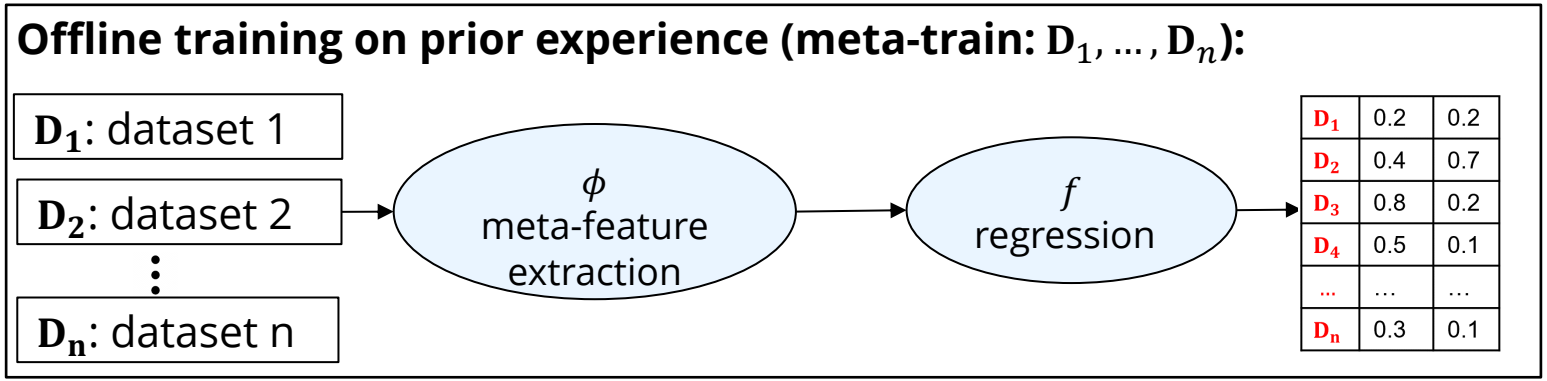
Statistical features (128): statistical characteristics

[new] ***OD features*** (72): OD-related characteristics

USC Viterbi School of Engineering

Predicting Dataset Embedding $\mathbf{U}_{\mathbf{test}}$

Training: predictor $f(\cdot)$ to predict dataset embeddings



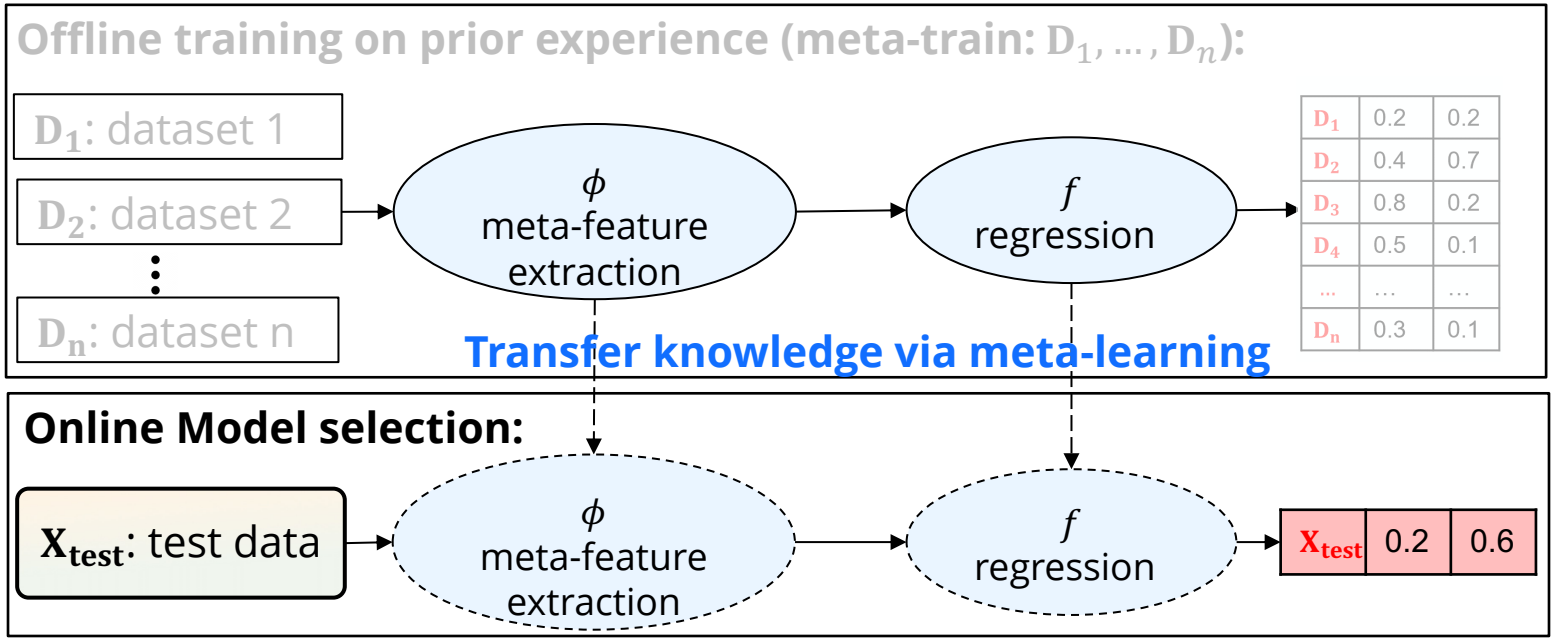
$\mathbf{U}$: dataset matrix

USC Viterbi School of Engineering

Predicting Dataset Embedding $\mathbf{U_{test}}$

Training: predictor $f(\cdot)$ to predict dataset embeddings

Prediction: predict test data embedding by $\mathbf{U_{test}} = f(\mathbf{X_{test}})$



U: dataset matrix

$\mathbf{U_{test}}$: test embed.

MetaOD: Predict Test Model Perf. Via Meta-Learning

Offline training

	M_1	M_2	…	M_m
D_1	0.8	0.7	0.3	0.6
D_2	0.5	1	0.6	0.2
…	…	…	…	…
D_n	0.1	0.2	0.3	0.2

P: performance matrix

≈

U: dataset matrix

D_1	0.2	0.2
D_2	0.4	0.7
D_3	0.8	0.2
D_4	0.5	0.1
…	…	…
D_n	0.3	0.1

•

V^T: model matrix

M_1	M_2	M_3	M_4	…	M_m
1	0.6	0.1	0.7	…	0.7
0.7	0.3	0.2	0.5	…	0.2

Transfer knowledge via meta-learning

Online Model selection on X_{test}:

X_{test}	0.2	0.6

U_{test}: test embed.

•

M_1	M_2	M_3	M_4	…	M_m
1	0.6	0.1	0.7	…	0.7
0.7	0.3	0.2	0.5	…	0.2

V^T: model matrix

≈

	M_1	M_2	…	M_m
X_{test}	0.7	0.3	…	0.2

Embedding U_{test} by prediction

$P_{test} := U_{test} V^T$: predicted model performance

USC Viterbi School of Engineering

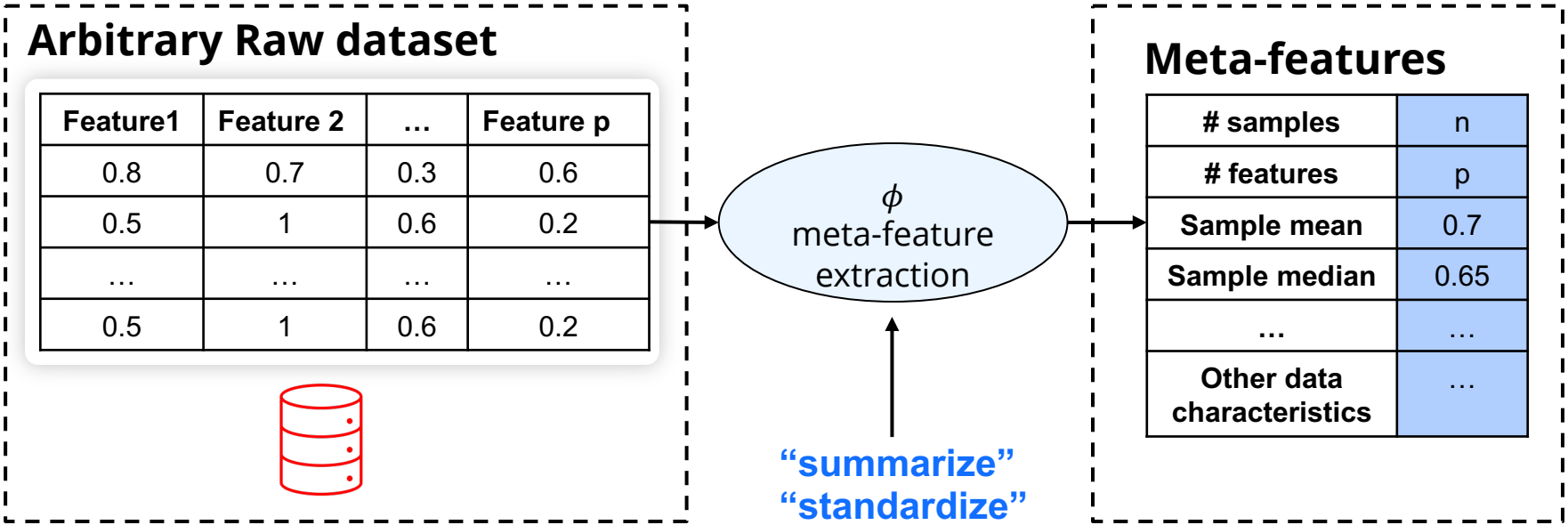
Part 2: Improve Unsupervised Model Selection via LLM-based Data Features

MetaOOD: Automatic Selection of OOD Detection Models

ICLR' 25 [Qin, Zhang, Nian, Ding, Zhao]

Characterize Raw Datasets by Meta-features

Meta-features: characteristics of a dataset



Automation

Limitations of Meta-features

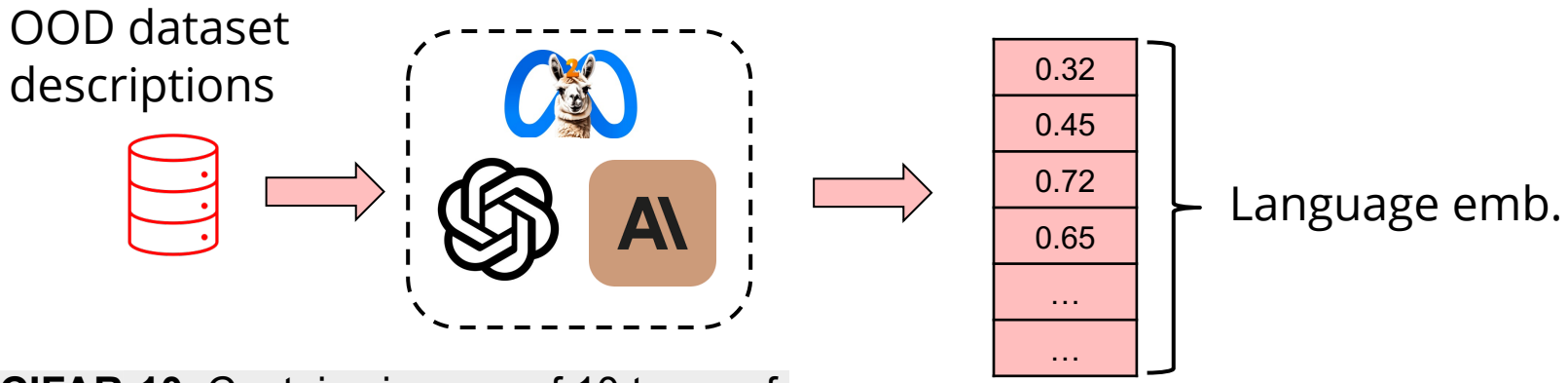
Meta-feature extraction for datasets: handcrafted, heuristic ☹

- Which statistical features should I use?

USC Viterbi School of Engineering

New: LLM-Generated Data Features

Standardize data representations via LLMs

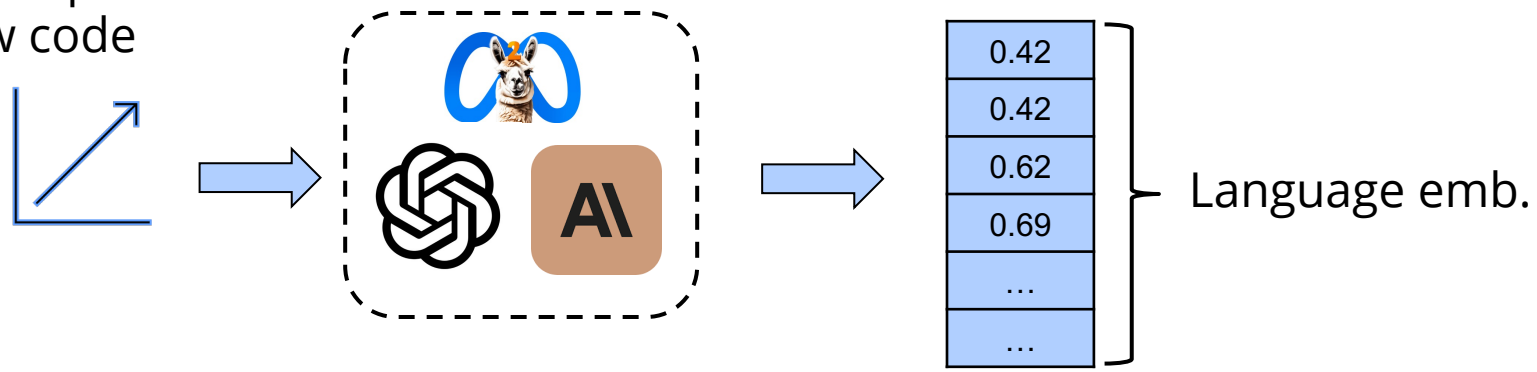


CIFAR-10: Contains images of 10 types of objects, including airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Each image is a small 32x32 RGB image.

New: LLM-Generated Model Features

OOD model descriptions + raw code

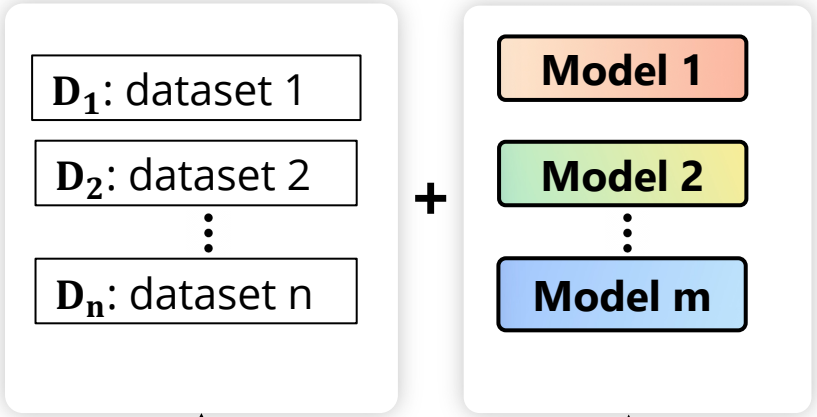
Standardize model representations via LLMs

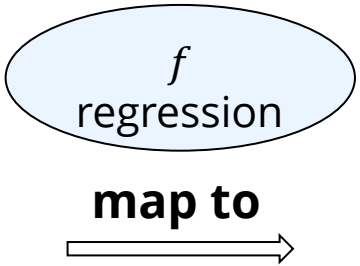


Openmax: Determines a center for each class in the logits space of a model, and then creates a statistical model of the distances of correctly classified inputs. It uses extreme value theory to detect outliers by fitting a Weibull function to the tail of the distance distribution. The activation of the unknown class is used as the outlier score.

Learning the Performance Mapping

Training: regressor $f(\cdot)$ to predict dataset embeddings





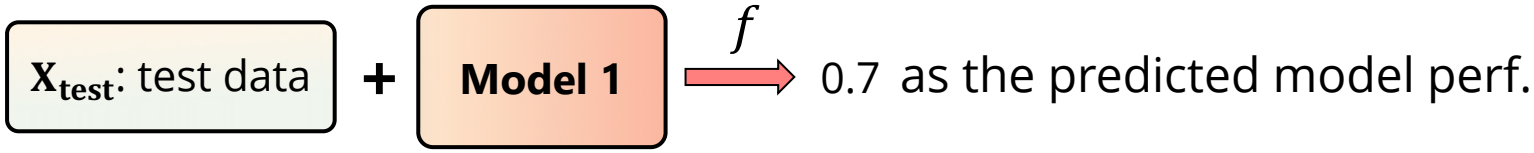
	M_1	M_2	…	M_m
D_1	0.8	0.7	0.3	0.6
D_2	0.5	1	0.6	0.2
…	…	…	…	…
D_n	0.1	0.2	0.3	0.2

Performance matrix

historical datasets **model set**

Embeddings by LLMs

Model Selection: Predict the Performance w/ f



[Zero Shot] Iterate over all models in $\mathcal{M}$, choose the one with the highest predicted performance.

$$\text{argmax}_{M \in \mathcal{M}} f(Emb(\mathbf{X}_{\mathbf{test}}), Emb(M))$$

Output the one w/ highest predicted performance

Predicted performance of M by $f(\cdot)$

USC Viterbi School of Engineering

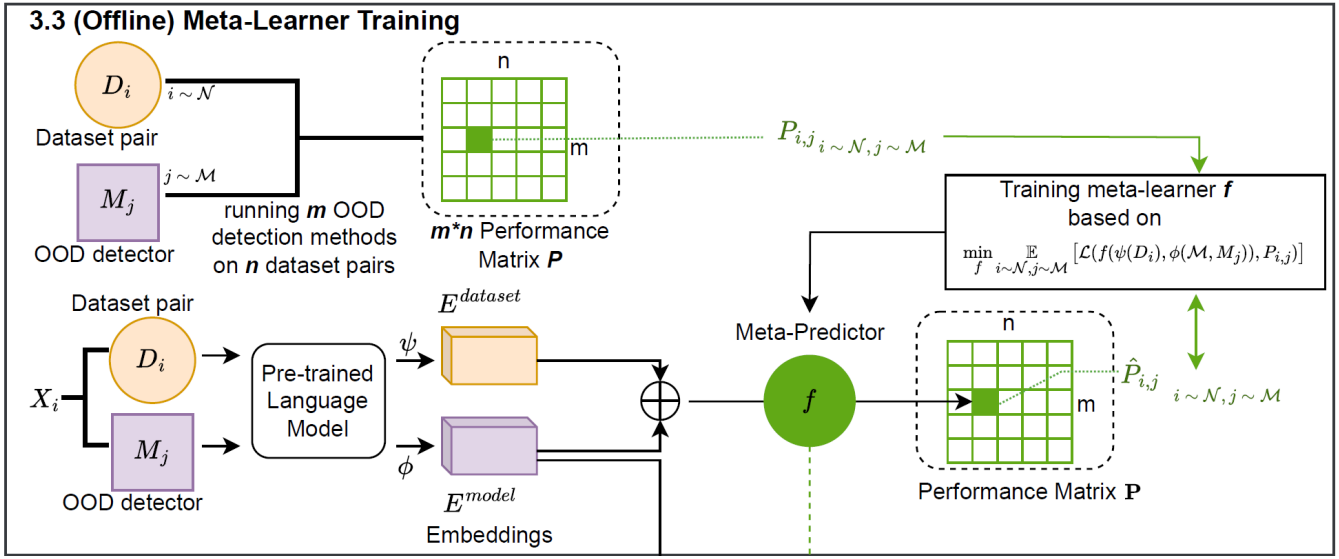


Figure 1: MetaOOD overview (§3.2); offline meta-training phase is shown on the top (§3.3)—the key is to train a meta performance predictor f (denoted in ●) to map language embeddings of the datasets and models to their performance $\mathbf{P}$; the online model selection (§3.4) is shown at the bottom by transferring the meta-predictor f to predict the test data paired with OOD detectors for selection.

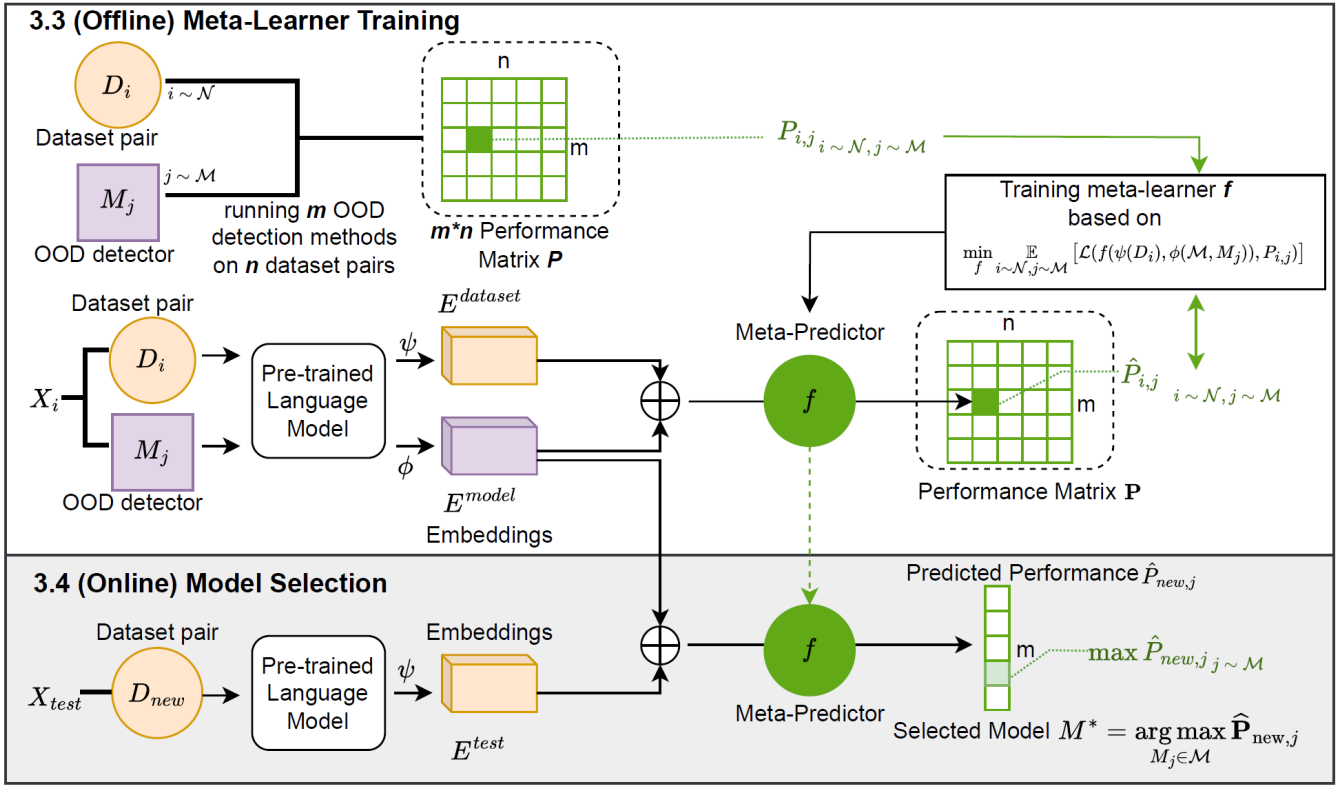


Figure 1: MetaOOD overview (§3.2); offline meta-training phase is shown on the top (§3.3)—the key is to train a meta performance predictor f (denoted in ●) to map language embeddings of the datasets and models to their performance $\mathbf{P}$; the online model selection (§3.4) is shown at the bottom by transferring the meta-predictor f to predict the test data paired with OOD detectors for selection.

Small Overhead of Model Selection

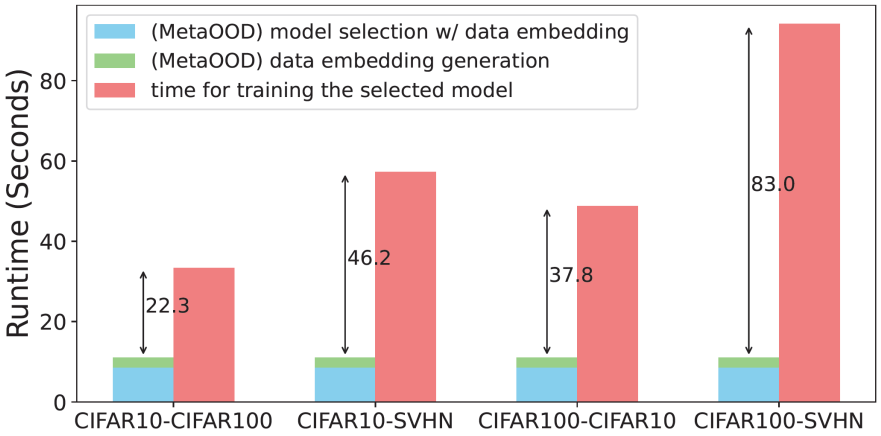


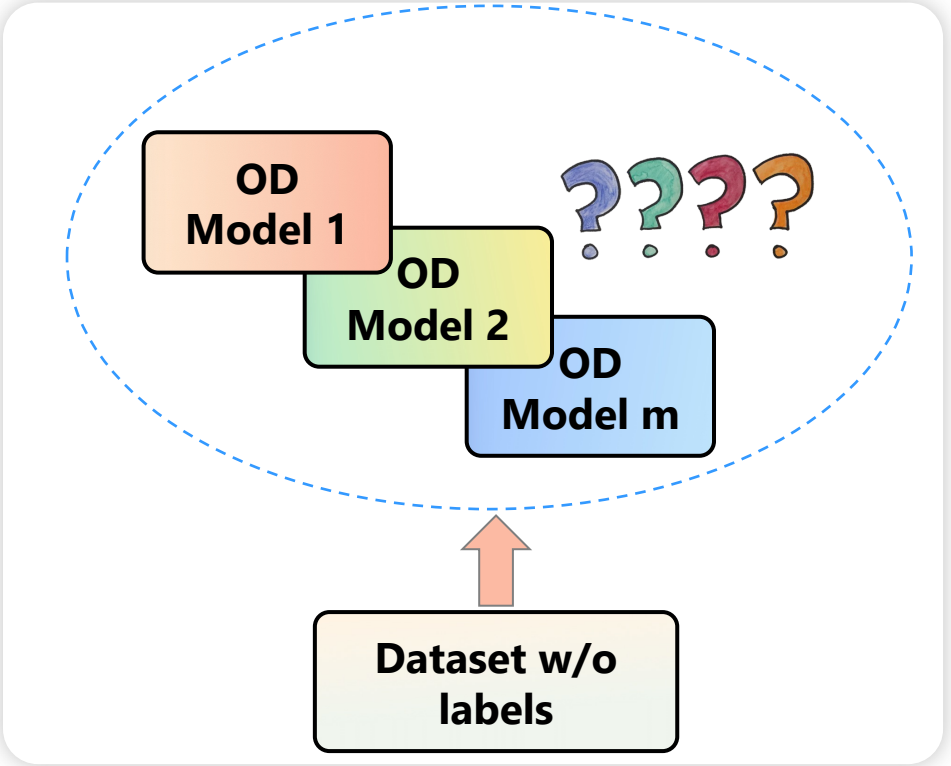
Figure 7: Runtime of MetaOOD vs. selected OOD detector. MetaOOD incurs a small overhead (difference shown with black arrows).

Model selection overhead negligible to the actual OOD detection fitting on the image datasets.

Recap: 1st OD/OOD Model Selection Framework

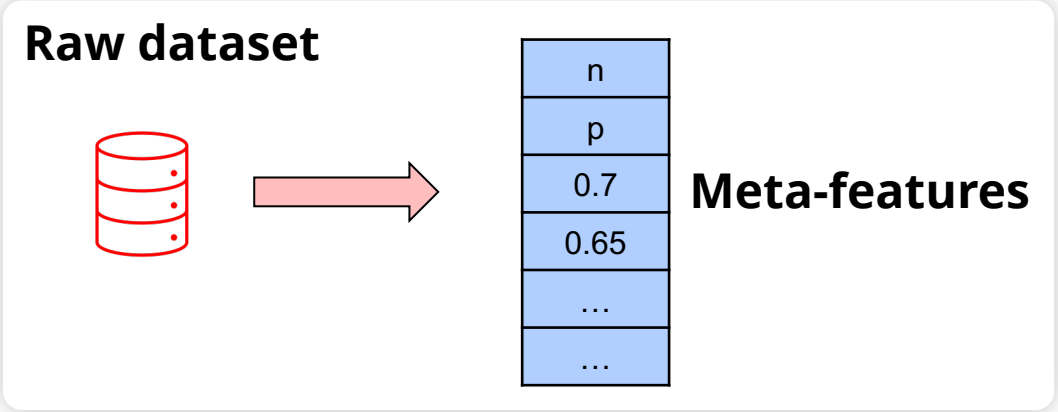
1. **Problem Formulation** of unsupervised OD model selection (UOMS) by meta-learning

- **Performance**: On avg. the **top 18%** model from 300+ models
- **Speed**: ≤**1s** for most datasets



Recap: 1st OD Model Selection Framework

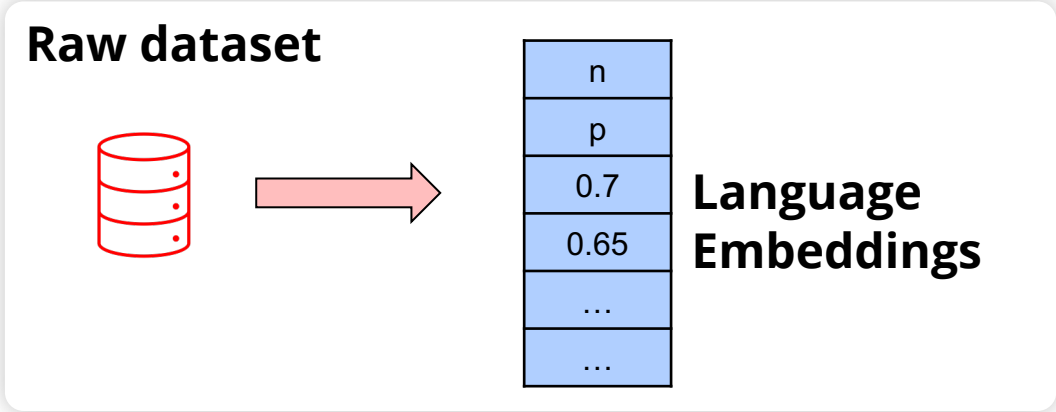
1. **Problem Formulation** of unsupervised OD model selection (UOMS) by meta-learning
2. **Data embedding and OD meta-features**



USC Viterbi School of Engineering

Recap: 1st OD Model Selection Framework

1. **Problem Formulation** of unsupervised OD model selection (UOMS) by meta-learning
2. **Data embedding and OD meta-features**
3. **Language embeddings for datasets and models**



USC Viterbi School of Engineering

Summary of Automated OD/OOD Algorithms

Work	Year	Algorithm Selection	Hyperparameter Optimization		Meta-Learning	Training-free at Test time
			continuous	discrete		
IPM	2021	✓	✓	✓		
MetaOD	2021	✓		✓	✓	✓
ELECT	2022	✓		✓	✓	
HPOD	2022	N/A	✓	✓	✓	
ADGym	2023	✓	✓	✓	✓	✓
Hyper	2024	N/A	✓	✓	✓	✓
MetaOOD	2025	✓			✓	✓

Model Selection and Architecture Search

MetaOD and MetaOOD illustrated choosing a model for a new dataset.

SELECT A MODEL

Dataset information

↓

Candidate models

↓

Recommended model

SELECT AN ARCHITECTURE

layer count
widths
connections

These choices define one candidate network.

Next: search the architecture as another model configuration.

Course reading route. AD-LLM case: Block 10.

P3: Neural Architecture Search

Neural nets may be considered as a computational graph, and most of time it is sequential. For **Neural architecture search (NAS)**, we are trying to decide:

1. Number of layers
2. Types of layers: dense, conv, pooling,
3. Hyperparameters of layers
4. Branching/join/skips

Approaches: HPO methods can search architecture choices.

Strategies: random, evolution, RL and gradient-based search.

AutoML Evaluation and Benchmarks

Compare searches under matched tasks, search spaces and compute budgets.

Search-space coverage	A benchmark measures a defined set of candidate architectures.
NAS-Bench-101	A public benchmark for reproducible architecture-search experiments.
NAS-Bench-201	Extends benchmarking to multiple datasets and richer diagnostics.

A benchmark result has limits set by its evaluation conditions.

Course schematic. Sources and assumptions in speaker notes.

Training Choices to Carry Into Your Project

Use a sequence of experiments that you can explain and reproduce.

Data and evaluation	A sound split and metric make results interpretable.
Training and adaptation	Choose updates that fit the task and available budget.
Next topics	Use the current course schedule for upcoming lectures.

Keep the evidence for why each change helped.

Course teaching example. Sources and assumptions in speaker notes.